Understanding the Odin Programming Language

by Karl Zylinski

Version 1.10 - April, 2026

Contents

1: Introduction

- 1.1: Why learn Odin?
- 1.2: Language overview
- 1.3: Why write a book?
- 1.4: How to read the book
- 1.5: Installing the Odin compiler
- 1.6: If you get stuck
- 1.7: Acknowledgements
- 1.8: Copyright notice
- 1.9: Let's go!

2: Hellope! A tiny program

- 2.1: Compile and run the code
- 2.2: Line-by-line
- 2.3: What does the period in `odin run .` mean?
- 2.4: Compile without running
- 2.5: No need for semicolons
- 2.6: Where is the code inside `core:fmt`?
- 2.7: Compile a single Odin source file
- 2.8: There's much more!

3: Variables and constants

- 3.1: Declaring variables and assigning values
- 3.2: Another variable type and casting
  - 3.2.1: Something strange?
- 3.3: Constants
- 3.4: Type inference defaults
  - 3.4.1: Constants of explicit type
  - 3.4.2: Benefits of Untyped Types
- 3.5: Basic types summary
- 3.6: Untyped Types summary
- 3.7: Video

4: Some additional basics

- 4.1: Procedure parameters and return values
- 4.2: If statements
- 4.3: Loops

4.4: Fixed arrays
4.5: All together

5: Making new types

5.1: Structs
5.1.1: Structs within structs
5.1.2: `using` on struct fields
5.1.3: `using` and "parenting"
5.1.4: No methods
5.2: Enums and switch
5.2.1: Switch
5.2.2: Explicit enum numbering
5.2.3: Backing type
5.3: Unions
5.3.1: The zero value of a union
5.3.2: The union's tag
5.3.3: Maybe
5.3.4: Raw union: C-style unions
5.3.5: Struct variants using unions
5.3.6: Using unions for state machines

6: Pointers

6.1: A procedure that modifies an integer
6.2: nil: the zero value of a pointer
6.3: A pointer to a struct
6.4: Copying a pointer and the address of a pointer
6.5: Under the hood: Addressables

7: Procedures and scopes

7.1: Default parameter values and named arguments
7.1.1: Example: Allocator parameter with default value
7.2: Multiple return values
7.3: Named return values
7.4: Nested procedures and captured variables
7.5: Parameters are always immutable
7.6: Don't use pointer parameters for the sake of optimization
7.6.1: Pointer and immutable reference aliasing
7.7: Explicit overloading
7.8: Run procedures on startup and shutdown
7.9: The scope and the stack
7.10: Scopes within scopes
7.11: defer: Making things happen at end of scope

7.11.1: Don't overuse defer

8: Fixed-memory containers

8.1: Fixed arrays revisited
8.1.1: Where does the memory live?
8.1.2: Blowing up the stack
8.1.3: Vectors and array programming
8.1.4: Passing fixed arrays to procedures
8.2: Enumerated arrays
8.3: Fixed Capacity Dynamic Arrays

9: Introduction to manual memory management

9.1: What is manual memory management?
9.2: Dynamic arrays
9.2.1: Creating a dynamic array and appending items
9.2.2: Deallocating a dynamic array
9.2.3: Removing items
9.2.4: Preallocated dynamic arrays
9.2.5: Under the hood: `append` and `Raw_Dynamic_Array`
9.2.6: Exercise: Understanding Dynamic Arrays
9.3: Dynamically allocated variables
9.3.1: Example: Dynamically allocated struct
9.3.2: Example: Dynamically allocated fixed array
9.4: The default allocator: `context.allocator`
9.4.1: The heap allocator
9.4.2: The WASM allocator
9.5: The separation of pointer and allocated memory
9.6: free and delete
9.7: Temporary allocator
9.7.1: Placing the `free_all`
9.8: "Scripts" that allocate memory

10: More container types

10.1: Slices: A window into part of an array
10.1.1: Creating a slice
10.1.2: Slice internals
10.1.3: Slice example: Considering 10 elements at a time
10.1.4: Prefer to pass slices
10.1.5: Slices with their own memory
10.1.6: Slice literals
10.2: Maps
10.2.1: Creating and destroying maps
10.2.2: Maps and allocations
10.2.3: Iterating maps

10.2.4: When to use maps
10.2.5: Making a set using a map
10.3: Custom iterators

11: Strings

11.1: String variables and string constants
11.1.1: Combining string constants
11.2: Iterating strings
11.3: Creating substrings: String slicing
11.4: Construct strings using `fmt`
11.5: Construct strings using a string builder
11.5.1: `fmt` and string builder
11.6: What's a string internally?
11.6.1: Ambiguities when talking about "string constants"
11.7: Deallocating strings mixed with string constants
11.8: Null termination and `cstring`
11.9: Grapheme clusters
11.10: Why does the slice operator count bytes?
11.11: Under the hood: Manually decoding UTF-8
11.12: UTF-16: Windows API example
11.12.1: Setting the window title using a string constant
11.12.2: Setting the window title using a string variable
11.12.3: Fetching the window title

12: Implicit context

12.1: `context.allocator`
12.1.1: Setting `context.allocator` vs passing explicit allocators
12.2: `context.temp_allocator`
12.3: `context.assertion_failure_proc`
12.4: `context.logger`
12.5: `context.random_generator`
12.6: `context.user_ptr` and `context.user_index`
12.7: How the context is passed

13: Making manual memory management easier

13.1: Tracking memory leaks
13.1.1: Setting up the tracking allocator
13.1.2: How does it work?
13.1.3: Making memory leak reports clearer
13.2: Arena allocators
13.2.1: No individual deallocations

13.3: Choosing an arena allocator
13.3.1: Growing virtual memory arena
13.3.2: Static virtual memory arena
13.3.3: Fixed buffer arena
13.4: Debugging memory mistakes
13.5: Improving your understanding

14: Parametric polymorphism: Writing generic code
14.1: Procedures with polymorphic parameters
14.2: Explicitly passing a type to a procedure
14.3: Parametric polymorphism with structs and unions
14.4: The unified concept of parametric polymorphism
14.5: Specialization
14.5.1: Specialization and distinct types
14.6: Video

15: Bit-related types
15.1: Binary representation of integers
15.2: Bit sets
15.2.1: The bits inside the `bit_set`
15.2.2: Force backing type
15.2.3: `bit_set` using letters and numbers
15.3: Bit fields
15.3.1: When to use bit sets and bit fields

16: Error handling
16.1: Errors using `bool`
16.2: Errors using `enum`
16.3: Errors using `union`
16.4: `or_return`
16.5: `#optional_allocator_error`
16.6: `#optional_ok`

17: Package system and code organization
17.1: What is a package?
17.2: Creating a package
17.3: The package name
17.4: Make sure your packages are independent
17.5: Breaking cyclic dependencies using callbacks
17.6: File and package private

17.7: Making your own collections
17.8: Video

18: You (probably) don't need a build system

18.1: Compiling your program
18.2: Importing packages
18.3: Importing non-Odin libraries
18.3.1: Caveats
18.4: Platform specific code
18.5: Compiling multiple binaries
18.6: More complicated build setups
18.7: Summary

19: Reflection and Run-Time Type Information (RTTI)

19.1: Example: Enum member from string
19.2: Case study: Enum dropdown menu
19.3: When to not use reflection
19.4: Learning more about reflection

20: Data-oriented design

20.1: Avoid doing many small heap allocations
20.1.1: Performance differences when iterating
20.1.2: How much faster is it?
20.1.3: Problem: Pointers get invalidated when the array grows
20.2: Structure of arrays (SoA)
20.2.1: How much faster is SoA?
20.2.2: When do I use SoA arrays?
20.3: Conclusion and further reading

21: Making C library bindings (Foreign Function Interface)

21.1: The C library
21.2: The Odin bindings
21.3: Testing our bindings
21.4: More examples
21.5: Video

22: Debuggers

22.1: VS Code
22.2: RemedyBG
22.3: RAD Debugger

23: Odin features you should avoid

- 23.1: `using` on variables and parameters
- 23.2: `any` type
- 23.3: Dynamic literals

24: A tour of the core collection

- 24.1: Reading and writing files (`core:os`)
- 24.2: Read and write structs as JSON (`core:encoding/json`)
- 24.3: Virtual memory arena allocators (`core:mem/virtual`)
- 24.4: Working with strings (`core:strings`)
- 24.5: Using the logger (`core:log`)
- 24.6: Start a new thread (`core:thread`)
- 24.7: Do things periodically (`core:time`)
- 24.8: Linear algebra math operations (`core:math/linalg`)
- 24.9: Create random numbers and shuffle arrays (`core:math/rand`)
- 24.10: C types and using the C standard library `core:c` and `core:c/libc`
- 24.11: Opening a window using the Windows API `core:sys/windows`
- 24.12: Load symbols from a dynamic library `core:dynlib`

25: Libraries for creating video games

- 25.1: Make a game using raylib `vendor:raylib`
- 25.2: Write platform-independent rendering code using SOKOL
- 25.3: Write rendering code using vulkan (`vendor:vulkan`)
- 25.4: Create 2D game physics using Box2D (`vendor:box2d`)

26: A few more things...

- 26.1: `#load` and `#load_directory`
- 26.2: `@rodata`
- 26.3: Matrices
- 26.4: Quaternions
- 26.5: Struct tags
- 26.6: WASM
- 26.7: Variadic procedures
- 26.8: `deferred_X` attributes
- 26.9: Making libraries compile cleanly
- 26.10: Code generation

27: Where to find more Odin resources

28: Thanks for reading!

29: Appendix A: Handle-based array

30: Appendix B: Using only fixed arrays

31: Appendix C: `gui_dropdown` from CAT & ONION

32: Appendix D: Box2D and raylib

33: About the author

Chapter 1

Introduction

Hello! If you've been trying to learn the Odin Programming Language, but feel like there are basic concepts that you don't fully understand, then this book is for you. Also, if you have never programmed in Odin before, but have some basic programming experience, then this book should be a good starting point as well.

By "basic programming experience", I assume you understand variables, procedures (functions), if-statements, basic arrays and loops. Don't worry though, I'll go over how to do those things in Odin.

I also assume that you know how to run programs from a command-prompt or terminal.

Why learn Odin? 1.1

I wouldn't say there is one big reason for learning Odin, but rather a big number of small reasons. Odin doesn't try to fundamentally solve any new problems. There are many "big agenda" languages out there, meaning languages that try to introduce completely new ways of thinking about programming. Odin is not one of them. The reasons for learning Odin may vary depending on your background. Here are two examples of such backgrounds:

If you come from a higher level language such as C#, JavaScript or Python, but want to learn a low level language, where performance and control is easily attainable, then Odin may be for you. I think it is a great first low level language. Odin's modern features and simplicity cuts out some noise you find in older low level languages. This makes the programming fun and lets you focus on the low level concepts, without getting overwhelmed. I usually say that Odin is "low level with high level feeling".

A low level language is "closer to the hardware" than a high level language. There are less layers of abstract things between your code and the computer. In a low level language you have more control, but less things are done automatically for you.

If you've used the C Programming Language, but don't find it as fun to program in as it could be, then Odin may be for you. I have worked on big video game engines written in C. I would say that Odin point-for-point addresses many of the issues I've had with C, while also incorporating some of my favorite C best practices, straight into the language.

From C / C++

For example, Odin makes heavy use of designated initializers and zero-is-initialized, which is a great way to program C. It also comes with built-in support for custom allocators and Structure of Arrays (SoA).

If you don't know C or C++, then don't be intimidated by the "From C / C++" bubble just before this paragraph. I try to keep most information that assumes you know C or C++ within those bubbles. They are safe to skip.

As a final answer to "Why learn Odin?", I'll give a short anecdote: In 2023-2024 I used Odin to create my own video game: **CAT & ONION** . Never before have I been able to finish and ship a whole game, all by myself. In the past I always felt some degree of friction from whichever programming language I used. That friction usually built up over time, until I dropped the project. With Odin enough friction was removed, making it possible for me to ship something.

You can also get it on **itch.io** . That version comes with the full Odin source code!

Language overview 1.2

Let's talk a bit about what Odin is, so you have some reasonable expectations before reading more.

Odin is a *compiled* language. You have to run the source code through the Odin compiler to produce an executable, that you can later run without having to involve the Odin compiler.

Odin is a *simple* language. Great care has been put into the design of the language, making the different parts work together as a simple whole.

Odin uses *manual memory management*. You manually decide the lifetime of dynamically allocated memory. This means that you are in control of when dynamic memory is deallocated. It is not done automatically for you. This gives benefits with regards to performance and memory usage. This may seem scary to programmers coming from languages that employ automatic memory management. But fear not, Odin comes with built-in ways to track memory leaks and practical ways to handle temporary memory allocations. In chapters 9 to 13 we will look at different aspects of manual memory management.

Odin is *not object-oriented*. Your code will mainly use procedures (functions), if-statements, loops and structs to get things done. This makes the code very simple. The object-oriented programmer will find new ways to write reusable and generic code, that in general has better performance characteristics than object-oriented code.

Odin is well suited for *data-oriented programming*. This means that it is easy to write Odin code that utilizes the CPU in an efficient manner. We will discuss data-oriented programming in chapter 20.

Finally, the language really *gets out of the way* and lets you do things your way. This is one of many reasons why "joy of programming" is marketed as an important feature of the language.

Why write a book? 1.3

I wanted to write a book about Odin for two reasons. Firstly: Odin's online documentation is missing some things and it is not always very pedagogical. By transforming several years of Odin programming experience into a book, I aim to bridge that gap.

Secondly: I've been active within the Odin community for several years. During that time I've seen some questions and confusions over and over again. Sometimes these questions can only be properly answered if you take a step back and look at the big picture of why something in the language works the way it does.

With that in mind I decided to write this book. I want to provide holistic explanations of things people often find confusing, and in doing so provide a deep understanding.

How to read the book 1.4

Each chapter deals with a certain part of the language. I don't fully cover every aspect of the language. Instead, each chapter tries to explain the basics of a certain language feature, and then follow that up with more advanced topics. Some of these advanced topics are actually not hard to understand, it's just that some things may not be apparent until you've used the language for a long time. I hope that I can save you some of that time.

> You'll see bubbles like this one every now and then. In these you'll find extra clarifications as well as some trivia that is interesting, but not vital.

This book is not meant to be a "tutorial" where you follow step-by-step instructions to reach a specific outcome. Instead, I've tried to make this book enlightening: On top of explaining *how* to write Odin code, I also provide explanations of *why* things work the way they do. Sometimes I'll talk about how things work "under the hood". I strongly believe that understanding your tools will make you a better craftsperson.

I've written this book in an informal style, hoping that it can be enjoyed in its entirety. That said, if you already know Odin fairly well, then I think you can skip directly to **chapter 6**.

Installing the Odin compiler 1.5

I will not go over how to install the Odin compiler. For that I refer to the official Getting Started guide on the Odin website.

After installing the compiler, I assume that you have the folder where you installed it in your path. Meaning that you can run `odin` from a command prompt or terminal anywhere on your computer.

Should you have any trouble installing the compiler, then I also have a video on the topic, targeted at Windows users:

https://www.youtube.com/watch?v=yq5VabsGz_4

Note that I install Odin from source in this video. This is not a requirement, you can use the pre-built releases that the Getting Started page mentions.

If you get stuck 1.6

If you can't figure out how to do something, and you can't find any help in this book, then I recommend checking these resources, in the order stated:

1. Check the official language overview. It is not exhaustive, but searching around in there will answer many questions.
2. Look inside demo.odin. It is an example that comes with the compiler. It showcases many different features of the language. You'll find a local copy in `<odin>/examples`.
3. Search in the core and base library collections. Pull the `<odin>/core` and `<odin>/base` folders into your text editor and just search around in there. Those library collections are well written and easy to read. See my video "Exploring Odin without leaving your editor" for more information.
4. Ask in the `#beginners` channel on the Odin Discord server. It is a chat server that has thousands of members. Many questions get answered very quickly.

When I write `<odin>` in a path I refer to the location where you installed the Odin compiler.

Finally, you're warmly welcome to join my own Discord server (not the same as the official Odin Discord server mentioned above). It's a friendly place where you can ask questions about Odin and also discuss game development, or tell me about that typo you found in this book!

There's also a channel for posting cat pictures!

Acknowledgements 1.7

Thanks to all my supporters on patreon.com for their donations! It truly helped a lot.

Thanks to my proofreaders: Tobias Möllstam, PythagoRascal, blob and Jaysen.

Thanks to Bill Hall (gingerBill) for proofreading, and for creating the Odin Programming Language!

Thanks to Geraldine Lee for proofreading and being a stellar partner while I've been stuck with my nose in the computer.

Copyright notice 1.8

Please don't redistribute this book without my explicit permission. If you somehow got hold of this book without paying for it, then please consider buying a copy at **odinbook.com** .

Let's go! 1.9

That's it for the introduction, enjoy the book!

/Karl Zylinski (April, 2026)

Chapter 2

Hellope! A tiny program

In this chapter we'll look at just about the smallest possible Odin program. It's a program that just shows the message "Hellope!" in a console. First I'll show the code and explain how to compile and run it. Then I'll talk about what each line of code does. Finally, I'll go over some points that I think are good insights related to this tiny program.

By *console* I mean a command prompt (Windows) or a terminal (macOS / Linux).

"Compilation" is the process by which code gets translated into an executable that your computer can run.

Compile and run the code 2.1

Here's the code of our tiny program:

```
package hellope

import "core:fmt"

main :: proc() {
    fmt.println("Hellope!")
}
```

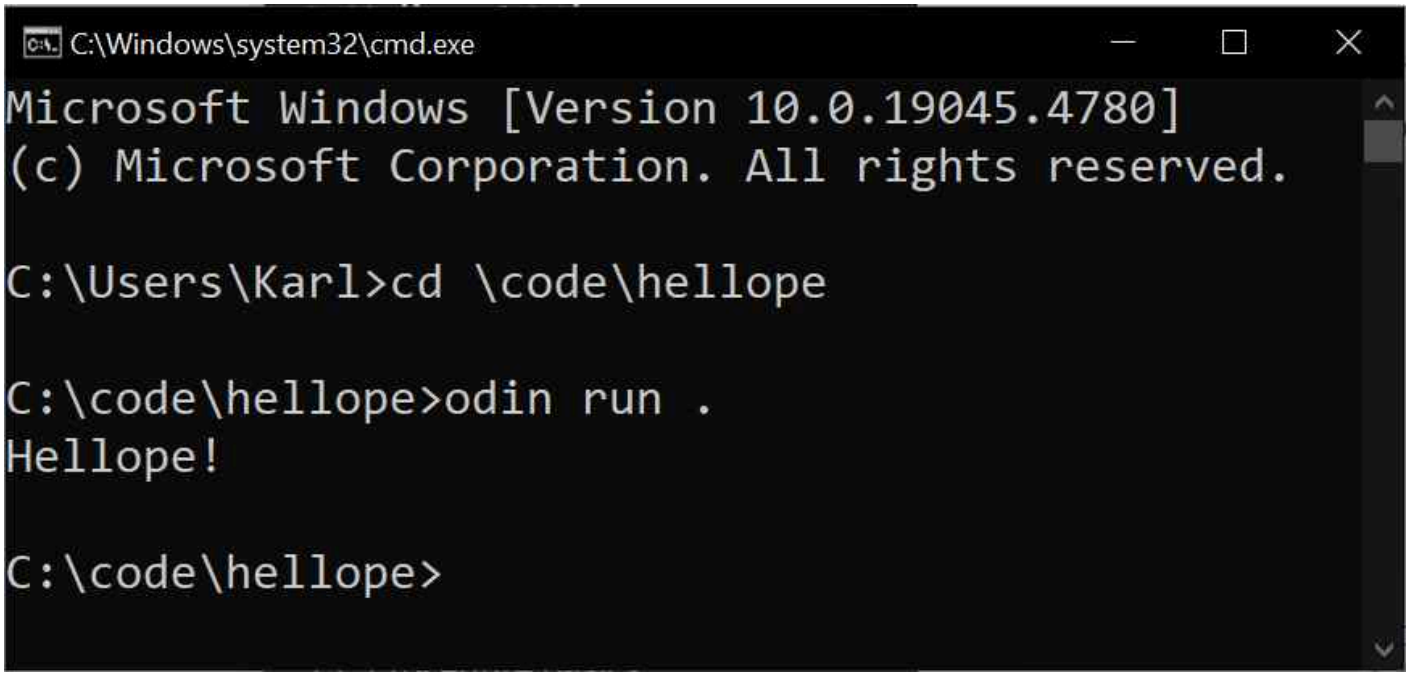
Open your favorite text editor and copy the code above into a file. Save the file as `hellope.odin` inside an empty folder.

Popular text editors for writing Odin code are Sublime Text, VS Code and Vim.

Use a command prompt or terminal to navigate to the folder where you saved `hellope.odin`. Once inside it, type:

```
odin run .
```

Please include the period, with a space before it! This will compile your program into an executable, and then run it. The program should print "Hellope!".



Shows how I use the command prompt `cmd` on Windows to navigate to the folder where I saved `hellope.odin`. I compile and run the program using `odin run .`

If you use macOS or Linux, then you need to use forward slashes in the path:

```
cd /path/to/your/code
```

Note that this is a console application. The message will be printed to the command-prompt or terminal. No graphical window will be opened.

Print is a funny word. It's an old hand-over from early programming languages and literally refers to a printer. These days print goes to a "standard output stream", which often means some kind of text console.

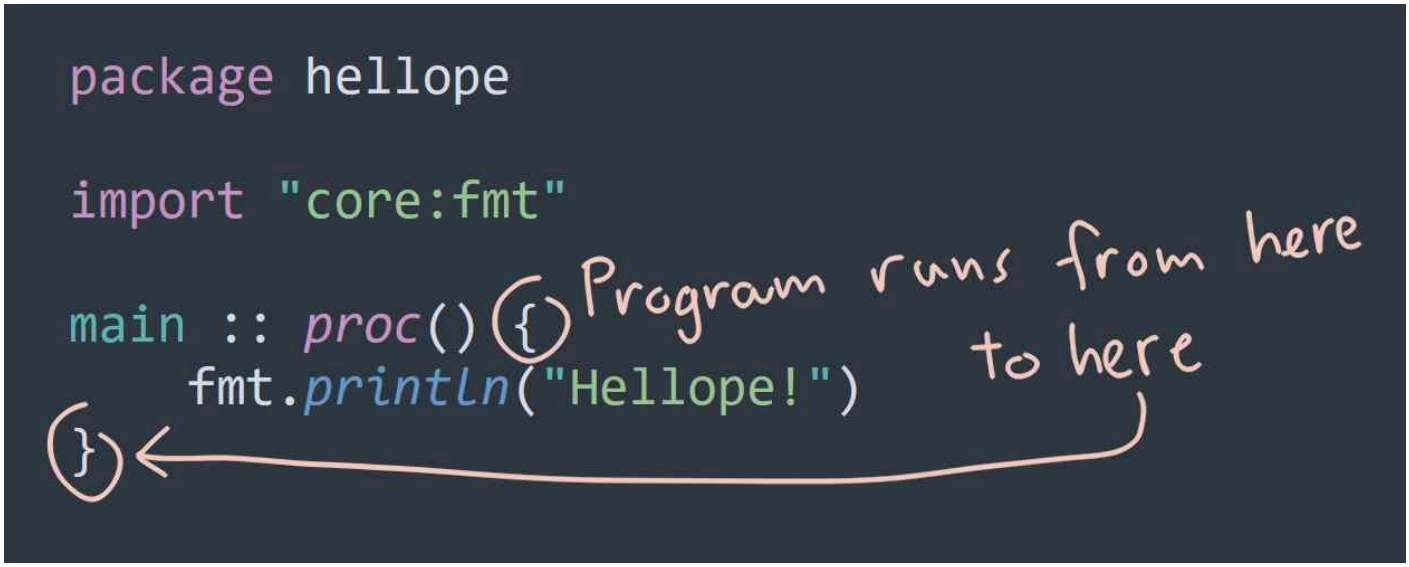
Line-by-line 2.2

Let's go through the program line-by-line, starting with this one:

```
main :: proc() {
```

This declares a new proc called `main`. Proc is short for *procedure*. Some languages refer to procedures as *functions*. The `main` procedure is where the program starts. The program runs from the `main` procedure's opening curly brace `{` to the closing curly brace `}`. I will talk more about procedures in chapter 7.

Strictly speaking, functions are *side-effect free* procedures, meaning that they do not modify anything outside the procedure. Since there's no promise of being side-effect free in Odin, the term *procedure* is used.



The only thing the `main` procedure does is print "Hellope!". It does this using the `fmt` *package*. The line

```
import "core:fmt"
```

fetches the `fmt` package from the *core collection*. The core collection comes with the Odin compiler and contains an enormous amount of useful code. The `fmt` package contains procedures that can print text and also *format* strings. The procedures inside `fmt` can be used by typing `fmt.some_procedure_name()`. In this case we use `fmt.println` and pass it the message to put on the screen:

```
fmt.println("Hellope!")
```

`fmt` is short for "format". Odin doesn't always try to keep names short, but I guess `fmt` is used so often that the name was made a bit more keyboard efficient.

The line at the top of the program says

```
package hellope
```

This is the package name. For now, just note that this line must be the same for all Odin files within a folder. The name must also be unique project-wide, in the sense that no other imported package uses the same name. I go over this in more detail in **chapter 17**.

Let's now look at some interesting details about this example program.

What does the period in `odin run .` mean? 2.3

The `.` in `odin run .` refers to the current folder. This means that the Odin compiler will take all files ending with `.odin` in the current folder, and compile them as a package. By default `odin run .` assumes that you want to create an executable program based on this package. It will look for a procedure called `main` inside the package, which will be the starting location of your program. When it has created the executable, it will run it.

> There are other options than creating executables. For example you could create a library. A library won't need a `main` procedure.
>
> For more information, run `odin build -help` and look into the `-build-mode` flag.

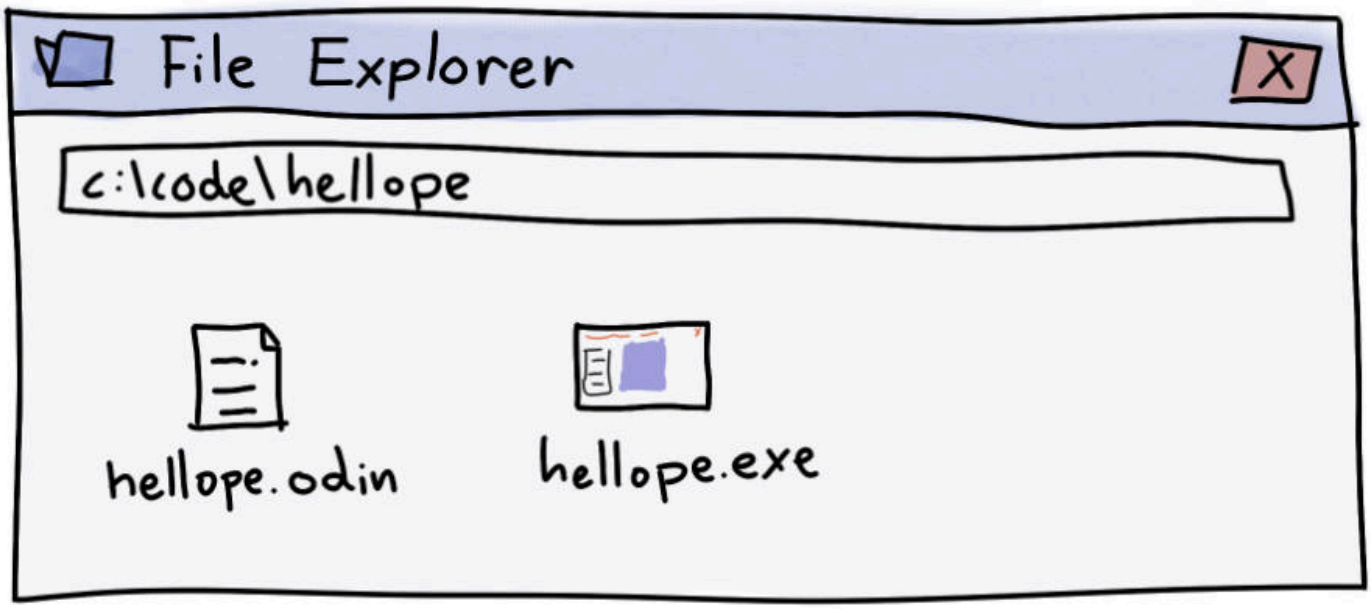
If you had more than one Odin source file in the current folder, then all of them would be compiled as part of your program.

Compile without running 2.4

You can replace `run` with `build`. Meaning that you execute the following:

```
odin build .
```

This will compile your code into an executable, without running it. You'll be able to find the resulting executable within the same folder as your code. The name of the executable will be the name of the folder.



The `odin build .` command results in an executable that you can later run, without involving the Odin compiler.

If you are on Windows but can't see the `.exe` or `.odin` file endings within the file explorer, then please go into the *View* tab and check *File name extensions*. Computers are pretty much unusable for software development without enabling this.

Actually, `odin run .` also creates an executable, and then runs it. But the executable is automatically deleted when it finishes running. To skip the deletion, add `-keep-executable`:

```
odin run . -keep-executable
```

No need for semicolons 2.5

There's no need to put a semicolon (`;`) at the end of each line, as is required in many other languages.

Semicolons are *optional*, but I would encourage to simply not use them. The absence of semicolons fits well with the aesthetics of Odin. However, you can use them if you really want to!

Also, if you really want to make sure that you don't accidentally use them by old habit, then there is a compiler flag to make any unnecessary semicolon into an error: `-vet-semicolon`

Compiler flags are added when you run the compiler:

```
odin run . -vet-semicolon
```

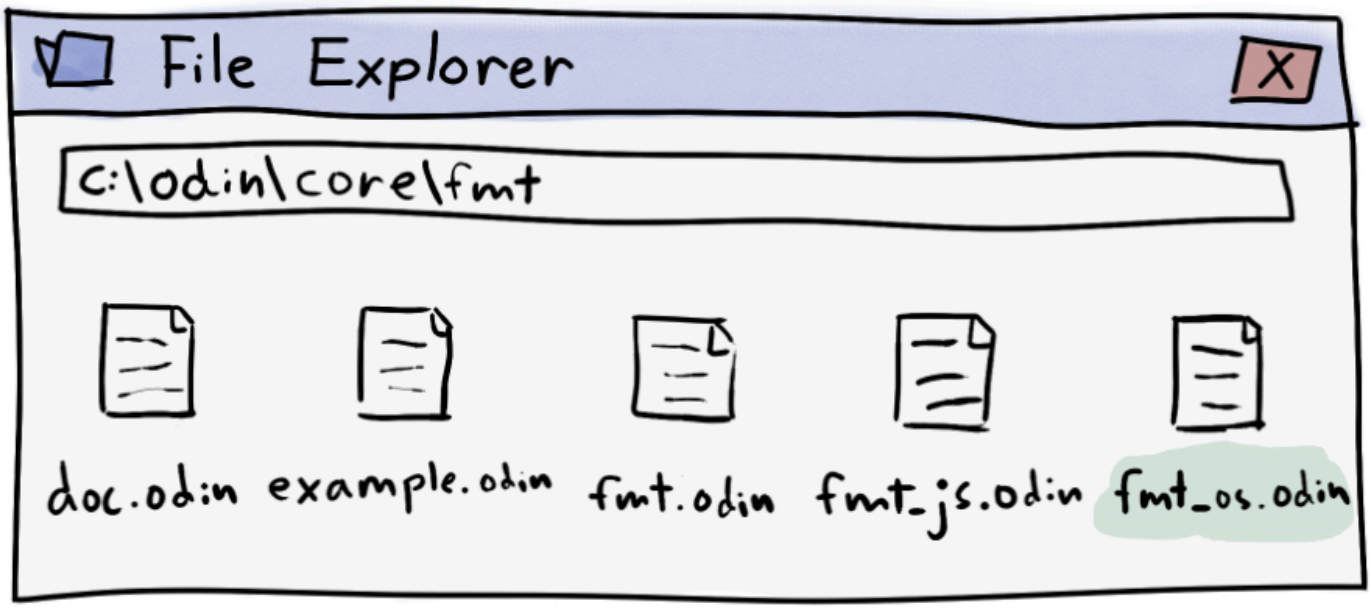
You can also try the `-vet` flag. That one includes a whole bunch of other helpful checks, such as making it an error if you have unused imports.

Personally, I like compiling with `-vet -strict-style`, which enables most vetting rules and also enforces the code style of `core`.

Where is the code inside `core:fmt`? 2.6

Packages are just folders with Odin files inside them. The folder with your `hellope.odin` file is one package. And from your `hellope` package you also import the `core:fmt` package, which is also just a folder. But where is that `fmt` folder?

It's part of the core collection. The core collection comes as source code along with the Odin compiler. This means that you can look at what the code inside it does. I encourage you to do this, as you can learn a lot from reading that code. You'll find the `core:fmt` package in `<odin>/core/fmt`. The `println` procedure you used can be found in `fmt_os.odin` within that folder.



The contents of `<odin>/core/fmt`*. The* `println` *procedure can be found inside* `fmt_os.odin`

Compile a single Odin source file 2.7

If you want to just compile a single Odin file instead of all the files within a folder, then add the `-file` flag:

```
odin run hellope.odin -file
```

This treats a single file as a complete package instead of fetching all the files within a folder.

There's much more! 2.8

This chapter was just a quick crash course, an appetizer. Throughout this book we'll return to many of these concepts in depth.

From here on, most chapters will focus on specific areas of the language.

Chapter 3

Variables and constants

Odin has a syntax for declaring and assigning variables that will look unfamiliar to people coming from JavaScript, C or C#. But it might look familiar to those who have programmed in Pascal or Go.

The syntax of a language defines the form of the language; how it looks.

In this chapter we'll look at how to create variables. We'll also look at how to convert variables from one type to another. As we shall see, there is something *strange* afoot with how plain numbers like `7` work in relation to variable types, which will bring us to talking about *constants*. By the end of the chapter, I hope you'll see how Odin's interplay between variables and constants creates an elegant whole.

Declaring variables and assigning values 3.1

Variables are, like the name suggests, possible to *vary*. Meaning that the code can change a variable's value.

You declare a variable like this:

```
number: int
```

This variable is named `number`. It is of type `int`, short for integer. Note the `:` between the name and the type. It is the symbol Odin uses for declarations.

Odin is a strongly typed language, meaning that the language is strict about types: You can *only* put integers into integer variables. You *cannot* change the type of a variable after it has been declared.

What value does `number` have? Since we didn't specify any, it will have the value *zero*. This is true for all variables of all types. Technically speaking, since we didn't specify any value, then the *underlying memory* that holds the variable's value will be all zeroes. As we'll see in the **summary at the end of this chapter**, this can have a few different meanings for non-numeric types.

From C / C++

In C, writing just `int number;` will leave the variable non-initialized. It can contain any garbage memory! This is almost never what you want, and a source of many bugs. In Odin it will be initialized to zero by

default. However, if you want to leave a variable non-initialized, then you can write:

```
number: int = ---
```

This is mostly useful in very specific, performance-sensitive situations.

You can assign a new value to an already existing variable:

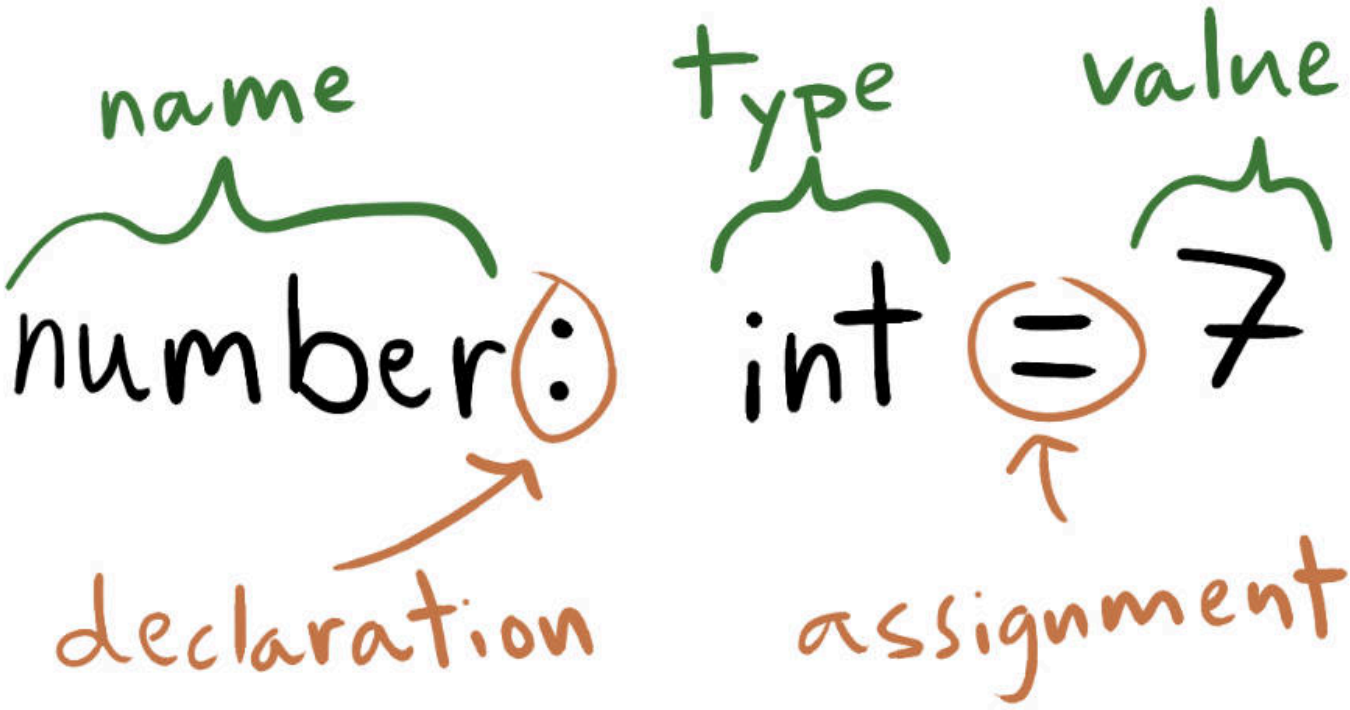
```
number: int
number = 7
```

Note that doing `number = 7` with no pre-existing variable called `number`, will fail to compile.

If you want to both declare a variable *and* assign a value to it, then you can combine the `:` and the `=` on a single line, like this:

```
number := 7
```

This creates a new variable `number` of type `int` and gives it the value `7`. In this case you didn't have to say what type `number` should have. Instead, the type is *inferred* from the value on the right hand side. This means that the compiler looks at the value on the right and decides what type the variable should have based on the value. In this case the type is inferred to `int`.



The anatomy of declaration and assignment. Remove the type `int` to infer the type from the value on the right. Remove `= 7` to make this a declaration without an assignment. Remove `: int` to make this an assignment to a previously declared variable.

Here's a small program that creates a variable, prints its value, changes it and then prints it again:

```
package variable_example

import "core:fmt"

main :: proc() {
    number := 7
    fmt.println(number) // prints 7
    number = 12
    fmt.println(number) // prints 12
}
```

Putting `//` in front of some text in your code file creates a comment. The comment goes from `//` to the end of the line. You can also write

`/* comment */`

to create a multi-line comment. Multi-line comments can be nested.

Note the usage of `:=` on the line where we declare and initialize `number` and that we use a plain `=` when we later change its value.

Another variable type and casting 3.2

We just saw the type `int`. A variable of that type can only store whole numbers. If we want to store any decimal number, meaning a number that can have a fractional part, then we can use the type `f32`:

```
decimal_number: f32
```

The `f` stands for "floating point number", usually called just "float". A floating point number is a way of representing a decimal number. The `32` means that it uses 32 bits of memory (or 4 bytes) to store the decimal number. Again, since we are not specifying a value, `decimal_number` will be zero by default.

Using more bits for your floats increases the biggest number it can store as well as its precision. However, more bits eat more memory. So many programmers use `f32` by default, even though most computers have 64 bit processors.

For example, video games use floats a lot, but very often the `f32` type is precise enough.

My use of the word *decimal* is a bit sloppy. A decimal number implies that the base 10 is used. That's not required. I just mean a number that can have a whole and a fractional part, which many people refer to as

decimal numbers.

When we previously wrote `number := 7`, then the compiler inferred `number` to be of type `int`. How can we declare a variable of type `f32`, but also initialize it to `7`, using a single line of code? There are two ways:

```
decimal_number: f32 = 7
```

or

```
decimal_number := f32(7)
```

Both ways have the same end result. The first one looks like `decimal_number: f32`, but we've added `= 7` at the end. In the second case we have skipped saying what type `decimal_number` is, and instead rely on the type being inferred from the right hand side: `f32(7)`. This means that we take `7` and *cast it*, or convert it, to the type `f32`.

`f32(7)` almost looks like we are running a procedure called `f32` and feeding it the value `7`. Whenever you see a *type name* followed by parenthesis like this, then it is a cast.

A decimal number can of course have a decimal point followed by a fractional part:

```
decimal_number: f32 = 7.42
```

But what happens if we do the following?

```
decimal_number := 7.42
```

In this case `decimal_number` will be inferred to have the type `f64`, *not* `f32`.

So when you write `decimal_number := 42.07`, then there is a default choice of type `f64`. We shall soon see where these default choices come from.

Something strange? *3.2.1*

Now we'll get to the stuff that may initially seem strange, but will end up being *very useful* once you get it. The following code compiles just fine:

```
number := 7
decimal_number: f32 = 7
```

However, the next snippet does *not* compile. It will complain that it cannot automatically convert `int` to `f32`:

```
number := 7
decimal_number: f32 = number
```

It will compile if you do an explicit cast:

```
decimal_number := f32(number).
```

Why is that? At a surface level, `7` seems to be of type `int` because typing `number := 7` makes a variable of type `int`, where the type `int` is inferred from the `7` on the right hand side. But this does not make any sense because you can also assign `7` to a variable of type `f32` without an error:

```
decimal_number: f32
decimal_number = 7
```

`decimal_number: f32 = 7` also works.

As I've said Odin is strongly typed, so if `7` was of type `int` then assigning it to an `f32` would be a compilation error.

The answer is that a plain `7` is *not* of type `int`. It is a *constant*, and constants have a slightly different system for types than variables do. To understand this fully, let's take a step back and talk about what constants are.

Constants 3.3

Constants are things that only exist at compile time. Since they only exist at compile time, they cannot be changed while the program is running.

By compile time I refer to things that happen as the Odin compiler is running. Anything that happens later, as the program is running, is referred to as run time.

I already said that a plain number like `7` is a constant. But you can also create *named* constants:

```
CONSTANT_NUMBER :: 12
```

We use `::` to create named constants, *not* `:=` that we used to create variables. You can assign a constant to a variable like this:

```
CONSTANT_NUMBER :: 12
number := CONSTANT_NUMBER
```

I use `SCREAMING_SNAKE_CASE` when naming constants. That's just **the naming convention used by the core collection** , which I have adopted.

The type of `number` will be inferred to `int`. Why? The two lines above are actually identical to just typing:

```
number := 12
```

As your program is getting compiled, you can think of named constants having their value "copy-pasted" into wherever you use their name. You can also think of a stand-alone `12` as a nameless constant.

Now, I said that constants have a slightly different type system compared to variables. One could say that constants have a slightly *weaker* type system. The type of `CONSTANT_NUMBER :: 12` is *Untyped Integer*. An Untyped Integer can be implicitly converted to any integer or floating point type, as long as that type can 'accommodate' the value.

This is why `decimal_number: f32 = 7` is allowed. `7` is a constant of type Untyped Integer, and that type allows for an implicit conversion (or cast) to the type `f32`.

I write Untyped Integer capitalized like this to make it easier to read. I know, the idea of an Untyped Type is quite *strange*. But as you'll find out by programming more Odin, this system is very comfortable without feeling vague.

What's up with the "as long as that variable can 'accommodate' the value" part above? Here's something that will *not* compile:

```
BIG_CONSTANT_NUMBER :: 100000000
small_number: i8 = BIG_CONSTANT_NUMBER
```

`small_number` is of type `i8` (8 bit signed integer), the biggest value it can contain (accommodate) is `127`. `BIG_CONSTANT_NUMBER` is a constant of type Untyped Integer. When the compiler tries to shove `BIG_CONSTANT_NUMBER` into `small_number`, then it sees that the value of this constant is bigger than `127`, so it refuses to do so.

"8 bit integer" means that the computer has eight binary numbers (bits) at its disposal to represent an integer. The biggest number one can represent with 8 bits is `255`. But since signed integers can be

negative, then only half of that range can be used. The range of the 8 bit signed integer is therefore `-128` to `127`.

There are more Untyped Types than just Untyped Integer. Constants containing a decimal point such as `DECIMAL_CONSTANT :: 27.12` are of type *Untyped Float*. An Untyped Float can only be assigned to a floating point variable such as `f16`, `f32` or `f64` (with one exception, which we'll see below). The code below will *not* compile, because we are trying to assign an Untyped Float to an integer variable. The integer variable *cannot accommodate* numbers with a fractional part.

```
DECIMAL_CONSTANT :: 27.12
my_integer: int = DECIMAL_CONSTANT
```

There's an exception to this. If you have an Untyped Float with all zero decimals, then it can be implicitly converted to an integer:

```
DECIMAL_CONSTANT :: 7.0
my_integer: int = DECIMAL_CONSTANT
```

Equivalent to

```
my_int: int = 7.0
```

The integer must still be able to accommodate the value. This will not compile:

```
my_int: i8 = 400.0
```

Again, an `i8` cannot store anything bigger than `127`.

We say that Untyped Floats are allowed to be implicitly converted to integers as long as they don't get *truncated*, meaning that no decimals get cut off.

At the end of this chapter I'll present a list of all the Untyped Types in Odin.

Type inference defaults 3.4

As I mentioned before, when we type something like

```
decimal_number := 13.38
```

then this variable will be of type `f64`. This default is specified by the Untyped Type system that constants use:

- `CONSTANT_NUMBER :: 7` or just a plain `7` are both of type *Untyped Integer*. The default inferred type is `int`.
- `DECIMAL_CONSTANT:: 42.09` or just a plain `42.09` are both of type *Untyped Float*. The default inferred type is `f64`.

These type inference defaults should not be confused with the *implicit conversions* that Untyped Types also allow:

```
decimal_number: f32
decimal_number = 7.42
```

This is allowed because `7.42` is an Untyped Float, which may be implicitly cast to any float type, such as `f64`, `f32` or `f16`. It has nothing to do with type inference defaults.

Constants of explicit type *3.4.1*

You can give a constant a specific type, if you really want to. Note how we wedge a type between the two `:` below.

```
DECIMAL_CONSTANT: f32 : 7.42
```

When using this constant, no implicit type conversions will happen. You'll need to cast the constant to be able to use it as a `f16` or `f64`.

Explicitly typed constants aren't used that much. I have never used them in my own code.

There is a similarity between this syntax and the `:=` syntax used with variables. Compare the line above to the one below:

```
decimal_variable: f32 = 7.42
```

In both cases the type comes after the first `:`. Then you put a `:` or a `=` after the type depending on if you want a constant or a variable.

Benefits of Untyped Types *3.4.2*

The system of Untyped Types means that both these assignments are valid:

```
decimal_32: f32
decimal_32 = 0.5
```

```
decimal_64: f64
decimal_64 = 0.5
```

This may not seem like a big deal, but if you have programmed in C or C#, then you may have been annoyed with having to type `0.5f` every time you want to assign `0.5` to a 32 bit floating point number.

That is because those languages have distinct types associated with literals like `0.5` and `0.5f`. In C `0.5` is a 64 bit floating point number and `0.5f` is a 32 bit floating point number. Odin delays that choice of distinct type, leaving `0.5` untyped for as long as possible, until it really needs to choose a type for it.

A *literal* refers to values written directly in the source code, such as `0.5` and `"cat"`.

Basic types summary 3.5

We've seen the types `f32`, `f64` and `int`. We refer to these as basic types, because they are built right into the language itself. There are more basic types in Odin. Here are some of them:

Signed integers

- Types: `int i8 i16 i32 i64 i128`
- Possible values: Negative and positive whole numbers.
- `int` is equivalent to `i64` on a 64 bit computer and `i32` on a 32 bit computer etc.

Unsigned integers

- Types: `uint u8 u16 u32 u64 u128 uintptr`
- Possible values: Positive whole numbers (including zero).
- `uint` is equivalent to `u64` on a 64 bit computer and `u32` on a 32 bit computer etc.
- `uintptr` is a special unsigned integer that is always of **pointer** size.

Floating point numbers

- Types: `f16 f32 f64`
- Possible values: Positive and negative decimal numbers (numbers with a whole and a fractional part).
- More bits make it possible to store bigger and more precise decimal numbers.
- Unlike integers, you can assign however large a number you want to floats. If it is too big, then the float will take on the value positive or negative infinity.

Boolean types

- Types: `bool b8 b16 b32 b64`
- Possible values: `false` or `true`.
- Zero value: `false`.
- `bool` is equivalent to `b8`. It uses 8 bits, or 1 byte, of memory.
- The specific `b32` etc variants are there mostly for interfacing with other programming languages. Just use `bool` in your Odin code.

By "zero value" I refer to what you get when you type `my_variable: bool`. The memory for this variable will be all zeroes, but what those zeroes are interpreted as varies from type to type. For a `bool` it means `false`. For a `string` it means `""` (empty string).

Strings

- Type: `string`
- Possible values: Text encoded as UTF-8
- Zero value: `""` (empty string)
- There's also a `cstring` type for interfacing with the C programming language. There are procedures in `core:strings` to convert back and forth between `string` and `cstring`.
- Additionally, there are `string16` and `cstring16` types that use UTF-16 instead of UTF-8. These are useful when interfacing with, for example, the Windows Operating System API.
- There's a whole chapter on strings.

From C / C++

In contrast to C, a `string` knows how long it is and does not use null-termination. Therefore `len(some_string)` can tell you the length of the string without having to scan through it in search for a null character. The `cstring` type uses a null-terminator.

Runes

- Type: `rune`
- A single UTF-8 string code point (in many cases a single character, but can be part of a *grapheme cluster*. More about that in the strings chapter).
- Internally a 32 bit signed integer.

You'll find some additional basic types in the official overview.

Untyped Types summary 3.6

Untyped Types are the types used by constants. Since constants only exist at compile time, so do Untyped Types.

There are six kinds of Untyped Types. Each has its own rules for what kind of implicit conversions it allows. Also, each Untyped Type has a default type to pick when type inference occurs.

Untyped Booleans

- The type of any constant of value `true` or `false`.
- Can be implicitly cast to any boolean type such as `bool`, `b32`, `b64` etc.
- Inferred type of `some_variable := true` is `bool` (equivalent to `b8`).

Untyped Integers

- The type of any numeric constant *without* a period, such as `7`.
- Can be implicitly cast to any integer or floating point type *given* that the type can accommodate the value. For example `some_variable: i8 = 10000` will not compile because the type `i8` cannot store numbers bigger than `127`.
- Inferred type of `some_variable := 7` is `int` (equivalent to `i64` on 64 bit systems).

Untyped Floats

- The type of any numeric constant *with* a period, such as `7.42`.
- Can be implicitly cast to any floating point type. Can also be implicitly cast to an integer type, but only if all decimals are zero.
- Inferred type of `some_variable := 23.12` is `f64` (on all systems, even on a 32 bit computer).

Untyped Strings

- The type of any string constant, meaning something between two quotation marks `"a_string"` or between two backticks `` `can be multi-line` ``.
- Can be implicitly cast to `string`, `cstring`, `string16` and `cstring16`.
- Inferred type of `some_variable := "Hellope"` is `string`.

Untyped Runes

- The type of any single character constant, such as `'A'`.
- Can be implicitly cast to the type `rune` but also to any integer type that can accommodate the underlying numerical value.
- Inferred type of `some_variable := 'A'` is `rune`.

Something that will not compile is `x: i8 = '猫'` because `'猫'` uses more than 8 bits to store its numerical value. The `i8` cannot accommodate this Untyped Rune.

Using `i16` would make it compile.

Untyped nil

- The type of `nil`.

- Can be implicitly cast to anything that supports being in a "nil state". Such as: pointers, enums and unions.
- Cannot be used with type inference. `some_variable := nil` will not compile.

Video 3.7

I've made a video where I explain how the Untyped Types system works:

https://www.youtube.com/watch?v=qLYga3iCmt0

Chapter 4

Some additional basics

As mentioned in the introduction, I assume that you have done some programming before. But it doesn't have to be any deep knowledge. Having seen variables, procedures (functions), if-statements, arrays and loops should be enough.

We've already looked at variables. But before we go on, let's take a quick look at procedures, if statements, loops and fixed arrays.

I will return to procedures and fixed arrays in more depth later, this chapter is just here to make sure everyone is on the same page. If you have programmed in Odin before, you can probably skip this chapter.

Procedure parameters and return values 4.1

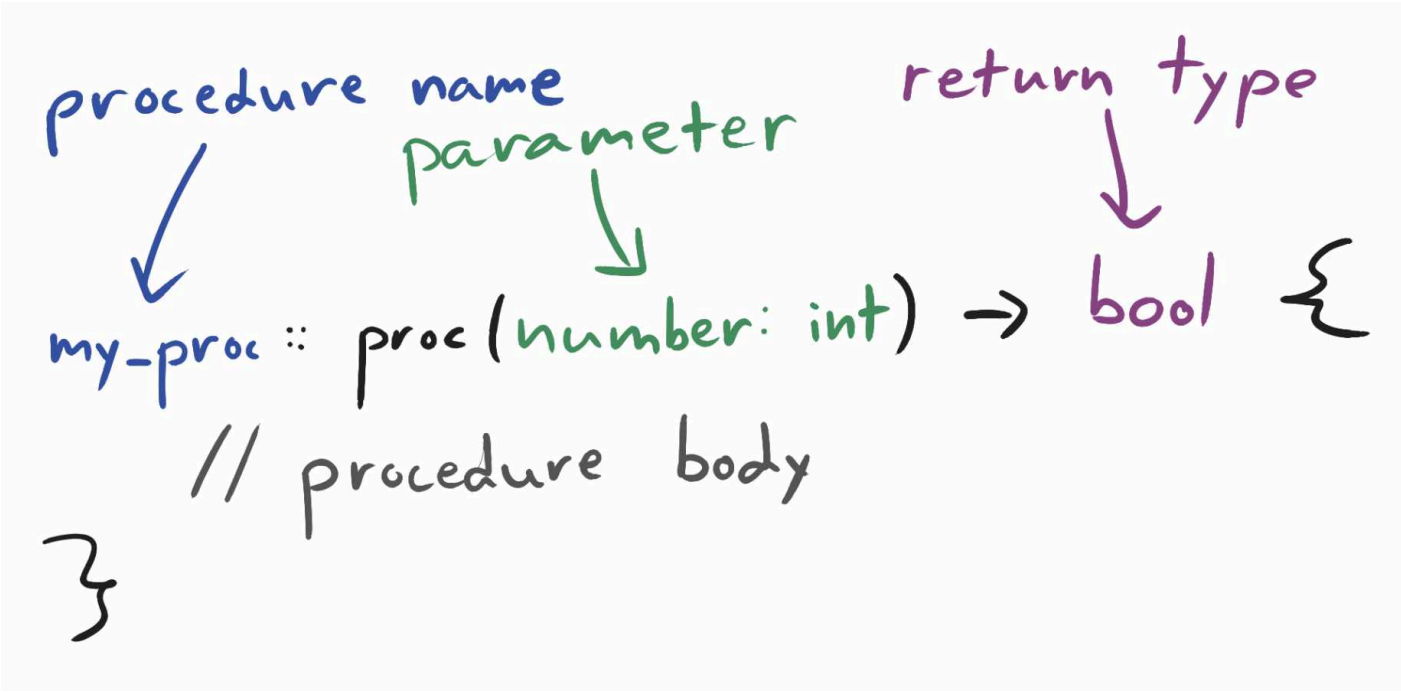
We've already seen the `main` procedure, where our program starts.

You can declare as many procedures as you want. You can add *parameters* to procedures, making it possible to supply data *into* the procedure. You can also add *return values* to procedures, so that the procedure can *output* data.

Here's a procedure with two parameters and a single return value:

```
is_bigger_than :: proc(number: int, compare_to: int) -> bool {
    return number > compare_to
}
```

This procedure accepts two integer parameters: `number` and `compare_to`. All it does is check if `number` is bigger than `compare_to` and then returns the answer. The return values are listed after the `->`. In this case the return value is of type `bool`.



We can run, or *call*, `is_bigger_than` and store the return value in a new variable:

```
result := is_bigger_than(3, 2)
```

In which case `result` will be a variable of type `bool` that contains the value `true` (since 3 is bigger than 2). The type `bool` is inferred from the right hand side, in this case from the return type of `is_bigger_than`.

In the **chapter on procedures** we'll look at multiple return values, named return values and what the scope of a procedure is.

If statements 4.2

Odin's if statements look like this:

```
if condition {
    // Code in here will run if `condition`
    // is true.
}
```

`condition` *must* be a value of type `bool`. If the condition is `true`, then the code between the curly braces `{}` will run.

From C / C++

Unlike C, you cannot use an integer as a condition, instead you have to do something like

```
if some_int > 0 {}
```

Also, note that you do not need parenthesis around the condition.

As an example, you can use an if-statement to check if a variable has a specific value:

```
some_variable := 7

if some_variable == 7 {
    fmt.println("It's seven!")
}
```

Here `==` is a *comparison operator*. The result of a comparison operator is a value of type `bool`. Available comparison operators are:

```
== // left and right are equal
!= // left and right not equal
<  // left is less than right
<= // left is less than or equal to right
>  // left is greater than right
>= // left is greater than or equal to right
```

You can use `left || right` to check if `left` *or* `right` are true. Also, you can use `left && right` to check if `left` *and* `right` are true. Note that `&&` is evaluated before `||`. We say that `&&` has *precedence* over `||`. You can use parenthesis to override the precedence:

```
if var1 == 7 && (var2 == 12 || var3 == 42) {
    // In here var1 is 7. And at least one
    // of the following is true:
    // var2 is 12 OR var3 is 42
}
```

Similarly, multiplication is considered before addition:

`x * y + z`

is different from

`x * (y + z)`.

The official overview has a full list of **operator precedence** .

Note that `&&` and `||` are *short-circuiting*. If you do `x && y` then `y` will not even be considered if `x` is `false`. Similarly: If you do `x || y` then `y` will not be considered if `x` is `true`. This is especially important when using conditions that are return values from procedures:

```
if var == 7 && some_proc() {
    // var is 7
```

```
    // some_proc() returned true
}
```

In this example `some_proc()` won't run at all if `var` is not equal to `7`.

You can "flip" the value of a `bool` using `!`, causing `true` to become `false` and vice-versa.

```
some_variable := 7
my_condition := some_variable == 7

if !my_condition {
    fmt.println("some_variable is not 7")
}
```

Note how we create a variable called `my_condition` here. It will be of type `bool` since the right side is `some_variable == 7`.

`if` statements can be followed up with `else if` and `else` branches:

```
if some_variable > 7 {
    // some_variable is larger than 7
} else if some_other_variable == 2 {
    // some_variable is less or equal to 7
    // and some_other_variable is 2
} else {
    // some_variable is less or equal to 7
    // and some_other_variable is not 2
}
```

If statements without curly braces are *not* allowed. This will not compile:

```
if some_variable == 7
    some_proc()
```

However, you can write:

```
if some_variable == 7 { some_proc() }
```

alternatively you can use `do`:

```
if some_variable == 7 do some_proc()
```

I tend to just write the full three lines:

```
if some_variable == 7 {
    some_proc()
}
```

Some people like putting the `{` on a separate line. If you do that then perhaps the `if X do stuff()` syntax is more useful, since 4 lines can be a bit much. However, I recommend keeping the `{` on the same line as `if` because it plays nicer with Odin's lack of semicolons.

Loops 4.3

Odin has many different types of loops, but they all use the same keyword: `for`

This loop runs forever:

```
for {
    fmt.println("Printed over and over, forever")
}
```

From C / C++

Equivalent to

`while(true) {}`

and

`for (;;) {}`

in C. You can see the Odin version as the C version with the two `;;` dropped.

In order to stop it, you can use the `break` keyword:

```
for {
    if condition {
        break
    }

    fmt.println("Printed over and over, until 'condition' becomes true")
}
```

Instead of using `break`, you can have a condition on the `for` line. Also, note that this "inverts" the condition: In the previous example the loop runs until `condition` becomes `true`. The following loop runs until `condition` becomes `false`.

```
for condition {
    fmt.println("Printed over and over, until 'condition' becomes false")
```

```
}
```

From C / C++

Same as

```
while (condition) {}
```

in C.

The following loop runs as long as the variable `i` is less than `10`. It increases `i` by `1` for each lap:

```
i := 0
for i < 10 {
    fmt.println(i) // 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
    i += 1
}
```

You can modify the loop above and move the `i := 0` and `i += 1` into the `for` line:

```
for i := 0; i < 10; i += 1 {
    fmt.println(i) // 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
}
```

From C / C++

This now looks similar to a for loop in C, but we do not need to surround `i := 0; i < 10; i += 1` with parenthesis.

Note: there are no `++` or `--` operators. Instead you have to type

```
i += 1
```

This removes some subtle bugs from the code. For example, if you do this in C: `some_array[i++] = other_array[i];` then you can easily make mistakes because the evaluation order may not be apparent.

The loop above is a classic "for loop" as you might have seen them in other languages. Odin provides a more compact way of doing the same thing:

```
for i in 0..<10 {
    fmt.println(i) // 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
}
```

Above `0..<10` specifies a *range*, we'll see ranges appear in some other situations too. Note the `<`. It is possible to replace it with an `=`

```
for i in 0..=10 {
    fmt.println(i) // 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
}
```

The first version will run as long as `i` is less than `10`. The second version will run as long as `i` is less than or equal to `10`.

If you have two nested loops but want to stop the outer loop, then you can do so using *labels*. Note the `outer:` label and the `break outer` in this code:

```
outer: for x in 0..<20 {
    for y in 0..<20 {
        if x == 5 && y == 5 {
            break outer
        }
    }
}
```

Without the label, the `break` would stop the inner loop, not the outer one. This avoids having to use an extra variable in order to stop the outer loop.

Use `continue` to skip a lap of a loop:

```
for i in 0..<10 {
    if i == 5 {
        // skips printing "5"
        continue
    }
    fmt.println(i) // 0, 1, 2, 3, 4, 6, 7, 8, 9
}
```

Similarly to `break`, you can also use `continue` with labels to skip a lap of a labeled loop:

```
continue some_label
```

Fixed arrays 4.4

A fixed array is a list of things where the list has a fixed length. You cannot make the list longer or shorter.

You can create a variable that holds a fixed array of 10 integers like this:

```
ten_ints: [10]int
```

All the 10 integers within `ten_ints` will have the value zero. You can use `:=` to both declare and assign a value to the array:

```
ten_ints := [10]int {
    61, 81, 12, 41, 5, 10, 1234, 8, 4, 1,
}
```

Use

```
ten_ints = { your ten numbers }
```

if you want to change the value of a previously existing array. Again, note how `:=` and `:` creates new variables while a plain `=` changes the value of something preexisting.

You can fetch an item from the array using brackets `[]`:

```
third_item := ten_ints[2]
```

In this case `third_item` will be a variable of type `int` with the value `12`.

You can also set items using brackets:

```
ten_ints[6] = 7
```

If you want to loop through all the 10 items, or *elements*, then you can use this kind of loop:

```
for n in ten_ints {
    fmt.println(n) // 61, 81, 12, 41, 5, 10, 1234, 8, 4, 1
}
```

This loop goes through all the elements of `ten_ints` and prints each element on a separate line.

You can also loop through the elements in reverse by adding `#reverse` in front of the loop:

```
#reverse for n in ten_ints {
    fmt.println(n) // 1, 4, 8, 1234, 10, 5, 41, 12, 81, 61
}
```

We'll look more at fixed arrays in **chapter 8**. In that chapter we'll discuss where their memory lives and how to copy them. We'll also look at what *array programming* is, and how it can be used to do *vector maths*.

All together 4.5

Here's a small program that combines most of what we've seen so far:

```
package just_the_basics

import "core:fmt"

main :: proc() {
    numbers := [10]int {
        6, 4, 7, 10, 1, -1, -9, 100, 1, 54,
    }

    cmp := 6

    for n in numbers {
        if is_bigger_than(n, cmp) {
            fmt.printfln("%v is bigger than %v", n, cmp)
        }
    }
}

is_bigger_than :: proc(number: int, compare_to: int) -> bool {
    return number > compare_to
}
```

Here we use `fmt.printfln`. The procedures in `fmt` with `printf` in the name (note the `f`) accept a so-called *format string*. It will replace the two `%v` in `"%v is bigger than %v"` with `n` and `cmp`, in the order they are stated.

Also, `is_bigger_than(n, cmp)` should just be replaced with `n > cmp`. It's just there to illustrate how to use procedures.

This program creates a fixed array of 10 numbers. It loops over the fixed array and checks if the elements of the array are bigger than 6. For any element bigger than 6, it prints a message saying so.

As an exercise to make sure you understood this chapter, look at the program above and try to figure out what it will print without running it. Then run it to get the answer.

Chapter 5

Making new types

So far we've seen basic types such as `int` and `f32`. Let's look at how you can group such basic types into bigger, more complex types called *structs*. The benefit of using structs is that you can reason about them as a single thing rather than a bunch of separate tiny pieces.

In this chapter we'll also look at some additional ways to create new types, such as *enums* and *unions*. As we shall see, unions can be used to save memory, but also to create variations within structs.

Structs 5.1

A struct consist of a number of fields, where each field has a type and a name. You can see structs as a way to group variables. Let's say that we want to store the data for a rectangle. A rectangle needs a position in two dimensions (x and y), a width and a height. We can define a new struct type that contains those four things:

```
Rectangle :: struct {
    x: f32,
    y: f32,
    width: f32,
    height: f32,
}
```

Each field looks like a variable declaration, but with a comma at the end of each line.

The comma after the last field is optional. But it's nice to have there. That way you don't forget to write it when you later add a new field. If you run the Odin compiler with the `-vet-style` or `-strict-style` flag, then these trailing commas are enforced.

Here we see the double colon (`::`) again. So far we have seen three different things use `::`. Those are procedures, constants and now struct definitions. All these things are known at compile time. So whenever the compiler sees anything defined using `::`, then it will be able to reason about it at compile time. In other words, `::` means that you are doing a compile-time declaration.

The above just defines a new *type* `Rectangle`. In order to actually use `Rectangle` to store any information, we can create a variable of that type:

```
rect: Rectangle
```

This is similar to writing `number: int`. But the type `Rectangle` is a more complicated type than just a single integer.

The four fields `x`, `y`, `width` and `height` will be zero initialized.

If you want to initialize the `rect` when you create it, then you can use the good old `:=` syntax:

```
rect := Rectangle {
    width = 20,
    height = 10,
}
```

From C / C++

These initializers where you mention fields by name are known in both Odin and C as designated initializers. They are one of my favorite features in C because any non-mentioned fields will be zero initialized. Unfortunately, this has only recently been added to C++, so you can't really use it in C++ yet. Not being able to use them was always an annoying trade-off when I needed to use C++ instead of C.

Odin's designated initializers also initialize any alignment-padding in-between fields, which is a big benefit over C.

This rect will have its `width` and `height` fields set to `20` and `10` respectively. Any field not mentioned will be zero-initialized. In this case the fields `x` and `y` will both be zero.

Note that we could also have written

```
rect: Rectangle = {
    width = 20,
    height = 10,
}
```

Here we have moved `Rectangle` between the `:` and the `=` instead of after the `=`. But doing `rect := Rectangle {` whenever possible is usually considered more idiomatic Odin. Doing so will make your code look more consistent since there will be cases when you must do `Rectangle { /* initializers */ }` without there being a variable.

We'll see this later when setting the value of a union and when passing a struct to a procedure.

You can replace the value of a whole struct by assigning to it, effectively re-initializing it:

```
rect := Rectangle {
    width = 20,
    height = 10,
```

```
}

// a bit later

rect = {
    x = 10,
    width = 5,
    height = 7,
}
```

Just like with basic types, we use `:=` on the line were we declare and set the initial value of `rect`. But when we want to give the preexisting `rect` variable a new value, then we *only* use `=`.

In both the `:=` and `=` case any non-mentioned fields will be zero-initialized.

Also note that we did not have to supply the type name `Rectangle` on the `rect = {` line. The compiler sees that we are assigning to a variable of type `Rectangle` so we do not need to type it.

You can also initialize structs by just writing the values directly, without providing the field names:

```
rect := Rectangle {
    20, 20,
    200, 200,
}
```

This sets `x`, `y`, `width` and `height` of our `Rectangle` to the values in the order they are listed. When initializing in this way, without using specific names, then you must list all the fields. In this case you must provide four numerical values since there are four fields in `Rectangle`.

Structs within structs *5.1.1*

In `Rectangle`, we only used the basic type `f32` for our struct fields. But the fields can also use struct types:

```
Person_Stats :: struct {
    health: int,
    age: int,
}

Person :: struct {
```

```
    stats: Person_Stats,
    name: string,
}
```

Here we declare a type `Person` that has two fields. The `stats: Person_Stats` field uses another struct type.

Just as before, if we type:

```
p: Person
```

then `p` will be of type `Person` and all the fields inside it will be zero-initialized. This means that all the fields within `stats` will also be zeroed.

The memory used by the `stats` field lives *directly within* the `Person` struct. There is no separation between the memory of the two.

Having `stats: Person_Stats` in there makes the memory of a `Person` struct bigger by however big a `Person_Stats` struct is.

`Person_Stats` uses `2 * 8 bytes = 16 bytes` of memory. This is because it has two fields of type `int`, and `int` uses `64 bits = 8 bytes` on a 64 bit computer.

You can get the size of a type using `size_of(Some_Type)`.

You can do nested initialization of the `stats` field when initializing a `Person` struct:

```
p := Person {
    stats = {
        health = 7,
    },
    name = "Bob",
}
```

All the non-mentioned fields will be zero.

You can fetch and set the fields inside the struct using `.` (a period):

```
p := Person {
    stats = {
        health = 7,
    },
    name = "Bob",
}

p.name = "Bobinski"
```

```
p.stats.age = 36
bobinskis_health := p.stats.health
```

Note the last line: We are creating a new variable by fetching the `health` field. The type of `bobinskis_health` will be inferred to `int`, since the `health` field uses that type.

`using` *on struct fields* *5.1.2*

In the previous example we used `p.stats.age` to fetch the `age` field of the `Person_Stats` within the `Person`.

You can make `age` directly accessible on `p`, so that you just have to write `p.age`. This is possible thanks to the `using` keyword:

```
Person_Stats :: struct {
    health: int,
    age: int,
}

Person :: struct {
    using stats: Person_Stats,
    name: string,
}
```

The only difference between this and the earlier definition of `Person` is that I've added `using` in front of `stats: Person_Stats`.

You can now use the fields of `stats` directly on a `Person` object:

```
p := Person {
    health = 20
}
p.age = 70

hs := p.health
```

This `using` is not to be confused with "`using` on variables and parameters", which I discourage you to use in the chapter on things in Odin to avoid.

`using` *and "parenting"* *5.1.3*

Let's say that we are making a video game where there is a concept of an *entity*. This entity can be represented by a `struct`. Our entity needs some kind of identifier and a 2-dimensional

position:

You can think of entities as objects that live inside a game world.

```
Entity :: struct {
    id: int,
    position: [2]f32,
}
```

Now, say that we have a player character in the game. It is controlled by whoever is playing the game. You might then want this player to also "be an entity", meaning that you want it to be possible to pass the player to procedures that expect a parameter of type `Entity`.

You can achieve this by putting `using entity: Entity` at the top of the `Player` struct, and then follow it with additional fields that are unique to the player:

```
Player :: struct {
    using entity: Entity,
    can_jump: bool,
}
```

If you don't want any extra fields other than those in `Entity`, then you could instead define a type alias:

```
Player :: Entity
```

We can now create a player and use the id and position on it directly:

```
p := Player {
    id = 7,
    position = {5, 2},
    can_jump = true,
}

fmt.println(p.position) // [5, 2]
```

Furthermore, you can use `p` *as if it was of type Entity*:

```
p := Player {
    id = 7,
    position = {5, 2},
    can_jump = true,
}

print_position :: proc(e: Entity) {
    fmt.println(e.position) // [5, 2]
```

```
}

print_position(p)
```

Notice how `print_position` has one parameter. That parameter is of type `Entity`. But we didn't need to do any conversion when we sent in `p` into `print_position`, even though `p` is of type `Player`. Instead, the `entity` field inside the player is implicitly sent along because of the `using entity: Entity` field.

There are many ways to organize entities for a video game. This is just one example. Find the way that is most suited for the video game that *you are making* and don't listen too much to general advice about this.

No methods *5.1.4*

If you come from something like C++ or C#, then you might be used to the concept of a `class`. You might also be used to being able to define procedures within classes and structs, usually referred to as *methods*. There are no classes or methods in Odin, only structs and separate procedures. A procedure cannot be defined within a struct. Use structs to store data and procedures to process data.

However, you *can* do something like this in order to store a procedure within a struct:

```
My_Interface :: struct {
    required_name_length: int,
    is_valid_name: proc(My_Interface, string) -> bool,
}

my_proc :: proc(i: My_Interface, name: string) -> bool {
    return i.required_name_length == len(name)
}

my_interface := My_Interface {
    required_name_length = 5,
    is_valid_name = my_proc,
}
```

From C / C++

This is like storing function pointers in a struct in C.

But in this case each object of type `My_Interface` can have a different procedure assigned to the field `is_valid_name`. It's just a field that is a procedure, and you can assign any compatible procedure to that field. It's not in the definition of `My_Interface` what that procedure actually does.

I do not recommend doing the above in order to "implement methods". It is useful in some cases where the actual implementation of a procedure is unknown or somehow abstract.

> An example of a good use case for such an abstract interface is the `Allocator` type in `<odin>/base/runtime/core.odin`.
>
> For another example, see my 2D game creation library **Karl2D**. There I use interfaces to create abstractions for **rendering**, **platforms** and **audio**.

In general, just use structs and separate procedures. Embrace this. It will make your code nice and simple.

Enums and switch 5.2

Sometimes you need to classify something by associating it with a name. But it would be good if that name had some simple underlying representation, such as just being represented by a number. This is what enums, short for *enumerations*, do.

> Having a simple numeric representation is great when comparing values, storing data on disk or sending data over a network.

You can define new enum types. Each enum type declares a series of *members*, which are actually just named numbers.

Create an enum type like so:

```
Computer_Type :: enum {
    Laptop,    // value 0
    Desktop,   // value 1
    Mainframe, // value 2
}
```

Create a variable of this enum type like so:

```
ct: Computer_Type
```

Since we didn't initialize `ct`, it is zero-initialized. As we see in the definition of `Computer_Type`, the first member (`Laptop`) has the value `0`. This means that `ct` was set to `Laptop` when it was zero-initialized.

You can assign any of the three members we listed in `Computer_Type` to `ct`:

```
ct = .Mainframe
```

Which is equivalent to writing:

```
ct = Computer_Type.Mainframe
```

But the compiler knows that the type of `ct` is `Computer_Type`, so it's OK to skip that part.

Switch *5.2.1*

You can use `switch` to do different things based on the enum's current value:

```
switch ct {
case .Laptop:
    fmt.println("It's a laptop")

case .Desktop:
    fmt.println("It's a desktop")

case .Mainframe:
    fmt.println("Wow, a mainframe, in 2024?")
}
```

From C / C++

In C you have to write `break;` after each case in a switch. In Odin you do not need to do that. Instead, if you want a case to fall through to the next case, then you write `fallthrough`.

You can use if-statements to achieve the same result:

```
if ct == .Laptop {
    fmt.println("It's a laptop")
} else if ct == .Desktop {
    fmt.println("It's a desktop")
} else if ct == .Mainframe {
    fmt.println("Wow, a mainframe, in 2024?")
}
```

However, the `switch` version is less chatty and usually faster since the compiler can generate simpler code for it.

A switch must list all members of the enum. If you want to skip some of them, then put `#partial` in front of `switch`:

```
#partial switch ct {
case .Laptop:
    fmt.println("It's a laptop")

case .Desktop:
    fmt.println("It's a desktop")
}
```

I avoid `#partial` unless I have a good reason to use it. By avoiding it I'll get a compile error whenever I add new members to the enum. This makes it easy to find all those `switch` statements and add in some new code. If you don't need to do anything for a specific enum member, then you can just add a `case` that does nothing.

Explicit enum numbering *5.2.2*

In the previous example, we had an enum with three members. The first one had value `0`, the second had `1`, etc. However, you can associate a member with an explicit value. Note how it now says `Laptop = 1` in the following example:

```
Computer_Type :: enum {
    Laptop = 1, // value 1
    Desktop,    // value 2
    Mainframe,  // value 3
}
```

Now the first member `Laptop` has the value `1`, and then it counts from there. `Desktop` has the value `2`, etc.

`ct: Computer_Type` will still be zero-initialized. This means that it holds an invalid value. `0` is not represented by any of the members. It is therefore good practice to have something sensible associated with the zero value.

If you wish, you can give all members explicit values:

```
Computer_Type :: enum {
    Laptop = 1,
    Desktop = 2,
    Mainframe = 7,
}
```

Any time you specify an explicit value, then the members that follow will count on from that explicit value (given that they don't have an explicit value themselves):

```
Computer_Type :: enum {
    Laptop = 0,    // value 0
    Desktop,       // value 1
    Mainframe = 5, // value 5
    Broken,        // value 6
    Gamer_Rig,     // value 7
}
```

Backing type *5.2.3*

By default enums use `int` as the 'backing type'. This means that behind the scenes an enum uses an `int` to store the current value. You can specify a custom backing type by adding a type name after `enum`:

```
Animal_Type :: enum u8 {
    Cat,
    Rabbit,
}
```

Note how it says `enum u8`. This enum will use an unsigned 8 bit integer as backing type.

In most cases, I recommend just going with the default of `int`. A situation where I *would* use a specific backing type is when I **create bindings to libraries written in other languages**, as the size of the enum must match what the library expects.

From C / C++

In C, you can directly compare enums to integers. In Odin, enums are distinct types. This means that you must cast your enum to `int` before comparing it with an integer:

```
if int(some_enum) == 5 { /* do stuff */ }
```

Unions 5.3

You can assign a value to an enum. But it does not let you store any additional data related to the current value. In Odin, *unions* let you store both *what something is* (like an enum), and also some data associated with the current value. Here's an example:

```
My_Union :: union {
    f32,
    int,
    Person_Data,
}
```

```
Person_Data :: struct {
    health: int,
    age: int,
}

val: My_Union = int(12)
```

The three fields inside `My_Union :: union {}` are *type names*. This means that `My_Union` can hold a value of any of the three types `f32`, `int` or `Person_Data`. The union stores both which type it currently holds as well as the actual data for that type. In the example above we create a new variable `val` of the union type `My_Union` and assign `int(12)` to it. The union then *knows* that it contains an `int` and it also knows that the value is `12`.

You can assign a new value to `val`, for example a `Person_Data` object:

```
val = Person_Data {
    health = 76,
    age = 14,
}
```

Note that you cannot do

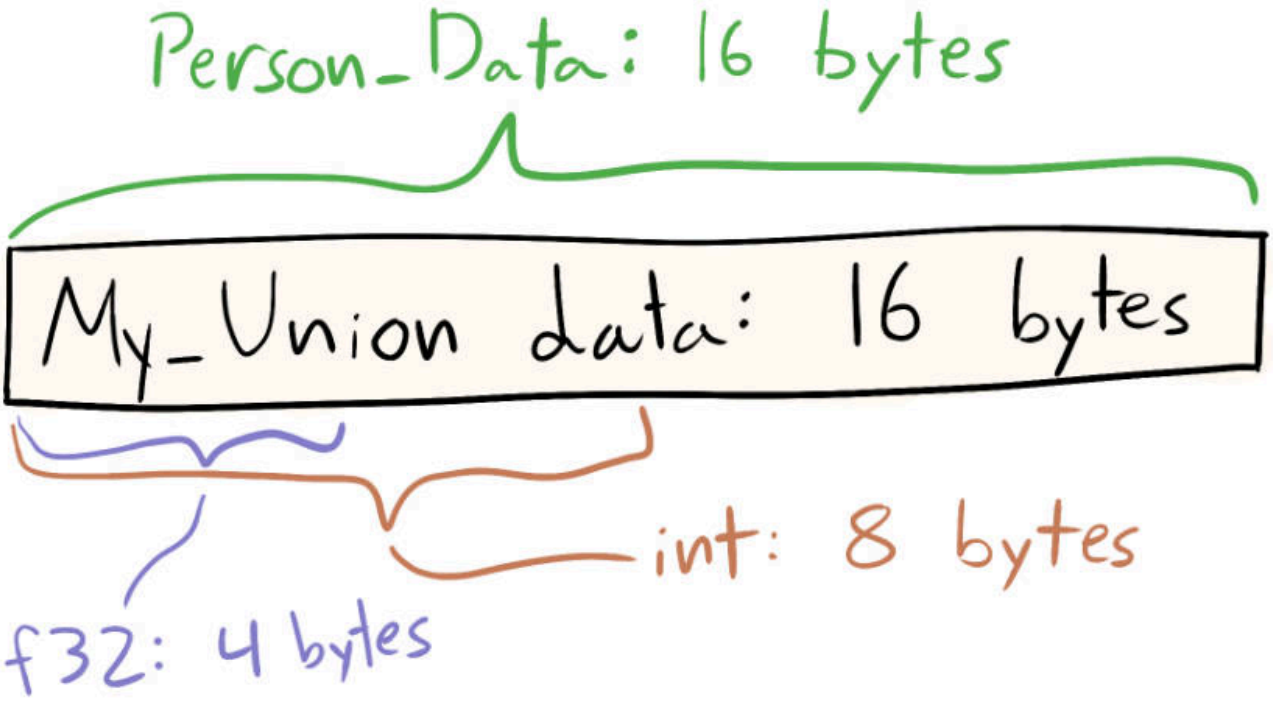
```
val = { health = 76, age = 14 }
```

You can't skip the `Person_Data` type name. Skipping the type works when assigning to a struct variable, but here it would be ambiguous, so we must provide the type name.

After this the union `val` knows that it contains a value of type `Person_Data` and that this `Person_Data` has the fields `health` and `age` set to `76` and `14` respectively.

The different possible types, such as `f32` and `Person_Data` above, are referred to as the *variants* of the union.

The `My_Union` type will only use as much memory as the biggest variant. It can only contain one of the variants at a time, so it can use the same block of memory for all of them. You can think of it as three different variables that all share the same memory, but you're only allowed to use one of them at a time.



`My_Union` memory layout: `f32`, `int` and `Person_Data` uses 4, 8 and 16 bytes of memory respectively, but they share the same memory. This is how the union only takes as much space as the biggest variant. Note that `int` is actually 4 bytes on 32 bit systems.

The biggest variant of `My_Union` is `Person_Data`, which uses `16` bytes of memory. Therefore, a variable of type `My_Union` uses `16` bytes to store its data. It also needs `8` additional bytes to store the *tag*. The tag keeps track of which variant it currently holds. More on the tag in a bit. This means that the total size of `My_Union` is `24` bytes, which you can confirm by printing `size_of(My_Union)`:

```
fmt.println(size_of(My_Union)) // 24
```

Just like with an enum you can use `switch` to branch on the value currently held by a union:

```
switch v in val {
case int:
    // You can use v, it is of type int.

case f32:
    // You can use v, it is of type f32.

case Person_Data:
    // You can use v, it is of type
    // Person_Data. You can use v to access
    // the fields of Person_Data:
```

```
    fmt.println(v.age)
}
```

When comparing this to the switch statement we used with an enum, there are two differences to note:

- It says `v in` on the line `switch v in val {`. For an enum we just say `switch some_enum {`.
- The `case` labels are the union variant type names.

Only the case that matches the union's current variant will run. So if the program ends up inside `case Person_Data:` then you know that the union `val` is of variant `Person_Data`, and you can also access the data of that variant by typing `v`.

You can choose any name instead of `v`.

If you wish to make `v` modifiable within the `case`, then you need to add an `&` in front of `v`:

```
switch &v in val {
case int:
    v = 7

case f32:
    v = 42

case Person_Data:
    v.age = 7
}
```

From C / C++

In most cases, putting `&` in front of things makes it into a pointer. But in this specific case `v` becomes an *addressable*. Addressables are similar to L-values in C. I'll talk about them at the end of the **pointers** chapter.

Sometimes you're only interested in checking if the union is holding one specific variant. In that case you don't need a whole `switch` statement, the following will be enough:

```
f32_val, f32_val_ok := val.(f32)

if f32_val_ok {
    // f32_val is OK to use in here.
}
```

We use `val.(f32)` to check if the variant of the union `val` is of type `f32`. If it is then `f32_val_ok` contains `true` and `f32_val` contains the value.

Note the `.(f32)`. It's syntax we haven't seen before in the book. It will ask the union if its current variant is of type `f32`. This syntax is only used with unions.

However, the code above is a bit awkward because it litters the surrounding code with the variables `f32_val` and `f32_val_ok`. To avoid that, I recommend this special type of if-statement:

```
if f32_val, f32_val_ok := val.(f32); f32_val_ok {
    // f32_val is OK to use in here.
}
```

This if-statement first creates the two variables `f32_val` and `f32_val_ok`, then the part after the `;` checks the actual condition. `f32_val` and `f32_val_ok` are only available within the if-statement, not outside of it.

You can also just do `f32_val := val.(f32)`, skipping the `f32_val_ok` variable. But this means that you are *expecting* `val` to have a variant of type `f32`. If it doesn't, then your program will assert, which is a fancy way to say "crash with an error".

Finally, if you wish to modify the value, then you can add in an `&` just before `val.(f32)`:

```
if f32_val, ok := &val.(f32); ok {
    f32_val^ = 7
}
```

In this case adding the `&` makes `f32_val` into *a pointer*. We haven't talked about pointers yet, but it's essentially an address to a value of type `f32`. The `f32_val^ = 7` above changes the value which the pointer `f32_val` points to. I will discuss pointers more **in the next chapter**.

In the previous bubble I said that the `&` in `switch &v in val {` just made `v` into an "addressable" so that it is possible to modify. I emphasized that in that case `v` *is not a pointer*. This might seem confusing. But `&` creating addressables as opposed to pointers is specific to `switch` and `for` loops. In all other cases `&` gets the memory address of something, creating a pointer.

But anyways, more about pointers and addressables in the next chapter.

The zero value of a union *5.3.1*

Say that you have a union of two structs, like so:

```
Shape :: union {
    Shape_Circle,
    Shape_Square,
```

```
}

Shape_Circle :: struct {
    radius: f32,
}

Shape_Square :: struct {
    width: f32,
}

// Zero value: nil
shape: Shape
```

Since we didn't provide a value when we wrote `shape: Shape`, it is default-initialized to zero. In this case the zero value is interpreted as `nil`. This means that your union does not have a value of any variant at all.

We can compare this to the zero value of an enum:

```
Shape :: enum {
    Circle,
    Square,
}

// Zero value: Shape.Circle
shape: Shape
```

This enum `shape` will have the value `0`, which is equal to `Shape.Circle`.

It is possible to make unions behave in the same way as enums, meaning that it is possible to make unions use the first variant as the zero value. If we add `#no_nil` to the declaration of our union. Then it cannot be `nil`, the zero value is then instead the first variant:

```
Shape :: union #no_nil {
    Shape_Circle,
    Shape_Square,
}

Shape_Circle :: struct {
    radius: f32,
}

Shape_Square :: struct {
    width: f32,
}
```

```
// Zero value:
// Variant Shape_Circle
shape: Shape
```

Note the `#no_nil`.

In this case `shape` will be of variant `Shape_Circle`.

The union's tag *5.3.2*

Let's talk a bit about how unions work under the hood. This will make us understand how `#no_nil` and the zero value of unions work. This is also important if you ever want to save the value of a union to a file.

The way a union keeps track of which variant it currently holds is by something known as a *tag*. This tag is part of the internal workings of the union. It is not readily accessible.

There are procedures in `core:reflect` to **get** and **set** the tag of a union.

So if we have a union like this one, that can be nil:

```
Shape :: union {
    Shape_Circle,
    Shape_Square,
}
```

then there are three possible tag values:

- `0 -> nil`
- `1 -> Shape_Circle`
- `2 -> Shape_Square`

When we write

```
shape: Shape
```

then the whole union is zero-initialized, including the tag. Which means that the union is interpreted as having the value `nil`.

On the other hand, if we add `#no_nil`:

```
Shape :: union #no_nil {
    Shape_Circle,
    Shape_Square,
}
```

then there are two possible tag values:

- `0 -> Shape_Circle`
- `1 -> Shape_Square`

So now when we write `shape: Shape` and get a zero-initialized union, then the tag says it is of variant `Shape_Circle` instead. Also, note that the data block that holds the actual memory for the variants is also zero initialized, so the fields inside `Shape_Circle` are also all zeroed.

Maybe *5.3.3*

There's a special type in Odin called `Maybe`. Variables of that type can either have *no value*, or *some value*. It's implemented using a union that has a single variant:

```
Maybe :: union($T: typeid) {
    T,
}
```

This uses **parametric polymorphism**, which we haven't talked about yet. But you can see it as the `int` in

`time: Maybe(int)`

being copy-pasted into where it says `T`.

So when you type:

```
time: Maybe(int)
fmt.println(time) // nil
time = 5
fmt.println(time) // 5
```

Then `time` can either be `nil` or have a value of type `int`. You can check if `time` has *some value* like so:

```
if time_val, time_val_ok := time.?; time_val_ok {
    // Use time_val.
}
```

This `time.?` syntax is the same as writing `time.(int)`. But since the `Maybe` only has a single variant then the compiler fills the type in for you.

You can also skip the `ok` value on the left side:

```
t := time.?
```

In that case the program will assert (crash with error) if `time` is nil.

Raw union: C-style unions *5.3.4*

The style of unions in Odin are known as *tagged unions*, because there is a tag inside it that keeps track of which variant it currently holds.

In C, `union` also exists, but it is a bit simpler. It has no tag, so you have to use an extra variable to keep track of which variant it currently holds. A common approach in C is to use both an `enum` and a `union`.

In Odin you can create these C-style unions, they are known as *raw unions*. You create them by putting `#raw_union` on a struct:

```
My_Raw_Union :: struct #raw_union {
    number: int,
    struct_val: My_Struct,
}

a_raw_union: My_Raw_Union
```

Note: We used `struct`, not `union`! There's no safety here that tells us which variant `a_raw_union` currently holds. You can access either variant by typing `a_raw_union.number` or `a_raw_union.struct_val`.

What `#raw_union` does is that it makes all the fields inside the struct start at the same point in memory, making them overlap. This is the same as the data block in the tagged `union`, but without the tag. So you need some extra variable to keep track of the current variant.

Struct variants using unions *5.3.5*

Again, say that you're making a video game and you have an `Entity` struct that represents an object in your game world:

Sorry non-game developers for all the video game examples. I'm a game developer and these examples come naturally to me!

Odin has gotten a reputation for being a "game development language". This is probably because the compiler comes with many libraries for game development, as part of the `vendor` collection (see

`<odin>/vendor`).

For some types of programs it may feel like there aren't as many useful libraries included. However, I am sure that more non-gamedev libraries will pop up as time goes on. There is nothing inherent about Odin that is stopping this.

```
Entity :: struct {
    position: [2]f32,
    texture: Texture,
    can_jump: bool,
    time_in_space: f32,
}
```

Here the first two fields sound nice and general: A 2D position that tells us where the entity is, and a texture (an image) that is used for drawing it on the screen.

However, let's say `can_jump` is only used when `Entity` represents the player character, and `time_in_space` is only used when `Entity` represents a rocket flying through space.

We can save space in the `Entity` struct by moving those things into a union:

```
Entity :: struct {
    position: [2]f32,
    texture: Texture,
    variant: Entity_Variant,
}

Entity_Player :: struct {
    can_jump: bool,
}

Entity_Rocket :: struct {
    time_in_space: f32,
}

Entity_Variant :: union {
    Entity_Player,
    Entity_Rocket,
}
```

OK. We're not saving much space at all. But imagine there being more fields in each union variant.

Some game developers just keep everything inside `Entity`! This can create interesting emergent gameplay since any entity can modify any field of any other entity. This approach is sometimes called a "Mega Struct". If you're making a small game with few entities, then a mega struct is probably feasible.

So now when you create an entity, you can set the variant of it to an appropriate type:

```
player_entity := Entity {
    position = starting_position,
    texture = player_graphics,
    variant = Entity_Player {
        can_jump = true,
    }
}
```

Earlier I showed how the `using` keyword can make `Entity` into a "parent type" of some other struct. If you compare the two approaches you see that they solve a similar problem, but in different ways. Whatever problem you're trying to solve, try several methods and see which works best for you.

Using unions for state machines *5.3.6*

I have a video on using unions to create state machines:

https://www.youtube.com/watch?v=bGc7C3U89-I

Chapter 6

Pointers

Sometimes you need your code to directly modify the value of a variable that lives somewhere else. You can do that using what's known as a *pointer*.

A pointer is a reference. It is used to "point out" something else in memory. Internally it contains the memory address of that "something else".

Let's look at a few examples. Each section in this chapter gets a little bit more advanced. If you're new to all these things, then don't worry if it takes a while to grasp. I will also discuss topics that involve pointers in later chapters. This is just an introduction.

The final section called **Under the hood: Addressables** is less about pointers, and more about demystifying how the compiler does operations involving pointers.

A procedure that modifies an integer 6.1

Say that we have an integer variable and we want to make a procedure that can directly add `1` to that variable for us. We can do that using a procedure that accepts a *pointer* to an integer. Such a procedure can then use that pointer to directly modify the value of the integer. It will go to the memory address that the pointer contains and add `1` to the number it finds there. Here's how you can write that:

```
increment_number :: proc(num: ^int) {
    num^ += 1
}

number := 7
number_pointer := &number
increment_number(number_pointer)
fmt.println(number) // 8
```

`number := 7` creates a variable `number` of type `int` that contains `7`. Note the `&` on the next line:

```
number_pointer := &number
```

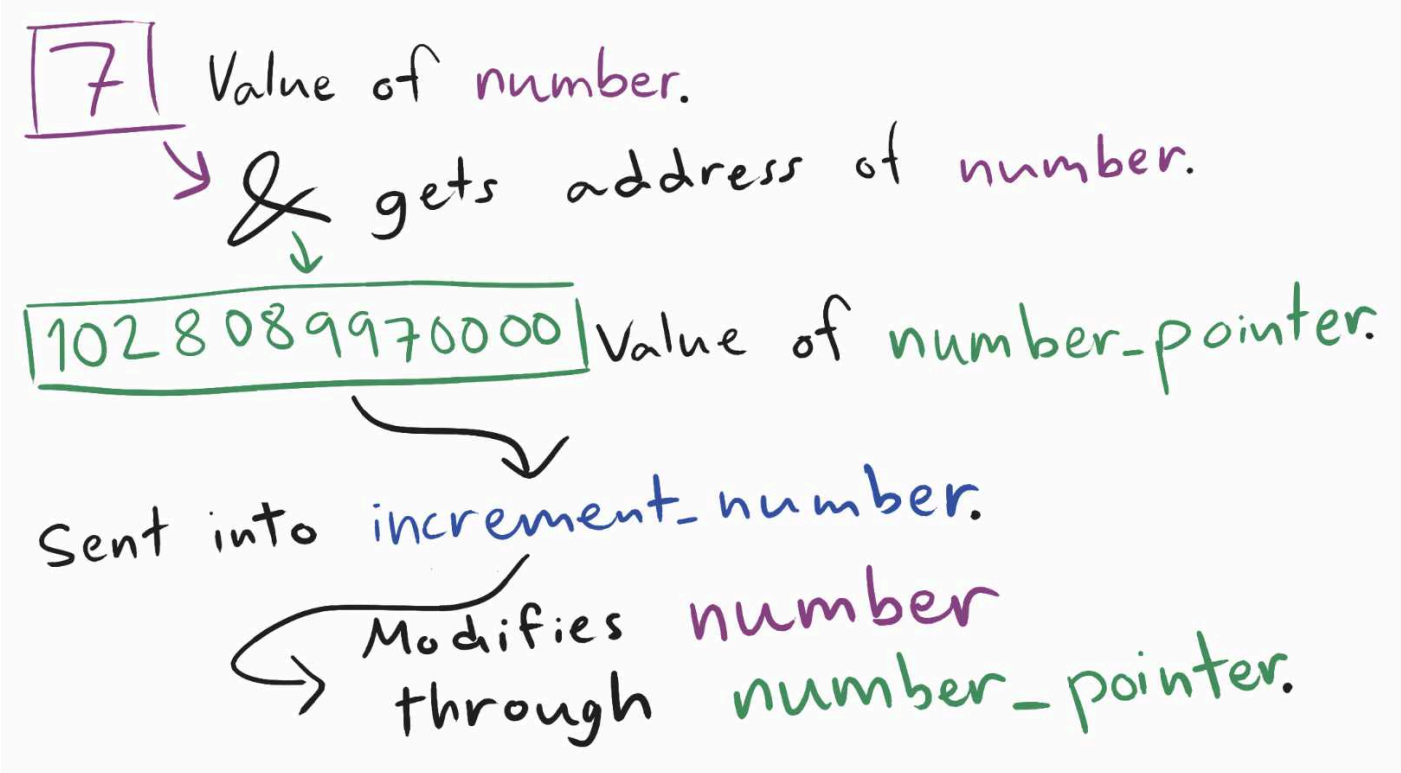
The `&` fetches *the memory address* of `number`. This means that the variable `number_pointer` now contains that address. This address tells us where in the computer's memory `number` is stored. We can use that address to access and modify `number` from other parts of our code.

You don't need a separate variable `number_pointer`, you could just have written `increment_number(&number)`.

I just wanted to have the pointer as a separate variable, to make it easier to talk about.

The type of `number_pointer` is `^int`. Any type that contains a `^` is a pointer. `^int` can be read as "pointer to integer", meaning that we expect this pointer to contain a memory address, and at that address in memory we expect to find an integer.

The procedure `increment_number` has a single parameter: `num: ^int`. That's the same type as `number_pointer`. So when we run `increment_number` and feed it `number_pointer`, then its parameter `num` contains the address of the variable `number`.



This big address `102808997000` is literally how a pointer can look when printed as an integer. The exact value will be different on your computer.

You can print pointers as integer numbers like this:

```
fmt.printfln("%i", some_pointer)
```

Inside `increment_number` we see this:

```
increment_number :: proc(num: ^int) {
    num^ += 1
}
```

The line `num^ += 1` fetches the integer at the address that `num` points to, adds `1` to it and stores it back at that address. That line is equivalent to writing:

```
num^ = num^ + 1
```

above we see that `num^` can be used for two different things:

- `num^ = some_value` writes the value on the right side of the `=` into that which `num` points to.
- `num^ + 1` reads the value that `num` points to and adds 1 to it.

From C / C++

`num^ += 1` is the same as writing `*num += 1` in C. Note that the `^` is after the pointer name instead of before it.

Compare the position of the `^` in these two lines:

```
increment_number :: proc(num: ^int) {
```

and

```
num^ = num^ + 1
```

- In the first case it is *to the left of type name*. This is how you denote a pointer type.
- In the second case it is *to the right of a pointer variable's name*. This is how you read or write through a pointer.

Whenever the `^` appears on the right side, we call it the *dereference operator*. A pointer is essentially a *reference* to something, meaning that it contains information about where to find something. So to *dereference* it means to fetch the thing it references.

Note how I use the word *through* in "read or write *through* a pointer". This is a good way to talk about pointers: You are trying to read or write something, but you must use the address inside the pointer to get there, which is like "going through the pointer".

nil: the zero value of a pointer 6.2

If you create a new variable of pointer type, like this:

```
my_pointer: ^int
```

then you are declaring a pointer to an integer, without giving it a value. As usual it will be initialized to zero. But what does zero mean for a pointer? What does the zero value of a memory address represent?

Internally a pointer is just a numerical value, comparable to an unsigned integer. On most 64 bit platforms, pointers can be seen as 64 bit unsigned integers. Or a 32 bit unsigned integer on most 32 bit platforms. This is because that's the biggest address that such a computer can reason about.

> The biggest value a 64 bit unsigned integer can contain is
>
> `18446744073709551616`.
>
> But that doesn't mean your computer can have that much memory. Currently, most CPU architectures only use 48 of those 64 bits. There are some architectures that use 52 bits.
>
> 48 bits still gives a memory limit of just over 256 terabytes (256000 gigabytes)! Your OS and computer's motherboard probably also has some limitations. So I would be surprised if it is possible to install much more than 100-1000 GB of memory (RAM) in your computer.

So there's nothing magical about pointers. A pointer of value zero can just be seen as the number zero. This zero value means "no address", meaning that the pointer is currently not referring to anything at all. There's a special keyword in Odin to denote pointers of value zero: `nil`.

Trying to read or write through a `nil` pointer will crash your program. The code below would crash, since it tries to write `10` through a `nil` pointer:

```
my_pointer: ^int
my_pointer^ = 10
```

Similarly, a procedure that reads or writes through a pointer parameter will crash if you feed `nil` into that parameter. In our earlier `increment_number` procedure, we can protect against such crashes by checking if the parameter `num` is not `nil` before trying to use it:

```
increment_number :: proc(num: ^int) {
    if num != nil {
        num^ += 1
    }
}
```

You can also use `==` instead of `!=` and instead do an *early return*:

```
increment_number :: proc(num: ^int) {
    if num == nil {
        return
    }

    num^ += 1
}
```

From C / C++

`nil` is the same as `nullptr` in C++ or `NULL` in older versions of C.

`nil` comes from the Latin word *nihil*. `null` comes from the Latin word *nullus*. Their meaning is similar, but nihil means "nothingness" while nullus means "nothing".

As you can see, `nil` and nihil is slightly more mystical and cool.

These examples, where we increment integers using a pointer, aren't all that useful since you'd probably just return a new number instead of bothering with a pointer. In the next section we'll look at modifying a struct through a pointer, which is a lot more useful.

A pointer to a struct 6.3

In this example we'll use a procedure in order to modify a struct. We'll feed a pointer to a struct into the procedure. The procedure will modify a field of the struct through the pointer.

Here's a struct that describes a cat. It contains its name, age and color:

```
Cat :: struct {
    name: string,
    age: int,
    color: Cat_Color,
}

Cat_Color :: enum {
    Black,
    White,
    Orange,
    Tabby,
    Calico,
}
```

Below we create a new cat called "Patches". It's Patches' birthday, so we need to increment its age and print a happy message. In this example, `process_cat_birthday` takes a pointer to a `Cat` struct and increments the `age` field.

```
process_cat_birthday :: proc(cat: ^Cat) {
    if cat == nil {
        return
    }

    cat.age += 1
}

my_cat := Cat {
    name = "Patches",
    age = 7,
    color = .Calico,
}

process_cat_birthday(&my_cat)

// Hooray, Patches is now 8 years old!
fmt.printfln("Hooray, %v is now %v years old!", my_cat.name, my_cat.age)
```

Output of running this program is:

```
Hooray, Patches is now 8 years old!
```

Just like previously we fetch the address of a variable using the `&`:

```
process_cat_birthday(&my_cat)
```

The result of `&my_cat` is a value of type `^Cat`. It's a pointer to a struct of type `Cat`.

As before, the pointer just contains an address. At that address we find the memory that the variable `my_cat` uses to store its data. How much memory does this `my_cat` variable use? It uses `32` bytes because that's the combined size of the fields of the struct `Cat`.

Within `process_cat_birthday` we increment the `age` field by writing through the pointer:

```
cat.age += 1
```

Note where the line that prints `"Hooray, Patches is now 8 years old!"` is: It's after we've called `process_cat_birthday`. This is an important point to understand: By passing `my_cat` by pointer, we let `process_cat_birthday` modify the variable `my_cat` directly. In the code that follows the line `process_cat_birthday(&my_cat)`, we can see the changes that `process_cat_birthday` did to `my_cat`.

Unlike the examples in the previous section, we didn't have to write

```
cat^.age += 1
```

When you have a pointer to a struct, and access a field of the struct through that pointer, then the `^` is implicit. `cat^.age` and `cat.age` do the exact same thing.

From C / C++

In C++ or C `(*cat).age` and `cat->age` are identical. The Odin compiler knows that `cat` is a pointer, and uses that knowledge make `.` do the same thing as `->` does in C.

On the other hand, if you want to replace the *the whole struct*, then you need to use `^`. Below we have a procedure called `replace_cat` that replaces all the data that its `cat: ^Cat` parameter points to. Note how it uses `cat^ = {}`.

```
replace_cat :: proc(cat: ^Cat) {
    if cat == nil {
        return
    }

    cat^ = {
        name = "Klucke",
        age = 6,
        color = .Tabby,
    }
}
```

You can't just do `cat = {}` without the `^`. This is because assigning to a pointer means changing the address the pointer contains. In other words, assigning to a pointer means re-directing what the pointer refers to. But we want to go *through* the pointer and modify the memory it points to. So we have to use `cat^ = {}`.

Copying a pointer and the address of a pointer 6.4

As I've mentioned throughout this chapter, you can think of a pointer as just containing a number. That number is an address that points to a location in memory. In this section we'll discuss what happens when you have two pointers containing the same address. The things we discuss here are meant to give you an intuition for what a pointer really is.

In the code below we have an integer variable `number`. We fetch the address of `number` and put it in `pointer1`. We then create another variable called `pointer2`, which is a copy of the variable `pointer1`. We then write the number `10` through `pointer2`.

```
number := 7
pointer1 := &number
pointer2 := pointer1
pointer2^ = 10
```

You can think of `pointer1` and `pointer2` as two variables containing the exact same number: They both contain the same memory address. To modify the original variable `number` you can go via any of the two pointers: `pointer1^ = 10` and `pointer2^ = 10` would both set the variable `number` to `10`, because they both go through the same address.

Let's think a bit about what `pointer1` and `pointer2` actually are. They are two separate variables. This means that they must both store a separate copy of `number`'s address somewhere. If you fetched the address of `pointer1` and `pointer2` and printed it, then you would see two different addresses:

```
fmt.println(&pointer1)
fmt.println(&pointer2)
```

Note the `&` in front of both. The type of `&pointer1` and `&pointer2` is in this case `^^int`, which you can read as "pointer to a pointer to an integer". The above would print something like:

```
0x445C6FF868
0x445C6FF860
```

Pointers are by default printed using hexadecimal notation. If you rather look at "normal" numbers, then you can change the print lines to:

`fmt.printfln("%i", &pointer1)`.

Note that the first one ends with `8` and the second one ends with `0`. The exact numbers will not be the same on your computer. But it will be two different addresses. This shows that pointers are variables just like any other: `pointer1` and `pointer2` have separate locations in memory for storing whatever they contain. In this case they both store a separate copy of the same memory address. At the address that they both store, you find the value of the integer variable `number`.

Under the hood: Addressables 6.5

We've seen how pointers can be used to get and set the value that the pointer refers to:

```
// Read a value through a pointer
read_value := some_pointer^

// Write a value through a pointer
some_pointer^ = 10
```

When `^` appears to the right of a pointer's name, we call it the *dereference operator*. But what is it that `^` actually does? As we'll see, there's more going on here than just reading and writing through the pointer.

Some of the things we talk about here aren't strictly necessary to know. But it may prove useful, interesting and demystifying. Here's an example of something we'll be able to understand at the end of this section:

How can the following code get a pointer that refers to the tenth element of `array`?

```
array: [20]int
element_pointer := &array[10]
```

After all, doesn't `array[10]` fetch something of type `int`? If you think about it, the above kind-of looks like it does the same as this:

```
array: [20]int
element := array[10]
element_pointer := &element
```

In which case `element_pointer` would not point to the tenth element of `array`. Instead, it would point to `element`, which is a copy of the tenth element. But that's *not* what happens when you do `&array[10]`. Somehow `&array[10]` gives you a pointer directly to the tenth element of the array. This section is all about understanding how.

From C / C++

When learning to program in C, I used to be scared of writing `&array[10]`. I thought that it would take the address of a copy of `array[10]`, instead of the giving me the address of the tenth element. I often did `array + 10` in C, because that felt more like it didn't use any "in-between copy". However, `&array[10]` works perfectly fine in both C and Odin.

Let's take a step back and look at some simple examples that will give us new insights and thereafter return to the array example.

Say that we again have an integer variable `number` and a pointer to that variable:

```
number: int
number_pointer := &number
```

We can now assign to `number` through `number_pointer` like this:

```
number_pointer^ = 10
```

The above *seems* to work like this: Whenever we find `number_pointer^` on the left side of an `=`, then it goes through the pointer and sets the value at that address.

We can also fetch the value of `number` through `number_pointer` like this:

```
another_number := number_pointer^
```

Similarly, the above *seems* to work like this: Whenever we find `number_pointer^` to the right of `:=` (or to the right of `=`), then that fetches the number the pointer refers to.

I put emphasis on *seems* above, because this is a surface-level explanation. It's an explanation that suffices in most cases.

But if we talk about what the compiler is actually doing internally, then we say that `number_pointer^` (the whole thing including the `^`) creates what is known as an *addressable*. As the name suggests, addressables are things that are possible to locate in memory. Addressables can be read, fetching their value. They can also be assigned to, writing their value.

From C / C++

Addressables are known as *L-values* in C. L-value is short for "Left-value". Historically L-values could only appear on the left side of an assignment, which is no longer true. Nowadays it just means that they *can* appear on the left side, meaning that it is possible to assign to them.

To reduce the confusion, some people re-brand L-value to instead mean "Locator-value", because it can locate the data when you assign to it. That sounds about equally confusing to me.

Some skip all of this and just say *addressable* instead.

So on a line like

```
number_pointer^ = 10
```

Then we write into the addressable `number_pointer^` because it is on the left side of the assignment. But on a line like

```
another_number := number_pointer^
```

then the addressable `number_pointer^` is read because it is on the right side of the assignment. Here `another_number` is actually also an addressable: It's something we can assign to.

I want to repeat and stress something here: To the compiler, `number_pointer^`, including the `^`, creates an addressable. An addressable is an internal thing that the compiler can use to read and write into some part of memory. I like to think of addressables as the compiler's own *internal version* of pointers. Meaning that when we write `number_pointer^`, then the compiler still retains the address of whatever `number_pointer` points to, so you can assign to it.

There are things that are not addressables, because they cannot be assigned to. A constant number like `7` is an example. You can never assign to it, because doing:

```
7 = some_variable
```

doesn't make any sense.

From C / C++

In C we call these non-addressables *R-values*, because they can only appear on the right side in an assignment.

Let's return to the initial example; creating a pointer to a value you've just fetched from an array:

```
array: [20]int
element_pointer := &array[10]
```

If you have an array and fetch the element at index `10` using `array[10]`, then you might think that you've already completely lost the original memory address of that value. After all, the result of doing `element := array[10]` is something of type `int`.

But in the example above `element_pointer` somehow contains the direct address to the tenth element in the array. This is because `array[10]` is an addressable that refers to the tenth element of the array. Taking the address of an addressable, like `&array[10]` does, gives you this "original address".

As a final example, if you write `some_pointer^` and immediately take the address of it again, then you get the original pointer back:

```
number := 7
number_pointer := &number
number_pointer_again := &number_pointer^
// Both these pointers refer to `number`!
```

Again, one could have thought that `number_pointer^` gives you some in-between value and that the pointer `number_pointer_again` would refer to that in-between value. But no, `number_pointer^` is an addressable, and taking the address of an addressable gives back the

original memory address. As long as the addressable's value hasn't crossed over the `=`, then you still have one last chance to fetch its original address.

Chapter 7

Procedures and scopes

We've already seen the basics of how to create procedures. Most of the procedure-related concepts we have seen so far can be summarized with this example:

```
is_bigger_than :: proc(number: int, compare_to: int) -> bool {
    return number > compare_to
}

result := is_bigger_than(5, 2)
```

- `is_bigger_than` is a procedure.
- It has two *parameters* of type `int`. Those parameters have the names `number` and `compare_to`.
- It returns a single `bool` value.
- `result := is_bigger_than(5, 2)` calls the procedure with the *arguments* `5` and `2`. It stores the return value in `result`.

Note how I use the word *parameter* when I talk about the things a procedure accepts, such as `number: int, compare_to: int`. I use the word *argument* when I talk about the values sent into the procedure when calling it, such as `5` and `2` in `is_bigger_than(5, 2)`.

Sometimes I'll say "run the procedure", sometimes "call the procedure" and sometimes "invoke the procedure". It's all the same.

Also, I don't always use the word *argument*. Sometimes I say stuff like "pass the variable into the procedure" and similar. I assume it's all clear from context what I mean.

Let's now dig deeper into procedures. We will look at things like default parameter values, multiple return values as well as some special properties of procedure parameters.

After that we'll also discuss what the scope of a procedure is and what *the stack* is. We'll finish off with talking about how the *defer* keyword works.

Default parameter values and named arguments 7.1

You can give a parameter a default value. This makes it possible to skip that parameter when calling the procedure.

The following procedure prints a message, but it also prints `Info:` in front of the message.

```
write_message :: proc(message: string, label: string = "Info") {
    if label != "" {
        fmt.print(label)
        fmt.print(": ")
    }

    fmt.println(message)
}
```

If you run the procedure like so:

```
write_message("Hellope!")
```

then it will print `Info: Hellope!`

However, you can replace the `"Info"` part by providing another parameter:

```
write_message("Hellope!", "Very important info")
```

which will make it output `Very important info: Hellope!`

This is possible because the second parameter looks like this:

```
label: string = "Info"
```

It looks like a normal string parameter, but with `= "Info"` added at the end. `"Info"` is referred to as the *default value* of the parameter. Note how this uses the exact same syntax as creating a string variable with initial value `"Info"`. You can even write

```
label := "Info"
```

and have the type of the parameter inferred from the right side.

If you have a procedure with multiple parameters that have default values, such as this one:

```
my_proc :: proc(a: int, b := 1, c := "hello") {

}
```

Then you can use *named arguments* to skip over `b` but still give `c` a value:

```
my_proc(7, c = "bye bye")
```

`7` goes into `a` and `"bye bye"` goes into `c`. `b` has default value `1`.

This also means that you don't have to provide the arguments in the order the parameters are stated.

These named arguments don't actually have anything to do with default parameter values. You can always use name arguments. For example, you could specify a value for `a` using named arguments, even though it does not have a default value:

```
my_proc(c = "bye bye", a = 7)
```

However, note that this will not compile:

```
my_proc(c = "bye bye", 7)
```

The non-named arguments are called *positional* arguments. All positional arguments must come before all the named arguments.

Example: Allocator parameter with default value *7.1.1*

This example comes from **`<odin>/core/bytes/bytes.odin`** and mentions both **allocators** and **slices**, which we will talk about later. It's a common pattern for passing allocators that you will see over and over.

```
clone :: proc(s: []byte, allocator := context.allocator, loc := #caller_location)
-> []byte {
    c := make([]byte, len(s), allocator, loc)
    copy(c, s)
    return c[:len(s)]
}
```

`clone` takes a slice of bytes and clones it.

We'll talk about slices later, but you can think of them as arrays for now.

In order to do this cloning it needs to *allocate memory*. Memory allocation is done using an *allocator*. Note the `allocator` parameter of `clone`:

```
allocator := context.allocator
```

Since it has the default value `context.allocator`, then we can just run `bytes.clone` with a single argument, like so:

```
copy := bytes.clone(some_bytes)
```

Giving a parameter called `allocator` the default value `context.allocator` is very common in the core collection. It gives a sensible default, while making the user understand that they can provide their own allocator, if they so wish.

`clone` also has a `loc` parameter with a default value:

```
loc := #caller_location
```

This is a way to fetch the source code position of where the procedure was called. This is used a lot in the core collection in order to give good error messages.

Multiple return values 7.2

To return more than one value, use multiple types after `->`:

```
divide_and_double :: proc(n: f32, d: f32) -> (f32, bool) {
    if d == 0 {
        return 0, false
    }

    return (n/d)*2, true
}
```

As you can see, there are now parentheses around the two return values:

```
(f32, bool)
```

When you have more than one return value, then you *must* have parentheses around them.

The parentheses are required because not having them would create ambiguities while the compiler is parsing the code.

`divide_and_double` divides the number `n` by `d` and then multiples the result by `2`.

However, in order to avoid division by `0`, it first checks if `d` is `0`. If it is, then it returns `0` in the first return value and `false` in the second return value. Returning `false` in the second return value indicates that there was an error.

If `d` is non-zero, then the computation is done as expected. The result is returned in the first return value. `true` is returned in the second return value, indicating that everything went OK.

If you wish to store the results of this procedure, then you use two separate variables. Below we assign the two return values to `res` and `ok`.

```
res, ok := divide_and_double(2, 4)

if ok {
    fmt.println(res)
}
```

You can also assign them to pre-existing variables:

```
res, ok = divide_and_double(2, 4)
```

Note the `=` instead of `:=`. You *cannot* mix creating some new variables and re-using some old ones. Either all are new or all are pre-existing.

You can also completely skip handling the return values:

```
divide_and_double(2, 4)
```

If you don't like that, then you can make it *required* to handle the return values. This is done by putting the attribute `@require_results` in front of the procedure declaration:

```
@require_results
divide_and_double :: proc(n: f32, d: f32) -> (f32, bool) {
    // ...
}
```

There are **many more attributes** .

Which makes this line an error:

```
divide_and_double(2, 4)
```

However, if you handle at least one of the return values, then you must handle all of them. The following would be an error, regardless of if `@require_results` is used or not:

```
res := divide_and_double(2, 4)
```

If you wish to skip assigning one of the return values, then you must explicitly assign it to the special symbol `_`, which means that we throw away that value:

```
res, _ := divide_and_double(2, 4)
```

`_` can also be used like this:

```
_ :: fmt
```

You might need to do that if you've imported `core:fmt` but haven't used it yet. That could trigger a `'fmt' declared but not used` error, given that you compile with the `-vet` flag.

Since it can be messy to litter the code with variables such as `res` and `ok`, then you can use this special if-statement, so that `res` and `ok` are only available within it:

```
if res, ok := divide_and_double(2, 4); ok {
    fmt.println(res)
}
```

We saw this type of if-statement before, when we looked at **unions**.

This method of returning a `bool` in the last return value is a common way to report errors in Odin. For more complicated errors one can also use enums and unions. We'll see more of that when we talk about **error handling**.

Named return values 7.3

You can give names to return values. In the procedure below, note how the return values look like a list of variable declarations: `(res: f32, ok: bool)`. In fact, these named return values act just like normal variables. You can assign to them and they are zero-initialized.

```
divide_and_double :: proc(n: f32, d: f32) -> (res: f32, ok: bool) {
    if d == 0 {
        return
    }

    res = (n/d)*2
    ok = true
    return
}
```

This procedure is similar to the one in the previous section. However, here's what the old one did when `d == 0`:

```
if d == 0 {
    return 0, false
```

```
}
```

now we just do

```
if d == 0 {
    return
}
```

This is called a "naked return". It's a return that doesn't specify what it is returning. It means that we are returning the named return values `res` and `ok`. Since `res` and `ok` were initialized to zero by default, they contain `0` and `false` respectively. So this procedure does the same thing as the procedure in the previous section.

Also, the old version of `divide_and_double`, did this when `d` was non-zero:

```
return (n/d)*2, true
```

Now we do:

```
res = (n/d)*2
ok = true
return
```

However, we are not forced to use the naked return. We can still do explicit returns like before: `return (n/d)*2, true`, which may in fact look cleaner.

Naked returns may seem a bit strange to some people. But when we later talk about **error handling**, then we will make use of named return values in combination with things like `or_return`. In that case we will see that being able to do naked returns is a good thing.

Nested procedures and captured variables 7.4

Procedures can be declared both directly at global file scope, but also within other procedures:

By "global file scope" I mean directly in the file, outside of all other procedures.

```
do_stuff :: proc() {
    print_message :: proc(msg: string) {
        fmt.println(msg)
    }
```

```
    print_message("Hellope!")
}
```

Above we see a procedure `do_stuff` that lives in the global file scope, but we also have a procedure called `print_message` within `do_stuff`. `print_message` is only available from within `do_stuff`. This is useful for reusing code within a procedure without having to make a globally accessible procedure. Keeping it within `do_stuff` keeps the global file scope tidy.

Let's look at which variables and constants are available within a *nested* procedure such as `print_message`. Say that we extend the example above with a global variable, a global constant and also a local variable within `do_stuff`:

```
global_state: int
CONSTANT_NUMBER :: 7

do_stuff :: proc() {
    local_variable: int

    print_message :: proc(msg: string) {
        fmt.println(msg)
    }

    print_message("Hellope!")
}
```

`print_message` can use these things:

- The parameter `msg`
- The global variable `global_state`
- The global constant `CONSTANT_NUMBER`

However, `print_message` cannot see or use `local_variable` that lives in the parent procedure `do_stuff`. Odin does not support any such *automatic capturing*.

The reason for not supporting it is that you either need automatic memory management or some complicated system for copying things from the parent procedure's scope. Odin does not do automatic memory management and strives for simplicity, so no automatic capturing at all ever happens.

The reason you can always use globals is because globals live for as long as the program runs. They are not bound to the lifetime of any procedure.

You can also create static variables within procedures:

```
@static my_variable: int
```

A static variable behaves like a global variable, but it's only available within the procedure where it was declared. That variable can also be seen from any nested procedure. The value it has lives on between different calls to that procedure. Beware of relying on this too much. If you ever write multi-threaded code, then multiple threads might end up using the same static variable!

Parameters are always immutable 7.5

All procedure parameters are always immutable. You can never change the value of a parameter. The following would not compile because we are trying to change the value of the parameter `n`:

"mutable" means "possible to change". It's possible to *mutate* it. So "immutable" means "not possible to change".

```
divide_numbers :: proc(n, d: f32) -> f32 {
    n = n / d
    return n
}
```

From C / C++

There is no concept of `const` in Odin. However, all parameters are always immutable. So it's as if all of them are `const`.

To make the above compile, add `n := n` as the first line of the proc:

```
divide_numbers :: proc(n, d: f32) -> f32 {
    n := n
    n = n / d
    return n
}
```

This creates a copy of the parameter `n` and stores it in a variable with the same name. After that we can modify `n` since it is no longer a procedure parameter. You can always create a new variable with the same name as a procedure parameter. This is handy because it gives you a mutable copy without having to come up with a new name.

Reusing a name like this is known as *shadowing*. The compiler flag `-vet` disallows shadowing. However, even if you compile with `-vet`, this special case where you "self-shadow" a procedure parameter in order to get a mutable copy is still allowed.

Although Odin's procedure parameters are immutable, you can still modify the value that pointers refer to. The following procedure modifies the value that the pointer `n` refer to:

```
increase_number :: proc(n: ^int, amount: int) {
    n^ += amount
}
```

What you're not allowed to do is change what `n` points to:

```
increase_number :: proc(n: ^int, amount: int) {
    some_other_int := 7
    n = &some_other_int
    n^ += amount
}
```

The address that the pointer contains is the thing that is immutable, not the memory it points to.

Don't use pointer parameters for the sake of optimization 7.6

Copying large amounts of data is slow. We say that there is an *overhead* related to copying. It may be tempting to use procedure parameters that are pointers, in order to avoid copying. It may be especially tempting when the parameter uses a type that is a very big struct or array. However, this is not necessary in Odin: A parameter that is larger than `16` bytes is automatically passed as an *immutable reference*. Let's look at what this means in detail.

Say that you have a big struct such as `Person` below:

```
Person :: struct {
    name: string,
    health: int,
    age: int,
    number_of_teeth: int,
    height_meters: f32,
}
```

Also, say that you have the following procedure that prints some info about a `Person` struct:

```
print_person_info :: proc(p: Person) {
    fmt.printfln("%v is %v years old and has %v teeth.", p.name, p.age,
```

```
p.number_of_teeth)
}
```

You might think that every time some code calls `print_person_info`, then the value you supply into the parameter `p: Person` is copied. In order to avoid that copying you might then change the parameter into a pointer:

```
print_person_info :: proc(p: ^Person) {
    fmt.printfln("%v is %v years old and has %v teeth.", p.name, p.age,
p.number_of_teeth)
}
```

Note how it says

`p: ^Person`

instead of

`p: Person`

This might seem like a very reasonable thing to do. After all, `Person` uses `48` bytes of memory. A pointer only uses `8` bytes (on a 64 bit computer). So it may seem like there would be way less copying if you used a pointer.

However, the above is *not necessary* in Odin. Why? Because the size of `Person` is larger than `16` bytes, so it will automatically be passed as an immutable reference.

You can find out how many bytes of memory a type uses using `size_of`, for example:

`size_of(Person)`

OK, so what does "immutable reference" mean? Say that you write the following:

```
print_person_info :: proc(p: Person) {
    fmt.printfln("%v is %v years old and has %v teeth.", p.name, p.age,
p.number_of_teeth)
}

hans := Person {
    name = "Hans",
    age = 65,
    number_of_teeth = 6,
    health = 20,
    height_meters = 1.69,
}

print_person_info(hans)
```

When you call `print_person_info(hans)`, then `hans` outside the procedure will use the exact same memory as `p` inside the procedure. Under the surface the compiler has passed it as a reference. However, the reference is also immutable, meaning that you cannot alter any of its fields. This makes sure that you can't accidentally modify the variable `hans`. This is why we say that `p` is an immutable reference.

You can do `p := p` inside the `print_person_info` if you want to create a modifiable local copy of `p`. That would be a new, unrelated variable.

If you want to actually make a procedure that modifies something, then you can use a pointer parameter. But whenever you're making a procedure that just uses some data without modifying it, then you do not need to worry about the overhead of copying.

From C / C++

If you come from C++ then you probably do a lot of `const Person& person` in order to specify that you want immutable (const) reference parameters. In Odin you just use a value type parameter like `person: Person` and instead rely on it being automatically passed as immutable reference if the type is larger than `16` bytes.

Pointer and immutable reference aliasing *7.6.1*

There is something called pointer aliasing. This means that you have two pointers that somehow reference the same data. This means that changing some data through one of the pointers alters what you see through the other pointer. Sometimes this was not intended by the programmer, and may therefore introduce bugs.

Procedure parameters larger than `16` bytes will be passed as immutable references. This means that they are susceptible to similar aliasing issues. Here's an example:

```
Person :: struct {
    name: string,
    health: int,
    age: int,
    number_of_teeth: int,
    height_meters: f32,
}

Couple_Data :: struct {
    person_1: Person,
    person_2: Person,
}

couple_data := Couple_Data {
    person_1 = {
```

```
            name = "Hans",
            age = 65,
        },
        person_2 = {
            name = "Maj-Britt",
            age = 50,
        },
    }

    aliasing_example :: proc(cd: ^Couple_Data, person: Person) {
        cd.person_1 = {
            name = "This modifies 'person' too",
        }

        fmt.println(person.name) // Prints "This modifies 'person' too"
    }

    aliasing_example(&couple_data, couple_data.person_1)
```

The procedure `aliasing_example` accepts two parameters: `cd: ^Couple_Data` and `person: Person`. We run `aliasing_example` like this:

```
aliasing_example(&couple_data, couple_data.person_1)
```

As you can see we feed `&couple_data` into the parameter `cd: ^Couple_Data` of `aliasing_example`. Into the parameter `person: Person` we feed `couple_data.person_1`. But since the struct `Person` uses `48` bytes of memory, then it is automatically passed as an immutable reference.

This means that within `aliasing_example` the parameter `person` is an immutable reference to the exact same memory as `cd.person_1`. Modifying `cd.person_1` will modify `person` as well.

This can cause bugs if you do not know about it. Note what happens in `aliasing_example`: It modifies the `name` field of `cd.person_1` and then prints `person.name`. This printing will print the new name we just gave to `cd.person_1`, because they share the same memory.

From C / C++

This can sneakily happen in `C++` too. There one often passes things by const reference to avoid copying. The parameters of `aliasing_example` would look like this in C++: `Couple_Data *cd, const Person& person`, which would be susceptible to the exact same issues.

In C this is perhaps less of a problem because there you have to pass things by explicit pointer, which is more visible at the call-site and may thus set off alarm bells in your head.

If you have a procedure that is susceptible to such aliasing, then you can make an explicit copy of the parameter that you think would have the issue. In our example you could put

`person := person` at the top of the procedure:

```
aliasing_example :: proc(cd: ^Couple_Data, person: Person) {
    person := person

    cd.person_1 = {
        name = "This no longer modifies 'person'",
    }

    fmt.println(person.name) // Prints the original name
}
```

Explicit overloading 7.7

Use explicit overloading to make two or more procedures exist under the same name. Here's an example of how to implement a procedure `length` that accepts both `[2]f32` and `[3]f32` arguments:

You can see these fixed arrays as 2 and 3 dimensional *vectors*. In the **Fixed arrays revisited** chapter, I will talk more about how fixed arrays can be used to provide vector types and how we can do vector maths operations on them.

By "vector" I mean the linear algebra term. It's not to be confused with how C++ uses the word "vector" to mean "dynamic array".

```
length :: proc {
    length_float2,
    length_float3,
}

length_float2 :: proc(v: [2]f32) -> f32 {
    return math.sqrt(v.x*v.x + v.y*v.y)
}

length_float3 :: proc(v: [3]f32) -> f32 {
    return math.sqrt(v.x*v.x + v.y*v.y + v.z*v.z)
}
```

This is just Pythagoras' Theorem in 2 and 3 dimensions. This can be implemented in more generic ways using **parametric polymorphism**.

Also, `math.sqrt` comes from `import "core:math"`

Note the special `length :: proc {}` syntax. That's how you make an explicit overload. Within those curly braces, list all the procedures you want to be available under the name `length`.

We can then use `length` with any value of type `[2]f32` or `[3]f32`:

```
v2 := [2]f32 { 3, 3 }
v3 := [3]f32 { 3, 3, 3}
len_v2 := length(v2)
len_v3 := length(v3)
```

A positive thing about explicit overloading is that we can still use a specific variant if we so wish:

```
v2 := [2]f32 { 3, 3 }
len_v2 := length_float2(v2)
```

From C / C++

C++ uses implicit overloading where all the overloads share the same name, so it's not possible to pick out a specific one later.

Run procedures on startup and shutdown 7.8

Put `@init` in front of a procedure to automatically run it when your program starts. Put `@fini` in front of a procedure to automatically run it when your program shuts down:

```
package init_fini_example

import "base:runtime"
import "core:fmt"

main :: proc() {
    fmt.println("Program running")
}

@init
startup :: proc "contextless" () {
    context = runtime.default_context()
    fmt.println("Program started")
}

@fini
shutdown :: proc "contextless" () {
    context = runtime.default_context()
```

```
    fmt.println("Program shutting down")
}
```

This program will print:

```
Program started
Program running
Program shutting down
```

Note that the `startup` and `shutdown` procedures are marked with `"contextless"`. This is a requirement for any procedure that has the `@init` or `@fini` attribute.

`contextless` is called a *calling convention*. A calling convention controls how a procedure is called (how is it passed parameters? How does it return things? etc). When using the default calling convention of Odin, all procedures are implicitly passed a variable called `context`. It contains things such as an allocator, a logger and many other useful things. `contextless` means that there is *no* implicit `context` passed into that procedure.

We'll discuss the implicit context more in **chapter 12**.

However, both `startup` and `shutdown` calls `fmt.println`. `fmt.println` expects to be implicitly passed a `context`; because it uses the default calling convention. So we cannot call it unless we first give `context` a value. In this case we set `context` to `runtime.default_context()`. That's the same value as `context` has when the `main` procedure begins.

See **the official overview** for a list of all supported calling conventions.

`@init` and `@fini` procedures must use the `contextless` calling convention for two reasons:

- `main` may set up a custom `context` when it starts. Procedures with the `@init` attribute run before `main`. So any such setup will not yet have happened. Enforcing the `contextless` calling convention makes you aware of this, so you can act accordingly.
- You may import a package that uses `@init` or `@fini` procedures. The `contextless` calling convention ensures that those procedures don't try to allocate memory using `context.allocator`. That's good because compiler flags such as `-default-to-nil-allocator` and `-default-to-panic-allocator` may give the context an allocator that can't allocate memory. In this case, enforcing `contextless` ensures that you don't get misbehavior or crashes on startup from just importing some package.

The scope and the stack 7.9

Let's look at what a scope is and how it relates to the concept of *the stack*. The scope of a procedure runs from the opening curly brace `{` to the closing one `}`:

```
my_proc :: proc() {
    number := 7
}
```

This means that `number` in the code above is only valid within that scope. This is why the following is a classic mistake:

```
my_proc :: proc() -> ^int {
    number := 7
    number_pointer := &number
    return number_pointer
}
```

The code above will compile, but it's going to misbehave. The type of the return value is `^int`, or "pointer to integer". The value we are returning is the address of the local variable `number`. But that variable is only valid within the scope of the procedure. So once the procedure is over, then the pointer we are returning is pointing at garbage memory. This memory may soon be used by other parts of the program.

If you instead wrote

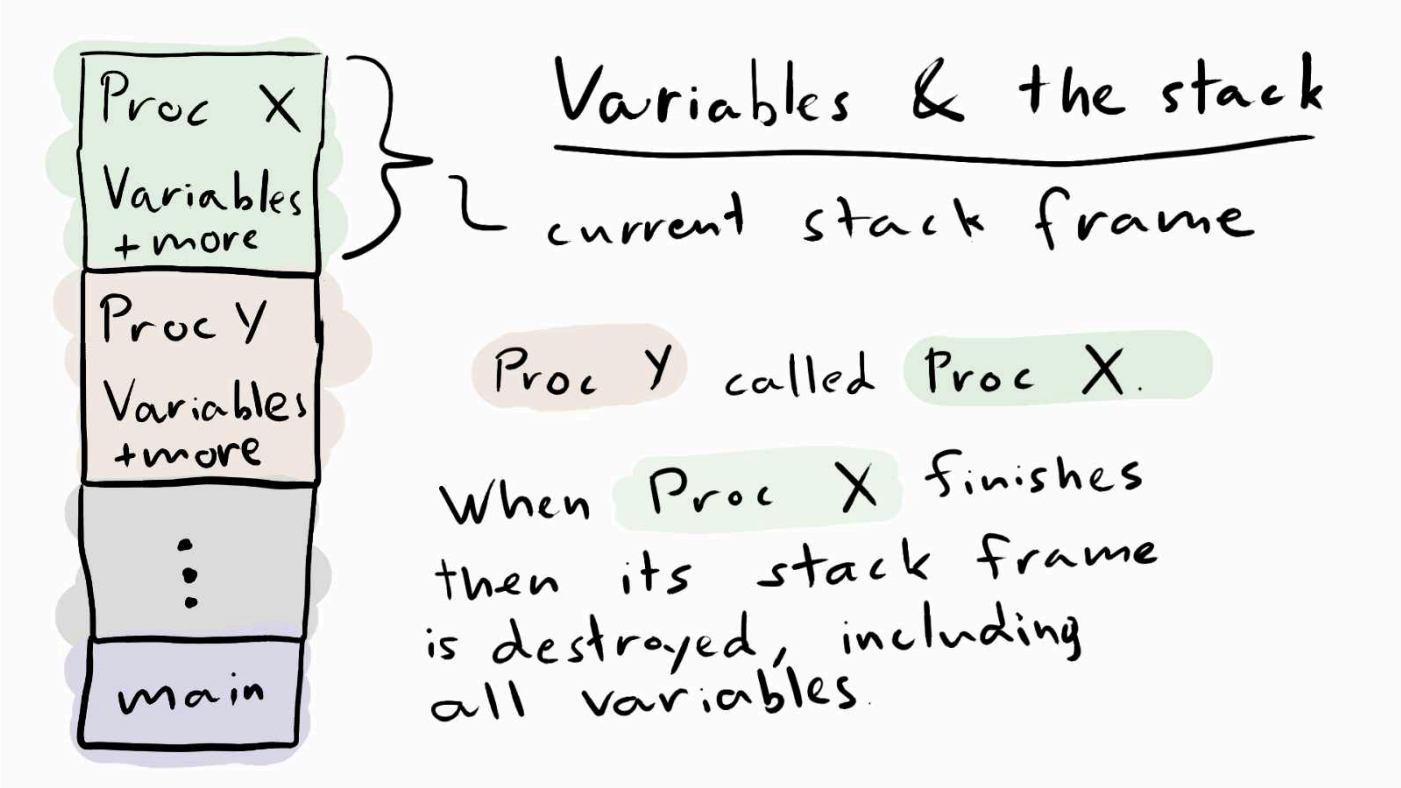
`return &number`

then it would no longer compile. The compiler would see that we are trying to return a pointer to a local variable. But if you put the pointer into an in-between variable such as `number_pointer`, then it does compile.

So what is this memory that is available within the scope of the procedure. Where does `number`'s memory live?

Each procedure has what is known as a *stack frame*. A stack frame is some memory that is big enough to hold the procedure's local variables, parameters and information about where to store the return values. This means that declaring a variable like `number: int` will reserve memory for that variable within the stack frame. How much memory? It depends on the size of the variable. In this case `number` is of type `int`, which is a 64 bit integer (on 64 bit systems). So something of type `int` will use 8 bytes of memory within the stack frame.

8 bits is equal to 1 byte of memory. So 64 bits is equivalent to 8 bytes.



Proc Y has called Proc X. Each box is a stack frame. The stack frame contains, among other things, all the local variables of each running procedure. When Proc X finishes, then its stack frame will be destroyed.

You do not need to deallocate any memory that lives within the stack frame. Each procedure that is being executed has a stack frame. These stack frames make up what is referred to as *the stack*. The top-most frame of the stack is the current procedure, the one just below it in the stack is the procedure that called the current procedure, and so forth, all the way down to the `main` procedure. When a procedure finishes, then its stack frame is destroyed, or "popped off" the top of the stack. Since all the local variables of a procedure live within the stack frame, there's no need for any kind of deallocation of those variables.

Later we shall look at **dynamically allocated memory**, which usually means allocating memory that lives *outside* of the stack. Such dynamically allocated memory lives on, independent of any scope.

Scopes within scopes 7.10

There can be scopes within scopes. Below we see an if-statement. The code between the curly braces of the if-statement is a scope of its own. The variable `message` is not available outside that scope.

```
my_proc :: proc() {
    number := 7

    if number > 5 {
        message := "It's more than 5"
        fmt.println(message)
    }

    // message is not available here
}
```

The code within the curly braces of any `if`, `for` or `proc` all have their own scope.

Something that does *not* have its own scope is the `when` statement. This is explained in a later chapter when I talk about **tracking memory leaks**.

You can also make anonymous scopes. That just means some code contained between two curly braces:

```
my_proc :: proc() {
    number := 7

    {
        message := "Hellope"
        fmt.println(message)
    }

    // message is not available here
}
```

The memory for `message` is actually part of the stack frame of the procedure and thus reserved outside of the anonymous scope. But the name `message` is only available within the scope where it is declared.

In theory, you could make a pointer that contains the address of `message` and use it outside of the anonymous scope. However, the backend of the compiler might do optimizations where it reuses stack memory. So you should never try to do this to save stack memory space.

Anonymous scopes can be useful in long procedures where you would otherwise get variable name collisions.

defer: Making things happen at end of scope 7.11

Sometimes you create something that needs to be destroyed at the end of the scope. Rather than manually putting the code that handles this destruction at the end of the scope, you can instead use `defer`. This means that you can write some code in the middle of a scope and slap `defer` in front of it, making it actually happen at the end of the scope.

Here's an example, based on an example from the **official overview** . It uses defer in order to close a file handle when the procedure finishes:

```
read_file :: proc() {
    f, err := os.open("my_file.txt")

    if err != os.ERROR_NONE {
        // handle error
    }

    defer os.close(f)

    // Put code here that uses `f`
    // to read data from file.

    // os.close(f) is run at the end of
    // the procedure.
}
```

The line `defer os.close(f)` will make `os.close(f)` run at the end of the scope.

As I've pointed out, a procedure can have additional scopes within it. If you put the `defer` within an anonymous scope or within a block of an if-statement, then the defer will execute at the end of that scope, not at the end of the procedure.

Here follows a bigger example. In the procedure `most_green_color` we load an image using raylib (`import rl "vendor:raylib"`) and then figure out which pixel within the image is "the most green". This means that it will figure out what pixel has the largest value in the green color channel.

`rl.UnloadImage(img)` is used to unload the image. We write `defer rl.UnloadImage(img)` near the start of the procedure, but due to the `defer` this unloading will wait until the procedure returns.

```
most_green_color :: proc(filename: cstring) -> rl.Color {
    img := rl.LoadImage(filename)

    if img.data == nil || img.width == 0 || img.height == 0 {
        return {}
```

```
        }

        defer rl.UnloadImage(img)

        most_green_color := rl.GetImageColor(img, 0, 0)

        for x in 0..<img.width {
            for y in 0..<img.height {
                color := rl.GetImageColor(img, i32(x), i32(y))

                if color.g == 255 {
                    // rl.UnloadImage(img) happens after this line
                    return color
                }

                if color.g > most_green_color.g {
                    most_green_color = color
                }
            }
        }

        return most_green_color
    } // <-- rl.UnloadImage(img) happens here.
```

Note something here: There are two lines that uses `return`. This one:

```
if color.g == 255 {
    // rl.UnloadImage(img) happens after this line
    return color
}
```

and this one:

```
    return most_green_color
} // <-- rl.UnloadImage(img) happens here.
```

The deferred code will run when any of these two returns happen. This is a powerful thing about defer: You can defer something once, but have it happen in different places depending on how the scope ends. It's very useful when returning early from procedures.

The defer happens after the last line of the scope. Or in the case of a `return`, after the `return`. Also, if you have multiple defers within a scope, then they happen in the reverse order of how they were created.

Don't overuse defer *7.11.1*

It may be tempting to use `defer` as much as you can. It reduces bugs related to having `return` in multiple places. But it also makes the code a bit harder to read: You can't read the code in a serial manner anymore. The opinions on how much to use defer varies. Personally I only use `defer` in procedures like `most_green_color`, where we want to run some "cleanup code" regardless of how the procedure ends. I recommend that you experiment a bit. Get a feeling for how it affects readability versus how it reduces bugs.

Chapter 8

Fixed-memory containers

Containers are things in your program that can contain several items of a certain type. We had a quick look at fixed arrays **earlier**. In this chapter we'll go over fixed arrays in more detail, including how they can be used to provide the foundations for doing *vector math*.

We will also look at enumerated arrays. Enumerated arrays are very similar to fixed arrays, but have a close relationship to enums.

The containers we look at in this chapter are simple and do not require any kind of manual memory management. This means that they cannot grow larger than the initial size we give them. We say that these are *fixed-memory* containers.

At the end of this chapter we'll look at the "Fixed Capacity Dynamic Array", FCD Array for short. It's a container that uses a fixed array under the hood, while also keeping track of how much space within the fixed array it has used. This makes it behave as if it can grow, even though it cannot grow past its initial capacity.

Fixed arrays revisited 8.1

As the name suggest, fixed arrays are created with a fixed size. They cannot grow or shrink. They are often useful when you just need a small temporary array. But as we shall see, they can also be used to store big amounts of permanent data.

This creates a fixed array containing 7 integers:

```
some_ints: [7]int
```

As always in Odin the fixed array is zero-initialized. All the 7 integers in this fixed array will be zero. Just like with any variable, you can also set the value of a fixed array:

```
some_ints = { 51, 21, 621, 1, 51, 7, 12 }
```

Note that you must specify all the values when you set it. Actually setting a fixed array like this is mostly useful when you have a fixed array of a small size.

You can also clear all the elements of the array, which will make all of them zero:

```
some_ints = {}
```

As usual, you can do both declaration and assignment at once:

```
some_ints := [7]int { 51, 21, 621, 1, 51, 7, 12 }
```

Note the `:=`

To set and fetch values in the array, use brackets `[]`. Put the *index* of the thing you want to set or fetch between the brackets.

```
some_ints[3] = 500
some_ints[4] = 200
some_ints[5] = 333

fmt.println(some_ints[4]) // 200
```

You can fetch the length of the array using `len(some_ints)`, but since it's fixed it will just return the fixed length, which in the example above will be `7`. Indices (indexes) start at `0`, meaning that the index of the last element is `len(some_ints) - 1`.

Bugs where you accidentally use index `i-1` or `i+1` instead of `i` are very common and are referred to as "off-by-one errors".

You can iterate over a fixed array, meaning that you can easily loop through all the items:

```
for v in some_ints {
    fmt.println(v)
}
```

If you need the index while looping, then add in another loop variable after `v`:

```
for v, i in some_ints {
    // This will print:
    // `value at 0 is <some number>`
    // etc
    fmt.printfln("value at %v is %v", i, v)
}
```

Note the `, i` after `v`

`v` is immutable (not possible to modify) in the loop above. If you want to modify the value, then add in `&` in front of it. For example, this loop multiplies each element in an array by 2:

```
for &v in some_ints {
    v *= 2
}
```

The `&` here does not give you a pointer to the element. It gives you an addressable that you can assign to. See the section on **addressables** for more info on what that means.

You can think of it as doing `some_ints[i] *= 2` where `i` is the index of the element.

Also, note that without the `&` each lap of the loop makes a copy of the element. This can be of concern if you iterate an array of big structs, as it can become slow to make all those copies.

You can of course make fixed arrays containing more complex types than just integers. For example, we can make a fixed array of a struct type:

```
Rectangle :: struct {
    x, y, width, height: f32,
}

rectangles: [10]Rectangle
```

This makes a fixed array of 10 "Rectangles". All of them have their fields zeroed by default.

Where does the memory live? *8.1.1*

If you create a fixed array within a procedure, like this:

```
main :: proc() {
    some_ints: [100]int
}
```

Then the memory of `some_ints` is part of the stack frame of the procedure `main`. **Earlier** I talked about what the stack is and how local variables live within the *stack memory* of the procedure. The same is true for fixed arrays. The array above lives and dies with the scope of the procedure.

Fixed arrays copy in the same way as basic variable types (such as `int` and `f32`). So if you assign a fixed array to a new variable, then the new variable has its own separate copy of all the array's data:

```
main :: proc() {
    some_ints: [100]int
    some_ints[10] = 2
    some_ints2 := some_ints
    some_ints2[10] = 5
```

```
    fmt.println(some_ints[10]) // 2
    fmt.println(some_ints2[10]) // 5
}
```

From C / C++

This is slightly different from C. In C a fixed array like `int some_ints[100];` lives on the stack just like in Odin, but if you try to assign a fixed array to something else, then you just get the pointer to the first element. To copy the data you'd have to use `memcpy` or similar. In Odin the data copies automatically.

If you want a pointer to the first element use `raw_data(&some_ints)` or `&some_ints[0]`.

This shows that there is no shared memory between these two fixed arrays. With regards to copying, they behave just like a variable of some basic type.

The above works the same if you have a fixed array that lives within a struct, and you copy the struct. For example:

```
My_Struct :: struct {
    some_ints: [100]int,
}

main :: proc() {
    a_struct: My_Struct
    a_struct.some_ints[10] = 2
    another_struct := a_struct
    another_struct.some_ints[10] = 5
    fmt.println(a_struct.some_ints[10]) // 2
    fmt.println(another_struct.some_ints[10]) // 5
}
```

This also shows that fixed arrays take up space directly within the struct. Fixed arrays don't keep the data anywhere else as some sort of reference. If you create a copy of a struct, then you get a copy of all the fields it contains, including fixed arrays.

However, if your struct has any kind of pointer, then the data which the pointer refers to will *not* get copied.

This kind of copy is sometimes known as a *shallow copy*. Only data that directly takes up space within the struct will get copied. The things that take up space within a struct are the same kind of things that take up stack memory.

Blowing up the stack *8.1.2*

Since the stack frame of each procedure has a limited amount of memory available, making a giant fixed array is a common way to run out of stack memory. This would lead to your program crashing. Beware of stuff like this:

```
main :: proc() {
    lots_of_ints: [1000000]int
}
```

The compiler will warn you when you try to create giant things that live on the stack.

While procedures are limited to the amount of available stack memory, what is less limited is the amount of memory you have available for global variables. The following should work fine:

```
lots_of_ints: [10000000]int

main :: proc() {
    // lots_of_ints is a global variable
    // that you can use for as long as your
    // program runs.
}
```

Global variables don't live within any procedure, so they aren't part of any procedure's stack memory. Instead they live inside what is known as the *data block* of the program, where you can store much bigger things.

By the "procedure's stack memory" I refer to the ***stack frame* of the procedure**.

The biggest global fixed array you can create is `268435455` items long. That said, you cannot have a too big data block in your program. On Windows (or at least on my computer) the linker will protest if have more than `80000000` bytes (`80` megabytes) in the data block.

However, if you really need a fixed array that big, then you are probably better off dynamically allocating memory, which you can read about in **the next chapter**.

Vectors and array programming *8.1.3*

Say that you are writing some 3D visualization software or a video game. Then you need to represent positions in 3D space. You can do so using a fixed array of three decimal numbers:

```
position: [3]f32
```

Use `position: [2]f32` if you're working in 2D.

You can assign to it, swapping out the three numbers that it contains:

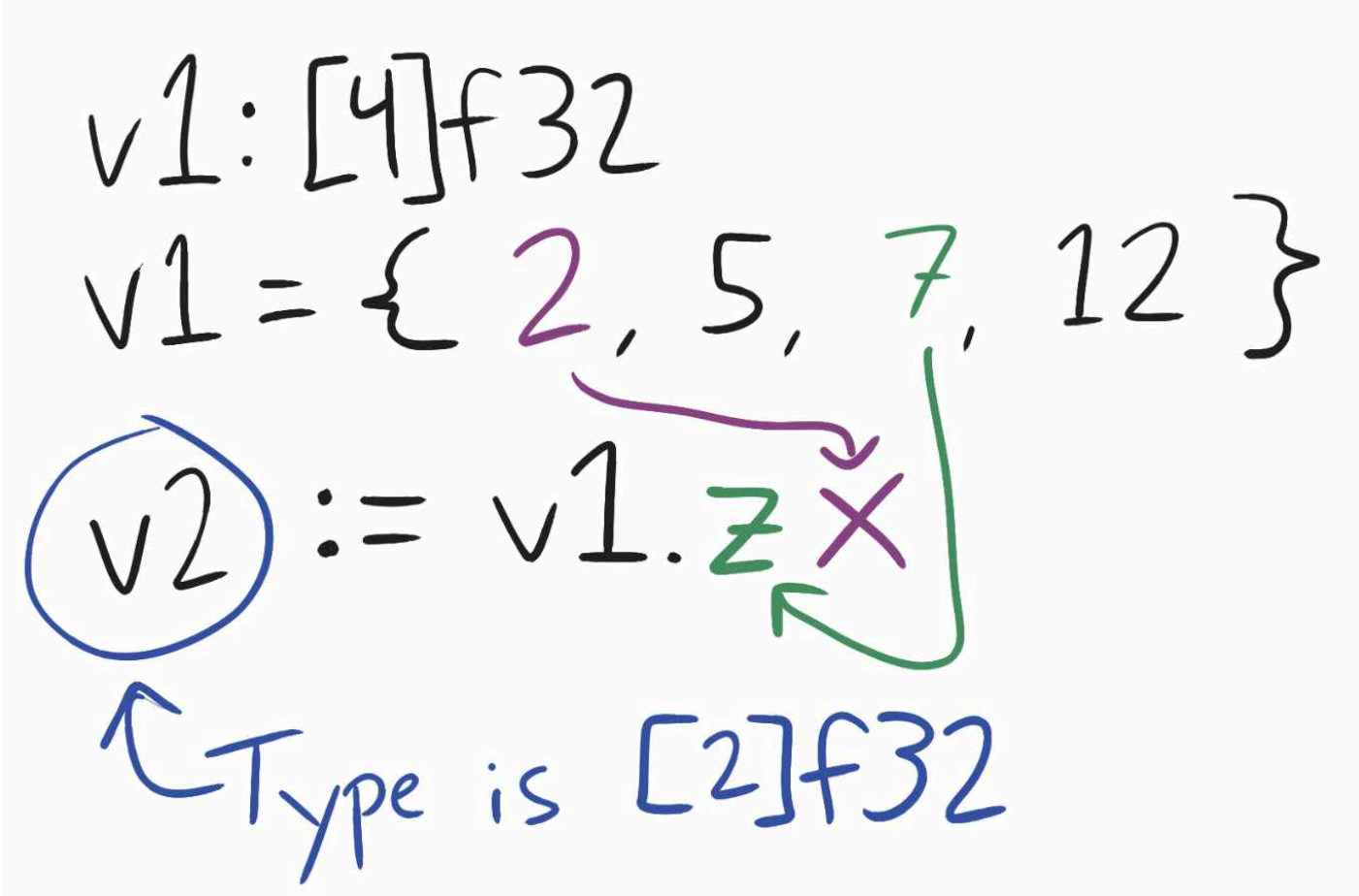
```
position = {7, 1, 2}
```

You can see the first, second and third element as x, y and z positions in 3D space

As usual, you can do it all in one line:

```
position := [3]f32 {7, 1, 2}
```

As we have seen, you can fetch the first element of `position` like so: `position[0]`. However, when arrays have 4 or fewer items, then you can also index them using `x`, `y`, `z` and `w`. So instead of `position[0]` you can write `position.x`. This is very handy because we can see this as the position along the x axis in 3D space!



You can also create a smaller array from a bigger one using these xyzw letters:

```
zx_pos := position.zx
```

This new variable `zx_pos` is of type `[2]f32` and contains the z and x parts of the variable `position`. You can also do stuff like `position.xxyy` or `position.zyx`. These kinds of operations are known as *swizzling*.

If you've written code in a shader language such as HLSL or GLSL, then you might have seen swizzling before.

Also, because some applications use 4 dimensional numbers to represent colors, you can also use the letters `.rgba` instead of `.xyzw`.

If you don't want to write `[3]f32` all the time, then you can make an alias called `Vector3`:

```
Vector3 :: [3]f32
```

This turns `Vector3` into a new type that can be used interchangeably with `[3]f32`. Why the name Vector3? Because Vector3 means "3D Vector". A 3D vector is a direction plus a length in 3D space, which is representable using 3 floating point numbers.

Even though vectors are thought to represent a direction plus a length, the same type is usually used for positions as well, even though positions are just a point in space. This is because we often add vectors to positions in order to move them. So many programs just use the same type for vectors and positions. However, there's nothing stopping you from making another alias:

```
Position3 :: [3]f32
```

You can add two Vector3 variables, like so:

```
position: Vector3
velocity := Vector3 {0, 0, 10}
position += velocity
fmt.println(position) // [0, 0, 10]
```

This is known as *array programming*. As long as two fixed arrays have the same element type (`f32` in this case) and the same number of elements (3 in this case), then they can be added.

You can also subtract fixed arrays of matching element type and length:

```
position1 := Vector3 {10, 21, 1}
position2 := Vector3 {9, 1, 3}
from_1_to_2 := position2 - position1
fmt.println(from_1_to_2) // [-1, -20, 2]
```

You can multiply them element-wise:

```
xz_vector := Vector3 {10, 0, -2}
z_scaled_up := xz_vector * {1, 1, 10}
fmt.println(z_scaled_up) // [10, 0, -20]
```

Finally you can also multiply a fixed array by a single number, which will multiply all the elements by that number:

```
one_meter_ahead := Vector3 {0, 0, 1}
ten_meters_ahead := one_meter_ahead * 10
fmt.println(ten_meters_ahead) // [0, 0, 10]
```

Such a "single number" is known as a scalar. This scalar must be of the same type as the fixed array's elements.

Odin does not have operator overloading. However, one of the most common use cases for operator overloading is doing vector maths. So I haven't missed operator overloading at all, because of how great array programming works.

Things like this are a good example of why I enjoy Odin so much for video games programming.

Also, as I've shown, fixed arrays live directly on the stack and making copies of them copies all the underlying data. This is good news, because you do want to be able to easily copy a vector by just assigning it to a new variable.

A fixed array declared within a procedure lives on the stack. In **chapter 9.3.2** we'll see how you can dynamically allocate memory for a fixed array, which would make it live outside of the stack.

Also, if you write

`arr: [10]int` in the global file scope, outside of any procedure, then that variable isn't stack allocated. Rather, it lives as a global in the *data block* of your program. But it still behaves like a stack allocated array, in the sense that it directly contains all its elements.

Passing fixed arrays to procedures *8.1.4*

Procedure parameters that are fixed arrays are easy to assign to, since you can just write the value you want within curly braces. As an example, say that you have a procedure that draws an image at a specific position on the screen. That 2D position can be represented by a parameter of type `[2]f32`:

```
draw_image :: proc(image: Image, position: [2]f32) {
    // Draw image using position.
}
```

If you want to pass in a hard-coded position, then you can simply do this:

```
draw_image(my_image, {100, 200})
```

There's no need to write the type name since the compiler can figure it out by looking at the type of the parameter.

The above is actually just the standard way of assigning a value to a fixed array:

```
position: [2]f32
position = {100, 200}
```

but you can think of it as assigning it to the parameter instead of a local variable.

Enumerated arrays 8.2

You can create arrays that map nicely to an enum and that always have the same number of elements as an enum:

```
Nice_People :: enum {
    Bob,
    Klucke,
    Tim,
}

nice_rating := [Nice_People]int {
    .Bob = 5,
    .Klucke = 7,
    .Tim = 3,
}

bobs_niceness := nice_rating[.Bob]
```

In the code above we have an enum `Nice_People`. The variable `nice_rating` is an enumerated array. Note how it uses `Nice_People` between the two brackets:

```
nice_rating := [Nice_People]int {
```

We can then use the values of `Nice_People` to index the array:

```
bobs_niceness := nice_rating[.Bob]
```

You could also skip initializing the enumerated array. It would then have the same number of items as the enum, but all of them would be zeroed:

```
nice_rating: [Nice_People]int
```

Finally, you can do a partial initialization. All non-mentioned items will be zero:

```
nice_rating := #partial [Nice_People]int {
    .Klucke = 10,
}
```

Note that we have to write `#partial` here, either you mention all enum members, none, or use `#partial`.

Enumerated arrays work just like fixed arrays. They live on the stack and copy in the same way. No extra memory allocations happen. It's just a convenient way to use enum member names as indices for a fixed array.

Fixed Capacity Dynamic Arrays 8.3

In the next chapter we'll take our first look at manual memory management. At that point we'll meet the *dynamic array*, which is an array that can grow and shrink. But before we go there, let's look at a type of container that is similar to a dynamic array, but doesn't require manual memory management.

It's called "Fixed Capacity Dynamic Array". The name contains "dynamic array", but it should not be confused with the dynamic array that we will see in chapter 9. It's also a long name, so I'll use the abbreviation FCD Array. It is possible to add and remove items from an FCD Array. Since you can add and remove items, it may *seem* like this array can grow. But it cannot actually become larger than the maximum size that we choose. And it will always use as much memory as the chosen maximum size.

Before the FCD Array existed in Odin, there was a type in `core` called `Small_Array`. The FCD Array replaced `Small_Array`.

Having a predetermined maximum size may seem limiting. But if you can make a reasonable upper estimate of the needed size, then an FCD Array can be a good idea. After all, there's the benefit of not having to do any manual memory management.

Before we look at a proper example, be aware that the syntax for a dynamic array and an FCD Array is similar. As we shall see in the next chapter, you declare dynamic arrays using the `[dynamic]T` syntax. FCD Arrays are declared using the `[dynamic; N]T` syntax. Note the

difference: The FCD Array has an additional `; N` within the brackets. These are two completely different types that differ vastly in the way they manage their memory.

Here's an example of how to use the FCD Array:

```
package fcd_array_example

import "core:fmt"

main :: proc() {
    arr: [dynamic; 1024]int

    append(&arr, 5)
    fmt.printfln("len: %v", len(arr))
    fmt.println(arr)

    append(&arr, 7)
    fmt.printfln("len: %v", len(arr))
    fmt.println(arr)

    append(&arr, 11)
    fmt.printfln("len: %v", len(arr))
    fmt.println(arr)

    // Remove element at index `0`. This
    // may reorder the other elements.
    unordered_remove(&arr, 0)
    fmt.printfln("len: %v", len(arr))
    fmt.println(arr)
}
```

This program creates an FCD Array called `arr`. It uses `append` to append things to the array. It also uses `unordered_remove` to remove things from the array.

> You can also remove items using `ordered_remove`. That procedure preserves the order of the elements, but may be slower. These procedures also work with dynamic arrays. Therefore, we'll save looking at them in detail for the next chapter. For now, just notice how `unordered_remove` modifies the order of the elements.

Running the program outputs:

```
len: 1
[5]
len: 2
[5, 7]
```

```
len: 3
[5, 7, 11]
len: 2
[11, 7]
```

So, what is this FCD Array really? Note `1024` and `int` on this line:

```
arr: [dynamic; 1024]int
```

This means that this FCD Array internally contains a fixed array of `1024` integers. You can verify this by investigating the file **`<odin>/base/runtime/core.odin`**. There you'll find the struct that is actually used to store the FCD Array's data:

```
Raw_Fixed_Capacity_Dynamic_Array :: struct($Capacity: uint, $T: typeid) {
    data: [Capacity]T,
    len:  int,
}
```

The things with a `$` are related to parametric polymorphism, which we'll discuss **in chapter 14**. But as you can see the FCD Array contains two things:

- A `data` field, which is a fixed array.
- A `len` field, which keeps track of how many elements of `data` are actually in use.

The FCD Array in our example cannot contain more than `1024` integers and cannot grow once `len` hits that number. When it is full, `append` will no longer do anything: It will then return `0` to indicate that nothing was added to the array.

Since it uses a fixed array, the FCD Array lives in stack memory. This means that assigning an FCD Array to a new variable copies all its data. Furthermore, the FCD Array always uses as much memory as was specified when you declared it. So the FCD Array with room for `1024` integers will use `1024*size_of(int) = 1024 * 8 = 8192` bytes of stack memory, regardless of how many things you append to it.

> It will actually use `8` bytes more (on a 64 bit system), since it also contains the `len` field: A total of `8200` bytes.

As we see, we can't "start small and grow over time" with an FCD Array. In the next chapter we will look at the *dynamic array*, which can start tiny and then grow as it runs out of space. The downside is that it requires manual memory management.

> In **appendix B** I outline some ideas for how you can use fixed memory to write code without any manual memory management at all.

Chapter 9

Introduction to manual memory management

This chapter will be our first look at manual memory management. I will start off with discussing what manual memory management is and how it relates to *dynamic memory allocations*.

As an example of something that requires manual memory management, we'll look at how the *dynamic array* works. Thereafter we'll look at how you can dynamically allocate a variable of any type and how that involves manual memory management.

Towards the end of this chapter we'll discuss the two default allocators in Odin: `context.allocator` and `context.temp_allocator`, as well as some other good-to-know things.

Note that this is just the first chapter of several that discusses manual memory management. In chapters 10, 11 and 12 we'll use what we learned in this chapter in order to understand other parts of the language. In chapter 13 we'll look at methods for making manual memory management easier.

What is manual memory management? 9.1

Odin uses manual memory management. This means that the programmer is responsible for deallocating memory when it is no longer needed. This can be put into contrast to languages that employ automatic memory management, which means that the language uses certain techniques in order to automatically deallocate memory.

> C# is an example of an automatically memory managed language. It uses a garbage collector in order to periodically deallocate memory that is no longer needed.

Manual memory management can be slightly harder than automatic. But we shall see throughout this book, with the right tools we'll be able to simplify and wrap our heads around it. When you've understood manual memory management then you'll also have an easier time understanding what the program is *actually doing*. Since you'll be forced to think of when memory is allocated and deallocated, you'll also be forced to structure your program in a more thoughtful way.

When I speak of manual versus automatic memory management, then I wouldn't blame you for saying the following: "But Karl, when you talked about the stack memory used by

procedures, you said that variables that live on the stack are **automatically deallocated** when a procedure ends. Doesn't this mean that Odin also has automatic memory management?"

The answer is no. Almost all languages that use either automatic or manual memory management have the concept of a stack frame, where variables that live within the stack frame are deallocated automatically. So we can draw the conclusion that the definition of manual vs automatic memory management refers to some other type of memory.

Assembly languages do not have automatic stack memory management, instead you are usually just handed a stack pointer. That stack pointer tells you where the stack is currently starting. Moving that pointer is how you in assembly languages allocate and deallocate stack memory.

But in languages like Odin, C and C# the stack pointer is moved automatically. Those languages set up and destroy the stack frames automatically as a procedure is called and later finishes. The rules for how a language does that is part of the *calling convention* of that language.

The type of memory allocations that manual memory management concerns itself with is *dynamic memory allocations*. These are the kind of allocations where the amount of memory we want to allocate does not have to be known at compile time. Dynamic memory has the property that it is usually allocated somewhere else than on the stack. It can therefore live on after a procedure ends. This makes it possible to manually control the *lifetime* of this memory.

And that is what manual memory management is really about: Manual control over the lifetime of dynamically allocated memory.

Dynamic arrays 9.2

As an example of something that requires manual memory management, let's look at the *dynamic array*. First I'll show you how to create one and under what circumstances memory is dynamically allocated for it. Then we'll look at how to deallocate its memory and also how to remove individual elements from the array. Finally we'll discuss how dynamic arrays work "under the hood".

I start this chapter with the dynamic array because it is many Odin programmers' first exposure to manual memory management.

When we **looked at fixed arrays** I said that they are, as the name suggests, of fixed size and cannot grow. If you want an array that *can* grow as your program runs, then you can use a dynamic array. But since dynamic arrays can grow, that means that their size is not known at compile time. Rather, it is only known when the program is running, and the size will change

as the array grows. Therefore dynamic arrays need to use dynamic memory and are subject to manual memory management.

Creating a dynamic array and appending items *9.2.1*

You create an empty dynamic array that holds integers like this:

```
dyn_arr: [dynamic]int
```

You can swap out `int` in `[dynamic]int` for any type you'd like to use. But for simplicity we'll use `int` in this chapter.

You can add the number `5` to this array of integers like so:

```
append(&dyn_arr, 5)
```

Note the `&`, `append` needs a pointer to make changes to the array.

The newly added `5` will be at index `0`. So this prints `5`:

```
fmt.println(dyn_arr[0])
```

You can give the element at index `0` a new value like this:

```
dyn_arr[0] = 7
```

You can, just like with fixed arrays iterate the array using

```
for v, i in dyn_arr {}
```

Initially this dynamic array is empty. It has no allocated memory in which to store any elements. So when you `append` a first element to it, then it needs to *grow*. Growing means that a dynamic memory allocation needs to happen. This allocated memory will *not* be deallocated automatically, we'll look into how to deallocate it soon.

When the dynamic array grows, it will use an allocator to somehow acquire the required amount of memory. There are different kinds of allocators, and each has its own way of acquiring memory. There is an allocator available everywhere: `context.allocator`. This is the allocator that `append` will use by default.

On most platforms the default value of `context.allocator` is a *heap allocator*, meaning that those allocations end up in something called the heap. The heap is an unordered area of memory that can be used for dynamic memory allocations. We'll discuss the heap allocator **later in this chapter**.

When some memory has been allocated, then the element you wanted to append will be stored within that memory. At this point your dynamic array contains a single element. We

can check the *length* of the dynamic array using the `len` procedure, this tells us how many elements the dynamic array currently contains. The following will print `1`:

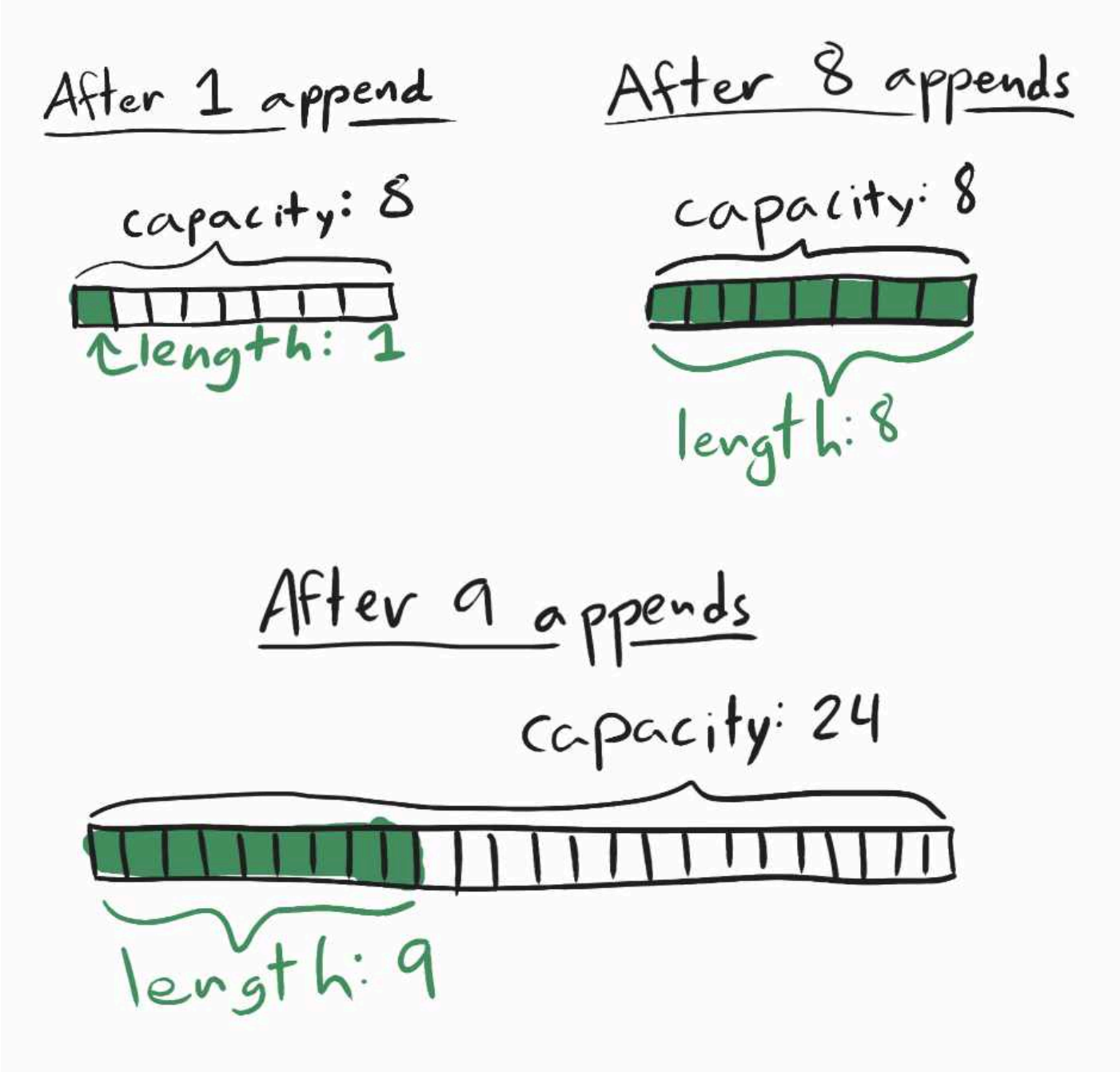
```
fmt.println(len(dyn_arr)) // 1
```

There is also a `cap` procedure which tells you the *capacity* of the dynamic array. The capacity says how many elements you can store in the dynamic array before it needs to grow again. The following will print `8`:

```
fmt.println(cap(dyn_arr)) // 8
```

As you can see, after appending a single element to an empty dynamic array, it has the capacity `8`. This means that the dynamic memory allocation that happened on the first call to `append` actually allocated space for `8` elements. If you append a second element to the dynamic array, `len(dyn_arr)` becomes `2`, but `cap(dyn_arr)` will stay the same. No memory allocations will happen until the length and capacity become equal. When they are equal and you try to append another element, then the dynamic array will grow. At that point more memory needs to be allocated, increasing the capacity.

The reason for only occasionally increasing the capacity is that dynamic memory allocations are computationally expensive, so you don't want them to happen too often.



The capacity tells you how much memory is currently allocated for the dynamic array. Once the dynamic array needs to grow, then the capacity will increase. Only when the capacity increases does the memory usage of the dynamic array increase.

The illustration above shows how the capacity increases from `8` to `24` when the length is `8` and you try to append another element. The illustration can be translated into code, like so:

```
package dynamic_array_example

import "core:fmt"

main :: proc() {
    dyn_arr: [dynamic]int
```

```
    // Will make `dyn_arr` grow:
    append(&dyn_arr, 5)

    fmt.println("After first append:")
    fmt.println("Capacity:", cap(dyn_arr)) // 8
    fmt.println("Length:", len(dyn_arr)) // 1

    // append 7 numbers to the dynamic
    // array. This will not make `dyn_arr`
    // grow since capacity is `8` after
    // first `append`.
    for i in 0..<7 {
        append(&dyn_arr, i)
    }

    fmt.println("\nAfter 7 more appends:")
    fmt.println("Capacity:", cap(dyn_arr)) // 8
    fmt.println("Length:", len(dyn_arr)) // 8

    // Capacity is 8, length is 8. This
    // call to `append` will make `dyn_arr`
    // grow:
    append(&dyn_arr, 5)

    fmt.println("\nAfter one more append:")
    fmt.println("Capacity:", cap(dyn_arr)) // 24
    fmt.println("Length:", len(dyn_arr)) // 9
}
```

If you run the code above, it will print:

```
After first append:
Capacity: 8
Length: 1

After 7 more appends:
Capacity: 8
Length: 8

After one more append:
Capacity: 24
Length: 9
```

Deallocating a dynamic array *9.2.2*

Any memory that was allocated when `append` ran can be deallocated using `delete`:

```
delete(dyn_arr)
```

You can `delete` a dynamic array that is empty. In that case it will not do anything.

`delete` works with any built-in container type that uses dynamic memory. Later we'll also use it for slices, maps and strings.

Some people say "free" instead of deallocate. This comes from the C procedure `free` that is used to deallocate raw memory. As we shall see soon, `free` also exists in Odin, but is not used for dynamic arrays.

So don't be confused when someone says "free the dynamic array", they just mean that you run `delete(dyn_arr)`

Be sure that you do not need anything in the dynamic array before you run `delete`. Choosing when to actually run `delete` is what some people find a bit tricky about manual memory management. It can be tricky because you need to start thinking about the *lifetimes* of objects. We will discuss techniques for thinking about memory lifetimes in the **Making manual memory management easier** chapter.

If you continuously create dynamic arrays but never `delete` them, then your program's memory usage will steadily grow. This is known as a memory leak.

Other than deallocating the memory, `delete` will *not* modify your dynamic array. If you wish to reuse the same dynamic array after the delete, then it might be a good idea to reset the dynamic array to its zero value:

```
dyn_arr = {}
```

This is because `dyn_arr` will still keep its internal state after the `delete`, so trying to `append` to it again after a `delete` will cause issues.

I'll show how this internal state looks **in a bit**.

However, in many cases a `delete` followed by zeroing it actually means that you just need to *clear* the array:

```
clear(&dyn_arr)
```

`clear` does *not* deallocate any memory. The capacity of the dynamic array remains the same and the allocated memory will still be around. It will just set length of the dynamic array to `0`, so it starts over from the beginning of the allocated memory block. Later, when you are actually done with your dynamic array, then you can `delete` it.

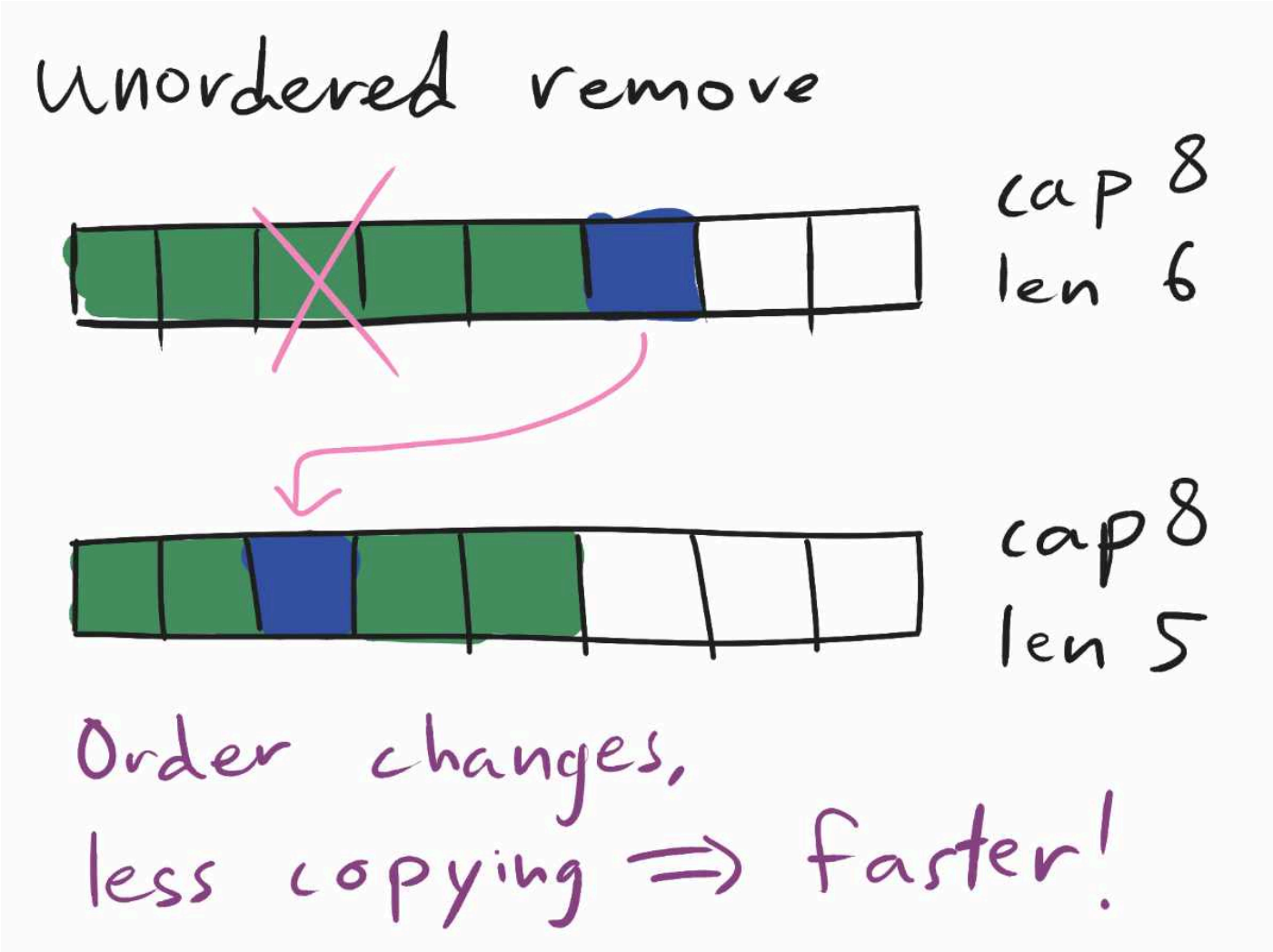
Note how `delete` takes a value and `clear` takes a pointer. This is because `clear` needs to modify the dynamic array itself (it sets length to `0`). However, `delete` will only deallocate memory, but it won't directly modify any of the dynamic array's state.

Removing items *9.2.3*

You can remove items from a dynamic array in two ways. Here's the first one:

```
unordered_remove(&dyn_arr, index)
```

`unordered_remove` removes an element at position `index`, but it does it in an *unordered way*. It copies the last element of the array to position `index` and then reduces the length of the array by `1`. This means that the order of the dynamic array might not be the same before as after the remove.

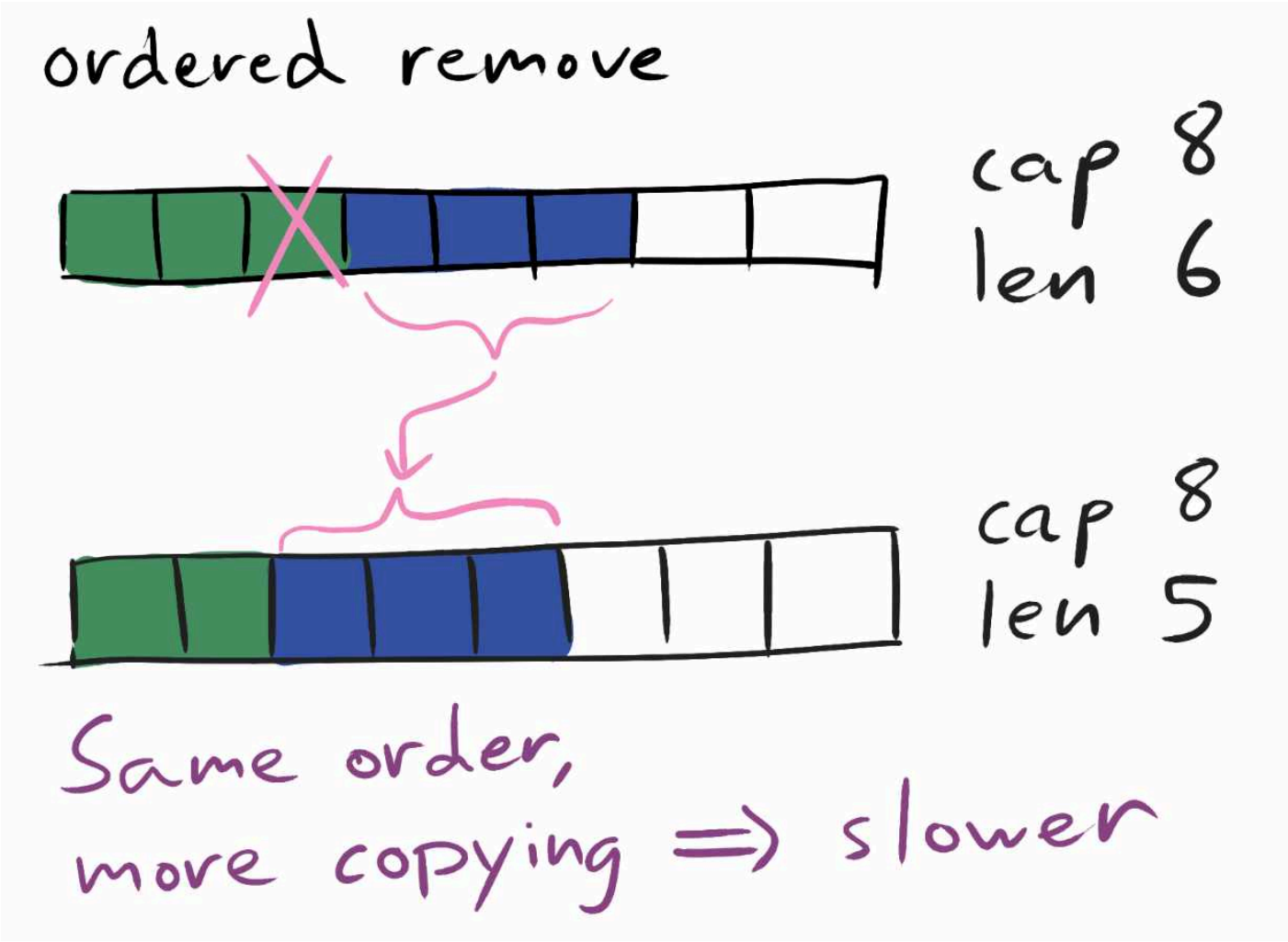


The `unordered_remove` procedure is fast but changes the order of the array.

If you need to preserve the order of your dynamic array, then instead use:

```
ordered_remove(&dyn_arr, index)
```

This procedure will move everything that resides at indices larger than `index`. It will move those things back a single index, so that the gap left by the thing you just removed is closed. Thereafter the length is decreased by `1`.



The `ordered_remove` *procedure is slower than* `unordered_remove`*, but keeps the order of the array the same.*

Always use `unordered_remove` unless the order matters! It is less computationally expensive since it only has to copy a single element.

Note that while both these procedures will reduce the length of the dynamic array, the capacity will remain the same. This means that the amount of allocated memory is not reduced. If you really want to reduce the amount of allocated memory, then you can run `shrink(&dyn_arr)`. `shrink` will reallocate the memory of the dynamic array so that the length and capacity becomes equal. Don't use `shrink` unless you have a good reason. Since you're using a dynamic array, you'll usually end up adding something new to it at some point, so there is often no point in shrinking the capacity.

Preallocated dynamic arrays *9.2.4*

As we've seen, this creates an empty dynamic array that will grow on the first call to `append`:

```
dyn_arr: [dynamic]int
```

You can also *preallocate* a dynamic array like this:

```
dyn_arr := make([dynamic]int, 0, 20)
```

This will create a dynamic array with the length `0`, but with capacity for `20` items. The `make` procedure will allocate memory immediately, not at the first `append`. This also means that the dynamic array won't have to grow until you've added 20 things to it.

`make([dynamic]int)` without any capacity or length does *not* cause any immediate memory allocations. It will happen at the first `append` like we've seen before.

So it is almost the same as

```
dyn_arr: [dynamic]int
```

But when using `make` an allocator is chosen for the dynamic array immediately instead of when `append` runs. More on how the dynamic array stores its allocator in the next subsection.

If you do this:

```
dyn_arr := make([dynamic]int, 20)
```

Then both the capacity and the length will be `20`. This means that the dynamic array will start with 20 zeroed out items. `dyn_arr := make([dynamic]int, 20)` is equivalent to `dyn_arr := make([dynamic]int, 20, 20)`

Under the hood: `append` *and* `Raw_Dynamic_Array` *9.2.5*

Let's look at how `append` actually makes changes to the dynamic array. This will make it easier for you to understand how the memory is allocated and demystify what dynamic arrays actually are.

Say that you run the following code:

```
dyn_arr: [dynamic]int
append(&dyn_arr, 7)
```

Since `append` will make changes to the dynamic array, we'll start with asking ourselves: "What does a dynamic array look like internally?". It is a struct that looks like this:

```
Raw_Dynamic_Array :: struct {
    data:      rawptr,
    len:       int,
    cap:       int,
    allocator: Allocator,
}
```

You'll find `Raw_Dynamic_Array` in **<odin>/base/runtime/core.odin** . As the 'Raw' part hints at, this is the *internal* format, you cannot access these fields except for the `allocator` field. If you look at how `append` works in `base/runtime/core_builtin.odin`, then you can see that it really does cast the dynamic array to the type `^Raw_Dynamic_Array`.

Also, `data` is of type `rawptr`. This is a pointer without any additional type information.

`rawptr` is like `void*` in C.

So when you write

```
dyn_arr: [dynamic]int
```

then you are creating something of type `[dynamic]int`. But you can also see `dyn_arr` as being of type `Raw_Dynamic_Array`. That's the internal representation that `append` will use. Since no value is given, the dynamic array is zero initialized. This means that the `data`, `len`, `cap` and `allocator` fields of the `Raw_Dynamic_Array` will all be initialized to zero.

So if you then append `7` to this empty array:

```
append(&dyn_arr, 7)
```

then `append` will cast `dyn_arr` from the type `^[dynamic]int` to the type `^Raw_Dynamic_Array`. It then looks inside the `Raw_Dynamic_Array` and sees that the field `cap` (the capacity) is zero. This means that the array has room to store zero elements. The array needs to grow in order to add something to it.

While growing, it will check if the `allocator` field is set and use that allocator to allocate the memory. But in our case the allocator field is zeroed, so it will instead fall back to using `context.allocator` and ask that allocator for some memory. It will also make sure to store `context.allocator` in the empty `allocator` field.

`context.allocator` is part of the `context` struct that is available everywhere and passed implicitly to all procedures, we'll talk more about the context in **chapter 12**.

How much memory does this first call to `append` ask for? It defaults to allocating memory for `8` elements. Since our dynamic array is of type `[dynamic]int`, then this means that it will

allocate room for `8` integers. When the allocator has allocated that amount of memory, then it will return a pointer to that memory. That pointer will be assigned to the `data` field of the `Raw_Dynamic_Array`. Since there is room for `8` elements in that data, then the `cap` field will be set to `8`. The `len` field (the length) will tell you how much of that capacity you've actually used up, which will be `1` after a single `append`. Finally, in the allocated memory that `data` now points to, the element that you wanted to append will be written.

Since the default initial capacity is `8`, then `data` will point to some memory where there is enough room for `8` elements. In the case of our `[dynamic]int` array, `data` points to `cap * size_of(int) = 8 * 8 = 64` bytes of memory.

Later, if you append more items to the dynamic array, then `len` is increased by `1` for each appended item. When `len` equals `cap`, `append` will try to allocate more memory. This time the `allocator` field has a value that `append` can use instead of defaulting to `context.allocator`. This new block of memory will be bigger than the old one, since the capacity is bigger. All the memory pointed to by `data` (the contents of the dynamic array) will be copied to this new block of memory. Thereafter `data` will be updated to point to the new block of memory.

However, `data` might or might not end up at a new memory address. With the default allocator you'll see `data` moving around fairly often. In cases where `data` does *not* move, then the contents of the array doesn't have to be copied to any new block.

The reason for the data moving varies. One common reason is that there isn't enough continuous space for the new block of memory at the old location, so it needs to move to a new region of memory.

In the **chapter on data-oriented design** we'll discuss problems related to the dynamic array's memory moving.

If `data` moves to a new address, then the old `data` will be deallocated. You do not have to worry about having to `delete` anything else than the array's current memory.

You can't access the `data` field of the dynamic array directly. But you can fetch it using `raw_data(array_name)`.

Exercise: Understanding Dynamic Arrays *9.2.6*

Try running this program:

```
package exercise_dynamic_array

import "core:fmt"

main :: proc() {
```

```
    my_ints: [dynamic]int

    for i in 0..<1024 {
        print_info(my_ints)
        append(&my_ints, 5)
    }
}

print_info :: proc(arr: [dynamic]int) {
    fmt.printfln("len: %v", len(arr))
    fmt.printfln("cap: %v", cap(arr))
    fmt.printfln("data: %p", raw_data(arr))
}
```

It will add one element at a time to a dynamic array and then print the `len`, `cap` and `data` fields of the array. It repeats this process 1024 times. From the output of the program, try to figure out these things:

- What capacity does the dynamic array get after the first append?
- When does it grow again and what is the new capacity? Can you come up with a formula for what the new capacity is based on the previous?
- The data pointer doesn't always move to a new address. How many appends did it take for your program to have the data pointer move to some other address?

Dynamically allocated variables 9.3

We just saw how to use the dynamic array and how it uses dynamic memory. You can also dynamically allocate a single variable of any type. Doing so will make that variable's memory live *outside* the stack. This means that it will live on after the current procedure ends. It is the programmer's responsibility to deallocate the memory when they no longer need it.

This variable `stack_number` is allocated on the stack:

```
stack_number: f32
```

We can use `new` to instead dynamically allocate it:

```
number := new(f32)
```

`new` will by default use `context.allocator` to allocate memory. It is equivalent to doing this:

```
number := new(f32, context.allocator)
```

The stack allocated `stack_number` will be of type `f32`. But the dynamically allocated `number` will be of type `^f32`, meaning that it is a pointer to a 32 bit floating point number. The memory that the pointer refers to is the memory that was allocated for us. An `f32` requires 4 bytes of memory, so at the address that `number` contains we can expect to find those 4 bytes of memory.

Since the allocated memory is not part of the stack frame, it will live on after the current procedure finishes. You'll need to manually deallocate it when you no longer need the memory. For anything you allocate using `new`, you deallocate it using `free`:

```
free(number)
```

`free` also takes an allocator of default value `context.allocator`. If your call to `new` used some custom allocator, then you'll need to pass the same allocator to `free`.

Just like with forgetting to `delete` dynamic arrays, the following is true: If you run `new` periodically, but forget to `free` the memory at some point, then your program's memory usage will continuously grow. You have what is known as a memory leak.

Example: Dynamically allocated struct *9.3.1*

The example we just saw, where we dynamically allocated a single `f32` is often not very useful. What is more useful is dynamically allocating a whole struct:

```
Cat :: struct {
    age: int,
    name: string,
}

make_cat :: proc(name: string, age: int) -> ^Cat {
    cat := new(Cat)
    cat^ = {
        name = name,
        age = age,
    }
    return cat
}

cat_simulation :: proc() {
    cat := make_cat("Fluffy", 12)

    // Cat simulation code goes here
```

```
    free(cat)
}
```

Note how `make_cat` uses `new(Cat)` to dynamically allocate a struct object. The pointer `cat` points to the memory where our Cat struct actually resides. This memory will stay allocated for us until we run `free(cat)`.

The type of the variable `cat` inside `make_cat` is `^Cat`. **As we saw in the chapter on pointers**, we need to use `cat^ = {}` to go through that pointer and replace the value of the whole `Cat` struct.

Another use case for dynamically allocating a struct is when you have very large global variables. There's a limit to how much memory you can put into global variables, but if you instead dynamically allocate that big global variable, then you can have much bigger global data. Here's an example:

```
Program_Memory :: struct {
    big_array_of_numbers: [10000000]int,
}

memory: ^Program_Memory

main :: proc() {
    memory = new(Program_Memory)

    // Rest of program can use
    // global variable `memory`.
}
```

As you can see `Program_Memory` contains a really big fixed array of integers. This will make `Program_Memory` use 80 megabytes of memory. If we used an ordinary global variable for `memory`, then we might run into the limit of how big global variables can be:

```
memory: Program_Memory
```

So instead we dynamically allocate `memory` when the program starts.

Note that there is no `free` in this case. If you dynamically allocate memory once on program startup and want that memory to be around for the whole execution of the program, then having a `free` at the end of the program is unnecessary. A program that shuts down has all its allocations freed automatically. Using `free` here would actually make the program shut down slightly slower.

But you can still `free` it if you wish to keep memory-analyzers such as Valgrind and the clang sanitizer happy.

Example: Dynamically allocated fixed array *9.3.2*

As we have seen, creating fixed arrays allocates their memory on the stack:

```
ints_stack: [128]int
```

However, you can pass the type `[128]int` to `new` and dynamically allocate the fixed array:

```
ints := new([128]int)
```

The type of `ints` is `^[128]int`. `new` dynamically allocated the memory for the fixed array, and gave us a pointer that says where that memory is. This fixed array does not live on the stack.

If you actually want a dynamically allocated array of fixed size, then I recommend that you instead use a slice, as I will explain in the **chapter on slices**. The example I'm going over here attempts to explain how `new` works.

Deallocate `ints` using `free` when you no longer need it:

```
free(ints)
```

When we've previously talked about fixed arrays, then I said that fixed arrays copy all their data when you assign them to a new variable. However, the type of the dynamically allocated array `ints` is `^[128]int`. This means that if you assign `ints` to a new variable, then you are just assigning the pointer to that variable. You do not get a copy of the array's data. Example:

```
ints := new([128]int)
ints[10] = 5
ints_2 := ints
ints_2[10] = 7
fmt.println(ints[10]) // 7
fmt.println(ints_2[10]) // 7
```

In this example I create a dynamically allocated fixed array `ints`. I then create a new variable `ints_2` that is set to the same value as `ints`. Both these variables have the type `^[128]int`.

I then modify the item at index `10` in both `ints` and `ints_2`. Note how `ints[10] = 5` happens first and `ints_2[10] = 7` happens just thereafter. Since both `ints` and `ints_2` are pointers

that contain the same memory address, those two lines are actually just modifying the same element in the same array.

There's **a section** in the pointers chapter that discusses similar things.

Note how the brackets `[]` automatically dereferences the pointer. You do not need to do `ints^[10] = 5`.

The default allocator: `context.allocator` 9.4

We have seen how dynamic arrays can grow when we use `make` and `append`. We've also seen how we can "manually" allocate memory using `new`. Let's look at the `allocator` parameter of those procedures, and how its default value looks.

For example, `new` looks like this:

```
new :: proc($T: typeid, allocator := context.allocator, loc := #caller_location)
-> (^T, Allocator_Error) #optional_allocator_error {
    return new_aligned(T, align_of(T), allocator, loc)
}
```

and the overload of `make` that handles dynamic arrays looks like this:

```
make_dynamic_array :: proc($T: typeid/[dynamic]$E, allocator :=
context.allocator, loc := #caller_location) -> (T, Allocator_Error)
#optional_allocator_error {
    return make_dynamic_array_len_cap(T, 0, 0, allocator, loc)
}
```

Both are from

`<odin>/base/runtime/core_builtin.odin`

This just shows one of the overloads of `make` for dynamic arrays. There are additional overloads for when we are supplying a length and / or a capacity.

Both these procedures have a parameter `allocator` with default value `context.allocator`. Whenever something in Odin needs a default method for allocating memory, then it will usually use `context.allocator`.

`context.allocator` is set up for you automatically when the program starts. Exactly what type of allocator it is depends on your system. At the time of writing there are two possible defaults:

- On desktop operating systems such as Windows, macOS and Linux it is a *heap allocator*
- On web-based systems, such as WASM, it is a *WASM allocator*

You can see the code that decides which allocator to use by default in

`<odin>/base/runtime/default_allocators_general.odin`

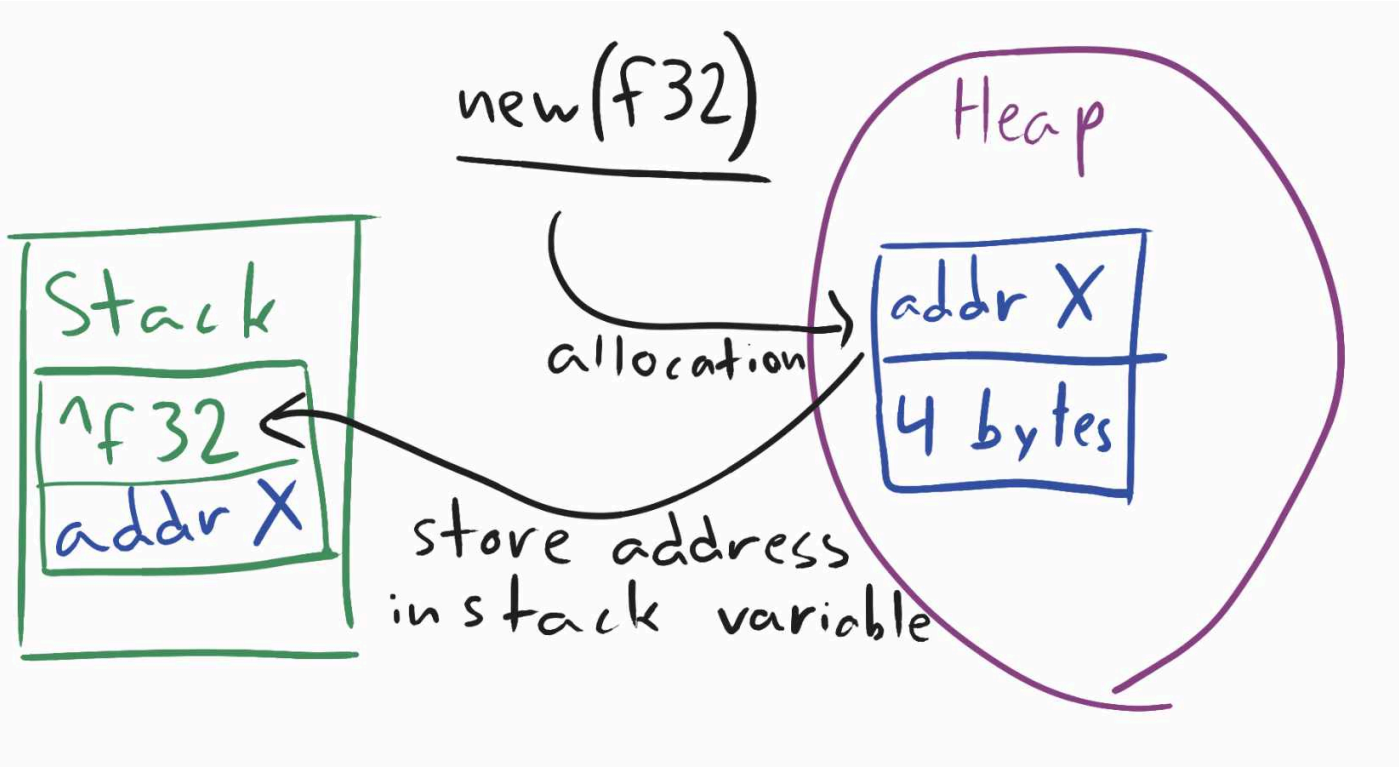
The heap allocator *9.4.1*

Whenever you allocate dynamic memory using the heap allocator, your memory will be allocated from what is known as a heap. A program can have several heaps, but the heap allocator uses the default heap assigned to the program by the operating system. The heap uses an area of memory that is separate from the stack. This means that when we allocate anything using the heap allocator, then that allocation ends up outside the stack.

So when you do something like

```
my_num := new(f32, context.allocator)
```

then `new` will ask `context.allocator` to allocate memory, which will in turn ask the operating system to do a heap allocation. The memory it allocates will reside at some address in the heap. `my_num` will be a pointer to that heap-allocated memory.



`new(f32)` *allocates 4 bytes on the heap and then creates a new stack variable that contains the address to that memory.*

The heap starts out small, but can grow as it runs out of space. It's essentially a list of memory blocks. These memory blocks are reserved using what is known as *virtual memory*. Virtual memory is a nifty invention where your program can use the whole giant 64 bit address space, even though the amount of physical memory in your computer is much, much smaller. Virtual memory is split into chunks called *pages*. When a page is in use it is mapped to some physical memory.

Going deep into virtual memory is beyond the scope of this book.

However, I'll talk a bit more about it when I discuss the **growing virtual memory arena**.

Your computer has a limited amount of memory. If you run out of memory then your allocations will fail. `new`, `make` and `append` all have an optional second return value that contains an error if the allocation fails:

```
my_dyn_arr, alloc_err := make([dynamic]f32, 100, context.allocator)

if alloc_err != nil {
    // There's an error, alloc_err could
    // for example be the "Out_Of_Memory"
    // error.
}
```

The WASM allocator *9.4.2*

The WASM allocator is made for when you build an Odin program that runs in a web browser. In this case `context.allocator` is not a heap allocator. Instead, the WASM allocator uses a series of blocks of memory that are handed to the allocator by the browser. The WASM allocator can ask the browser for additional memory when it runs out of memory blocks.

So when you allocate memory with for example `new`, then you get a pointer that contains the address of some memory within one of those blocks that the browser has handed the allocator.

Going into WASM is outside the scope of this book. I give some suggestions material for learning WASM at **the end of the book**.

The separation of pointer and allocated memory 9.5

Note that the result of

```
number := new(f32)
```

consists of two parts:

1. `number` is of type `^f32`. That's a pointer. It contains a memory address which tells us the location of the allocated memory.
2. At the address `number` points to, there are 32 bits (4 bytes) of memory that is allocated for us to use.

So within a procedure, the pointer itself lives on the stack, but the dynamically allocated memory that it points to does not. In other words, these two variables

```
number := new(i32)
number2 := number
```

are both pointers. The pointers themselves are two separate stack variables. But they both point to the same memory.

Another important thing to understand is this: What happens when you copy a variable that is a dynamic array? Say that you do this:

```
dyn_arr: [dynamic]int
append(&dyn_arr, 5)
dyn_arr_2 := dyn_arr
```

Remember, a dynamic array looks like this internally:

```
Raw_Dynamic_Array :: struct {
    data:      rawptr,
    len:       int,
    cap:       int,
    allocator: Allocator,
}
```

After the call to `append`, then the field `data` inside `dyn_arr` will point to the memory that was allocated.

When we do `dyn_arr_2 := dyn_arr`, then `dyn_arr_2` becomes a new struct that has a copy of all the fields of `dyn_arr`. But just like `len` will be `1` in both `dyn_arr` and `dyn_arr_2`, `data` in `dyn_arr_2` will contain the same memory address as `data` in `dyn_arr`.

Therefore `dyn_arr` and `dyn_arr_2` refer to the same memory. However, if you now try to add more things to `dyn_arr`, which can make it grow, then this situation becomes dangerous.

Why? Say that you do this:

```
dyn_arr: [dynamic]int
append(&dyn_arr, 5)
dyn_arr_2 := dyn_arr

// This loop will make
// `dyn_arr` grow again.
for i in 0..<1024 {
    append(&dyn_arr, 5)
}
```

This example starts off like the previous one. But after creating `dyn_arr_2` based on `dyn_arr`, we use a loop to add 1024 new items to `dyn_arr`. This will make `dyn_arr` grow several times. After this has happened several things may be wrong:

- When appending to `dyn_arr`, the data of `dyn_arr` might move to some other address. When such a move happens, then it also deallocates the old data. This would mean that `dyn_arr` now has some other pointer `data` than `dyn_arr_2` has. It also means that the pointer `data` inside `dyn_arr_2` now points to deallocated memory.
- Even if `data` didn't move, then the `cap` and `len` fields of `dyn_arr_2` will be wrong.

Instead, if you really want a clone of a dynamic array, then do this:

```
dyn_arr_2 := slice.clone_to_dynamic(dyn_arr[:])

import "core:slice"
```

The above makes a clone of `dyn_arr` with its own memory. Now `dyn_arr` and `dyn_arr_2` work independently. This uses the `slice` package and the `[:]` syntax, which we'll go over in the next chapter when we talk about slices.

free and delete 9.6

We've seen `free` and `delete` now. But why do both of these exist and when should we use which?

`free` is very simple: Whenever you allocate memory using `new` you are getting a plain pointer directly to that memory. So `new(int)` gives you something of type `^int`. You deallocate that memory using `free`.

However, `delete` takes some additional explaining.

Let's look at dynamic arrays as an example. Dynamic arrays are internally represented by a struct that in turn contains a pointer to some allocated memory:

```
Raw_Dynamic_Array :: struct {
    data:      rawptr,
    len:       int,
    cap:       int,
    allocator: Allocator,
}
```

So you can't use `free` directly on a dynamic array because the allocated memory is pointed out by the `data` field. There is "one step of indirection" if you will. So instead we use `delete`, which is an explicit overload that looks like this:

```
delete :: proc{
    delete_string,
    delete_cstring,
    delete_dynamic_array,
    delete_slice,
    delete_map,
    delete_soa_slice,
    delete_soa_dynamic_array,
}
```

In this overload you see all the other built-in containers that may need manual deallocation using `delete`. We'll talk about many of these in the subsequent chapters.

You'll find all these in `<odin>/base/runtime/core_builtin.odin` .

The overload `delete_dynamic_array` will deallocate the memory that the `data` field in the `Raw_Dynamic_Array` points to using the `allocator` that it stores.

All the types that are compatible with `delete` have an internal representation similar to `Raw_Dynamic_Array`, that in turn contains a `data` field. For example `delete_string` will deallocate the memory that the `data` field within the `Raw_String` refers to.

Temporary allocator 9.7

Many times you need to allocate some dynamic memory, but you only need it for "a short while". Here "a short while" could mean during the execution of an algorithm. Or in a video game, it could mean until the "end of the frame".

In these cases using a temporary allocator can simplify your code. There's a default temporary allocator: `context.temp_allocator`. You can use the temp allocator to make temporary dynamic arrays, **temporary strings** and do temporary data processing.

Here's an example of how you can make a dynamic array that uses the temp allocator:

```
great_algorithm :: proc() {
    numbers := make([dynamic]int, context.temp_allocator)

    for i in 0..<100 {
        append(&numbers, rand.int_max(1000))
    }

    for n in numbers {
        if n > 500 {
            fmt.println(n)
        }
    }
}
```

Note how `make` is fed `context.temp_allocator`. After this, the variable `numbers` will be a dynamic array with the `allocator` field set to `context.temp_allocator`. So when `append` is run and the dynamic arrays needs to grow, it will use `context.temp_allocator` to allocate memory.

Since we do not specify any capacity or length when calling `make`, then no allocation will be made at this point. The first allocation happens as part of `append`.

The procedure adds 100 random numbers in the range `0` to `1000` to the dynamic array (excluding `1000`: The biggest possible number is `999`). Thereafter it loops over `numbers` again and prints the ones that are bigger than 500.

As you can see, there is no `delete(numbers)` at the end of this procedure. When using the temp allocator you do not need to individually deallocate memory. Instead, you need to make your program periodically free everything that allocated using the temp allocator. You do that by running the following line every now and then:

```
free_all(context.temp_allocator)
```

This means that all the memory that was allocated using the temp allocator is now deallocated. Before doing this, make sure you don't need anything allocated using the temp allocator!

Technically, it is only half-true that it deallocates all the memory. Internally, the temp allocator uses blocks to put your temp allocations in. If the current block is full, then it makes a new block. When `free_all` is run, then it frees all the blocks except for the first one, since it can reuse that block in future temp allocations.

See `<odin>/base/runtime/default_temp_allocator_arena.odin`

Do not forget to put the `free_all(context.temp_allocator)` somewhere in your program. Forgetting to do so will, if you use the temp allocator, lead to memory leaks.

Placing the `free_all` *9.7.1*

Placing the `free_all(context.temp_allocator)` somewhere can be easy or tricky depending on the type of program. It's especially easy to place in a program that runs in a loop. Such an example is a video game. Video games usually have a 'main loop' where each lap of that loop is one frame. You can then put the `free_all(context.temp_allocator)` at the end of that loop:

```
main :: proc() {
    init_game()

    for game_should_run() {
        game_update()
        game_draw()
        free_all(context.temp_allocator)
    }
}
```

This makes anything allocated using the temp allocator valid until the end of the frame. We say that the lifetime of the temporary allocations is "one frame".

It is more tricky to place the `free_all(context.temp_allocator)` in event-driven programs such as a desktop applications. In those kinds of programs there may not be a well-defined 'main loop' like above. This means that you may need to be a bit more careful with what you use the temp allocator for. If you use it for only temporary things within a procedure, then it's fine. But if you use it to return dynamically allocated data, or dispatch dynamically allocated data for later processing, then you might end up in trouble if you place the `free_all` in a location where it deallocates your data before you were done with it. In that case you might be better off using separate *arena allocators*. Read more about arena allocators in **chapter 13**.

The temporary allocator is a type of arena allocator. When using arena allocators it is important to think in terms of what the *lifetime* of an allocation is. Ask yourself: For how long does the memory need to stay allocated? If you find it hard to use the temp allocator, then it might be a sign that you are mixing things of different lifetimes. You may then need several separate arenas instead. Everything within an arena needs to have the same lifetime.

This is why video games have such an easy time using the temporary allocator: "One frame" is an easy lifetime to reason about. The deeper you get into manual memory management, the more you'll see that it's mostly about finding well-defined lifetimes.

"Scripts" that allocate memory 9.8

You can use Odin to write programs that are similar to "scripts". By script I mean a program that does some processing and then shuts down. An example would be a program that goes through a few files and prints some statistics about them.

Your script may need to use dynamic memory. Perhaps it uses dynamic arrays. However, if you come from something like Python, which has automatic memory management, then writing scripts in Odin can feel a bit overwhelming: You don't want to deal with manual memory management when doing something quick and dirty.

Here's the thing: You probably don't have to deal with it. All dynamically allocated memory is deallocated when a program shuts down. So for a short-lived script-like program, just let it do whatever dynamic allocations it needs without care for when the memory needs to be deallocated. The program will shut down soon anyway.

You don't have to use the temporary allocator either. Just pretend allocators don't exist and write your script.

There are limits to this. If you make a script that runs for a long time and processes a lot of large data, then you should still do proper memory management.

But for most script-like programs you can simply ignore memory management.

This book is generated by an Odin program that takes markdown and converts it into an HTML document. The program never does any memory management since it runs for less than a second.

Chapter 10

More container types

There are two built-in container types left for us to cover: Slices and maps. Let's look at them now. Maps require dynamic memory allocations and slices can optionally be allocated, so we are in a good position to talk about these container types now that we've gotten an introduction to manual memory management.

At the end of this chapter we'll also look at how to create custom iterators, making it possible to easily loop through custom container types.

Slices: A window into part of an array 10.1

When programming Odin you will use slices *a lot*. Slices provide a way to look at a section of an array. There are two very nice things about slices:

- You can create slices that represent a subsection of another array without any extra memory allocations.
- You can form slices from most array types, such as fixed arrays and dynamic arrays. This means that slices can be used as a generic way to represent data within a more specific type of array.

Creating a slice *10.1.1*

Suppose you have a fixed array of 50 integers:

```
my_numbers: [50]int
```

You can create a slice that only "sees" the first 20 items of `my_numbers`:

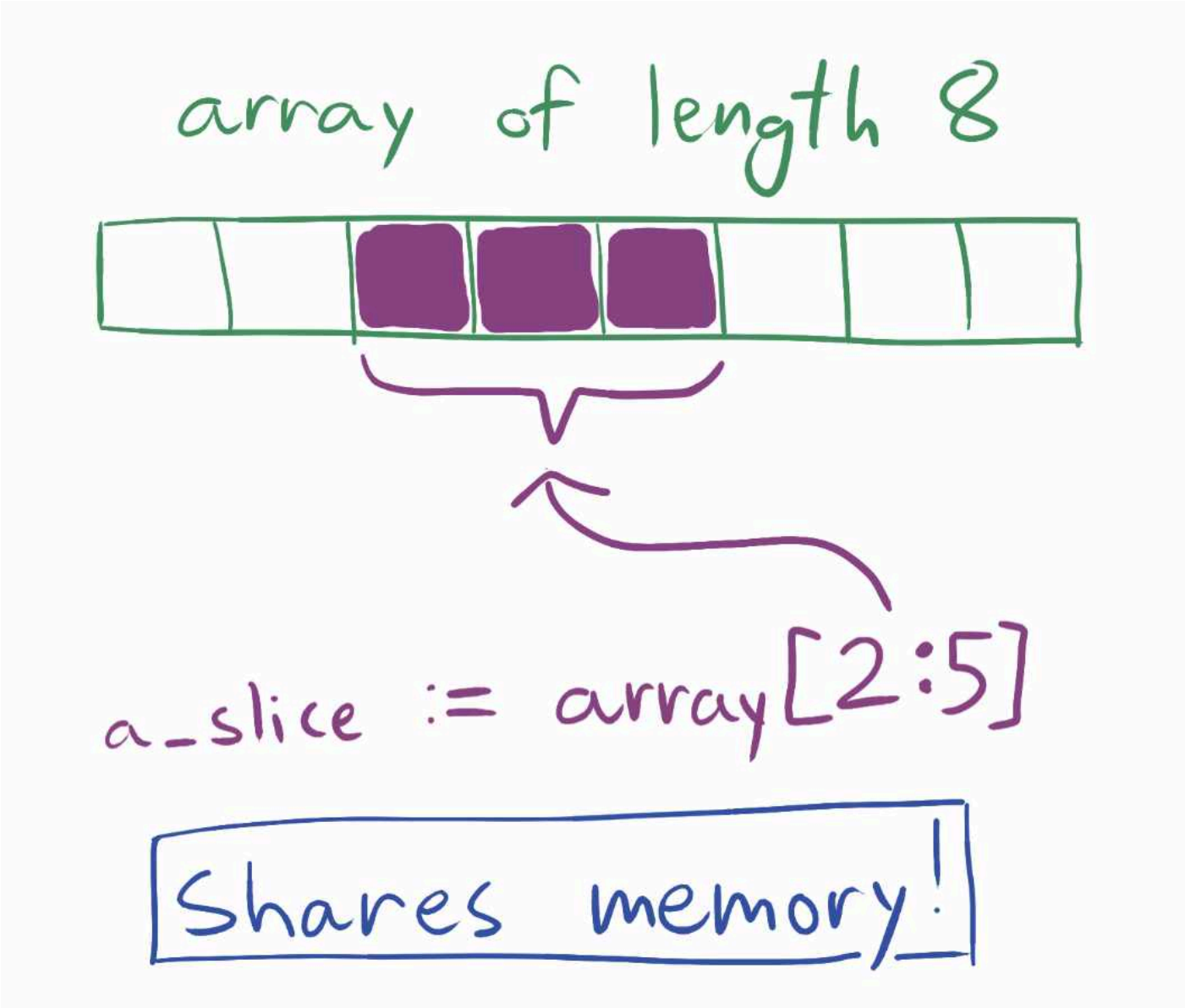
```
first_20 := my_numbers[0:20]
```

Note the `[0:20]` syntax: We create slices using the `[:]` operator, where you put in the indices you want it to span like this: `[start_index:end_index]`. The type of `first_20` is `[]int`. The fixed array above has type `[50]int`, that's a different type than `[]int`. They just look similar:

- `[]int` is a slice.
- `[50]int` is a fixed array.

- `[dynamic]int` is a dynamic array.

Creating a slice does not allocate any memory, it uses the same memory as the thing you slice, but looks into just a part of it.



Slicing using the `[:]` syntax does not allocate any memory. The slice looks into the memory of the source array.

Note how the end index `5` is non-inclusive: `a_slice` sees the elements at index 2, 3 and 4 of `array`.

You can also skip some of the indices:

```
my_numbers: [50]int
last_20 := my_numbers[30:]
```

Note the absence of the second index in `[30:]`. In this case we create a slice that runs from index `30` to the last index. Skipping the first index would make it run from index `0`, for example: `first_20 := my_numbers[:20]`.

Odin has built in bounds checking of arrays, slices and dynamic arrays. This means that you get a nice error immediately when you go out of bounds. This can happen if you for example use an index that goes past the end of a slice. This bounds checking does have a slight performance impact, so for release / production builds you can choose to disable bounds checking using the compiler flag `-no-bounds-check`.

Whether you should disable it depends on the type of program you are making. If you are making a server that is vulnerable to buffer-overrun attacks, then do not disable it. If you are making a game without any online functionality, then you can probably disable it. In the case of the offline game, it's fine to disable since you've probably caught most out-of-bounds errors while testing your development build.

If you skip both indices, then you get a slice that looks at the whole thing:

```
my_numbers: [50]int
full_slice := my_numbers[:]
```

You can slice most kinds of arrays in Odin. Slicing a dynamic array uses the same syntax:

```
dyn_arr: [dynamic]f32

for i in 0..<200 {
    append(&dyn_arr, f32(i*i))
}

half_dyn_arr_size := len(dyn_arr)/2
half_the_dyn_arr := dyn_arr[:half_dyn_arr_size]
```

This code adds 200 items to `dyn_arr` using a loop. `half_the_dyn_arr` is a slice that looks at the first `100` of those items.

When you've sliced something, you can loop through the slice just like you can with fixed and dynamic arrays:

```
for v, i in half_the_dyn_arr {

}
```

You can as usual use `&v` to make `v` mutable. Since the slice and the original array shares memory, modifying `v` will modify the element in the original array.

Slice internals *10.1.2*

As I mentioned before, when you create a slice using the `[:]` syntax, then there are no extra dynamic memory allocations to worry about. Slices are actually very simple. They are essentially a pointer plus a length. Here's how the internal representation of a slice looks:

```
Raw_Slice :: struct {
    data: rawptr,
    len:  int,
}
```

See **<odin>/base/runtime/core.odin**

So when you do something like this:

```
numbers: [50]int
my_slice := numbers[10:30]
```

Then the `data` field will be a pointer containing the memory address of the tenth element of `numbers`, equivalent to `&numbers[10]`. The `len` field will be 20, because we wrote `[10:30]`, which makes it run from element 10 to element 30, and `30 - 10 = 20`.

Slice example: Considering 10 elements at a time *10.1.3*

In this example we use slices to display 10 numbers at a time from a bigger list of numbers:

```
display_numbers :: proc(numbers: []int) {
    fmt.println(numbers)
}

my_numbers: [128]int

// Set the elements of my_numbers to
// interesting values
for i in 0..<len(my_numbers) {
    my_numbers[i] = i*i
}

// Slice my_numbers, 10 numbers per slice.
// Send each slice into display_numbers for
// printing on the screen.
for i := 0; i < len(my_numbers); i += 10 {
    slice_end := min(i+10, len(my_numbers))
    ten_numbers := my_numbers[i:slice_end]
```

```
    display_numbers(ten_numbers)
}
```

This loop:

```
for i := 0; i < len(my_numbers); i += 10 {
```

steps through 10 items at a time. Inside each lap of this loop we create our slice of ten elements:

```
slice_end := min(i+10, len(my_numbers))
ten_numbers := my_numbers[i:slice_end]
```

`min(i+10, len(my_numbers))` picks the smallest number of `i+10` and `len(my_numbers)`. It is there to ensure we don't accidentally try to slice outside of `my_numbers`. This would otherwise happen at the last lap of the loop because `i` would be `120` and `i+10` is then `130`, but `my_numbers` has only `128` items, so this would crash the program.

We can then call `display_numbers(ten_numbers)` and let it print our 10 numbers.

Prefer to pass slices *10.1.4*

It's a good idea to use slices as the type for procedure parameters whenever possible. If you are creating a procedure that needs some kind of array parameter, then prefer to use a slice. Let's look at why.

As an example, below is a small program that has a dynamic array of cats. There are three procedures to manage the cats:

- `add_cat_of_random_age`: Takes a pointer to a dynamic array and a name. It creates a cat with that name and a random age and adds it to the dynamic array.
- `print_cats`: Takes a slice of cats and prints each cat's name and age.
- `mutate_cats`: Takes a slice of cats and gives each a new random age.

```
package cat_simulation

import "core:fmt"
import "core:math/rand"

Cat :: struct {
    name: string,
    age: int,
}
```

```
add_cat_of_random_age :: proc(cats: ^[dynamic]Cat, name: string) {
    random_age := rand.int_max(12) + 2
    append(cats, Cat {
        name = name,
        age = random_age,
    })
}

print_cats :: proc(cats: []Cat) {
    for cat in cats {
        fmt.printfln("%v is %v years old", cat.name, cat.age)
    }
}

mutate_cats :: proc(cats: []Cat) {
    for &cat in cats {
        cat.age = rand.int_max(12) + 2
    }
}

main :: proc() {
    all_the_cats: [dynamic]Cat
    add_cat_of_random_age(&all_the_cats, "Klucke")
    add_cat_of_random_age(&all_the_cats, "Pontus")

    print_cats(all_the_cats[:])
    mutate_cats(all_the_cats[:])
    print_cats(all_the_cats[:])
}
```

`add_cat_of_random_age` has a parameter `cats` that is a pointer to a dynamic array:

```
add_cat_of_random_age :: proc(cats: ^[dynamic]Cat, name: string) {
```

The parameter `cats` must be of this type because the procedure uses `append`, and that procedure in turn expects a pointer to a dynamic array.

However, `print_cats` can just use a slice, since it only iterates over the cats and prints some information about them:

```
print_cats :: proc(cats: []Cat) {
    for cat in cats {
        fmt.printfln("%v is %v years old", cat.name, cat.age)
```

```
    }
}
```

Note how we call it:

```
print_cats(all_the_cats[:])
```

We make a slice that looks into the whole dynamic array using `[:]`. This is cheap, no extra memory allocations need to happen when `[:]` is used.

Some people would make `print_cats` use a parameter of type `[dynamic]Cat`:

```
print_cats :: proc(cats: [dynamic]Cat) {
```

Some would perhaps even use `cats: ^[dynamic]Cat`. But we do not need a dynamic array since the slice gives us all the information we need. Also, even if we did pass a dynamic array, it would not need to be a pointer since we would not be modifying the fields of the dynamic array.

> Remember: You do not need to pass pointers in order to optimize away unnecessary copy operations. I explained why in the **procedures chapter**.

Using a slice instead of the dynamic array makes it possible to use the procedure `print_cats` with most types of arrays. We can make slices that look into both fixed arrays and dynamic arrays. So making a procedure accept a slice makes the procedure more generally useful!

The third procedure might be slightly surprising to some:

```
mutate_cats :: proc(cats: []Cat) {
    for &cat in cats {
        cat.age = rand.int_max(12) + 2
    }
}
```

This procedure takes a slice and modifies each cat in the slice so that it gets a new age. Note the `&` in `for &cat in cats {`, which makes it possible to modify the elements.

But now someone might say: "Hang on, the parameter `cats: []Cat` doesn't have a `^` in front of the type. How can you modify anything within it? Aren't all procedure parameters immutable?"

It's just the fields directly within the slice that are immutable. Remember, a slice looks like this internally:

```
Raw_Slice :: struct {
    data: rawptr,
    len:  int,
}
```

You are not allowed to modify the `len` field or the address that `data` contains. However, the loop above does not modify any of those two. It goes to the memory that `data` points to and modifies the data that lives there, and that is allowed.

We can summarize all of the above like so:

- If you need to modify a container, as in adding or removing items from it, then pass a pointer to the container.
- If you need to modify the existing elements of a container, then pass a slice to the contents of it.
- If you need to display (read) the information inside a container, then pass a slice to the contents of it.
- Using slices wherever possible will make your procedures more reusable, since you can create slices that look into several different types of arrays.

I hope this example illustrated why slices are used so much in Odin.

Slices with their own memory *10.1.5*

I have said that the syntax `some_array[:]` creates a slice that sees all the elements of `some_array`. I also said that slicing it doesn't cause any dynamic memory allocations.

The above is always true, `[:]` never does any automatic memory allocations.

However, you *can* create a slice that looks into memory that doesn't belong to any other array. This means that it is possible to create slices that have "their own memory".

For example, the following will make `first_20_clone` have its own memory, independent of `my_numbers`:

```
my_numbers: [128]int
first_20 := my_numbers[:20]
first_20_clone := slice.clone(first_20)
```

Needs `import "core:slice"`.

Note the usage of `slice.clone(first_20)`. This `clone` procedure will allocate memory using `context.allocator` and copy the data of `first_20` into that memory. This will make `first_20_clone` have its own, dynamically allocated memory. It will live on, independent of `my_numbers`. This means that you'll have to run `delete(first_20_clone)` in order to deallocate that memory.

This is useful in situations where you have some temporary memory, but you wish to permanently clone it (or a section of it).

You can also create a slice from scratch that has its own memory:

```
ints := make([]int, 128)
```

This uses `context.allocator` to allocate space for 128 integers and gives you a slice that looks into that memory. You'll need to run `delete(ints)` to deallocate it.

It might be a bit confusing that slices don't necessarily have to be a slice of something else. But as I've shown before, slices look like this internally:

```
Raw_Slice :: struct {
    data: rawptr,
    len:  int,
}
```

There is nothing stopping the program from having a `data` field in this struct that points to memory not used by anything else.

You can see slices that have their own memory as a way to dynamically allocate an array with a fixed size. Let's compare creating a standard fixed array to creating a slice of the same size:

```
my_numbers: [128]int
```

The above creates a fixed array and will not cause any dynamic memory allocations. However, this fixed array lives on the stack and will only be valid within the scope where it was created. Also, the size `128` must here be known at compile time. You cannot use a variable to set the size of a fixed array.

Compare this to the following:

```
my_numbers := make([]int, 128)
```

This does a dynamic memory allocation using `context.allocator`. Its memory will live on after the current scope ends. The size `128` does not have to be known at compile time. You could even be silly and create a slice of random size:

```
my_numbers := make([]int, rand.int_max(200) + 10)

import "core:math/rand"
```

Comparison to dynamically allocated fixed arrays

In a **previous example** I showed that you can use `new` in combination with a fixed array type to dynamically allocate fixed arrays:

```
ints_fixed := new([128]int)
```

But, what is then the difference between this:

```
ints_fixed := new([128]int)
ints_fixed_slice := ints_fixed[:]
```

compared to this?

```
ints_slice := make([]int, 128)
```

The first one dynamically allocates a fixed array of `128` integers and then makes a slice that looks at that whole fixed array. The second one makes a slice that has its own dynamically allocated memory with room for `128` integers.

In practice, there's no difference at all. The two examples are for most intents and purposes identical.

In fact, you can even use `delete` on `ints_fixed_slice`, as opposed to using `free` on `ints_fixed`:

```
ints_fixed := new([128]int)
ints_fixed_slice := ints_fixed[:]

// Some time later:
delete(ints_fixed_slice)
```

This only works if your slice starts at the first index of the fixed array. If it doesn't, then `delete` will try to deallocate using the wrong pointer, and your program may crash.

This makes sense since the data field of the slice is a pointer to the start of the fixed array. `delete` will deallocate the memory that `data` points to. So `delete(ints_fixed_slice)` ends up doing the same thing as `free(ints_fixed)` would do.

With this in mind, whenever you need a dynamically allocated array that doesn't need to grow, then I recommend just using `make` with a slice type.

Slice literals *10.1.6*

Sometimes you want to run a procedure that needs a slice, but you only have a fixed array. So you slice the fixed array:

```
print_numbers :: proc(numbers: []int) {
    for n in numbers {
        fmt.println(n)
    }
}

my_numbers := [3]int { 1, 2, 3 }
print_numbers(my_numbers[:])
```

There is a special way to do exactly the above, but in a more compact way:

```
my_numbers := []int { 1, 2, 3 }
print_numbers(my_numbers)
```

Note how we dropped the `3` inside the `[]int` and we no longer need the `[:]` after `my_numbers`.

`[]int { 1, 2, 3 }` is known as a slice literal. This:

```
my_numbers := []int { 1, 2, 3 }
```

does exactly the same thing as this:

```
my_numbers_arr := [3]int { 1, 2, 3 }
my_numbers := my_numbers_arr[:]
```

A slice literal creates a fixed array of integers that lives on the stack and then makes a slice that looks at the whole thing.

Note that `[]int { 1, 2, 3 }`

is a slice literal. You can also do this:

```
[?]int { 1, 2, 3 }
```

However, the one with the `?` creates a normal fixed array. It's just a convenient way to avoid having a `3` between the brackets. Instead, it infers the number of elements needed for the fixed array from the fact that there are three numbers between the curly braces.

You can even use slice literals directly as an argument to a procedure:

```
print_numbers :: proc(numbers: []int) {
    for n in numbers {
        fmt.println(n)
    }
}

print_numbers({ 1, 2, 3 })
```

Note how we now have no in-between variable before calling `print_numbers`. We just write `{ 1, 2, 3 }` as an argument to `print_numbers`. This makes a slice literal in the same way as above. Meaning that there is a hidden stack allocated fixed array that holds these three numbers, and a slice of that whole array is sent into `print_numbers`.

Slice literals pitfalls

Since slice literals use a hidden fixed array that lives on the stack, one can easily make mistakes when using them. Let's look at such a mistake:

```
package slice_literal_pitfall

import "core:fmt"

numbers: []int

set_numbers :: proc() {
    numbers = {
        7, 42, 13
    }
    fmt.println(numbers)
}

main :: proc() {
    set_numbers()
    fmt.println(numbers)
}
```

This program has a `set_numbers` procedure that sets a global variable `numbers: []int` to a value that is a slice literal.

If you run the program above, then it will print something like:

```
[7, 42, 13]
[140700727785616, 67460135296, 72339069014638605]
```

The first print happens inside `set_numbers` and the second one happens in `main` after `set_numbers` has finished. The big numbers of the second line (`140700727785616` etc) may be different on your computer. What happened here is the following: The global variable `numbers` is a slice that is looking into deallocated stack memory. We can see why more clearly if we change `set_numbers` to look like this:

```
numbers: []int

set_numbers :: proc() {
    numbers_arr := [3]int {
        7, 42, 13
    }
    numbers = numbers_arr[:]
    fmt.println(numbers)
}
```

The above is identical to the old code. When you set `numbers` to a slice that looks into `numbers_arr` then you are effectively creating a slice that looks into memory that lives on the stack of `set_numbers`. That memory is only valid until the procedure ends. So the `data` field of the slice would contain the address you'd get by doing `&numbers_arr[0]`, which points to stack memory.

Maps 10.2

Maps are in some languages referred to as dictionaries. You use them to map *keys* to *values*. If you have a key then you can lookup which value it is associated with.

Creating and destroying maps *10.2.1*

Declare a new map like this:

```
age_by_name: map[string]int
```

This map has keys of type `string` and values of type `int`.

Add entries by writing `age_by_name[key] = value`:

```
age_by_name["Karl"] = 35
age_by_name["Klucke"] = 7
```

Remove entries using the `delete_key` procedure:

```
delete_key(&age_by_name, "Klucke")
```

You also use `[]` to retrieve values:

```
karls_age := age_by_name["Karl"]
```

If there was no value with that key, then `karls_age` will have the zero value. If this isn't good enough, then you can also get a second value of type `bool` that says if the item existed or not:

```
if karls_age, ok := age_by_name["Karl"]; ok {
    // There was a value for this key! In
    // here you can safely use `karls_age`.
}
```

There are also two special operators called `in` and `not_in` that you can use to check if a key exists in the map:

```
has_karl := "Karl" in age_by_name
does_not_have_klucke := "Klucke" not_in age_by_name
```

If you make a loop that uses this `in` syntax, then you have to put the condition in parenthesis:

```
for ("Karl" in map) {}
```

This is because the compiler would otherwise confuse this with the syntax for iterating arrays:

```
for x in array {}
```

Maps and allocations *10.2.2*

The initial declaration of a map does not allocate any memory:

```
age_by_name: map[string]int
```

However, when you add items to it, like so:

```
age_by_name["Karl"] = 35
```

Then the map will grow if needed. This will cause a dynamic memory allocation. This is similar to how `append()` can cause a dynamic array to allocate memory.

Since the map uses dynamic memory, you'll need to destroy it manually when you no longer need it:

```
delete(some_map)
```

If you want to create a map that uses some other allocator than `context.allocator`, then you can use `make`:

```
age_by_name := make(map[string]int, context.temp_allocator)
```

This call to `make` does not cause any immediate memory allocation, it just saves the allocator inside the map for future usage. Later, when you add elements using for example `age_by_name["Bob"] = 89`, then memory may be allocated using the allocator you provided to `make`.

Similar to how dynamic arrays are just a struct under the hood, maps are represented by `Raw_Map` under the hood.

See **<odin>/base/runtime/core.odin** .

You can also provide an initial capacity to `make`:

```
age_by_name := make(map[string]int, 64, context.temp_allocator)
```

In this case an allocation will occur immediately.

Iterating maps *10.2.3*

You can iterate maps. Note how we have two loop variables below. One for the key, one for the value:

```
for key, value in some_map {
}
```

You can make the value modifiable by adding in a `&` in front of `value`:

```
for key, &value in some_map {
}
```

Note that you cannot make the key modifiable while iterating. Map keys are immutable.

When to use maps *10.2.4*

Use maps when you really don't know what kinds of keys you might need. Often this is because the user of the program can specify their own data. Perhaps you need to associate one user-specified value with another user-specified value. That's a good use case for a map.

If you *know* all possible keys, then just make an `enum` and use an **enumerated array**. There's no point in using a map for that. Enumerated arrays are much faster. With an enumerated array it can look up what element it needs very quickly, since the enum acts like an index. When fetching values from a map using a key, it instead has to search using the key, which is considerably slower.

Making a set using a map *10.2.5*

A set is a collection where each item can only appear once. Using a set, you can easily check if an item is in the set or not.

This is unrelated to `bit_set`, which we'll talk about in **chapter 15**.

Odin does not have any special support for sets. However, you can still create a set by making a map where the *keys* of the map are of the type you wished to store in a set. For the values of the map you use an empty struct. An example:

```
set_of_names: map[string]struct{}

// Add to set
set_of_names["Pontus"] = {}

// Check if set contains "Pontus"
if "Pontus" in set_of_names {

}

// Iterate set
for key in set_of_names {

}

// Remove from set
delete_key(&set_of_names, "Pontus")
```

Note how we declare the set using the type `map[string]struct{}`. This means that we are making a map that has strings as keys, but the value is an empty type `struct{}`

You can see `struct{}` as an "anonymous type". This means that this type does not have a name.

I don't blame you if you think the method for adding things to the set looks awkward:

```
set_of_names["Pontus"] = {}
```

This means that we are associating the key `"Pontus"` with a value that is an empty struct. That means that `"Pontus"` now exists in the set and is associated with a value, although that value is just an empty struct. A `struct{}` takes 0 bytes of memory. So no memory is wasted in defining a set in this way.

The `map` code has special cases internally for taking care of `0` byte types being used as values.

If you can't live with it, then you can introduce an `add_to_set` procedure that hides the weird `= {}` part:

```
add_to_set :: proc(s: ^map[$T]struct{}, v: T) {
    s[v] = {}
}
```

This procedure uses **parametric polymorphism**, which we will talk about later. For now, just note that it will work with any kind of map that uses `struct{}` as the value type.

Custom iterators 10.3

If you need to iterate over a collection in a special way, then you can define your own iterator. This means that you can create your own iteration loops, similar to this one:

```
for v, i in some_array {

}
```

As we shall see, using custom iterators need slightly more code than the loop above. Nevertheless, they are still very useful, since custom iterators walk through the collection in very specific ways.

In the following example there is a struct called `Slot`. It has a `used` field of type `bool`. I've made an iterator that iterates over a slice of `Slot`s. But this iterator only considers those elements that have `used` set to `true`. This means that the loop at the end of the example will automatically skip all the elements where `used` is `false`.

```
Slot :: struct {
    important_value: int,
    used: bool,
}

Slots_Iterator :: struct {
    index: int,
    data: []Slot,
}

make_slots_iterator :: proc(data: []Slot) -> Slots_Iterator {
    return { data = data }
}

slots_iterator :: proc(it: ^Slots_Iterator) -> (val: Slot, idx: int, cond: bool)
{
    cond = it.index < len(it.data)

    for ; cond; cond = it.index < len(it.data) {
        if !it.data[it.index].used {
            it.index += 1
            continue
        }

        val = it.data[it.index]
        idx = it.index
        it.index += 1
        break
    }

    return
}

slots := make([]Slot, 128)

slots[10] = {
    important_value = 7,
    used = true,
}
```

```
it := make_slots_iterator(slots)

for val in slots_iterator(&it) {
    fmt.println(val)
}
```

The program will only print the slot at index `10`, despite `slots` having `128` entries. This is because the slot at index `10` is the only slot that has `used` set to `true`.

These lines do the actual iteration:

```
it := make_slots_iterator(slots)

for val in slots_iterator(&it) {
    fmt.println(val)
}
```

First it creates an iterator. At the beginning of each lap of the loop, `slots_iterator` is called and fed the iterator. Note how `slots_iterator` is defined:

```
slots_iterator :: proc(it: ^Slots_Iterator) -> (val: Slot, idx: int, cond: bool)
{ }
```

It has three return values. An iterator procedure can be any procedure that has the following three return values:

- A value: The element it wants the loop to consider when `some_iterator` returns. This is `val` in `for val in some_iterator(&it) {}`.
- A 'second loop variable', often this is an index, but it doesn't have to be. This is the optional extra thing you can add after `val` in the loop: `for val, index in some_iterator(&it) {}`
- The condition that the iteration should continue. If this return value is false then the loop will stop.

Now let's focus on what's inside `slots_iterator`:

```
slots_iterator :: proc(it: ^Slots_Iterator) -> (val: Slot, idx: int, cond: bool)
{
    cond = it.index < len(it.data)

    for ; cond; cond = it.index < len(it.data) {
        if !it.data[it.index].used {
            it.index += 1
            continue
```

```
            }

            val = it.data[it.index]
            idx = it.index
            it.index += 1
            break
        }

        return
    }
```

It uses a `for` loop to skip all slots that are unused. Having a loop in there might seem wasteful, but it is fine since the loop always starts from `it.index`. This means that it doesn't loop from the start of the slice every time, it only loops until it finds the next element that has `used == true`. At that point `slots_iterator` returns, informing the loop that it found a relevant element. Then the loop's body can run, in this case `fmt.println(val)`. After that the loop continues, which means calling `slots_iterator` again. It continues to search through the slice for the next element.

Sometimes you need to modify the value from within the loop. To support that, make a second version of the iterator procedure. That version should return a pointer to a value instead of just a value:

```
slots_iterator_ptr :: proc(it: ^Slots_Iterator) -> (val: ^Slot, idx: int, cond:
bool) {
    cond = it.index < len(it.data)

    for ; cond; cond = it.index < len(it.data) {
        if !it.data[it.index].used {
            it.index += 1
            continue
        }

        val = &it.data[it.index]
        idx = it.index
        it.index += 1
        break
    }

    return
}
```

We've only changed three things from the previous implementation of `slots_iterator`:

- The new procedure is named `slots_iterator_ptr` instead of `slots_iterator`
- The type of return value `val` is now `^Slot`.
- The line `val = &it.data[it.index]` now contains an `&`.

You use it just like before, but use `slots_iterator_ptr` instead of `slots_iterator`:

```
it := make_slots_iterator(slots)

for val in slots_iterator_ptr(&it) {
    // val is a pointer.
}
```

This is a bit different from writing `for &v in array {}`. In our custom iterator case, then `val` is a pointer. But when writing `for &v in array {}` then `v` is an **addressable**, not a pointer.

Chapter 11

Strings

Text strings in Odin use Unicode. Unicode is a standard that makes it possible to use characters from most languages. You can mix different languages within the same string, and even use exotic things such as emojis!

Odin has two primary types for representing text: `string` and `cstring`. You'll mainly use the `string` type in your Odin code. The `cstring` type is for working with libraries written in the C programming language. Both types use the *UTF-8* encoding, short for *Unicode Transformation Format - 8 bit*. UTF-8 is the most popular way of storing Unicode text in memory.

There are two more string types called `string16` and `cstring16`. Those use the *UTF-16* encoding.

In this chapter we'll learn how to work with strings. At the same time, we'll build an understanding of UTF-8, and how it relates to Unicode. We'll also learn how to dynamically build new strings and how the `string` type looks on the inside. Towards the end of the chapter we'll look at how to manually decode UTF-8, so you understand how Odin does it for you. In the final section we'll see an example of when UTF-16 is needed.

String variables and string constants 11.1

Create a string variable and assign a value to it like this:

```
my_string: string
my_string = "Hellope!"
```

Or, on a single line with type inference:

```
my_string := "Hellope!"
```

The type of `my_string` is inferred to `string`.

`"Hellope!"` is a string constant, or string literal. It's called a literal because it's there, literally, in the code. String literals are valid for as long as the program runs and do not require any manual memory management.

Your program may crash if you write `delete(my_string)` where the variable `my_string` was created from a string literal.

You can create a named string constant, like so:

```
A_STRING_CONSTANT :: "Hellope!"
```

Both the literal `"Hellope!"` and `A_STRING_CONSTANT` are of type Untyped String. In other words, a string literal is just a nameless string constant.

As we shall see in **section 11.6.1**, talking about "string constants" can be ambiguous.

Untyped Strings can be implicitly cast to the variable types `string`, `cstring`, `string16` and `cstring16`. These lines are all valid:

```
STR :: "Hellope!"
str: string = STR
cstr: cstring = STR
str16: string16 = STR
cstr16: cstring16 = STR
```

The cast is done at compile time, including any needed UTF-8 or UTF-16 encoding.

Combining string constants *11.1.1*

String constants can only be created from values that are available at compile time. You can combine string constants into longer strings at compile time using `+`. Here's an example:

```
A_CONSTANT :: "Hellope!"
my_string := A_CONSTANT + " How are you?"
fmt.println(my_string) // Hellope! How are you?
```

You cannot use `+` in combination with non-constant strings. The following will not compile because `a_variable` is not a string constant:

```
A_CONSTANT :: "My name is"
a_variable := "Patches"
combined := A_CONSTANT + " " + a_variable
```

Error message: "String concatenation is only allowed with constant strings".

If you need to combine string variables into bigger strings (or turn non-strings into strings), then use a procedure from `fmt` or a string builder. We'll look at those things in **section 11.4**

and **section 11.5**.

To understand why `+` does not work with string variables, you can think of the compiler as "copy-pasting" together the strings you put to the left and right of the `+`. It cannot do such "compile-time copy-pasting" with something in a variable, because it is only known at run time what that variable contains.

This discussion is only related to strings. Adding numeric variables using `+` works fine.

Iterating strings 11.2

In the following example we iterate a string. For each lap of the loop, the loop variable `r` will contain a character from the string.

```
str := "Important Words"

for r in str {
    // r is of type `rune`
}
```

The loop variable `r` is of type *rune*. A rune is just a 32 bit (4 byte) number. In many cases (but not all!), a rune represents a single character. For example the letter `K` is represented by a rune of value `75`. The name "rune" is specific to Odin. Outside of Odin, a rune is called a Unicode *code point*.

The numeric values of code points are decided by the organization known as the Unicode consortium. They try to make a wide variety of characters, from many languages, possible to express using code points. For example, they've decided that the letter `Q` has the code point value `81` and that the Chinese character `猫` has the code point value `29483`.

When we talk about **grapheme clusters** you'll understand why I wrote "In many cases (but not all!), a rune represents a single character".

When our program stores a string in memory, such as in the variable `str`, then it doesn't store it as a series of 4 byte rune values. Instead, it stores each rune using 1 to 4 bytes. That's the UTF-8 encoding. As the loop in our example runs through the UTF-8-encoded string, it decodes sequences of 1-4 bytes into runes.

You can see it as a type of compression: A rune is a 4 byte integer. But in many cases you can store such a number using fewer bytes. For example, the letter `W` has the code point value `119`. UTF-8 encodes that using just a single byte, saving you three whole bytes of memory!

In UTF-8 encoded strings, all English characters need just a single byte. But characters from other languages may use more than a single byte. In the following example we iterate a string written in Simplified Chinese:

```
str := "小猫咪"

for r in str {
    fmt.println(r)
}
```

Which will print:

```
小
猫
咪
```

Within the memory of the UTF-8 string `str`, these Chinese characters use 3 bytes each. As the loop runs through the string, it decodes those groups of 3 bytes into proper 4 byte runes.

The UTF-8 decoder knows if it is processing a 1, 2, 3 or 4 byte rune. A few bits in each byte is dedicated to signaling if the rune continues, or if it is complete. In **section 11.11** I'll show how to manually decode UTF-8 by inspecting the value of each byte.

A note for Windows users: You may see garbage in the command prompt when printing the Chinese characters. In that case, try running `chcp 65001` in the command prompt, before executing the program. Alternatively, add `windows.SetConsoleOutputCP(.UTF8)` at the top of the `main` proc (`import "core:sys/windows"`). This works fine on Windows 11. If you are on Windows 10, then you might need a better command prompt program than `cmd`. I recommend **ConEmu**, which wraps `cmd` with some nice features. I don't recommend PowerShell, it is buggy.

It's important to understand that a rune, or Unicode code point, is independent of how the string is encoded. There are several different encodings, such as UTF-8 and UTF-16. Those encodings just decide how the string is stored in memory and how to turn that memory back into runes. The decoded value of the rune is the same, regardless of encoding.

Creating substrings: String slicing 11.3

You can cut strings up, or create substrings, using the same syntax that is used when slicing arrays: `[:]`. Because of this, I tend to say "string slice" instead of "substring". Here's an example where we slice a string and print the slice's value:

```
str := "Little Cat!?"
sub_str := str[7:11]
fmt.println(sub_str) // Cat!
```

The slicing operation does not cause any extra memory allocation. `sub_str` looks into a section of `str`'s memory.

The above creates the string slice `"Cat!"` by slice the string `"Little Cat!?"`. Please note that the slice operator `[start:end]` uses *byte indices*. You say at what byte your slice should start, and at what byte it should end. In this case our slice starts at byte `7` and ends at byte `11`. This works fine for English text, since English characters use a single byte.

But if we take the Chinese text we saw earlier and assume that we can slice it in the same way, then we'll end up in trouble:

```
str := "小猫咪"
sub_str := str[1:]
fmt.println(sub_str)
```

Skipping the index to the right of `:` in `str[1:]` means "go all the way to the end of the string".

the above prints:

```
◆◆猫咪
```

One could think that `str[1:]` would make a string that goes from the second character to the last one. But each one of these Chinese characters use 3 bytes of memory. So `str[1:]` actually makes a string slice where the first character's data has gotten cut up.

Running `fmt.println(len("小猫咪"))` will print `9`. `len` does not report how many runes the string contains, but rather the byte length.

`strings.rune_count("小猫咪")` will report `3` (import `core:strings`). Note that `rune_count` has to loop through the string and count the runes, whereas `len` just returns a stored number.

If you instead start the slice at index `3`, then you get the wanted string:

```
str := "小猫咪"
sub_str := str[3:]
fmt.println(sub_str) // 猫咪
```

However, it is also possible to slice using *rune indices*. You can do that using **`strings.substring_from`** :

```
str := "小猫咪"
sub_str, _ := strings.substring_from(str, 1)
fmt.println(sub_str) // 猫咪
```

`strings.substring_from` will make a substring that starts at the rune with index `1` and then goes to the end of the string. This also gives us the expected `猫咪` string. But note that a procedure such as `substring_from` needs to iterate over the string. Why? In this case we want a substring that starts at rune index `1`. But the string doesn't know how many bytes the UTF-8 encoded rune at index `0` uses. So in order to skip the rune at index `0`, it must loop from the start and decode that rune.

There are also procs such as **`strings.substring`** and **`strings.substring_to`**.

If you need the byte index of the current rune while iterating a string, then add a second loop variable:

```
str := "小猫咪"

for r, i in str {
    fmt.println(i, r)
}
```

Note the added `, i` on the `for` line.

which prints

```
0 小
3 猫
6 咪
```

This is useful for constructing slices that start or end at the current rune:

```
str := "小猫咪"

for r, i in str {
    from_start := str[:i]
    fmt.println(from_start)
}
```

The above will output

```
[blank line]
小
```

```
小猫
```

If you want the current rune included in the substring. Then you need to add the byte size of it. You can fetch the byte size using `utf8.rune_size(r)`:

```
str := "小猫咪"

for r, i in str {
    from_start := str[:i + utf8.rune_size(r)]
    fmt.println(from_start)
}
```

```
import "core:unicode/utf8"
```

`rune_size` may sound like a weird procedure name. `size_of(rune)` is always 4. But this procedure is from the `utf8` package. It tells us the size of the rune when encoded as UTF-8.

The above will print:

```
小
小猫
小猫咪
```

These pitfalls related to UTF-8 and string slicing may seem daunting. However, if you create a big piece of software, you'll be surprised how little string slicing you actually do. And when you do need to slice something, you'll often have the byte index of the relevant runes at hand. Don't be afraid!

Construct strings using `fmt` 11.4

One commonly needs to construct a string by combining a few variables and values. There are a couple of ways to do this.

A simple method is to use the `fmt.tprint` or `fmt.aprint` procedures.

```
name := "Pontus"
age := 7
str := fmt.tprint(name, "is", age)
fmt.println(str) // Pontus is 7.
```

Note the difference between `fmt.print` and `fmt.tprint`. `fmt.print` prints to "stdout", which by default means the console. `fmt.tprint` constructs a string and returns it so you can use it for whatever you

want.

This will combine `name`, `"is"` and `age` with a space between each. The result is `"Pontus is 7"`. The `t` in front of `print` means "temporary": This procedure allocates the string's memory using `context.temp_allocator`. You can swap out the space that `tprint` puts between each argument for something else. In the following example we put `" really "` between each word using the `sep` parameter of `tprint`:

```
name := "Pontus"
age := 7
str := fmt.tprint(name, "is", age, sep = " really ")
fmt.println(str) // Pontus really is really 7.
```

`aprint` is just like `tprint`, but you can specify which allocator you want to use. So this:

```
str := fmt.tprint(name, "is", age)
```

is identical to this:

```
str := fmt.aprint(name, "is", age, allocator = context.temp_allocator)
```

Something that is often more useful than `tprint` and `aprint` are the `tprintf` and `aprintf` procedures. Note the added `f` at the end of the name. These procedures take what is known as a *format string*:

```
name := "Pontus"
age := 7
str := fmt.tprintf("%v is %v", name, age)
```

This has the same result as when we used `tprint`. But here we see `%v` appear two times in the format string. Those `%v` are replaced by the subsequent arguments `name` and `age`, in the order they are stated. There's an `aprintf` version as well, which lets you specify your own allocator.

There are lots of different format strings you can use. The `%v` used in this example does whatever makes the most sense for the value we pass into it. But you can be more specific. For example, if you want to print a floating point number and only show two decimals, then you can replace `%v` with `%.2f`. See **`<odin>/core/fmt/doc.odin`** for a comprehensive list of format strings.

There are variants of all these procedures that add in a line break at the end of the string, such as `fmt.tprintfln`. All of them end with `ln` (Line Break).

Finally, if you do not want to use any allocator at all, then you can use a buffer that lives on the stack combined with `fmt.bprintf`:

```
name := "Pontus"
age := 7
buf: [128]byte
str := fmt.bprintf(buf[:], "%v is %v", name, age)
```

`str` will be valid as long as `buf` is valid, which will be until the end of the current scope. Note that this string has a fixed limit of 128 bytes. Only the first 128 bytes will make it into the string. If you need to support a string of any length, then I recommend using one of the `fmt.tprint` or `fmt.aprint` procedures I showed before. You can also use a string builder, which we will talk about in the next section.

Construct strings using a string builder 11.5

Perhaps you need to construct a string that contains several lines. Or perhaps you need to construct a string over many lines of code, putting it together piece-by-piece. In that case I recommend using a string builder. Here's an example:

```
lines := []string {
    "I like",
    "I look for",
    "Where are the",
}
b := strings.builder_make()

for l, i in lines {
    strings.write_string(&b, l)
    if i != len(lines) - 1{
        strings.write_string(&b, " cats.\n")
    } else {
        strings.write_string(&b, " cats?")
    }
}

str := strings.to_string(b)
fmt.println(str)
strings.builder_destroy(&b)
```

Available `write_xx` procedures can be found in **<odin>/core/strings/builder.odin** .

Note the `\n` that is manually added:

```
strings.write_string(&b, " cats.\n")
```

This is another way to add a line break.

In this example `lines` is a slice containing three different strings.

The loop goes over `lines` and writes each of them to the builder. It adds a different ending to each line, depending on if it is the last line or not.

After the loop is done, it extracts the string `str` from the builder:

```
str := strings.to_string(b)
```

It then prints the constructed string and destroys the builder, which will deallocate `str`.

The output of the program is:

```
I like cats.
I look for cats.
Where are the cats?
```

It's important to understand where the allocations happen in this example. This line just sets up the builder:

```
b := strings.builder_make()
```

That line does not allocate any memory. However, when you call `strings.write_string`, then the builder may need to grow. Growing would result in a dynamic memory allocation. This is similar to using **`append` with a dynamic array**. In fact, the builder uses a dynamic array internally.

The final string is fetched by running `str := strings.to_string(b)`. That call does not cause any extra allocations, it just gives you a string that looks into the data of the builder. So after the builder is destroyed then `str` is no longer valid.

If you need a temporary string, then you can pass `context.temp_allocator` to `builder_make`:

```
b := strings.builder_make(context.temp_allocator)
```

The builder stores this allocator internally. It'll use it when it needs to grow. You can then skip the `strings.builder_destroy(&b)` line.

But remember to run `free_all(context.temp_allocator)` somewhere in your program! See **chapter**

9.7.1.

Using the temp allocator with the string builder is my favorite way of constructing strings, even if I need them around long-term. Here's an example of how I'd do it:

```
b := strings.builder_make(context.temp_allocator)

strings.write_string(&b, "I have ")
strings.write_int(&b, 7)
strings.write_string(&b, " cats.")

str := strings.clone(strings.to_string(b))
```

Here the builder uses the temp allocator for the construction of the string. At the end I use `strings.clone` in order to clone the builder's final string. Since this cloning uses `context.allocator` by default, then the final `str` will live on even after the temp allocator is reset.

There's a slight difference between this approach and the previous one. By using `context.temp_allocator` for the builder and cloning at the end, I get a string that only uses as much memory as the string needs. If I use `context.allocator` for the builder and use the string from the builder directly without cloning it, then the string might use more memory because the builder may have a bigger internal capacity than the length of the string.

I tend to do things in this fashion quite often: Construct intermediate values using a temp allocator and only allocate the final result on a non-temp allocator for long-term storage. For example, I tend to load entire files on the temp allocator and do intermediate processing of the file's data using the temp allocator. Only at the end do I use `context.allocator` in order to get a non-temporary copy of the final result.

`fmt` *and string builder* *11.5.1*

There are a few procedures in `fmt` named **`sbprint`**, `sbprintf`, etc that let you print into a string builder (hence the prefix `sb`).

Those procedures enable you to easily combine string builders with format strings. Here's an example:

```
b := strings.builder_make()
num_cats := 7
fmt.sbprintfln(&b, "I have %v cats.", num_cats)

fmt.sbprintln(&b, "Time for some random numbers!")
```

```
for i in 0..<10 {
    fmt.sbprintfln(&b, "Number %i: %i", i + 1, rand.int_max(100))
}

str := strings.to_string(b)
fmt.println(str)
strings.builder_destroy(&b)
```

The output of this program is:

```
I have 7 cats.
Time for some random numbers!
Number 1: 52
Number 2: 38
Number 3: 59
Number 4: 4
Number 5: 49
Number 6: 94
Number 7: 19
Number 8: 8
Number 9: 28
Number 10: 94
```

What's a string internally? 11.6

Strings use the type `string`. But internally, `string` is represented by the type `Raw_String`. That type consists of a pointer and a length. It looks like this:

```
Raw_String :: struct {
    data: [^]u8,
    len:  int,
}
```

From C / C++

The type `[^]u8` is almost like a pointer to a u8 (a byte), but it is also indexable. It is called a "multi-pointer". Normal pointers in Odin cannot be indexed. In C, pointers can be indexed. So the multi-pointer works like a C pointer.

Because multi-pointers behave like C pointers, they are often used when creating bindings to libraries written in C.

Here it is used because `Raw_String` needs to have a pointer it can index like an array, without having to use a slice type.

This looks very similar to a slice! There's a `data` field and a length, just like in a `Raw_Slice`. Both slices and strings support the slice operator `[:]`, which makes sense given how similar they are. You can see a `string` as a slice of bytes (`[]u8`) that the language knows how to decode from UTF-8 into runes.

Note: Memory-wise, there is no difference between a variable of type `string` and its internal `Raw_String` representation. Within a procedure, you can see a `string` variable as a `Raw_String` struct. Within some procedures in `core` and `base`, strings are transmuted directly into the `Raw_String` type.

Depending on how you create your string, the `data` field inside the `Raw_String` may point at memory that lives in very different places. Let's look at three examples.

Example 1: The data of a string literal

When you assign a string literal to a string variable, like this:

```
my_string := "Hellope!"
```

then `my_string` is of type `string`. But you can also see it as a `Raw_String` struct that lives on the stack. However, where does the `data` field inside `Raw_String` point? In order to understand that, we need to understand *string interning*.

String literals are part of the source file. So they are known at compile time. Therefore, the compiler can give string literals special treatment. The compiler will store the data of all string literals in the *read-only data block* of the program, commonly referred to as `rodata`. It will also de-duplicate the strings it stores in `rodata`: If your code contains two `"Hellope!"` literals, then they'll use the same memory. This process is known as string interning.

> `rodata` is an immutable chunk of memory. In there, the program can store things that should never be modified. The things in `rodata` are valid for the whole execution of the program.

Therefore, the `data` field within a string such as `my_string := "Hellope!"` points at an address in `rodata`.

No manual memory management is required for string literals. Never try to run `delete` on a string initialized from a string literal. Doing so will crash your program, because the data is not dynamically allocated!

Example 2: The data of a dynamically allocated string

Strings can be dynamically allocated in many different ways. In the following example, we construct a string that contains some text combined with a randomly generated number.

```
random_number := rand.int_max(1000)
my_string := fmt.aprintf("A random number: %v", random_number, allocator =
context.allocator)

import "core:math/rand"
```

In this case, `my_string` can also be seen as a `Raw_String` that lives on the stack. But the `data` field within the `Raw_String` will point at dynamically allocated memory that lives on the heap (assuming that `context.allocator` is a heap allocator).

These strings should be deallocated using `delete(my_string)`.

There are many different allocators you could use here, and if you used the temporary allocator or an arena allocator, then you would not need the `delete(my_string)`. The main thing to realize is that the `string`/`Raw_String` type itself behaves like any struct that lives on the stack. The main difference between these examples lies in understanding what the `data` pointer inside `Raw_String` points at.

Example 3: String data pointing at the stack?

You can also create a fixed array of bytes and use something like `fmt.bprint` to create a string that lives completely on the stack:

```
buf: [128]byte
my_string := fmt.bprint(buf[:], "Hellope")
```

Not only does the `string`/`Raw_String` struct live on the stack, but in this case the `data` field within the `Raw_String` points at stack memory as well. This is because the string was written into `buf` by `bprint` and `buf` is a fixed array that lives on the stack. When the current scope ends, then `my_string` is no longer valid. Trying to run `delete(my_string)` would crash the program because the string's data is not dynamically allocated.

Ambiguities when talking about "string constants" *11.6.1*

As we saw above in Example 1: The data of a string literal, strings that originate from string literals will have their data stored in `rodata`.

`rodata` is immutable, or as some people say "constant". This can cause some ambiguity regarding what one actually means by "string constant". Take these two lines for example:

```
CONSTANT_STRING :: "Hellope!"
str := CONSTANT_STRING
```

Here `CONSTANT_STRING` is a constant (in the Odin sense). It has the type Untyped String. `str` is a variable of type `string`. The assignment of `CONSTANT_STRING` to `str` does the following:

- The variable `str` is inferred to be of type `string`. That's the default to use when doing type inference based on an Untyped String.
- Because the type is `string`, the compiler knows that it needs a UTF-8 encoded version of the string literal `"Hellope!"`.
- The compiler checks if it already has placed an encoded string with this value in `rodata`. It can then reuse that data. This makes repeated string literals reuse the same memory. This is known as string interning. Otherwise, it puts a freshly encoded UTF-8 string into `rodata`.
- Under the hood the string `str` has the type `Raw_String`. It has a `data` field. That field will be set to point at the UTF-8 encoded string in `rodata`.

So after assigning `CONSTANT_STRING` to `str`, the variable refers to immutable data. This makes it awkward to talk about "string constants": `str` is a variable but some would refer to it as a "string constant" because of how the data is stored.

You could also just do `str := "Hellope!"` and end up in the same situation. A string literal like `"Hellope!"` is like a "nameless constant".

So whenever you see the term "string constant", make sure to understand from the context what "constant" refers to.

A common work-around is to just say "string literal" whenever the value *originates* from a string literal. In the example below, we would then say that `CONSTANT_STRING`, `"Hellope!"`, `str1` and `str2` are *all* "string literals":

```
CONSTANT_STRING :: "Hellope!"
str1 := CONSTANT_STRING
str2 := "Hellope!"
```

It's not really correct: Only `"Hellope!"` is a literal here. But in this case we are more worried about "where is it stored?" rather than "how was it declared in the code?".

Deallocating strings mixed with string constants 11.7

A common problem is figuring out how to `delete` dynamically allocated strings that are mixed with string constants.

Here's an example of how it can go wrong:

```
str_to_number: map[string]int

str_to_number["Hellope!"] = 5
str_to_number["Hi!"] = 1
str_to_number[strings.clone("Dynamically allocated!")] = 7
str_to_number["Constant!"] = 42
str_to_number[strings.clone("Dynamic!")] = 8

// Use str_to_number for something important

for key, value in str_to_number {
    delete(key)
}

delete(str_to_number)

fmt.println("Probably didn't get here due to `delete(key)` crashing")
```

```
import "core:strings"
```

This code creates a `map` that associates a `string` with an integer. We say that it uses the type `string` as *key*. Some of the keys are string constants, such as `"Hellope!"`. There are also two dynamically allocated keys in there, such as this one: `strings.clone("Dynamically allocated!")`.

In order to not leak memory, the code needs to deallocate those dynamically allocated keys. That's what this loop attempts to do:

```
for key, value in str_to_number {
    delete(key)
}
```

But it has no good way of knowing which keys are constants and which are dynamically allocated. This program will crash when it tries to run `delete` on one of those string constants.

I'm aware that this is a synthetic example where those calls to `strings.clone` are pointless. But it's a problem many people run into when making a big program: One often sets up a map with some hard-coded initial data, where the keys are string constants. Then there is some separate code that also adds a few keys that are dynamically allocated, which puts you in this situation.

We'll look at two ways of fixing this. The simplest way wastes a bit of memory, but is very straightforward; dynamically allocate all the keys:

```
str_to_number: map[string]int

str_to_number[strings.clone("Hellope!")] = 5
str_to_number[strings.clone("Hi!")] = 1
str_to_number[strings.clone("Dynamically allocated!")] = 7
str_to_number[strings.clone("Constant!")] = 42
str_to_number[strings.clone("Dynamic!")] = 8

// Use str_to_number for something important

for key, value in str_to_number {
    delete(key)
}

delete(str_to_number)

fmt.println("Probably got here!")
```

In this version, all the keys of the map are dynamically allocated. Now `delete(key)` will no longer crash.

The second method will use a concept we haven't talked about yet: Memory arenas. I will talk about arenas in **chapter 13**. Here I just present this second method with some overview commentary about how it works:

```
arena: vmem.Arena
arena_allocator := vmem.arena_allocator(&arena)
str_to_number: map[string]int

str_to_number["Hellope!"] = 5
str_to_number["Hi!"] = 1
str_to_number[strings.clone("Dynamically allocated!", arena_allocator)] = 7
str_to_number["Constant!"] = 42
str_to_number[strings.clone("Dynamic!", arena_allocator)] = 8

// Use str_to_number for something important

vmem.arena_destroy(&arena)
delete(str_to_number)

fmt.println("Probably got here!")
```

```
import vmem "core:mem/virtual"
```

We set up a *virtual memory arena* called `arena`. The `arena_allocator` we send into `strings.clone` will allocate the string clones into `arena`. The benefit of an arena is that we can destroy the arena in order to deallocate everything within it. Note how we have replaced this:

```
for key, value in str_to_number {
    delete(key)
}
```

with this:

```
vmem.arena_destroy(&arena)
```

This is less wasteful since we didn't have to clone *all the strings*. The ones that we did have to clone, we cloned into the arena. The deallocation of the strings is simpler: We just destroy the arena. But in other ways it's more complicated: We had to set up the arena and now we have both a map *and* an arena to worry about.

Also, note that I didn't allocate the map itself using the arena. I explain why in chapter 13 when I talk about how arenas **do not support individual deallocations**.

For the sake of simplicity I tend to choose the first method.

There's some **string interning functionality in `core:strings`** that you can also use. That way you can do the simple "just clone everything" approach, but also avoid duplicating strings.

That interning code is not the same that the compiler uses for interning string literals.

Null termination and `cstring` 11.8

In `<odin>/vendor` there are bindings to a bunch of libraries written in the C programming language. Such libraries may use the type `cstring` when they expect a string.

For example, here's the signature of a procedure from `vendor/raylib/raylib.odin` that draws text on the screen:

```
DrawText :: proc(text: cstring, posX, posY: c.int, fontSize: c.int, color: Color)
```

The `text` parameter is of type `cstring`. The `cstring` type requires its data to be *null terminated*. A null-terminated string has a zero at the end of the string's data. To figure out the length of a null-terminated string, the code needs to iterate through the string and count until it hits the null terminator. That's what the C function `strlen(a_string)` does.

Null termination needs a zero, not the character `0`. A rune that represents the character `0` has the value `48`, which is not zero.

Odin's `string` type does not need a null terminator. Instead, it has a `len` field (see **Raw String**). When you write `len(some_string)`, it simply gives you that value.

However, Odin can cast all string constants to both `string` and `cstring`. They'll use the same interned string data (data placed in `rodata` by compiler). For example, the following two string variables will use the same data, even though the `cstring` version requires null termination:

```
STR :: "Hellope!"
str: string = STR
cstr: cstring = STR

// Print the address of the string's data
fmt.println(raw_data(str)) // 0x7FF7F6C1D7E0
fmt.println(rawptr(cstr))  // 0x7FF7F6C1D7E0

// The value will differ on your computer,
// but both will have the same value.
```

Thanks to de-duplication of interned strings, they'll share the same data even if you write:

```
str: string = "Hellope!"
cstr: cstring = "Hellope!"
```

This is achieved by simply storing all interned string literals with a null terminator.

However, don't assume that all variables of type `string` are null terminated! For example, the result of `my_str := strings.clone("Hellope!")` will be of type `string`. Its data is a dynamically allocated clone of the literal `"Hellope!"`. But there will be no zero at the end of the data. A variable of type `string` *might* be null terminated, since it might originate from something like `my_str := "Hellope!"`. But there is no guarantee.

You might think: Why doesn't Odin just include a null terminator in all strings? Among other issues, it would break *string slicing*. The following works without any allocations:

```
sub_str := str[5:10]
```

If it needed to insert a null terminator at the end of the slice, then an allocation would be required each time you slice a string.

What all this means in practice is: When you interface with C libraries, you can use string constants directly:

```
rl.DrawText("Hellope!", 50, 50, 10, rl.WHITE)
```

```
import rl "vendor:raylib"
```

But if you have a variable of type `string`, then you need to convert it, which can be done like so:

```
str: string = "Hellope!"
cstr := strings.clone_to_cstring(str)
rl.DrawText(cstr, 50, 50, 10, rl.WHITE)
delete(cstr)
```

Here `cstr` is allocated using `context.allocator` and manually deallocated. A more efficient and less error-prone method is using the temporary allocator:

```
str: string = "Hellope!"
rl.DrawText(strings.clone_to_cstring(str, context.temp_allocator), 50, 50, 10,
rl.WHITE)
```

Using the temp allocator is often efficient since it allocates into a growing arena. Using the default heap allocator (`context.allocator`) is slower because it results in many separate heap allocations.

Also: Remember to **clear the temp allocator** at some point in your program!

Finally, if you can *100% absolutely really promise* that a `string` variable has a null terminator, then you can convert it to a `cstring` without any allocation using **`strings.unsafe_string_to_cstring`**.

Grapheme clusters 11.9

In **section 11.2**, I mentioned that there are some cases where one rune (one Unicode code point) is not equivalent to one character. In the same section, we saw some English and Chinese examples. In those examples, every character was a single rune. However, this is not true for *grapheme clusters*. A grapheme cluster is something that appears as a single character on the screen, but actually consists of several runes. Here's an example:

```
str := "g̈"

for r in str {
    fmt.println(r)
}
```

Within this block of code, g̈ may not show up correctly. It's supposed to be a g with two dots on top. I've seen e-readers that can't render it properly. I'm trying to write a bit about the tricky parts of UTF-8, and I run into compatibility issues. The irony!

The code will loop through `str` and print each rune on a separate line. The output is:

```
g
̈
```

So this g̈ is a grapheme cluster: It consists of two separate runes! There are procedures to work with grapheme clusters in `core/unicode/utf8/grapheme.odin`. For example, you can use `utf8.decode_grapheme_clusters(str)` to get an array of all the grapheme clusters in a string. Beware that `decode_grapheme_clusters` allocates memory and is expensive to run.

A letter like `ä` may also look like a grapheme cluster. And it might be. There is a single-rune representation of `ä`, which is by far the most common way to write it. But you can also write it as two runes by first writing an `a` and then writing the rune that represents the two dots.

Some text editors do "grapheme cluster normalization". It will replace any grapheme cluster that is possible to represent with a single rune.

You may or may not have to worry too much about grapheme clusters. Are you making a text editor? Then you'll need to worry. Are you making a game where all the text is pre-defined? Then you might not need to worry, unless you localize your game into a language that depends on grapheme clusters. Writing code that makes it possible to edit text containing grapheme clusters, is tricky. Writing code that renders text containing grapheme clusters, is less tricky (but not trivial).

Why does the slice operator count bytes? 11.10

When slicing a string, such as `sub_str := str[1:5]`, then it may seem strange that the slice operator uses byte indices. Why doesn't it use rune indices? Let's discuss why using rune indices would be problematic.

The `Raw_String` needs to know the address of where the string starts. Because of the UTF-8 encoding, each rune is encoded using 1 to 4 bytes of memory. If the slice operator used rune

indices, then it could not directly grab the wanted memory address. Instead, it would have to iterate over the string and decode runes, counting them.

Odin could avoid such rune counting by having the string store an additional array containing the byte index of each rune. But that would complicate the `Raw_String` type and any code that creates strings and string slices. But there are other problems as well.

As we have seen, runes don't always represent a single character on the screen. A string may contain grapheme clusters, which use multiple runes for a single character. With this in mind, all the effort spent on making it possible to slice using rune indices would still be in vain. Making strings aware of their grapheme clusters and using some kind of grapheme-cluster indices is a very bad idea due to the complexity of just decoding grapheme clusters.

With all this in mind, it is not so weird that the string slice operator uses byte indices: It is the only alternative that doesn't make weird assumptions. It is still possible to reason about runes and grapheme clusters when you need to.

Under the hood: Manually decoding UTF-8 11.11

Let's take a look at what the memory of a UTF-8 string looks like and how you can manually decode it into runes. A loop such as `for r in str {}` does this decoding for us. This section will help us understand what work that loop actually does.

Here is some code that iterates a string and prints each rune on a separate line:

```
str := "Cät=猫"

for r in str {
    // Prints the characters of `str` on separate lines.
    fmt.println(r)
}
```

We want to investigate the bytes of the UTF-8 encoded string, without decoding them into runes. Therefore, we transmute the string to a slice of `u8` (byte). We can then loop through the bytes and print them in binary:

```
str := "Cät=猫"
str_bytes := transmute([]u8)(str)

for b in str_bytes {
```

```
    fmt.printfln("%8b (%v)", b, b)
}
```

`transmute([]u8)(str)` takes the memory of the string and "pretends" it to be of type `[]u8`. A slice of bytes and a string are more or less the same thing. But no automatic UTF-8 decoding will happen when we loop through the `[]u8` version. This makes it possible for us to inspect the encoded bytes.

The `%8b` format string tells `printfln` to print the number in binary. The `8` in `%8b` tells it to add zero padding on the left, so we always see 8 binary bits.

The output is:

```
01000011 (67)
11000011 (195)
10100100 (164)
01110100 (116)
00111101 (61)
11100111 (231)
10001100 (140)
10101011 (171)
```

Let's manually go through the bytes, construct Unicode runes from them and print each one to verify that we did it correctly.

Rune C (byte `01000011`)

`01000011`: The first bit (the left-most bit) is zero. This means that this is a single-byte rune. The byte just needs to be cast into a rune in order to complete the decoding:

`fmt.println(rune(0b01000011))` prints `C`.

The `0b` prefix tells the compiler that we have provided a binary value.

Note that a rune is just a 32 bit number. But we cast to `rune` instead of `i32`. This is so `fmt.println` shows it as a character instead of a number. But memory-wise, `i32` and `rune` are identical.

Rune ä (bytes `11000011` and `10100100`)

`11000011`: Start with `110`. This means that this is the beginning of a multi-byte rune. The number of bytes to expect for this rune is two, because there are two `1`s. We throw away the initial `110` and remember the rest: `00011`. Then we move to the next byte.

`10100100`: Starts with `10`. This means that this is a continuation of a multi-byte rune. The previous byte told us to expect two bytes (including the initial one), so this is the last byte. We throw away the `10` and combine the remainder with the result from the previous byte: `00011` and `100100`. Simply copy-paste them together: `00011100100`. Try printing it:

`fmt.println(rune(0b00011100100))` prints `ä`.

As you see, the main work in decoding UTF-8 is the following: Check if the byte starts with a `1`. If it does, then you count how many `1`s there are until the first `0`. Then use that information to classify the byte.

Rune t (byte `01110100`)

`01110100`: The first bit is zero, this is a single-byte rune. Just cast into `rune` and print:

`fmt.println(rune(0b01110100))` prints `t`

Here you also see why UTF-8 is backwards compatible with the first 128 values of ASCII. ASCII uses a single byte for each character. UTF-8 can also represent some characters using a single byte, if the first bit is zero. But then there are only 7 out of 8 bits left. 7 bits gives us a numeric range of `0-127`, or 128 values. Those 128 values have been chosen to represent the same characters as in ASCII.

Rune = (byte `00111101`)

`00111101`: The first bit zero. Just cast into `rune` and print:

`fmt.println(rune(0b00111101))` prints `=`

Rune 猫 (bytes `11100111`, `10001100` and `10101011`)

`11100111`: Starts with `1110`: Three `1`s mean that this is the start of a triple-byte rune. Throw away `1110` and remember the rest: `0111`.

`10001100`: Starts with `10`: This is a continued multi-byte rune. Throw away the `10` and remember the rest: `001100`

UTF-8 is said to be "self-synchronizing": Imagine looking at a random byte in a UTF-8 string and noticing that it starts with the bits `10`. Those starting bits are only used for "continuation bytes". You can always find where the rune starts by stepping backwards in the string, byte-by-byte, looking for the first byte that doesn't start with `10`.

`10101011`: Starts with `10`: Continued multi-byte rune. Throw away the `10` and remember the rest: `101011`. This is the last byte in the three-byte sequence. Copy-paste the bits together:

`0111, 001100, 101011 -> 0111001100101011`. Cast to `rune` and print:

`fmt.println(rune(0b0111001100101011))` prints 猫.

And we are done!

We didn't see any four-byte runes. But it's the same dance: The first byte starts with `11110` (four `1`s). Then follows three bytes starting with `10`.

As an exercise, try adding an emoji to the string and manually decode it. Many emojis are encoded using four bytes. However, some emojis may use multiple runes (they are **grapheme clusters**).

You can verify our results by looping through the string as usual (loop through `str`, not `str_bytes`) and printing the automatically decoded runes as binary numbers:

```
str := "Cät=猫"

for r in str {
    fmt.printfln("%b", r)
}
```

It will print the same binary numbers that we found:

```
1000011 (C)
11100100 (ä)
1110100 (t)
111101 (=)
111001100101011 (猫)
```

Any padding of zeroes on the left is not present here. Zero padding on the left of a binary number can often be thrown away, unless you want to keep the zeroes to hint at a specific bit width.

A real UTF-8 decoder would do some additional error checks while decoding. But the knowledge in this section is sufficient to investigate the memory of most UTF-8 strings.

There's a procedure in `core:unicode/utf8` called **`decode_rune`**. Check that one out for an actual implementation of what we have discussed in this chapter.

I've also made a video on this topic:

https://www.youtube.com/watch?v=Zl1Gs8iUpi0

UTF-16: Windows API example 11.12

UTF-8 is by far the most common standard for encoding text. For historical reasons, there are some technologies and popular APIs that use UTF-16. UTF-16 is similar to UTF-8, but represents the text as a series of 16 bit values instead of a series of 8 bit values. This means that a single character uses at least 2 bytes (16 bits).

An important expression to introduce here is *code unit* (note: different from *code point*). In UTF-8 the code unit is 1 byte (8 bits). In UTF-16 the code unit is 2 bytes (16 bits). It's the smallest part of an encoded Unicode string.

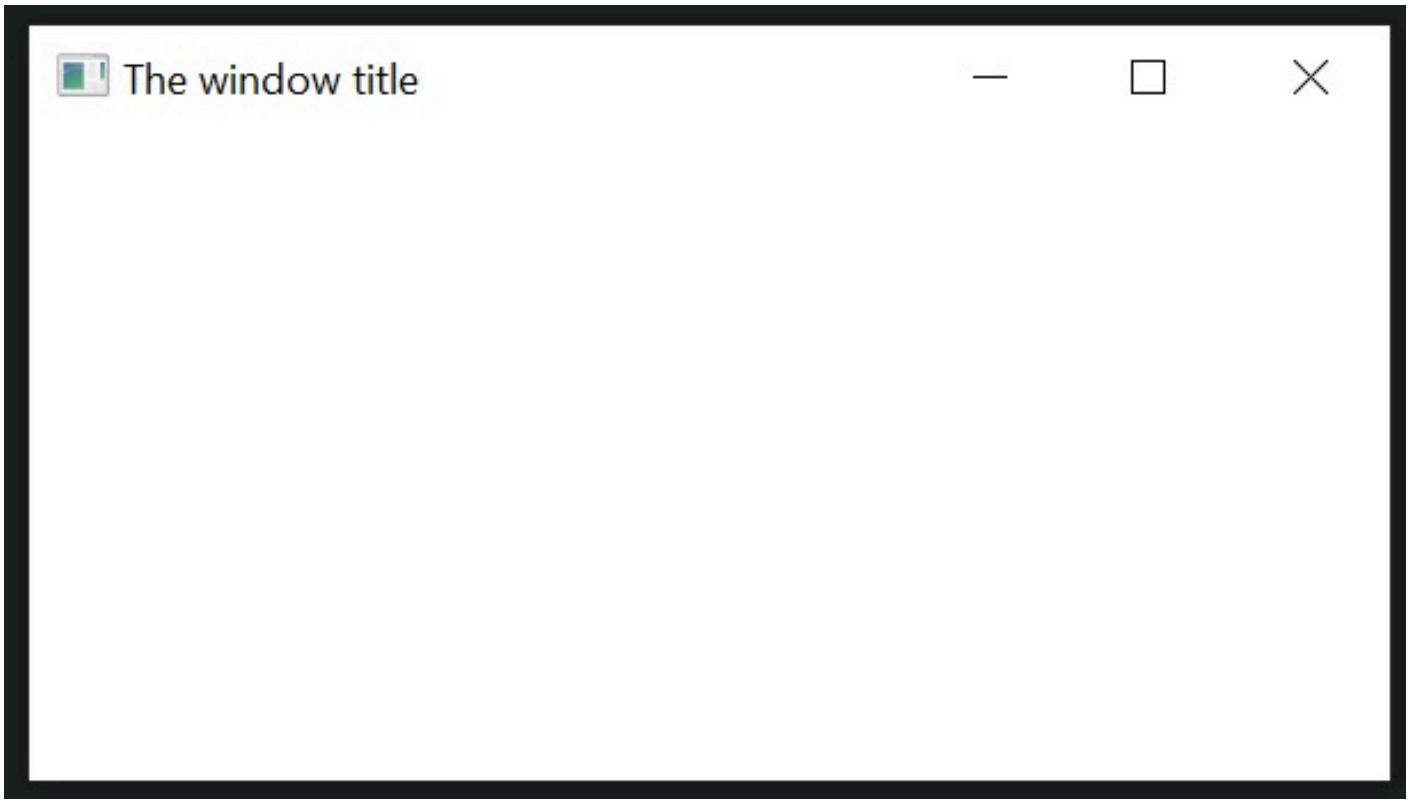
> We haven't talked about code units before because it's easier to just say "byte" when talking about UTF-8.

Regardless of if you use UTF-8 or UTF-16, a decoded rune is still 4 bytes. UTF-8 encodes the rune using 1, 2, 3 or 4 bytes of memory. UTF-16 encodes the rune using 2 or 4 bytes of memory. Again, it's just a way to save memory, so you don't have to store the full 4 bytes for each single rune.

The rune `A` has the value `65`. In UTF-8 this can be encoded using 1 byte of memory. But it requires 2 bytes of memory in UTF-16. Because of these memory layout differences, UTF-16 and UTF-8 are incompatible. You have to convert the string data from one format to the other. Odin has two types called `string16` and `cstring16` that can be used to represent UTF-16 encoded text. Most programs will primarily use the normal UTF-8 `string` type and only convert to `string16` when you need to interface with some API that requires UTF-16 strings.

The Windows OS API (also known as *win32*) uses UTF-16. Additionally, it uses null-terminated strings (C strings). In other words, we need to use the `cstring16` type when interfacing with it. Let's look at some examples where we set and get the window title (the name at the top of the window).

> These examples aren't trying to teach you the Windows API. Rather, they are good examples of how to convert between UTF-8 and UTF-16.



A window in the Windows OS. The code in this section is concerned with setting and getting the window title and doing the appropriate UTF-8 to UTF-16 conversions.

The following procedure from `core:sys/windows` changes the title of a window:

```
SetWindowTextW :: proc(hWnd: HWND, lpString: LPCWSTR) -> BOOL
```

The Windows API refers to these UTF-16 strings as *wide strings*, or wstrings. Hence the WSTR in the type name. Also, that's why there is a `W` at the end of `SetWindowTextW`.

The `hWnd` parameter is a window handle. The `lpString` parameter is assumed to be one of those null-terminated UTF-16 strings (`cstring16`). The type is `LPCWSTR`, but that is just an alias of `wstring`, which in turn is just an alias of `cstring16`.

Setting the window title using a string constant *11.12.1*

You can set the window title to a hard-coded string literal like so:

```
windows.SetWindowTextW(hwnd, "My Great Program")
```

This works without any kind of explicit conversion because "My Great Program" is a constant of type Untyped String. Such a string can be implicitly cast to `cstring16` (as well as `string`, `cstring` and `string16`). This is equivalent to using a named constant like so:

```
TITLE :: "My Great Program"
windows.SetWindowTextW(hwnd, TITLE)
```

Setting the window title using a string variable *11.12.2*

However, the following will not work:

```
title := "My Great Program"
windows.SetWindowTextW(hwnd, title)
```

You'll get this error:

```
Error: Cannot assign value 'title' of type 'string' to 'cstring16' in a procedure
argument
```

In this example, `title` is not a constant, but a variable. Its type is `string`, which uses UTF-8. One way to fix it is to force `title` to be of type `cstring16`:

```
title: cstring16 = "My Great Program"
windows.SetWindowTextW(hwnd, title)
```

Again, this works because the constant `"My Great Program"` is of type Untyped String, which can be implicitly cast into a `cstring16`.

However, say that you are calling `SetWindowTextW` within a procedure where you are not allowed to change the parameter type:

```
set_title :: proc(title: string) {
    // Assume `hwnd` is a global variable.
    windows.SetWindowTextW(hwnd, title)
}
```

You'll again get the error `Cannot assign value 'title' of type 'string' to 'cstring16'`. Adding a `cstring16` cast would *not* help:

```
set_title :: proc(title: string) {
    windows.SetWindowTextW(hwnd, cstring16(title))
}
```


You may wonder why the cast from Untyped String to `cstring16` works, but the cast from `string` to `cstring16` does not. As noted before, the underlying data formats of `string16` and `string` are incompatible. However, at compile-time it is possible for the compiler to create a `cstring16` based on the string constant, and put the data of that `cstring16` into the read-only data block of the program. But

a cast such as `cstring16(title)`, where `title` is a *variable* (*not* a constant), does not happen at compile-time. The compiler does not yet know what value `title` contains.

The data of the `string` parameter is incompatible with `cstring16`. We have a UTF-8 string, but a UTF-16 string is required. In order to convert between them we need an explicit conversion step, which may involve an allocation.

We can use `windows.utf8_to_wstring` to do this conversion. The following procedure will work:

```
set_title :: proc(title: string) {
    title_16 := windows.utf8_to_wstring(title)
    windows.SetWindowTextW(hwnd, title_16)
}
```

`utf8_to_wstring` both does the conversion to UTF-16 and also adds in the null terminator, which results in a string of type `cstring16`. It allocates dynamic memory, but uses the temp allocator by default.

Fetching the window title *11.12.3*

You can get the current window title using `GetWindowTextW`:

```
GetWindowTextW :: proc(hWnd: HWND, lpString: LPWSTR, nMaxCount: INT) -> INT
```

This procedure wants some memory to write the window title into. That's `lpString`. `nMaxCount` says how many characters it is possible to fit into `lpString`.

Note how the type of `lpString` is `LPWSTR`. Compare it to `SetWindowTextW`, which has the parameter `lpString: LPCWSTR`. One has `C` in the type name, and the other does not. As stated before, `LPCWSTR` is just an alias of `cstring16`. However, `LPWSTR` is an alias of `[^]u16`. The extra `C` stands for "constant", as in "the procedure won't modify it" or "immutable". Odin strings (any value of type `string`, `cstring`, `string16` or `cstring16`) should always be considered immutable. If you want to modify some string data, then you'd reason about an array of *code units*. That means `[]u8` in the case of UTF-8 and `[]u16` in the case of UTF-16. Here we are interfacing with a C library, so we pass `[^]u16` rather than `[]u16`.

Fetch the title and turn it into an Odin string like this:

```
title_data: [128]u16
title_len := windows.GetWindowTextW(hwnd, raw_data(&title_data), len(title_data))
title_str, title_str_err :=
windows.wstring_to_utf8(cstring16(raw_data(&title_data)), int(title_len))
```

```
if title_str_err == nil {
    fmt.println(title_str)
}
```

`title_data` is a stack allocated fixed array with room for `128` UTF-16 code units. We send `title_data` into `GetWindowTextW`, which will copy window title into it. If it does not fit within `128` code units, then we'll only get part of the title.

Then we use `windows.wstring_to_utf8` to convert this UTF-16 string into an Odin UTF-8 string. `windows.wstring_to_utf8` uses the temp allocator by default.

If you want to be sure you always get the full window title, then you could temporary allocate a slice with enough room for all of the code units. Here's how we can do that:

```
title_data_len := windows.GetWindowTextLengthW(hwnd) + 1
title_data := make([]u16, title_data_len, context.temp_allocator)
title_len := windows.GetWindowTextW(hwnd, raw_data(title_data), title_data_len)
title_str, title_str_err :=
windows.wstring_to_utf8(cstring16(raw_data(title_data)), int(title_len))

if title_str_err == nil {
    fmt.println(title_str)
}
```

In the code above, `GetWindowTextLengthW` returns the size of the current window title. That number is how many UTF-16 code units you need to store the text. We also add `1` to the result, because we need room for a null terminator at the end.

> In both UTF-8 and UTF-16 the null terminator is a single code unit of value `0`. This just means a single `u8` of value `0` in the case of UTF-8, and a single `u16` of value `0` in the case of UTF-16.

`title_data := make([]u16, title_data_len, context.temp_allocator)` makes a slice of `u16` elements that has room for the whole window title, plus the null terminator. It is temporary allocated. The rest of the code is mostly the same.

You might notice that we write `raw_data(title_data)` in this example and `raw_data(&title_data)` in the previous example. This is because **the `data` field of the slice already is a pointer**. But the fixed array is just a stack-allocated value. So we add in the `&` so that `raw_data` can return a pointer to the first element of that array. Don't worry though, `raw_data` would error if you used without the `&` in combination with a fixed array.

> `raw_data(title_data)` is equivalent to doing `&title_data[0]`, but it won't crash if the length of the slice is zero. Instead it would return `nil`.

Chapter 12

Implicit context

We've seen `context.allocator` and `context.temp_allocator` a bunch of times. It has become time to talk about what this `context` thing is!

The `context` is often referred to as the Implicit Context. We say that it is implicit because it is automatically passed along to any procedure that you call. This means that when you do something like this:

```
fmt.println("Hellope!")
```

then `fmt.println` is automatically fed a hidden parameter called `context`. Its value is based on whatever value `context` has in the current scope.

If you do *not* want a procedure to be fed the context you can write `some_proc :: proc "contextless" () {}`. For some super-often run procedures it can be a slight optimization. `"contextless"` is one of several possible *calling conventions*, another one is `"c"`, for interfacing with C libraries. I talk more about the C calling convention in the chapter on **making C library bindings**.

The type of `context` is a struct that looks like this:

```
Context :: struct {
    allocator:              Allocator,
    temp_allocator:         Allocator,
    assertion_failure_proc: Assertion_Failure_Proc,
    logger:                 Logger,
    random_generator:       Random_Generator,

    user_ptr:   rawptr,
    user_index: int,

    // Internal use only
    _internal: rawptr,
}
```

See **base/runtime/core.odin**

The fields in there are:

- `allocator`: Default memory allocator.
- `temp_allocator`: Default temporary memory allocator.

- `assertion_failure_proc`: The procedure that is run when the `condition` in `assert(condition, "Error message")` evaluates to false.
- `logger`: Default logger, decides what happens when you use the procedures in `core:log`.
- `random_generator`: Defines the random generator used by for example `core:math/rand`.
- `user_ptr`, `user_index`: Custom extra data that you can use in any way you want.

`context` is automatically passed along to any procedure you call. You can modify fields in the context before calling a procedure, in order to make that procedure use different allocators, loggers etc. As we shall see, there are situations where this is a good idea and situations where it is better to pass explicit parameters.

When a scope ends, then any changes you've made to the context are reverted. The context will again look like it did at the beginning of that scope.

The rest of this chapter will talk about each of the fields of `context` and how you use them.

`context.allocator` 12.1

Say that you have this code:

```
do_work :: proc() {
    make_lots_of_ints :: proc() -> []int {
        ints := make([]int, 4096)

        // Give the numbers of `ints` some
        // values.
        for &v, idx in ints {
            v = idx * 4
        }

        return ints
    }

    my_ints := make_lots_of_ints()
}
```

`make_lots_of_ints` dynamically allocates a slice of 4096 integers and sets them to some interesting values, and thereafter returns the slice. Say that you'd prefer to have this slice allocated using the temp allocator. We know that we can do that by changing

```
ints := make([]int, 4096)
```

to

```
ints := make([]int, 4096, context.temp_allocator)
```

However, let's pretend `make_lots_of_ints` is a procedure that lives somewhere else. What if we don't have access to that code, so we can't modify it. How can we then make it use the temp allocator?

We can modify `context.allocator` before calling it:

```
do_work :: proc() {
    make_lots_of_ints :: proc() -> []int {
        ints := make([]int, 4096)

        // Give the numbers of `ints` some
        // values.
        for &v, idx in ints {
            v = idx * 4
        }

        return ints
    }

    context.allocator = context.temp_allocator
    my_ints := make_lots_of_ints()
}
```

Note the line

```
context.allocator = context.temp_allocator
```

And also remember that `context` is automatically passed along into `make_lots_of_ints`. So within `make_lots_of_ints`, whenever `context.allocator` is used, it is actually using the temporary allocator. Since `make` defaults to `context.allocator`, then it will now end up using the temporary allocator.

Swapping out the allocator like this makes it possible to inject an allocator into a procedure where you normally don't have access to changing the allocator.

Setting `context.allocator` *vs passing explicit allocators* *12.1.1*

I recommend that you make procedures that allocate memory have an explicit `allocator` parameter. The example in the previous section would then look like this:

```
do_work :: proc() {
    make_lots_of_ints :: proc(allocator := context.allocator) -> []int {
        ints := make([]int, 4096, allocator)

        // Give the numbers of `ints` some
        // values.
        for &v, idx in ints {
            v = idx * 4
        }

        return ints
    }

    my_ints := make_lots_of_ints(context.temp_allocator)
}
```

Here we see that `make_lots_of_ints` has an allocator parameter with the default value `context.allocator`. This allocator is then passed to `make`. This means that we can call `make_lots_of_ints` without specifying any allocator if we are happy with the default. But we can also override it, which we do when we call `make_lots_of_ints`:

```
my_ints := make_lots_of_ints(context.temp_allocator)
```

Having an allocator parameter on a procedure that allocates memory is a good *documentation* to the programmer that uses the procedure. It gives the programmer a hint that the return value will be allocated. This is clearer than setting `context.allocator` before calling `make_lots_of_ints`.

Also, when you modify `context.allocator` then that will affect the remaining code in the scope. Using an explicit allocator parameter avoids that.

I recommend only overriding `context.allocator` in two cases:

1. When you call a procedure that allocates memory using `context.allocator` and you want to use some other allocator, but it doesn't have any `allocator` parameter that you can override.
2. When you really want *all* the code within a scope to use that allocator. A common example is to swap out `context.allocator` in the first few lines of the `main` procedure. This effectively makes your whole program use some other allocator. We'll see this when we talk about how to set up the tracking allocator in **chapter 13.1**.

context.temp_allocator 12.2

In chapter 9.7 we saw how to use `context.temp_allocator`. But what about modifying that field on the context? Just like modifying `context.allocator` before calling a procedure, you could also swap out the temporary allocator. In practice this doesn't happen as often.

The most common reason to modify `context.temp_allocator` would be because you want your whole program to use some other temporary allocator. This would mean that you assign some other allocator to `context.temp_allocator` in the beginning of the `main` procedure, so that the rest of the program uses that temporary allocator.

In most cases, the default temporary allocator is fine.

context.assertion_failure_proc 12.3

Assertions ensure that something that you expect to be true actually is true.

If you do this:

```
number := 5
assert(number == 7, "Number has wrong value")
```

Then the program will crash *on purpose* because the condition of the assert was false (`number` wasn't `7`).

You can modify `context.assertion_failure_proc` to alter what happens when an assertion fails. Here's a small program that modifies it when `main` starts, so that the rest of the program uses the procedure `assert_fail` when an assertion fails:

```
package assertion_test

import "core:fmt"
import "base:runtime"

assert_fail :: proc(prefix, message: string, loc := #caller_location) -> ! {
    fmt.printfln("Oh no, an assertion at line: %v", loc)
    fmt.println(message)
    runtime.trap()
}

main :: proc() {
```

```
    context.assertion_failure_proc = assert_fail
    number := 5
    assert(number == 7, "Number has wrong value")
}
```

Running this program will print something like:

```
Oh no, an assertion at line: C:/code/assertion_test/main.odin(15:2)
Number has wrong value
```

Note the `!` at the end of

```
assert_fail :: proc(prefix, message: string, loc := #caller_location) -> ! {
```

This means that this procedure will not return, because we are expecting it to crash the program. We say that this procedure is a *diverging* procedure. This means that the procedure must end with calling some other diverging procedure. In this case we call `runtime.trap()`, which is a diverging procedure that will tell the operating system that some kind of failure has occurred.

In many 'release builds' of software, asserts are disabled to speed things up. That can be done using the `-disable-assert` compilation flag. That will make all calls to `assert` always succeed. If you need an assert that always works, then you can use `ensure(condition, "Failure message)`. `ensure` will always work, even when `-disable-assert` is used. It also uses the `assertion_failure_proc`.

`context.logger` 12.4

Odin comes with logger functionality in `core:log`. Instead of using `fmt.println` everywhere you can use `log.info` and `log.error` etc to log events. You can set up your logger to go to a file, or to a console or to wherever you want.

> Note that `context.logger` is *not* set by default when the program starts. So the stuff in `core:log` won't do anything unless you set it up.

To make the logger messages appear in the console, just add `context.logger = log.create_console_logger()` at the start of your program:

```
package logger_test

import "core:log"
```

```
main :: proc() {
    context.logger = log.create_console_logger()

    log.info("Program started")

    // Rest of program goes here,
    // it will use context.logger whenever
    // you run `log.info`, `log.error` etc.

    log.destroy_console_logger(context.logger)
}
```

Since we set `context.logger` at the start of `main`, all subsequent procedure calls (your whole program), will use this logger as well.

Logging to a file requires a bit more setup:

```
package main

import "core:log"
import "core:os"

main :: proc() {
    logh, logh_err := os.open("log.txt", {.Create, .Trunc, .Read, .Write })

    if logh_err == os.ERROR_NONE {
        os.stdout = logh
        os.stderr = logh
    }

    logger := logh_err == os.ERROR_NONE ? log.create_file_logger(logh) :
log.create_console_logger()
    context.logger = logger

    // Rest of program goes here, it will
    // use `context.logger` whenever you
    // run `log.info`, `log.error` etc.
    //
    // Also, fmt.println will be redirected
    // to the logger, due to the
    // `os.stdout = logh` lines above.
    all_the_code_in_your_program()

    if logh_err == os.ERROR_NONE {
        log.destroy_file_logger(logger)
```

```
    } else {
        log.destroy_console_logger(logger)
    }
}
```

This uses the `condition ? if_true : if_false` syntax. It's a quick way to choose between two values based on a condition.

The code above opens `log.txt` for writing using `os.open`. It will be cleared each time the program starts, thanks to the `.Trunc` flag. `logh_err` tells us if `os.open` successfully opened the file. In the case of success the code goes ahead and creates the file logger using `log.create_file_logger(logh)`. If it fails to open the file then it uses a console logger instead.

`Trunc` refers to Truncate, another word for "shorten". In this case it means that we start over with an empty file.

Regardless of which logger it was able to create, `context.logger` is set to that value.

This part of the code is optional:

```
if logh_err == os.ERROR_NONE {
    os.stdout = logh
    os.stderr = logh
}
```

What it does is redirect `fmt.printfln()` and similar procedures, so they instead go to the log file. However, if you have the logger setup properly, then I recommend just using the procedures in the `core:log` package whenever you can.

Available logging procedures are:

```
log.debug("message")
log.info("message")
log.warn("message")
log.error("message")
log.fatal("message")
log.panic("message")
```

These are in order for "severity". `log.panic` is special in the sense that it logs, but unlike the others it also crashes the program on purpose.

All those logging procedures also have a version that ends with `f`, such as `log.errorf`. Those versions use a **format string** so you can do stuff like `log.errorf("The thing called %v is`

```
broken", name_of_thing).
```

Have a look in `<odin>/core/log/log.odin` to see how the log systems works in detail.

context.random_generator 12.5

There is a random number generator set up by default. However, you can modify `context.random_generator`, changing how any code called after that point generates random numbers. While you can write your own random number generator, a common thing is to use the default one, but seed it differently:

```
package random_generator_example

import "base:runtime"
import "core:fmt"
import "core:math/rand"

main :: proc() {
    random_state := rand.create(42)
    context.random_generator = runtime.default_random_generator(&random_state)
    fmt.println(rand.int_max(100))
}
```

The seed of a random generator controls the "starting state" of the generator. If you use the same seed on two different computers and then run some code that is completely controlled by the random generator, then both computers will get the same result.

If you run the above then it will print `48`, just like it did for me. This is because we are forcing the seed of the random generator to be `42`.

If we don't override the random generator, then `rand.int_max()` will probably give a different value each time you run the program. This is because the seed is automatically set when the program starts up.

`rand.int_max` uses `context.random_generator` by default, but you could also explicitly feed it any other random number generator.

Instead of replacing `context.random_generator`, you can also just call `rand.reset` with your seed:

```
rand.reset(42)
// Should also print 48
fmt.println(rand.int_max(100))
```

It resets `context.random_generator` to use the seed `42`.

`context.user_ptr` and `context.user_index` 12.6

`context.user_ptr` and `context.user_index` are two extra fields on the context that you can use to implicitly pass a pointer or an integer. They are useful when you need to send along some data to a callback. This way we can provide extra information into the callback. In the example below, note how we set `context.user_ptr` before calling `slice.sort_by`. We can thereby use that pointer within the `sorting_proc` callback.

```
numbers := []int { 2, 7, 42, 1}

Sort_Settings :: struct {
    put_at_end: int,
}

sorting_proc :: proc(i, j: int) -> bool {
    sort_settings := (^Sort_Settings)(context.user_ptr)

    if sort_settings != nil {
        if i == sort_settings.put_at_end {
            return false
        }

        if j == sort_settings.put_at_end {
            return true
        }
    }

    return i < j
}

sort_settings := Sort_Settings {
    put_at_end = 2,
}

context.user_ptr = &sort_settings
```

```
slice.sort_by(numbers, sorting_proc)
fmt.println(numbers) // [1, 7, 42, 2]
```

```
import "core:slice"
```

How the context is passed 12.7

Each time you call a procedure the current context is passed into it. However, it does not copy the current context, it just passes a pointer to it.

If you do something like this:

```
context.allocator = context.temp_allocator
my_ints := make_lots_of_ints()
```

Then there is some behind-the-scenes magic that takes the current context and copies it to a new stack variable. All subsequent procedure calls within that scope will be fed a pointer to this new context instead of the old one. When the scope ends, it reverts to using the old context pointer when calling procedures.

Chapter 13

Making manual memory management easier

Manual memory management can be quite overwhelming for the beginner. Let's take a look at two tools that can make it easier:

- Tracking memory leaks using the *tracking allocator*.
- Using *arena allocators* to group allocations according to their lifetime.

When it comes to the arena allocator we'll also discuss the different types of arena allocators that come with Odin.

Before we begin I want to mention that we've already discussed one concept that makes manual memory management easier: The **temporary allocator**. By doing short-lived allocations using the temporary allocator, you can greatly reduce the number of manual deallocations in your program.

Tracking memory leaks 13.1

Let's look at how to use Odin's tracking allocator. It's an allocator that helps you find memory leaks. This makes memory management easier since it's one less thing for us to worry about. Let's first talk about what a leak *is* and what it *is not*, and whether or not we can track them:

- If your program *periodically* allocates memory by for example creating dynamic arrays or allocating using `new`, but then never deallocates that memory using `delete` or `free`, then the program's memory usage will continuously grow. This is a memory leak and the tracking allocator can help you find these.

There are some commercial programs and video games that will eventually crash due to memory leaks, given that you just let them run for long enough.

- If your program allocates memory once at startup and forgets to free it, then I would not label this as a memory leak since the memory usage isn't continuously growing. Nothing is "leaking"! The memory allocated once at startup will be deallocated when the program shuts down. A program can't continue to hog memory after it has shut down, so you don't have to worry about that.

In fact, it can be good to skip deallocating these things, since deallocating them just makes your program take longer to shut down.

- Say that your program contains a dynamic array that is created at startup. It is meant to exist until your program shuts down. If the program runs for a long time and periodically adds things to this dynamic array, but erroneously never removes anything, then your program's memory usage will grow over time. This is a memory leak, but the tracking allocator cannot help with finding this. It's more of a logic error in your code. These kind of memory leaks can happen in both manually and automatically memory managed languages.

Setting up the tracking allocator *13.1.1*

Let's now set up the tracking allocator. Odin's core collection comes with one, you'll find it in the **`core:mem`** package. Below I show how to set it up:

This example is **adapted from the official overview**.

```
package main

import "core:fmt"
import "core:mem"

main :: proc() {
    when ODIN_DEBUG {
        track: mem.Tracking_Allocator
        mem.tracking_allocator_init(&track, context.allocator)
        context.allocator = mem.tracking_allocator(&track)

        defer {
            if len(track.allocation_map) > 0 {
                for _, entry in track.allocation_map {
                    fmt.eprintf("%v leaked %v bytes\n", entry.location,
entry.size)
                }
            }
            mem.tracking_allocator_destroy(&track)
        }
    }

    // rest of program goes here
}
```

In order for the tracking allocator to be set up, you must compile this program with the `-debug` compiler flag. This is because of the `when ODIN_DEBUG {}` block in `main`. I'll explain that in more detail soon.

Do something like

```
odin run . -debug
```

To test the tracking allocator, replace the line `// rest of program goes here` with the following line:

```
ints := make([]int, 4096)
```

Thereafter, run the program. When it shuts down, it should print the following:

```
C:/code/my_project/main.odin(27:10) leaked 32768 bytes
```

This message says that there is a leak of `32768 bytes` in `main.odin` at line `27`, column `10`. In order to fix the leak, go to the line it reported and try to figure out where to best add in your deallocation, in this case `delete(ints)`.

This isn't really a memory leak since it won't make our program's memory usage continuously grow. I said earlier that allocating something once at program startup and not freeing it isn't really a memory leak. This example is just meant to illustrate how the memory leak report looks.

In order to make these at-startup allocations not appear in the leak report, you can perform the allocations before setting up the tracking allocator.

However, still deallocating everything can be good if you want to keep third-party memory analysis programs happy. Just be aware that it is harmless to skip this deallocation.

How does it work? *13.1.2*

Let's go through what the code above does.

First we feed the value of `context.allocator` into `tracking_allocator_init`:

```
track: mem.Tracking_Allocator
mem.tracking_allocator_init(&track, context.allocator)
```

Since this code runs at the top of `main`, then `context.allocator` will have its default value, which is a heap allocator on **desktop platforms**. `tracking_allocator_init` "wraps" the allocator we give it and stores it inside the `track: mem.Tracking_Allocator` variable.

This line sets the tracking allocator on the context:

```
context.allocator = mem.tracking_allocator(&track)
```

> The procedure `mem.tracking_allocator` returns a value of type **`runtime.Allocator`**, which is the interface that allocators are expected to have.

Since this happens near the top of `main`, the rest of the program will use this allocator, due to the context being implicitly passed to any called procedures.

So now, whenever any allocation happens using `context.allocator` (for example, when a dynamic array tries to grow as part of `append` being called), then those allocations will go through the code of the tracking allocator. For each allocation, the tracking allocator will do these things:

1. The allocation is done like usual using the wrapped allocator (also known as the tracking allocator's *backing* allocator).
2. The source code location at which the allocation was done is recorded inside `allocation_map`, which is **a field of `mem.Tracking_Allocator`**.

Later, when you do some deallocation, such as `delete(some_dynamic_array)`, then the memory deallocation will also go through the tracking allocator. It will do the following:

1. The deallocation is done using the wrapped allocator.
2. The info about the allocation is removed from the `allocation_map`.

Therefore, whatever is left on the `allocation_map` at the end of `main` can be considered memory leaks. You can iterate over the `allocation_map` just before shutting down the program, and print a leak report. In our setup example this leak-report printing is done within a `defer` block:

```
defer {
    if len(track.allocation_map) > 0 {
        for _, entry in track.allocation_map {
            fmt.eprintf("%v leaked %v bytes\n", entry.location, entry.size)
        }
    }
    mem.tracking_allocator_destroy(&track)
}
```

I've shown before how `defer` can be used to **defer single lines of code**. But as shown here, we can also defer a whole block of code by putting `{}` after `defer`. That whole block of code will run at the end of the current scope. The code in there will check if anything is still left in `track.allocation_map`. For each thing left in there, it will print how many bytes and at what code location the leak happened.

Note that the whole tracking allocator setup, including the defer is wrapped in a `when` block:

```
main :: proc() {
    when ODIN_DEBUG {
        // Set up tracking allocator

        defer {
            // Check for leaks here
        }
    }

    // rest of program goes here
}
```

This is the layout of the tracking allocator example from the previous section, with the actual code omitted.

The code within a `when` block will only be included in the program if the condition is true. It's like a compile-time if statement. If the condition is false, then the code within the block will not be part of the program, at all.

In this case the condition is that `ODIN_DEBUG` is true. That constant is true if you run the compiler with the `-debug` flag set. So in non-debug mode the tracking allocator will *not* be set up. This is probably what you want, as the tracking allocator slows the program down a bit.

From C / C++

`when ODIN_DEBUG {`

is similar to doing

`#if defined(ODIN_DEBUG)`

in C.

There's a thing here that may confuse some readers: The `when ODIN_DEBUG {}` block looks like a scope. And the `defer` is within this `when` block. So wouldn't the `defer` happen at the end of that `when` block, before even reaching the comment that says `// rest of program goes here`? The answer is no. The curly braces associated with a `when` block do *not* set up a "real scope". It is only a block of code. So `defer` does not care about the `when` block ending. Also, any variables declared within a `when` block will exist after the `when` block ends.

Unless you get noticeably bad performance from using the tracking allocator, then I advise you to always run with the tracking allocator on in the 'development build' of your program. Disable it when you send out releases to users, which will happen automatically since those release builds should not use the `-debug` compiler flag.

The tracking allocator will also inform you if your code does any *bad frees*. A bad free means that the code tries to free memory that isn't allocated. The tracking allocator will panic on a

bad free (crash on purpose). But before crashing it will print a message telling you the file and line at which the bad free happened. You can override this behavior by assigning a different procedure to `track.bad_free_callback`. This is the default callback, but there is an alternative callback which adds the bad frees to an array instead of panicking. If you use the alternative callback, then you must make sure to check `track.bad_free_array`, otherwise the bad frees will go unnoticed.

> Fix your bad frees! They may cause crashes and memory corruption in the non-debug build of your program.
>
> Bad frees often happen because you try to deallocate memory that has already been deallocated. This is often referred to as a "double free". This can happen when you have two systems that both try to deallocate the same memory. Note that it is OK to `free` nil pointers and `delete` zeroed containers. So some bad frees can be avoided by zeroing out pointers and containers after deallocating them.

By using a tracking allocator I hope that you'll find dynamic memory allocations less scary. You'll be able to look at the reports, investigate where the memory leak started and in so doing learn more about when and how memory is actually allocated.

Finally, if you are still confused by anything, then you can watch this video I made about the tracking allocator:

https://www.youtube.com/watch?v=dg6qogN8kIE

Making memory leak reports clearer *13.1.3*

If you set up the tracking allocator like in the previous section, then there may still be some situations where it is unclear where a memory leak actually started. Here's an example of such a situation:

```
allocate_7 :: proc() -> ^int {
    number := new(int)
    number^ = 7
    return number
}

n1 := allocate_7()
n2 := allocate_7()
n3 := allocate_7()
```

If you add the code above at the end of the `main` procedure in the previous example and then run the program, then you'll get a memory leak report that looks like this:

```
C:/code/my_project/main.odin(29:13) leaked 8 bytes
C:/code/my_project/main.odin(29:13) leaked 8 bytes
C:/code/my_project/main.odin(29:13) leaked 8 bytes
```

As you see there are three leak reports, but they are all pointing at the same line: Line `29` (on my computer). At that line we find `number := new(int)`, which is inside `allocate_7`. It would be more useful if the leak report pointed at the three lines where we ran `allocate_7()`, since it is the responsibility of the code that called `allocate_7` to deallocate the memory.

You can fix this by adding the parameter `loc := #caller_location` to `allocate_7` and passing on `loc` to `new`:

```
allocate_7 :: proc(loc := #caller_location) -> ^int {
    number := new(int, loc = loc)
    number^ = 7
    return number
}
```

`loc := #caller_location` is a parameter that is automatically set to the source code location at which the procedure was called.

If you re-run the program now, you'll get this:

```
C:/code/my_project/main.odin(35:7) leaked 8 bytes
C:/code/my_project/main.odin(37:7) leaked 8 bytes
C:/code/my_project/main.odin(36:7) leaked 8 bytes
```

As you see, it now reports leaks at three different lines. These are the three calls to `allocate_7`. I recommend adding a `loc` parameter whenever you have a procedure that returns dynamically allocated memory, especially when the caller of the procedure is responsible for deallocating the memory.

The perceptive reader might have noted that in my case it reported line `35`, `37` and then `36`. They are out of order because the `allocation_map` inside `Tracking_Allocator` isn't sorted by line number, but rather by memory address. The dynamic allocations done in this example will end up at different addresses each time we run the program, so the order will change.

They are sorted by memory address because `allocation_map` is a `map` that uses `rawptr` as key, and maps are sorted by key.

Finally, and this is a bit off-topic: I would add in an allocator parameter to the `allocate_7` procedure and pass it on to `new`:

```
allocate_7 :: proc(allocator := context.allocator, loc := #caller_location) ->
^int {
    number := new(int, allocator, loc)
    number^ = 7
    return number
}
```

A good rule of thumb here is:

- If you make a procedure that dynamically allocates a return value; then add in an allocator parameter and use it when allocating the memory. A procedure having an allocator parameter is a good *hint* that its return value may be dynamically allocated. It also makes it possible to change the allocator without modifying `context.allocator`.
- Whenever you do have an allocator parameter, then also add a `loc := #caller_location` parameter and pass that on when allocating memory. The programmer that looks at memory leak reports is probably mostly interested in where they called a procedure that allocated memory, not how that procedure went about allocating memory.

Arena allocators 13.2

Arena allocators can be used to group allocations that share a common *lifetime*. A shared lifetime means that a couple of allocations should all be deallocated at the same time. This can simplify memory management since you only have a single arena to deallocate, compared to deallocating several objects one-by-one.

The **temp allocator** is actually an arena allocator: We destroy everything on it at the same time using `free_all(context.temp_allocator)`. However, you can create as many arena allocators as you want and use those arenas to define your own "groups" of allocations. These groups can then be deallocated with a single procedure call.

> If you have trouble using the temp allocator in your program because some things need to live on longer than other things, then it is a sign that those things do not share a common lifetime.
>
> Then it is probably better to use several separate arenas.

I've already shown one use case of arena allocators in Chapter 11's **"Deallocating strings mixed with string constants"**. Let's look at another example of a situation where an arena can be helpful.

Say that you have the concept of a *level* in a video game. A level is something that describes the current world that the player is exploring. In our example `Level` is a struct that contains the name of the level, a list of objects and some tiles. Something like this:

Tiles are grid-based, repeatable ground/wall/ceiling pieces than can be used to build a game world in a modular fashion.

```
Level :: struct {
    name: string,
    objects: []Game_Object,
    tiles: []Tiles,
}
```

Assume that `Game_Objects` and `Tiles` are structs that have some kind of position and some data that describes how they look. Exactly what they contain is not important. Now, say that you have a procedure that generates a level:

```
generate_level :: proc(name: string) -> Level {
    tiles := generate_tiles(context.allocator)
    objects := generate_objects(tiles, context.allocator)
    return {
        name = strings.clone(name, context.allocator),
        objects = objects,
        tiles = tiles,
    }
}
```

Here `generate_tiles` and `generate_objects` are two procedures that do some procedural generation, they return a slice each. Those slices are allocated using `context.allocator`. So `Level` then contains three separately allocated things: A clone of the string `name`, the `tiles` slice and the `objects` slice.

When you no longer need this level, then you probably want to deallocate those three dynamically allocated fields:

```
destroy_level :: proc(level: Level) {
    delete(level.name)
    delete(level.objects)
    delete(level.tiles)
}
```

We allocated these three things at the same time, and we are deallocating them at the same time. That means that they share a common lifetime.

> Three deallocations isn't a big deal. But sometimes this kind of "struct destroy procedure" can get quite messy. In a big struct there may be additional allocations. Perhaps each object has an allocated name as well. It can get out of hand quite easily.

Let's look at how we can use a *memory arena* to group the allocations and then deallocate them with a single procedure call. Here we will use the *growing virtual memory arena*. Let's first look at how to set it up, I'll explain how it works under the hood in the next section.

You'll need to import `core:mem/virtual`, I usually alias that as `vmem`:

```
import vmem "core:mem/virtual"
```

We then change the code so it looks like this:

```
Level :: struct {
    name: string,
    objects: []Game_Object,
    tiles: []Tiles,
    arena: vmem.Arena,
}

generate_level :: proc(name: string) -> Level {
    level_arena: vmem.Arena
    arena_allocator := vmem.arena_allocator(&level_arena)

    tiles := generate_tiles(arena_allocator)
    objects := generate_objects(tiles, arena_allocator)

    return {
        name = strings.clone(name, arena_allocator),
        objects = objects,
        tiles = tiles,
        arena = level_arena,
    }
}

destroy_level :: proc(level: ^Level) {
    vmem.arena_destroy(&level.arena)
}
```

There are a couple of differences: Inside `generate_level` we now create a `level_arena: vmem.Arena` variable. The line

```
arena_allocator := vmem.arena_allocator(&level_arena)
```

sets up `arena_allocator`, which is of type `runtime.Allocator`. It lets you allocate memory into the arena. Then we pass the arena allocator to `generate_tiles`, `generate_objects` and `strings.clone`, making them all use this allocator instead of `context.allocator`.

You could also set `context.allocator = arena_allocator` and skip the explicit passing of `arena_allocator`.

In cases like this, when you expect *all* allocations within this scope to use a certain allocator, then I think assigning to `context.allocator` is justified.

Note that we also assign `arena = level_arena` when we return the `Level` struct. The arena is the important thing to save somewhere, it contains the information about what was allocated. You should not attempt to save the `arena_allocator` in `Level`, as it will contain a pointer to the stack variable `level_arena`.

As you can see the `destroy_level` procedure is now *very* simple:

```
destroy_level :: proc(level: ^Level) {
    vmem.arena_destroy(&level.arena)
}
```

It just destroys the level's arena. Since all the memory of the `name`, `objects` and `tiles` fields were allocated into that arena, all of them are now deallocated. Note that I've changed the parameter `level` into a pointer. This is because `arena_destroy` requires a pointer. If `level` wasn't a pointer then doing `&level.arena` would not compile: Fetching a pointer to something contained in an immutable procedure parameter is not allowed.

A good take-away from all this is: Arenas are great for grouping allocations that have the same lifetime. Meaning that if you need several things to be deallocated at the same time, then perhaps using an arena is a good idea.

I have never tried to combine arenas with tracking allocators. This means that I don't get any help with remembering to destroy my arenas. Since there are often few enough of them, it means that it's not a big issue.

However, I'm sure you could make a custom tracking allocator that is informed whenever an arena is created. At program shut down, it could then print a report of all arenas that have not been destroyed.

No individual deallocations *13.2.1*

Please note that you can't deallocate individual allocations made into the arena, you can only destroy the entire arena. In our previous example, `delete(level.name)` doesn't really do anything, it will just return a "Not Implemented" error.

Arenas not being able to destroy individual allocations make them problematic when combined with growing containers, such as dynamic arrays. When the dynamic array grows, then it will allocate a bigger block of memory, but since it uses an arena allocator, the old block of memory can't be deallocated. So a dynamic array (or any other growing container) may end up using more memory than necessary when used in combination with an arena. What you can do to work around this is:

- Preallocate your dynamic array with a sensible capacity: `dyn_arr := make([dynamic]My_Type, 0, 128, arena_allocator)`.
- Figure out ways to not use dynamic arrays, instead allocate fixed-size slices of a reasonable size.
- Don't use an arena for your dynamic array, just use the default allocator instead.

I've written a blog post on this topic called **"Mistakes and cool things to do with arena allocators"** .

Choosing an arena allocator 13.3

Let's take a look at the different types of arena allocators. I'll explain the different situations where they might be useful.

Odin comes with three different arena allocator implementations:

1. One in **`core:mem/virtual`**
2. One in **`core:mem`**
3. Another one in **`base:runtime`**

In most cases the one in `core:mem/virtual` is the only one you need. The only time you'll need the one in `core:mem` is if you are on platforms that do not support virtual memory, such as WASM.

> In fact, the one in `core:mem/virtual` can do the same stuff as the one in `core:mem`, if you configure it properly, just that the `core:mem/virtual` package won't compile on platforms without virtual memory.

The arena in `base:runtime` shouldn't be used by anything other than the temporary allocator. It is similar to the growing virtual memory arena, but it does not use virtual memory, so it is less memory-efficient.

So "looking at the different types arena allocators" actually means that we are going to look at the three different modes of operation of the virtual memory arena allocator that you find in `core:mem/virtual`. That arena is a struct that looks like **this** :

```
Arena :: struct {
    kind:                Arena_Kind,
    curr_block:          ^Memory_Block,

    total_used:          uint,
    total_reserved:      uint,

    default_commit_size: uint, // commit size <= reservation size
    minimum_block_size:  uint, // block size == total reservation

    temp_count:          uint,
    mutex:               sync.Mutex,
}
```

The field `kind: Arena_Kind` says which mode of operation this `Arena` uses. The type `Arena_Kind` looks like **this** :

```
Arena_Kind :: enum uint {
    Growing = 0, // Chained memory blocks (singly linked list).
    Static  = 1, // Fixed reservation sized.
    Buffer  = 2, // Uses a fixed sized buffer.
}
```

Note that `Growing` is equivalent to `0`. That's why you get a growing virtual arena when you type:

```
arena: vmem.Arena
```

Since `arena` is zero-initialized, then the `kind` field inside it is also zero-initialized, which means that it defaults to the `Growing` kind.

Let's go through the three kinds in the order they are stated in `Arena_Kind`.

Growing virtual memory arena *13.3.1*

The growing virtual memory arena is useful when you have a couple of separately allocated things that have the same lifetime, but you are not sure how much memory they need.

In the **previous section** we looked at an example on how to use this arena. So here we'll discuss how it works under the hood. You create the arena and a compatible arena allocator like this:

```
arena: vmem.Arena
arena_allocator := vmem.arena_allocator(&arena)
```

Which it equivalent to doing this:

```
arena: vmem.Arena
err := vmem.arena_init_growing(&arena)
assert(err == .None)
arena_allocator := vmem.arena_allocator(&arena)
```

`arena_init_growing` will set the `kind` field of `arena` to `.Growing`.

Note that `arena_init_growing` returns an error, we *assert* that it has the value `.None`, meaning that we treat it as a fatal error if the creation of our arena failed.

The virtual growing arena consists of blocks where each block knows the location of the previous block. When allocations happen, those allocations go into these blocks. When the current block is full, then a new block is reserved.

Note the word *reserved* above. This arena uses *virtual memory*. Virtual memory allocations can happen in two steps: First you can reserve memory, which just tells the operating system that you want `X` bytes of continuous virtual memory. No physical memory is allocated at this step. This means that the memory usage of the program will not go up when you reserve virtual memory. It is only when you *commit* already reserved memory that physical memory is allocated, and you can commit it in small chunks.

What this means is that the blocks of memory that the arena uses are initially just reserved. Then, for each allocation it may commit parts of that reserved memory. So even though we have nice continuous chunks of virtual memory ready, the program's memory usage will only go up when allocations actually happen and virtual memory gets committed. The virtual memory arena will by default reserve blocks of 1 megabyte, however if you try to allocate something really big then it will reserve a big enough block to accommodate your allocation.

Destroy the arena using

```
vmem.arena_destroy(&arena)
```

Since each block knows the location of the previous block, then `arena_destroy` can walk through all those blocks and deallocate them all.

There's an optional second parameter of `arena_init_growing` that can be used to *pre-reserve* the arena with an initial block of that size:

```
arena: vmem.Arena
err := vmem.arena_init_growing(&arena, 100*mem.Megabyte)
assert(err == .None)
arena_allocator := vmem.arena_allocator(&arena)
```

The `mem.Megabyte` constant lives in `core:mem`

Static virtual memory arena *13.3.2*

You get this type of arena using the `arena_init_static` proc:

```
arena: vmem.Arena
err := vmem.arena_init_static(&arena, 1*mem.Gigabyte)
assert(err == .None)
arena_allocator := vmem.arena_allocator(&arena)
```

`arena_init_static` will set the `kind` field of `arena` to `.Static`.

A static virtual memory arena also uses virtual memory, in the sense that it reserves a block of virtual memory and then commits parts of it as you do your allocations. However, this static arena will never reserve new blocks. The block size you tell it to use at the start will be all the memory it has available. Above we reserve a big 1 Gigabyte block.

Just like with the growing virtual arena, this arena only does a virtual *reservation* of the memory. The actual memory usage of your program will only go up as parts of the reserved virtual memory is committed, which occurs when allocations happen.

You destroy this arena in the same way as the growing arena. This will deallocate all its memory:

```
vmem.arena_destroy(&arena)
```

Compared to the growing virtual memory arena, this static arena is useful when you want to put stricter constraints on your memory usage. Some use it to allocate a big arena when the program starts and then replace `context.allocator` with an arena allocator that uses that arena. This means that all allocations will go into a pre-reserved memory block. This forces you to consider all your memory allocations more carefully.

Fixed buffer arena *13.3.3*

Fixed buffer arenas are, like the static virtual memory arena, good for when you want to create an arena of a fixed size. The difference is that this one does not use virtual memory. Instead you can feed it any block of memory that you have laying around.

Yeah. It's confusing that it is in `core:mem/virtual` when it does not use virtual memory.

You get this type of arena using the `arena_init_buffer` procedure:

```
arena: vmem.Arena
buf := make([]byte, 10*mem.Megabyte)
err := vmem.arena_init_buffer(&arena, buf[:])
assert(err == .None)
arena_allocator := vmem.arena_allocator(&arena)
```

`arena_init_buffer` will set the `kind` of `arena` have the value `.Buffer`.

Above we allocate a 10 megabyte buffer and feed it into `arena_init_buffer`. Now any allocations done using this arena allocator will go into that buffer.

To destroy this kind of arena, you should not call `vmem.arena_destroy`. The only allocation happened on the `make` line, so you should instead deallocate that memory. This means that this arena is destroyed by running:

```
delete(buf)
```

The fixed buffer mode of this `core:mem/virtual` arena works mostly like the arena allocator you **find in `core:mem`** . So you do not need to bother with the one in `core:mem`, unless you are on a platform that doesn't support virtual memory, in which case the `core:mem/virtual` package won't compile.

A funny thing you can do with the buffer arena is *allocating into stack memory*:

```
arena: vmem.Arena
buf: [4096]byte
err := vmem.arena_init_buffer(&arena, buf[:])
assert(err == .None)
arena_allocator := vmem.arena_allocator(&arena)
```

Since the buffer `buf` is a fixed array, and since fixed arrays are allocated on the stack, then this means that any allocations done using this arena allocator will go into stack memory. It will not need any deallocation, since the buffer will die when the current scope dies. However, stack space is limited, using anything bigger than `10000` or `100000` bytes for the buffer is probably a bad idea as the program *will crash* if you run out of stack space.

This stack-based-semi-dynamic-allocation has been following me like a ghost as I've been writing this book. I've often said that allocations that use `context.allocator` end up *outside of stack memory*. However, I've been thinking of this edge case over and over. I'm just happy you finally read about it. Let's all move on.

Debugging memory mistakes 13.4

Manual memory management isn't just about manually deallocating things and finding memory leaks. It's also about writing safe code that doesn't crash. A common way to make your program crash is by writing into, or reading from, *the wrong location in memory*.

Here's a tiny, but very buggy program:

```
package writing_memory_bug

import "core:fmt"

main :: proc() {
    number: f32 = 7
    number64_ptr := cast(^f64)(&number)
    fmt.println(number64_ptr^)
}
```

This program creates a 32 bit float called `number`. It then fetches the address of `number`. That will result in a pointer of type `^f32`. It then casts that pointer to the type `^f64`. This means that `number64_ptr` points to the memory of `number`, but if you read or write through that pointer it will try to read or write `64` bits instead of `32` bits. This could cause all kinds of bugs!

The last line of the program fetches the value that `number64_ptr` points to, and prints it. Since it is a pointer to a 64 bit float, it will fetch 64 bits (8 bytes) from that address.

If you just run this program with

```
odin run .
```

Then it may run and print some number, without any error at all. The number it prints may be zero, or it may be something bizarre like:

```
-1.7851420269952399e-190
```

In any case, what the program did was the following: It used `number64_ptr` to go to the location in memory where `number` lives and read it as if it was an `f64` instead of an `f32`. This will both misinterpret the data at that address and also read twice the amount of data it should read.

But your program finished without any error messages. When working with a combination of manual memory management and casting pointers, it's not unlikely that you sometimes cause similar bugs. In order to locate such errors you can use the *LLVM address sanitizer*. Try running the same program, but add `-debug -sanitize:address` as compiler flags:

```
odin run . -debug -sanitize:address
```

This may not work on some computers. These sanitizers are a bit shaky.

You need the `-debug` compiler flag to make the filenames and line numbers appear correctly in the error below.

If you use raylib and this command fails, then try also adding the compilation flag `-define:RAYLIB_SHARED=true`. There are sometimes conflicts between raylib and the address sanitizer, especially on Windows. This compilation flag makes raylib use a DLL, so the raylib code is kept separate from the executable. You may need to copy `raylib.dll` from `<odin>/vendor/raylib/windows`.

Re-running our buggy program with those compilation flags is likely to produce a giant error, similar to the one below. Try to not get overwhelmed by it.

```
=================================================================
==16220==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x00ed11def664
at pc 0x7ff60e019a8c bp 0x00ed11deccf0 sp 0x00ed11decd38
READ of size 8 at 0x00ed11def664 thread T0
    #0 0x7ff60e019a8b in __asan_memcpy C:\src\llvm_package_18.1.8\llvm-
project\compiler-rt\lib\asan\asan_interceptors_memintrinsics.cpp:63
    #1 0x7ff60e019e23 in __asan_memset C:\src\llvm_package_18.1.8\llvm-
project\compiler-rt\lib\asan\asan_interceptors_memintrinsics.cpp:67

Address 0x00ed11def664 is located in stack of thread T0 at offset 36 in frame
    #0 0x7ff60e003dbf in writing_memory_bug.main
C:\code\playground\playground.odin:5

  This frame has 3 object(s):
    [32, 36) 'number' <== Memory access at offset 36 overflows this variable
    [48, 64) ''
    [80, 96) ''
HINT: this may be a false positive if your program uses some custom stack unwind
mechanism, swapcontext or vfork
      (longjmp, SEH and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow C:\src\llvm_package_18.1.8\llvm-
```

```
project\compiler-rt\lib\asan\asan_interceptors_memintrinsics.cpp:63 in
__asan_memcpy
Shadow bytes around the buggy address:
  0x00ed11def380: f2 f2 f2 f2 f2 f2 f2 f2 00 00 00 00 00 00 00 00
  0x00ed11def400: 00 f2 f2 f2 f2 f2 00 00 f2 f2 00 00 f2 f2 00 00
  0x00ed11def480: f2 f2 00 00 f3 f3 f3 f3 00 00 00 00 00 00 00 00
  0x00ed11def500: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x00ed11def580: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x00ed11def600: 00 00 00 00 00 00 00 00 f1 f1 f1 f1[04]f2 00 00
  0x00ed11def680: f2 f2 00 00 f3 f3 f3 f3 00 00 00 00 00 00 00 00
  0x00ed11def700: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x00ed11def780: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x00ed11def800: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x00ed11def880: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
  Array cookie:            ac
  Intra object redzone:    bb
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
==16220==ABORTING
```

Breathe! First, look at these lines:

```
==16220==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x00ed11def664
at pc 0x7ff60e019a8c bp 0x00ed11deccf0 sp 0x00ed11decd38
READ of size 8 at 0x00ed11def664 thread T0
```

Here, `stack-buffer-overflow` means that the program read from parts of stack memory it shouldn't. `READ of size 8` means that it tried to read 8 bytes of memory.

Then look at these lines:

```
Address 0x00ed11def664 is located in stack of thread T0 at offset 36 in frame
    #0 0x7ff60e003dbf in writing_memory_bug.main
C:\code\playground\playground.odin:5

  This frame has 3 object(s):
    [32, 36) 'number' <== Memory access at offset 36 overflows this variable
```

We see that the error happened in the file `C:\code\playground\playground.odin` at line `5`. It also tells us which variable that was *overflowed*: `number`.

The word overflow pops up here and there in programming. In this case it means that we read too many bytes.

We are thus able to directly see where an incorrect memory read happened and we can go in and fix our bug, which in this case means not casting it to a too big type.

This so-called *address sanitizer* is part of LLVM, which is the infrastructure that the Odin compiler is built upon. See `odin build -help` and look for `-sanitize` to see some additional sanitation modes. Some of them may not work on all platforms.

Some people refer to the Address SANitizer as ASAN.

When debugging random glitches and crashes, I suggest the following:

- If the program is crashing every now and then, first try running it from within a debugger, without any sanitizer. I show how to setup a debugger in **chapter 22**. Most crashes can be found by just having a debugger attached when the crash occurs. The debugger might directly show you the line that caused the crash.
- If your program is just randomly *misbehaving*, such as random glitches showing up after a while, then it may be a sign of *memory corruption*: Some memory that shouldn't be written to, has been written to. Or that some memory that shouldn't be read, has been read. In these cases I recommend trying the sanitizer.
- If your program crashes and you catch the crash in a debugger, but the crash doesn't make any sense at all, then try using the sanitizer. Crashes that don't make any sense are sometimes due to memory having been erroneously modified a while ago, but that memory wasn't used until a while later. So the crash your debugger is showing is just a symptom, not the cause. The sanitizer can sometimes catch these bugs at their origin.

The address sanitizer makes your program take a long time to compile and it also makes it very slow. Only use it when you suspect that some kind of memory corruption has occurred!

Improving your understanding 13.5

You may wonder what the "right way" to manage memory is. You might have questions like: When should I use a dynamic array? Should I sometimes prefer a fixed array? Should I always use arena allocators or is the default allocator good enough? And many more such questions.

I will give a short beginner-friendly summary below. But first just let me say that there is no consensus on any "right way" to manage memory. There are no universal solutions. You need to learn how your code really works, so that you can create good solutions for whichever problems you have at hand. Try to really understand what the program is actually doing. Think about the lifetimes of memory as you write your code. Write small test programs in order to verify that certain concepts work the way you've assumed. If you keep challenging your assumptions, then your understanding will quickly improve and things will over time become a lot easier.

> You've probably noticed that I often encourage you to look at the code in `core` and `base`. Don't be afraid to do so. Understanding what procedures such as `make` and `delete` actually do is a great step towards getting a very clear picture of how your program works. This clarity will enable you to write better code.
>
> Understanding the tools makes you a better craftsperson.

With this improved understanding you'll unlock the ability to deeply understand what your program is doing as well as being able to structure it in such a way that it runs very efficiently.

In any case, if you are a beginner looking for advice, then here is one general approach to manual memory management:

- Use the **tracking allocator**. Learn from the leak reports where memory is allocated. Look for patterns in how those procedures manage memory to get inspiration for your own work.
- Use the **temporary allocator** for most intermediate data processing. Use it for creating strings you need to display on the screen. Use it when you load a file, but only need a part of it: Then load the file on the temporary allocator and just extract what you need from that data.
- Use **arena allocators** if you feel the need to group allocations that should be deallocated at the same time. But you don't *have to* use arena allocators. If you're not sure if you need one, then don't use one and instead just stick to the first two points of this list. Usually one will discover applications of arena allocators over time.
- If you encounter crashes. Try running your program from within a debugger. I show how to set up a debugger in **chapter 22**. If the crashes are very random or you see lots of strange glitches, then try running your program with the flags `-debug -sanitize:address`. This may help finding certain memory corruption bugs. Only use the

`-sanitize:address` flag when you are investigating possible memory corruption issues, as it makes your program run very slow.

Chapter 14

Parametric polymorphism: Writing generic code

In this chapter we'll learn how to use *parametric polymorphism*, or *parapoly*. It makes it possible to write generic code. One thing we'll look at is how to make procedures that accept parameters of different types, without having to write separate code for each type. Odin accomplishes this by letting the compiler *generate* variations of procedures for us.

This is different from explicit overloading. **Explicit overloading** lets you provide several different procedures under a single name. Parapoly lets you write a single procedure that can be used as a template. This template is used by the compiler to generate variations of the procedure based on how you use it.

We'll meet three concepts that may at first glance seem unrelated. But as we shall see they are all part of parametric polymorphism. These three concepts are:

- Generic procedure parameters
- Procedure parameters that are compile-time constants
- Structs and unions with fields of generic types

Throughout this chapter you'll see `$` appearing a lot. Any time you see `$`, then you are dealing with code that uses parapoly. Sometimes the `$` will be part of a procedure parameter *type*, sometimes it will be part of a procedure parameter *name*, sometimes both. It'll also appear on struct and union definitions. Make sure you pay attention to where the `$` is throughout this chapter.

From C / C++

The thing we'll see in this chapter is similar to templates in C++, but less complicated and not as bad for the compile times.

Procedures with polymorphic parameters 14.1

Say that we have this procedure that implements "clamping" for numbers of type `f32`. It forces a value of type `f32` to be in a range between `min` and `max`:

```
clamp :: proc(val: f32, min: f32, max: f32) -> f32 {
    if val <= min {
        return min
    }
```

```
    if val >= max {
        return max
    }

    return val
}
```

There is a built-in `clamp` procedure in Odin, so you do not need to implement your own. I re-implement it here just because it is a simple enough example.

However, we would like `clamp` to work with `int` and `f64` numbers as well. Here's how we can make `clamp` accept a parameter of a generic type:

```
clamp :: proc(val: $T, min: T, max: T) -> T {
    if val <= min {
        return min
    }

    if val >= max {
        return max
    }

    return val
}
```

You could also type

`val, min, max: $T`

instead of

`val: $T, min: T, max: T`.

The type of the `val` parameter has been replaced with `$T`. The `T` without the dollar sign re-appears on the `min` and `max` parameters as well as on the return type.

What will happen when the compiler is compiling your code is this: When it sees that your code uses `clamp`, then it will generate a version of `clamp` where it replaces `T` with the type of the argument you fed into the parameter `val`. In other words, when we do something like this:

```
number: int = 7
clamped_number := clamp(number, 2, 5)
```

then the compiler sees that we use `clamp` with an integer as parameter. It will then, at compile time, generate a version of `clamp` that accepts integers.

Note that we only type `$T` with a dollar sign once:

```
clamp :: proc(val: $T, min: T, max: T) -> T {
```

The dollar sign is on the type of the `val` parameter. This tells the compiler that we want `T` to be based on the type of the value we feed into the parameter `val`. You cannot write `$T` multiple times, because the compiler needs to know which parameter it should use to figure out the type `T`.

The `$` also tells the compiler that `T` must be a *compile-time constant*. All types are compile-time constants, so this might not seem important in this specific example. But as we look more at parapoly, we'll meet other examples of `$` enforcing something to be a compile-time constant. As we shall see, it is a powerful and important concept.

I will write "compile-time constant" *a lot* in this chapter.

If you call `clamp` with a parameter of a certain type, but the compiler fails to generate that code, then you'll get a compilation error. In the case of our `clamp` example, it could fail to generate the code if you try to use the procedure with types that don't support the `<=` and `>=` operators that the procedure uses:

```
if val <= min {
    return min
}

if val >= max {
    return max
}
```

To enforce that this procedure can only be used with numeric types, you can add

```
where intrinsics.type_is_numeric(T)
```

after the return value, meaning that the declaration of `clamp` now looks like this:

```
clamp :: proc(val: $T, min: T, max: T) -> T where intrinsics.type_is_numeric(T) {
```

You may ask "but doesn't using `<=` and `>=` already implicitly require a numeric type?". No, there are non-numeric types that support those operators, such as strings.

Here we demand that `T` is a numeric type. This uses **`base:intrinsics`**, which you'll have to import. The condition of the `where` must be possible to evaluate at compile time. Some of the

procedures in the `intrinsics` package can be used at compile time, in particular the ones that "ask questions about types", such as `type_is_numeric`.

Explicitly passing a type to a procedure 14.2

In the previous example, our procedure `clamp` figured out the type `T` based on what type a procedure argument had. We can also use parapoly to pass a type into a procedure, without having a value of that type. We will be able to use that type in order to generate variations of the procedure.

Here's an example that allocates a slice of a random size:

```
make_random_sized_slice :: proc($T: typeid) -> []T {
    // `random_size` will be an integer
    // between 0 and 1024.
    random_size := rand.int_max(1024)
    return make([]T, random_size)
}
```

`rand.int_max` comes from `core:math/rand`

We can use this procedure like so:

```
my_slice := make_random_sized_slice(f32)
```

`my_slice` will then be a slice of type `[]f32` with a size between 0 and 1024. Note the parameter when we call the procedure: `my_slice := make_random_sized_slice(f32)`. It is just a type!

There's an important difference between this and the `clamp` example. In this one I moved the `$` to the parameter *name* instead of having it on the *type*. In other words, our `clamp` example did this:

```
val: $T
```

The `make_random_sized_slice` example does this:

```
$T: typeid
```

The `$` has moved to the left side of the `:`. What is this `typeid` thing we use here? It is the "type of types". We can then use `T` in the code to refer to this type. For example, we use it as the procedure's return type.

Note that we cannot just write `T: typeid` in the parameter list, the `$` has to be there. Again, the `$` enforces that `T` is a compile-time constant, which makes it possible for the compiler to use `T` as it is compiling the program.

You *can* make procedures that accept a parameter `T: typeid` without a dollar sign appearing anywhere, but then you could *not* do this:

```
create_and_set_name :: proc(T: typeid, name: string) -> T {
    v: T
    v.name = name
    return v
}
```

it would not be able to use `T` for any of these two things:

```
-> T
```

or

```
v: T
```

Both those usages require `T` to be known at compile time. Adding in that `$` on the parameter name means that we can only pass compile-time constants into this parameter. Since the compiler can be certain that `T` is a compile-time constant, it can then use it for compile-time code generation.

One additional important thing to realize is that the `T` in the parameter `$T: typeid` and the `T` in the parameter `val: $T` are both typeids. In the first case we explicitly send in a typeid. In the second case the typeid `T` is figured out implicitly from the argument passed.

This can be summarized like so:

- Use `$T: typeid` when you need to reason about a type at compile-time, but you got no value of that type.
- Use `val: $T` when you need a value of a type, but you also want to be able to reason about the type `T` at compile time.

We can also use this `$`-on-the-parameter-name to pass compile-time constants of other types than `typeid`, a common example is to pass a compile-time constant integer. Here's a

procedure that creates a fixed array of size `N` with element type `T` and sets all the elements to `7`:

```
make_array_of_7 :: proc($N: int, $T: typeid) -> [N]T {
    res: [N]T
    for &v in res {
        v = 7
    }
    return res
}

arr := make_array_of_7(128, f32)
```

You can also do `$N: $I` instead of `$N: int`, which will make it possible to use this procedure with `N` of any type.

The parameter `N` is an integer, but by putting `$` in front of it we are forcing it to be a compile-time constant. We can therefore use `N` when we are specifying the return value `[N]T` and when we are declaring a fixed array `res` of size `N`.

Fixed arrays must have their size known at compile time. You can't write

```
arr: [size]f32
```

where `size` is a variable, because the value of variables are unknown at compile time. But if `size` was a procedure parameter like `$size: int`, then you could declare a fixed array using it.

Parametric polymorphism with structs and unions 14.3

We've seen two different ways of using parapoly with procedures. But we can also use it when defining struct types, making it possible to generate variations of the struct. Here's an example:

```
Special_Array :: struct($T: typeid, $N: int) {
    items: [N]T,
    num_items_used: int,
}
```

Compare to how you normally type it:

```
Some_Name :: struct {
```

Also, this example is quite similar to the **Fixed Capacity Dynamic Array**.

Note: This is a `struct` not a `proc`. We see that this struct has a list of parameters. The first parameter is `$T: typeid`, which is used to choose the element type of the `items` array. The second parameter is `$N: int`, which gives us a compile-time value to use for the size of the `items` array.

We can then use this generic struct as a type, like so:

```
array: Special_Array(f64, 128)
```

`array` will then be a variable of type `Special_Array`. The field `items` inside `array` will be a fixed array of `128` elements of type `f64`. You can see it as the compiler taking the `f64` and the `128` and "pasting" it into the struct where ever `T` and `N` are mentioned, generating variations of this struct type for you.

Also, here's how you write procedures that accept these generic structs as parameters:

```
find_random_thing_in_special_array :: proc(arr: Special_Array($T, $N)) -> T {
    return arr.items[rand.int_max(arr.num_items_used)]
}

array: Special_Array(f64, 128)

// Fill `array` with 32 items of random data.
for i in 0..<32 {
    array.items[i] = rand.float64_range(-10, 10)
    array.num_items_used += 1
}

random_thing := find_random_thing_in_special_array(array)
```

Note that the procedure `find_random_thing_in_special_array` has a parameter of type `Special_Array($T, $N)`. You can therefore use it with any specific type of `Special_Array`. In our case we feed it something of type `Special_Array(f64, int)`. Therefore, the compiler will generate a procedure which accepts that specific type.

It will only generate one procedure per unique combination of parapoly parameters. If you call `find_random_thing_in_special_array` twice, both times with a parameter of type `Special_Array(f64, 128)`, then only one procedure will be generated.

Finally, you can also use parapoly with unions. This is similar to how you use it with structs. It's not something that is done terribly often, but Odin's **built-in type `Maybe`** uses it:

```
Maybe :: union($T: typeid) { T }
```

The `Maybe` type is used for checking if something has *no* value or *some* value. A union with a single variant provides this functionality since it can be of value `nil` or have a variant of type `T`.

You can create a Maybe that can hold an `f64` like this:

```
m: Maybe(f64)
m = 4.32

if m_val, m_val_ok := m.?; m_val_ok {
    fmt.println(m_val) // 4.32
}
```

`m` initially has the zero-value `nil`. After assignment it is of variant `f64` and holds the value `4.32`. Note the special `m.?` syntax used inside the if statement. That's equivalent to `m.(f64)`. But since the Maybe only has a single variant, the compiler fills it in for you.

The unified concept of parametric polymorphism 14.4

We have seen `$` used in four ways:

- `val: $T`
- `$T: typeid`
- `$N: int`
- `$N: $I`

In all these cases the thing with the `$` in front is a compile-time constant.

You may find it strange that all these things fall under the name *parametric polymorphism*. Some people might argue that only the `val: $T` example really is polymorphism. But the name aside, it is important to understand that all these things are part of the same system. And in a nutshell we can define this system like so:

Parametric polymorphism is the system by which the compiler looks at all the `$`-decorated parameters of a procedure/struct/union and uses them as a template for generating variations of the procedure/struct/union. One variation will be generated for each unique combination of arguments passed into those `$`-decorated parameters.

That's a mouthful. Personally, I like to think of it as "the dollar-sign-code-generation machine".

Specialization 14.5

Let's finish this chapter by talking about how you can limit which types are allowed when working with parametric polymorphism. This is known as specialization. Let's look at a few examples.

This first one is from **base/runtime/core_builtin.odin** :

```
delete_dynamic_array :: proc(array: $T/[dynamic]$E, loc := #caller_location) ->
Allocator_Error {
    return mem_free_with_size(raw_data(array), cap(array)*size_of(E),
array.allocator, loc)
}
```

This is the procedure that is run whenever you do `delete(some_dynamic_array)`. Let's focus on the `array` parameter:

When you run `delete(dyn_arr)` it ends up inside `delete_dynamic_array` because **`delete` is an explicit overload** .

```
array: $T/[dynamic]$E
```

Note the `/` after `$T`. Here `T` can be any type as long as it falls within the constraints of whatever we put on the right side of the `/`. We say that we are *specializing* the type `T`.

In this case we are enforcing that this polymorphic parameter *must be a dynamic array*. However, note the `$E` at the end of `[dynamic]$E`. This means that we allow the dynamic array to have elements of *any type*. Different variations of `delete_dynamic_array` will be generated when you use it with dynamic arrays that have elements of different types.

Also, note how the procedure is able to use the type `E` within the procedure body: `cap(array)*size_of(E)`.

Here's another example, this time **from the official overview** :

```
make_slice :: proc($T: typeid/[]$E, len: int) -> T {
    return make(T, len)
}
```

This is similar to the `make_slice` overload of the `make` procedure group.

This procedure makes slices of a certain type. Note the difference between the parameter we saw in `delete_dynamic_array`:

```
array: $T/[dynamic]$E
```

and this one from `make_slice`:

```
$T: typeid/[]$E
```

Look at what is before the `/`. In the first case the specialization is done on `$T`, but in the second case the specialization is done on a `typeid`. This might seem strange: From the `delete_dynamic_array` example you might assume that `/` only works with polymorphic parameters (something with a `$`). But we can actually specialize both polymorphic types such as `$T` and also compile-time constants, given that their type is `typeid`. This actually makes sense, since I said earlier that when you type `val: $T`, then `T` is actually a `typeid`. So in both cases we are specializing a `typeid`, just that in the `array: $T/[dynamic]$E` it is implicit that `T` is a typeid.

Specialization and distinct types *14.5.1*

The `delete_dynamic_array` example looked like this:

```
delete_dynamic_array :: proc(array: $T/[dynamic]$E, loc := #caller_location) ->
Allocator_Error {
    return mem_free_with_size(raw_data(array), cap(array)*size_of(E),
array.allocator, loc)
}
```

You might wonder why it says `array: $T/[dynamic]$E` instead of `array: [dynamic]$E`. Why is `$T` needed? If you removed the `$T`, the following code would actually still work:

```
d := make([dynamic]int, 128)
delete_dynamic_array(d)
```

However, the following code would *not* work:

```
Int_Array :: distinct [dynamic]int

d := make(Int_Array, 128)
delete_dynamic_array(d)
```

It would give an error like this:

```
Error: Cannot determine polymorphic type from parameter: 'Int_Array' to
'[dynamic]$E'
```

```
    delete_dynamic_array(d)
```

What we've done is introduce a new type called `Int_Array`. If `Int_Array` was just an ordinary alias of `[dynamic]int`, then things would still work. But in our case we've added `distinct` in front:

```
Int_Array :: distinct [dynamic]int
```

This makes `Int_Array` into a distinct type from `[dynamic]int`. No automatic conversions between the two will happen.

We can therefore see why `delete_dynamic_array` needs the parameter `array: $T/[dynamic]$E` instead of `array: [dynamic]$E`. The automatic conversion of `Int_Array` to `[dynamic]$E` is *not allowed*. However, in the case where we use `array: $T/[dynamic]$E`, then `array` can be of *any type* `T`, including `Int_Array`. But the specialization says that this type must be some kind of dynamic array.

Video 14.6

A video I've made about parapoly:

https://www.youtube.com/watch?v=3X2IzOfzepA

Chapter 15

Bit-related types

Bit sets and bit fields are two types in Odin that make it possible to store information in a very compact way. They do this by directly manipulating bits. Bits are the binary building blocks that your computer uses to store information. Let's first talk about how to interpret binary numbers and then look at what bit sets and bit fields are.

Binary representation of integers 15.1

A bit is a single binary digit, meaning that it can be either `1` or `0`. It's the smallest piece of information the computer can reason about. The computer combines multiple bits in order to store information. As an example, an 8 bit integer uses 8 bits to describe an integer. An 8 bit integer holding the value `2` can be written like this:

```
00000010
```

An 8 bit integer holding the value 13 can be written like this:

```
00001101
```

The bits are "worth" `1`, `2`, `4`, `8`, `16`, `32` etc, counted from the right. This is why `00001101` represents 13: Counting from the right, the first bit is worth `1`, the third bit is worth `4` and the fourth bit is worth `8`. If you add them you get `1 + 4 + 8 = 13`.

Bit sets 15.2

A `bit_set` provides a memory-efficient way of storing several boolean-style values. Internally, it uses the bits of an integer to store those values, meaning that each bit is associated with one value. However, the `bit_set` needs some way to identify the different bits. For that you can use an enum, a range of letters or a range of numbers. Let's start off with an example that uses an enum.

This is unrelated to **using a `map` to create a set**. The methods we'll see here are very fast and memory efficient, but less general than using a map. If you use a map, then you can use many different types for the key. With `bit_set` you must use types that map nicely to a numeric range, such as enums.

In this example we have a `Keycard` enum that represents a colored keycard. We use a `bit_set` to store which keycards someone has:

```
Keycard :: enum {
    Red,
    Green,
    Blue,
    Yellow,
}

keycards: bit_set[Keycard]
keycards += { .Red, .Blue }
fmt.println(keycards) // prints bit_set[Keycard]{Red, Blue}
```

From C / C++

In C you can achieve the same thing using an enum where you manually associate each member with the values 1, 2, 4, 8 etc and then you use an integer plus some bit-wise operations. In Odin we don't need to do any of that extra work, we just wrap the enum type in `bit_set` and get all that functionality.

We see here that `keycards` is of the type `bit_set[Keycard]`. This `keycards` variable uses one bit per enum member. Each bit acts like a separate boolean. We set the bits associated with `Red` and `Blue` like so:

```
keycards += { .Red, .Blue }
```

You can achieve the same result using four separate boolean variables:

```
red, green, blue, yellow: bool
red = true
blue = true
```

But then you have four variables to worry about instead of one. Also, as we shall see, there are many nice properties of `bit_set`.

We can add and subtract values from the `bit_set`, which essentially is like setting and clearing bools:

```
keycards := bit_set[Keycard] { .Red, .Blue, .Green }
keycards += { .Yellow }
keycards -= { .Red, .Green }
fmt.println(keycards) // bit_set[Keycard]{Blue, Yellow}
```

Here we initialize `keycards` directly upon creation, instead of using `keycards += { .Red, .Blue }`.

You can also check if certain values are in the `bit_set` or not:

```
keycards := bit_set[Keycard] { .Red, .Blue, .Green }
fmt.println(.Red in keycards) // true
fmt.println(.Yellow not_in keycards) // true
```

The bits inside the `bit_set` *15.2.1*

Let's look at how the `bit_set` in the previous example uses bits to store information.

```
Keycard :: enum {
    Red,
    Green,
    Blue,
    Yellow,
}

keycards := bit_set[Keycard] { .Red, .Blue }
```

`bit_set[Keycard]` uses an integer as the *backing* value. That's the integer used behind the scenes to store the bits. The type `bit_set[Keycard]` uses an 8 bit integer. Why an 8 bit integer? The types of integers available in Odin are 8, 16, 32, 64 and 128 bit. The `Keycard` enum has 4 members, so 8 bits is the nearest integer size that has the minimum amount of bits we need. If our enum had between 9 and 16 members, then a 16 bit integer would automatically be used.

It will associate `Red` with the first bit, `Green` with the second bit and so on. `keycards` is initialized with `Red` and `Blue` set, this makes the binary representation look like this:

```
00000101
     | |
     | + Red
     |
     + Blue
```

There are operators available for combining bit sets in different ways. For example, you can produce a new `bit_set` based on the values that two bit sets have in common. This is done using the `&` operator:

```
cards1 := bit_set[Keycard] { .Red, .Blue }
cards2 := bit_set[Keycard] { .Yellow, .Blue }
```

```
in_common := cards1 & cards2
fmt.println(in_common) // bit_set[Keycard]{Blue}
```

You can use such operators with integers too, they are called *bit-wise operators*.

The `&` operator takes the binary representations of `cards1` and `cards2` and produces a new `bit_set` where only the bits they have in common are set to `1`:

```
  00000101
& 00001100
= 00000100
```

This will result in a `bit_set` where only the third bit (counted from right) is `1`. That bit is associated with `Blue`, because it's the third member of the `Keycard` enum.

There is a full list of all operators compatible with `bit_set` in the official overview .

Force backing type *15.2.2*

If you write `bit_set[Some_Enum; i64]` then you force the `bit_set` to use a 64 bit integer as backing type, regardless of how many values the enum has.

`bit_set` *using letters and numbers* *15.2.3*

Instead of using enums, you can also use a range of letters or numbers to create bit sets:

```
numbers: bit_set[0..=5]
numbers += {4, 2}
fmt.println(numbers) // bit_set[0..=5]{2, 4}
fmt.println(2 in numbers) // true
```

The type `bit_set[0..=5]` is a `bit_set` where one bit each is given to the values 0, 1, 2, 3, 4 and 5, meaning that they are presented by these bits:

```
0 000001
1 000010
2 000100
3 001000
4 010000
5 100000
```

Other than that it works just like with enums. The backing type will be an 8 bit integer.

You can do the same with a range of letters:

```
letters: bit_set['A'..='Z']
letters += {'B', 'Q'}
fmt.println('E' in letters) // false
```

This will make a `bit_set` that needs 26 bits, because that's how many letters there are from `A` to `Z`. The backing type of this `bit_set` will therefore be the nearest larger integer type: a `32` bit integer.

Bit fields 15.3

Bit fields are like structs, but you can say how many bits each field should use. Bit fields can be used to create very compact and optimized data. A use case for such a thing is sending things over a network in a compact way. Here's some mockup code of how that could look:

```
Packet_Type :: enum {
    Handshake,
    Message,
    Disconnect,
}

Network_Packet_Header :: bit_field i32 {
    type: Packet_Type  | 8, // Some room for future packet types
    receiver_id: uint  | 8,
    payload_size: uint | 16,
}

h := Network_Packet_Header {
    type = .Message,
    receiver_id = 4,
    payload_size = 1200,
}

// type of size is int
size := h.payload_size
```

Here we create a `bit_field` called `Network_Packet_Header`. We must say which *backing type* it uses. That's decided by the type to the right of `bit_field`: in this case it's `i32`. This means that behind the scenes, the information of this `bit_field` is stored in a 32 bit integer.

Note how each field has `| some_number` at the end of the line. The number there says how many bits of these maximum 32 bits this field can use. For example, the field

```
payload_size: uint | 16
```

can use a maximum of 16 bits, which means that the largest value we can store in it is `65535`. But why do we then say that the field type is `uint`? After all, `uint` is a 64 bit unsigned integer (on a 64 bit platform). The answer is that `uint` is the type that will be used when you fetch the value of this field:

```
size := h.playload_size
```

Here `size` is of type `uint`. You could also use `int` as the type of `payload_size` within `Network_Packet_Header`, but then you'd be limited to a maximum value of `32767` since it must keep one bit to keep track of the number's sign.

In Odin, `int` is usually seen as a better default type for integers compared to `uint`. `uint` has no sign, so subtracting `1` from a `uint` with the value `0` would give you `max(uint)`. This often causes bugs when making loops that run backwards and similar. Therefore the language defaults to using `int` when you write for example `number := 5`. However, in a situation like this, where we are trying to cram as much data into a network packet as possible, then being specific about the sign is useful, as we otherwise lose half the range of our already tiny integer.

When to use bit sets and bit fields *15.3.1*

Use bit fields whenever you want a struct-like type, but you want to be specific about how much data each field uses. However, if you make a `bit_field` that only uses fields of type `bool` that are all of size `1`, then use a `bit_set` instead. This means that instead of this:

```
Keycards :: bit_field i8 {
    red: bool    | 1,
    green: bool  | 1,
    blue: bool   | 1,
    yellow: bool | 1,
}
```

do this:

```
Keycard :: enum {
    Red,
    Green,
    Blue,
    Yellow,
```

```
}

keycards: bit_set[Keycard; i8]
```

Using a `bit_field` with only `bool | 1` fields gives you the same data layout as using a `bit_set`, but you can't use nice `bit_set` operators such as `+`, `-`, `&` etc.

Chapter 16

Error handling

Error handling in Odin is very simple. It is done by making procedures return an extra value that says if anything went wrong. Since Odin supports **multiple return values**, adding an extra return value for the error is an easy thing to do.

These errors are usually a `bool`, `enum` or union, depending on how much information your error needs.

In some languages errors are implemented using "exceptions". In those languages you throw exceptions and then have to catch them later. Odin does not have exceptions. Errors are just normal return values that you are expected to check if you want to handle the errors.

Always return an error as the *last* return value of a procedure. You can have as many return values as you like, but make sure to put errors as the last one in order to make it work nicely with procedures such as `or_else` and `or_return`, which we will talk about shortly.

Errors using `bool` 16.1

A `bool` error just says if everything went OK or not.

As an example, below is a procedure that looks through a list of cats and tries to find one of a matching name. If it finds a match, then it returns the matching struct in the first return value. In the second return value it return `true`, indicating that something was found.

In case nothing is found, then an empty struct is returned in the first return value, and `false` in the second return value, indicating an error.

```
Cat :: struct {
    name: string,
    age: int,
}

find_cat_with_name :: proc(cats: []Cat, name: string) -> (Cat, bool) {
    for c in cats {
        if c.name == name {
            return c, true
        }
    }
```

```
    return {}, false
}
```

The procedure above can then be used like so:

```
cat, cat_ok := find_cat_with_name(cats, "Molly")

if cat_ok {
    fmt.printfln("Found %v! It is %v years old.", cat.name, cat.age)
}
```

As we've seen before, you can also declare and check the variables as part of the if-statement itself:

```
if cat, cat_ok := find_cat_with_name(cats, "Molly"); cat_ok {
    fmt.printfln("Found %v! It is %v years old.", cat.name, cat.age)
}
```

These kind of `bool` errors can be seen as a way to communicate if it is OK to use the other return values.

Errors using `enum` 16.2

Enums are often used for errors. A procedure might have several possible errors, which you can express using an enum. As an example, say that you have a procedure which loads a configuration file, containing settings for your program:

```
Load_Config_Error :: enum {
    None,
    File_Unreadable,
    Invalid_Format,
}

Config :: struct {
    fullscreen: bool,
    window_position: [2]int,
}

load_config :: proc(filename: string) -> (Config, Load_Config_Error) {
    config_data, config_data_err := os.read_entire_file(filename,
context.temp_allocator)
```

```
    if config_data_err != nil {
        return {}, .File_Unreadable
    }

    result: Config

    json_error := json.unmarshal(config_data, &result, allocator =
context.temp_allocator)

    if json_error != nil {
        return {}, .Invalid_Format
    }

    return result, .None
}
```

What the code inside `load_config` actually does is not very important. But we see that it returns two values: The first one is a struct called `Config` and the second one is the error enum. Depending on what happens inside `load_config`, the error will either be `None`, `File_Unreadable` or `Invalid_Format`.

We can call `load_config` and check the error like so:

```
DEFAULT_CONFIG :: Config {
    fullscreen = false,
    window_position = { 100, 100 },
}

main :: proc() {
    config, config_error := load_config("config.json")

    if config_error != .None {
        fmt.eprintln("Failed loading config with error", config_error)
        config = DEFAULT_CONFIG
    }

    // rest of program that uses config
}
```

Note the `e` in `fmt.eprintln`. It means that this procedure prints to the "standard error output stream", which on most operating systems is a separate output stream from the one used by `fmt.println`. They should both end up in the console however, but they are possible to re-route separately.

If you've **set up your logger** then you can also use

```
log.error("message")
```

The code loads a config from a file with the name `config.json`:

```
config, config_error := load_config("config.json")
```

An error has occurred if `config_error` is not `Load_Config_Error.None`. In that case the code sets the `config` variable to the constant `DEFAULT_CONFIG` and also prints a warning. This way you always have some sensible values in the variable `config`.

While Odin does not have any *special types* for handling errors (such as exceptions), what it does have are some special procedures for working with errors. One such special procedure is `or_else`. Using it, we can change the code above to this:

```
DEFAULT_CONFIG :: Config {
    fullscreen = false,
    window_position = { 100, 100 },
}

main :: proc() {
    config := load_config("config.ini") or_else DEFAULT_CONFIG

    // rest of program that uses config
}
```

Note that we use `or_else` after `load_config`. What happens here is:

- `or_else` looks at the last return value of `load_config`, which is of type `Load_Config_Error`.
- If the error has the *zero value* (in this case `.None`) then that means that *no error* has occurred. In that case it will assign the return value of `load_config` to the variable `config`.
- If the error is non-zero, then it will instead use the value on the right side of `or_else`, in this case `DEFAULT_CONFIG`. This is a way to provide a default value in case of an error.

This way we can be sure that `config` always has a valid value.

Note however that we no longer print a warning when there is an error. It differs from situation to situation if you actually need to output any error or if just using some default value is good enough.

Errors using `union` 16.3

Throughout the `core` collection you'll find **`unions`** being used as error types. This example is from the `core:flags` package:

`core:flags` implements a command-line argument parser.

```
Error :: union {
    Parse_Error,
    Open_File_Error,
    Help_Request,
    Validation_Error,
}
```

`Error` is a union with four variants. Two of the variants, `Help_Request` and `Validation_Error`, look like this:

```
Help_Request :: distinct bool

Validation_Error :: struct {
    message: string,
}
```

`distinct bool` means that this type is not implicitly convertible to `bool`.

We see that `Validation_Error` contains a `message` field. By using `union` for your error type you can both return different errors, and also attach some extra data to each error, such as `message` in this case.

`or_return` 16.4

We saw `or_else` earlier. There are more procedures for working with errors, whose names also start with `or_`.

A commonly used one is `or_return`. Below is an example from **`<odin>/core/strings/strings.odin`** of how `or_return` can be used. The purpose of `strings.clone` is to clone a string, but that requires a memory allocation, and things can go wrong when allocating memory. Most procedures that allocate memory return an optional error of type `runtime.Allocator_Error`.

`runtime.Allocator_Error` is often aliased as `mem.Allocator_Error`.

```
clone :: proc(s: string, allocator := context.allocator, loc := #caller_location)
-> (res: string, err: mem.Allocator_Error) #optional_allocator_error {
    c := make([]byte, len(s), allocator, loc) or_return
    copy(c, s)
    return string(c[:len(s)]), nil
}
```

The first thing to note here is that this procedure uses *named return values*:

```
(res: string, err: mem.Allocator_Error)
```

If you want to use `or_return` in a procedure with more than one return value, then they must be named.

On the line

```
c := make([]byte, len(s), allocator, loc) or_return
```

we see that `make` is used. `make` has two return values: a slice and an error of type `mem.Allocator_Error`.

If the second return value of `make` is non-zero, meaning that `make` returned some kind of error, then:

- The last return value of `clone`, which is named `err`, will be set to the error returned by `make`.
- `clone` immediately returns.

This shows why `or_return` requires named return values: `or_return` never sets the return value `res: string`. It will be whatever value it was initialized to prior to `or_return`.

On the other hand, if `make` returns an error that has the zero value, then everything is OK. In that case the return value of `make` is assigned to the value `c` for further usage in the procedure.

Note how `or_return` consumes the error. `make` returns two values: The result and an error. But after `or_return` the error is gone and we only have the result left.

The final line is:

```
return string(c[:len(s)]), nil
```

It returns `nil` as the error, which means no error.

`mem.Allocator_Error` is actually an enum. For an enum `nil` means the same thing as the zero value.

This combination of named return values and `or_return` can make it easy to write readable code and at the same time pass on errors to the code that called this procedure.

When should you pass on errors using `or_return`, versus handling them directly? It depends. In this case the calling code supplies an allocator. The error is of type `runtime.Allocator_Error`. So it is reasonable that the caller handles any allocation-related errors.

But beware of overusing `or_return`. It is easy to just pass on all errors by having a big error union and always using `or_return`. This means that you have just passed on all possible errors to the user of your code, even though your code should probably handle some of those errors internally.

Similar to `or_return`, there are also `or_continue` and `or_break` procedures that can be used within loops to skip laps of loops or stop them completely based on an error.

`#optional_allocator_error` 16.5

The example in the previous subsection used the `#optional_allocator_error` tag:

```
clone :: proc(s: string, allocator := context.allocator, loc := #caller_location)
-> (res: string, err: mem.Allocator_Error) #optional_allocator_error {
    ...
```

This tag means that the code that called this procedure can ignore the error. That's why it is possible to do the following:

```
cloned_string := strings.clone("Hellope!")
```

Without the `#optional_allocator_error` this would not work. Usually you have to assign all the return values. Note that `#optional_allocator_error` can only be used with `runtime.Allocator_Error`, which `mem.Allocator_Error` is an alias of. This tag exists because one often neglects allocation errors in lightweight programs running on powerful systems: They are very unlikely to happen, and if they do happen the program probably needs to crash anyways, since it usually means there is no more memory.

#optional_ok 16.6

Similarly to `#optional_allocator_error` there is the tag `#optional_ok`. This makes it possible to ignore the last `bool` a procedure returns:

```
divide_and_double :: proc(n: f32, d: f32) -> (f32, bool) #optional_ok {
    if d == 0 {
        return 0, false
    }

    return (n/d)*2, true
}

main :: proc() {
    num := divide_and_double(25, 5)
    fmt.println(num) // prints 10
}
```

Note how we didn't have to assign the second return value of `divide_and_double`. This is not recommended in most cases. In this case, ignoring the error makes `divide_and_double(0, 10)` and `divide_and_double(10, 0)` indistinguishable since both give the result `0`, but the second one is actually returning an error. The `bool` return value helps us distinguish these two cases.

Chapter 17

Package system and code organization

Throughout this book we've talked about packages here and there. In chapter 2 I gave a quick overview of what packages are and how you import them. We've seen a bunch of different packages from the core collection. Let's now take a deeper look at packages, and answer these questions:

- What is a package?
- How do you create packages?
- What happens when you import packages?
- What is the purpose of the `package` line at the top of a file?
- When is a good idea to split things into separate packages?
- How do you make your own package collections?

What is a package? 17.1

A package in Odin is just a folder. All the Odin files within that folder are considered part of the package. Packages have two use cases:

- Creating libraries
- Internal organization of a program

All the things you import from `core` such as `core:fmt` are packages.

As we shall see creating libraries using packages is very straightforward. However, slightly more care must be taken when using packages for internal program organization. This is because packages must be independent, meaning that you cannot have circular dependencies between packages. More about that in a bit.

Creating a package 17.2

You can create local packages within your project by creating a folder and importing it. If you have a folder called `xml`, then you can import it as a package like so:

```
import "xml"
```

Now you can use the things from within the `xml` folder by typing:

```
xml_parser: xml.Parser
parsed_result := xml.parse(&xml_parser, some_data)
```

If you want an import to have a different name than the folder name, then you can give it an alias. This is common when importing the raylib package (a library for creating video games):

```
import rl "vendor:raylib"
```

`vendor` is a collection, just like `core` and `base`. You find the `vendor` folder in your Odin compiler folder. It contains many popular third-party packages.

Now all those things from the folder `<odin>/vendor/raylib` are available under the name `rl`. We say that `rl` is the *namespace* given to those things.

The namespace given to imported packages is based on the last folder name of the import. Meaning that if you import `core:unicode/utf8`, then you'll be able to access the imported code under the namespace `utf8`. This can lead to conflicts. If two packages use the same folder name, then you'll get a compilation error:

```
// This will not compile because
// the import name `fmt` is used twice.
import "core:fmt"
import "localization/fmt"
```

You can fix this by giving one of the namespaces a local alias:

```
import "core:fmt"
import lfmt "localization/fmt"
```

The above means that you can never end up with unfixable naming conflicts between packages: All the stuff inside the package is kept within the package's namespace and the namespace itself is possible to rename.

From C / C++

Those who have programmed C a lot can testify that name collisions between libraries do happen. Because of this C libs often have prefixes on all the stuff they expose. In Odin the prefix (namespace) is instead chosen at the import site, which is more robust.

The package name 17.3

At the top of each file you must have a line like:

```
package some_package
```

This is referred to as the *package name*. All the files within a package must start with the same package name line.

This package name has nothing to do with the namespace a package is given within your code. Meaning that when you import for example `core:fmt`:

```
import "core:fmt"
```

Then the namespace `fmt` has nothing to do with whichever package name the `fmt` package uses internally.

The package name is used to make the code within the package uniquely identifiable across all packages that make up your program. The package name is not something you use within your code, but something the compiler and linker uses to uniquely identify every single thing within every single package, regardless how it was imported and regardless of who imported it.

You can investigate this by compiling your program into an *intermediate representation* (IR). Try this:

```
odin build . -build-mode=llvm
```

You'll then get a bunch of `.ll` files as output. There's one for each package name.

You can see how the different `.ll` files reference things in other `.ll` files: The package name will always be used as a prefix.

About LLVM: The Odin compiler uses LLVM as the *compiler backend*. The Odin compiler processes the Odin source code and tells LLVM what the code does. LLVM generates the IR, which it can then turn into machine code. The LLVM IR is similar to assembly language.

Because of this you should try to choose a unique package name for each package. It does not have to be the same as the directory name, so you can keep the directory name short and simple and try to come up with something more unique for the package name.

Choosing a unique package name is extra important if you are going to distribute your code as a library for others to use. An Odin program cannot import two packages that have the same package name. This might sound like a scary limitation, but it really isn't. Odin libraries tend to be distributed in source code form, so if anyone uses your library and has a package name conflict, then they can easily just change the name by editing the `package package_name` lines in all the files. But in any case, let's try to avoid that by choosing a unique package name!

However, say that you are *not* going to distribute your code as a library. Perhaps you're just making a program that consists of a few packages. In that case the package names are less important, you can choose whatever you like without thinking too much about it. You just have to make sure that all packages you use within your program have unique package names.

For example, in the video games I've made, my main game package just has the name `game`. Since I am not distributing that code as a library, I don't need to worry about anyone else naming their package `game`.

Make sure your packages are independent 17.4

The Odin package system does not allow cyclic dependencies. This means that package `A` can import package `B` only if package `B` does not import package `A`. With this in mind, it is important that the code you try to move into a separate package is *truly independent*.

A common problem is that you take a part of your program and make it into a package by putting those things in a subfolder, just to later notice that you need a cyclic dependency to continue working.

Say that you have this folder structure, where `game` is a folder that contains two subfolders `json` and `renderer`:

```
game
    json
    renderer
```

The `json` folder implements reading and writing a JSON data format. The `renderer` package implements some kind of 2D or 3D renderer for a game.

The `game` package uses the `renderer` package to draw things on the screen. If you notice a need to reference things from the `game` package within the `renderer` package, then you will not be able to do so, because that will create a cyclic dependency. An example of a thing one might want to reference from such a "parent package" are some commonly used *types*. Therefore one might introduce a `types` package:

```
game
    types
    json
    renderer
```

`types` can then be imported from `game`, `json` and `renderer` given that `types` does not import any of them.

The need for this kind of `types` package is already a slight warning sign that your packages aren't truly independent, but many have created Odin programs in this way.

If you cannot untangle the dependencies and it just feels like you're fighting the package system, then don't split those things into separate packages. Many people just make their whole program as a single big package. One downside of this is that Odin has no other concept of a namespace: The only way to get a namespace such as `fmt` in `fmt.println()`, is by importing a package called `fmt`. This means that if you want to compartmentalize things without splitting things into packages, then you'll instead have to add prefixes to procedure names, such as putting `renderer_` in front of `renderer_init :: proc() {}`. But honestly, that's not a real problem: `renderer_init()` and `renderer.init()` is the same amount of typing.

I've made whole video games as a single big package. It works fine!

For example, you could put all those rendering-related procedures in `renderer.odin` within your big package. By default everything within a package is visible to everything else within a package, so moving things internally within a package is very simple.

A good idea is to start out with your program as a single big package and only split things into a package if you find truly independent things. Don't try to compartmentalize for the sake of compartmentalizing. Early-on in a project, it is very hard to know where to draw sensible boundaries between packages. More often than not will you realize that you put the boundary in the wrong place.

You should think of packages as independent libraries.

Breaking cyclic dependencies using callbacks 17.5

Say that you have a program with this file structure:

```
main.odin
printer/printer.odin
```

This is two packages, because it is two separate folders. Say that the `main` package imports `printer`. This means that `printer` cannot import `main`. In other words, the following will not work:

File: `main.odin`

```
package main

import "core:fmt"
import "printer"

print_message :: proc(msg: string) {
    fmt.println(msg)
}

main :: proc() {
    printer.do_work()
}
```

File: `printer/printer.odin`

```
package printer

import "core:fmt"

import main ".."

do_work :: proc() {
    fmt.println("First")
    main.print_message("Middle")
    fmt.println("Last")
}
```

`import main ".."` means that we are importing the parent directory (denoted by `..`). We give the imported package the alias `main`. When importing something like `..` we must provide an alias because the path `..` contains no word to use as import name.

This is a cyclic dependency! We want the `do_work` procedure in the `printer` package to be able to run the procedure `print_message` that lives inside the `main` package. But the `main` package imported the `printer` package. We can break the cycle by removing the line `import main ".."` and instead using a callback:

A callback is just a procedure assigned to some variable, that can later be run.

File: `main.odin`

```
package main
```

```
import "core:fmt"
import "printer"

print_message :: proc(msg: string) {
    fmt.println(msg)
}

main :: proc() {
    printer.print_callback = print_message
    printer.do_work()
}
```

File: `printer/printer.odin`

```
package printer

import "core:fmt"

print_callback: proc(msg: string)

do_work :: proc() {
    fmt.println("First")

    if print_callback != nil {
        print_callback("Middle")
    }

    fmt.println("Last")
}
```

The line `print_callback: proc(msg: string)` declares a global variable of type procedure. In the `main` procedure we assign a procedure to it. The `print_callback` variable can later be used to run the assigned procedure. There's a `nil` check inside `do_work`, in case the programmer hasn't given `print_callback` a value.

Run this program by navigating into the folder of `main.odin` and executing

```
odin run .
```

It will print:

```
First
Middle
Last
```

As we see, the `printer` package was able to run code in the `main` package using the callback procedure.

If you want to group some callbacks, then you can put several of them in a struct. Some people refer to such a struct as an *interface*:

```
Printer_Interface :: struct {
    print_callback: proc(msg: string),
    end_of_work: proc(),
}
```

You can use this interface like so:

File: `main.odin`

```
package main

import "core:fmt"
import "printer"

print_message :: proc(msg: string) {
    fmt.println(msg)
}

end_of_work :: proc() {
    fmt.println("Let's all go home")
}

main :: proc() {
    interface := printer.Printer_Interface {
        print = print_message,
        end_of_work = end_of_work,
    }
    printer.interface = interface
    printer.do_work()
}
```

File: `printer/printer.odin`

```
package printer

import "core:fmt"

Printer_Interface :: struct {
    print: proc(msg: string),
```

```
    end_of_work: proc(),
}

interface: Printer_Interface

do_work :: proc() {
    fmt.println("First")

    if interface.print != nil {
        interface.print("Middle")
    }

    fmt.println("Last")

    if interface.end_of_work != nil {
        interface.end_of_work()
    }
}
```

Running this program will print:

```
First
Middle
Last
Let's all go home
```

Such interfaces are common in many libraries. It makes the library independent while still allowing for flexibility and extensibility.

File and package private 17.6

By default, everything within a package sees everything else in the package, across all files. Splitting things into files within a package is therefore mostly for logical organization. However, you can add

```
#+private file
```

at the top of a file (before the `package` line) to make the contents of that file inaccessible to the rest of the package. Then, in front of some procedures you can add `@(private="package")` to make those visible to the rest of the package. There's an example that does this:

```
#+private file
package my_package

// Can only be used in this file
some_int: int

// Can be used in whole package
@(private="package")
some_proc :: proc () {

}
```

If you just write `#+private` at the top of a file, then it is the same as writing `#+private package`. This makes the contents of that file only accessible within the package. Any other package that later imports this package will not be able to use the contents of that file.

You can also use `@(private="file")` to make specific things private to the file:

```
package my_package

// Can only be used in this file
@(private="file")
some_int: int
```

These things we put in front of procedures that start with `@` are called *procedure attributes*. We've previously seen **`@require_results`** and **`@init`**/**`@fini`**, which are also procedure attributes. You can use multiple procedure attributes:

```
@(private="file", require_results)
get_number :: proc() -> int {
    return 5
}
```

Note that you must have parentheses around the attributes if you use multiple attributes or if they contain something like `="file"`. You can skip the parentheses if you have a single, simple attribute, like `@init`.

Making your own collections 17.7

We've seen the `core`, `base` and `vendor` collections. You import packages from them by writing

```
import "core:fmt"
import "base:runtime"
import "vendor:vulkan"
```

These collections are automatically set up by the Odin compiler, they point to the `core`, `base` and `vendor` folders within the Odin compiler folder.

You can define your own collections when running the Odin compiler, like this:

```
odin run . -collection:libs=c:\cool_odin_libs
```

Now you can put all your favorite third-party Odin libs in the folder `c:\cool_odin_libs` and then import them like this:

```
import "libs:aseprite"
```

Off-topic: I love the aseprite pixel art editor. blob has made a **wonderful package** to import aseprite files.

Video 17.8

I have made a video about Odin's package system:

https://www.youtube.com/watch?v=Ntt73Zdoztc

Chapter 18

You (probably) don't need a build system

If you come from a language like C or C++, then you've probably run into build systems such as CMake or premake. Build systems are usually used to make sure that all the parts of your program are compiled and that required dependencies and libraries are imported correctly. However, build systems are annoying to use. I find myself often avoiding C++ projects that depend on something like CMake.

A simple script can also be seen as a build system. But when I say build system in this chapter, then I mean "Big Build Systems" such as CMake.

When programming Odin, you probably don't need a build system. Why? Let's look at how to build your program, import libraries and write platform specific code. I'll summarize our findings at the end of the chapter.

Compiling your program 18.1

First off, in order to compile a program you usually navigate to the folder where the program lives and run

```
odin build .
```

This takes all the `.odin` files in the folder and compiles them as a single package.

If you wish to skip some file, then add `#+build ignore` at the top of that file. This is useful for avoiding compilation of examples etc.

This takes care of one use case for build systems: Making sure that newly created files are "picked up" by the compiler. In Odin all files within a folder are considered as part of the package, so they are "picked up" automatically.

Importing packages 18.2

When you type something like

```
import "core:fmt"
```

then you are importing all the files in `<odin>/core/fmt`. That Odin code is *not* pre-compiled. Therefore, when importing libraries written in Odin, there are no dependencies on any pre-compiled binaries. Since the packages are folder-based, all those files inside the `fmt` folder are available as part of that `fmt` package.

So any package your program imports also has all its files "picked up" by the compiler. There's no need to have a build system for that either.

Importing non-Odin libraries 18.3

Your program may use libraries that are not written in Odin. Those libraries will need some kind of pre-compiled binary plus some Odin code that describes how to interact with that binary. The Odin code that describes this interaction is called the *bindings* for the library.

In the `vendor` collection you'll find bindings for many libraries written in other languages than Odin. One such library is **raylib** , which you import like so:

```
import rl "vendor:raylib"
```

Raylib is written in C and has pre-compiled binaries, but you do not need to manually link them. Let's look at why. If you open **vendor/raylib/raylib.odin** , then you'll see this:

By "link" I refer to what the *linker* does. The linker is a program that runs after the compiler. It combines separately compiled things into a single executable or library.

So if we use raylib, then the linker needs to combine the compiled Odin code and raylib's pre-compiled binaries.

```
when ODIN_OS == .Windows {
    @(extra_linker_flags="/NODEFAULTLIB:" + ("msvcrt" when RAYLIB_SHARED else
"libcmt"))
    foreign import lib {
        "windows/raylibdll.lib" when RAYLIB_SHARED else "windows/raylib.lib" ,
        "system:Winmm.lib",
        "system:Gdi32.lib",
        "system:User32.lib",
        "system:Shell32.lib",
    }
} else when ODIN_OS == .Linux  {
    // etc etc
```

When raylib is imported, then the `foreign import lib {}` block tells the compiler which libraries to link for you. It's also possible to specify extra linker flags. The raylib bindings come with binaries for Windows, Linux and macOS. You do not need to download any extra binaries for those platforms.

Note that the word `lib` from the `foreign import lib {` line is just a name, it is reused further down in `raylib.odin`:

```
@(default_calling_convention="c")
foreign lib {
    // Bindings to procs go here, such as:
    InitWindow :: proc(width, height: c.int, title: cstring) ---
}
```

The `foreign import lib {}` block tells the compiler which library files to link. The `foreign lib {}` block describes which procedures to make available from within the linked library. You list all the procedures and their signatures. Here "signature" refers to what parameters and return values the procedure has.

> Note the `---` at the end of the `InitWindow` line, it is required when you omit the body of the procedure. This means that we are leaving the procedure uninitialized, since it will get a value from library.

The `@(default_calling_convention="c")` line is there because raylib is written in C, so the procedures in there must be called using the C calling convention. Odin and C calls procedures in different ways. For example, remember that Odin passes along an implicit `context`. Trying to call a C procedure and forcing the context into it is essentially like calling a procedure with the wrong number of parameters.

> `@(default_calling_convention="c")` is actually the default whenever you use `foreign lib` blocks, but it is often included for clarity.

In chapter 21 we'll take a deeper look at making bindings to libraries written in C.

Caveats *18.3.1*

The provided binaries for non-Odin libraries tend to work well on Windows and macOS and most Linux distributions. However, on some Linux distributions you might see linker errors due to things like mismatching glibc versions. In order to fix this you may need to replace the library file with one that works on your distribution.

Also, some libraries have configuration parameters. For example, when you compile raylib from C source, then there are some configuration flags that are compiled into the library. In

order to change those you must recompile raylib from source. If you do, then you'd have to replace the binary library files distributed with Odin, such as `<odin>/vendor/raylib/windows/raylib.lib`

Whenever you make such a change to a package in `vendor` or `core`, then I'd recommend that you make your own local copy of the package. That way you can still upgrade the compiler without your package changes getting in the way. Create your copy by simply copying the whole package folder to your project directory. For example, you can copy the `raylib` folder from `<odin>/vendor/raylib` to `<your_project>/raylib` and then just write `import rl "raylib"` instead of `import rl "vendor:raylib"`. Now you can make any changes you need inside your own copy of raylib without worrying about future compiler updates.

If you use source control, such as Git, then you can just include this whole copy of raylib in your repository. It's a good way to make sure everyone on the project has the same version.

Platform specific code 18.4

If you want to make a file compile only on specific platforms, then there are two ways:

- Add `#+build linux, darwin, netbsd` at the top of a file to make it compile only on Linux, macOS (darwin) and NetBSD.
- Add a suffix to the filename such as `logger_windows.odin`. Then that file will only compile on Windows.

If you want a file to not compile on Windows *and* not compile on Linux, then add this to the top of the file:

```
#+build !windows
#+build !linux
```

You should *not* do this:

```
#+build !windows, !linux
```

That means "compile not on Windows *or* not on Linux", which doesn't make any sense.

It would compile on both Windows and Linux. The comma between them is an "OR". So it's like writing an if-statement with the condition `!windows || !linux`, which would be true on both Windows and Linux.

You can also use architecture names with the `#+build` tag:

```
#+build !amd64 windows
```

This will skip compilation if you are on Windows but don't have an amd64 architecture.

You may also want to create platform-specific abstractions. Perhaps you want a procedure called `get_log_file_path()` to be implemented differently on different platforms. In that case you can call `get_log_file_path()` in our platform-independent code and in the same package have `log_windows.odin` and `log_linux.odin` files and in those provide the `get_log_file_path` procedure for each platform.

Compiling multiple binaries 18.5

You might have a project where you need to run the Odin compiler multiple times. For example you might need to first compile a DLL (shared library) and then an executable. In that case you need to run two build commands:

```
odin build game -build-mode:dll
odin build main
```

Here we are (on Windows) compiling the `game` folder into a `game.dll` file, and the `main` folder into a `main.exe` file.

You can't do that with a single invocation of the Odin compiler. You need to use the two separate commands. However, this does not mean that you need a build system. Just make a batch or bash script that runs the two commands. I tend to do one `.bat` script for Windows and one `.sh` for mac / Linux. Since these scripts are so simple it's usually not a big deal to have two of them.

More complicated build setups 18.6

In some cases build scripts may get more complicated than the one in the previous section. An example is when you want to provide multiple different build modes, or perhaps download some assets from the internet before building.

In such cases you *could* use a build system like CMake. But in most cases you're probably better off with a script. The rest of the things we've seen in this chapter simplifies building enough that a script is usually good enough.

For example, in my **Odin + Sokol + Hot reload template** I provide a Python script that both builds the Odin code and also downloads and sets up the latest Sokol bindings (a library for cross-platform graphics programming). You can see that script **here** .

I could have used Odin itself to write this script. But I wanted the script to download files from the internet without adding in too many dependencies, and there is currently no such library available in the `core` or `vendor` collections. Python comes with such capability without having to install any additional dependencies.

Summary 18.7

To summarize, here's what we saw in this chapter:

- `odin build .` compiles all files in the current directory as a package. This can be thought of as your 'main program package'.
- As your program compiles, all Odin packages it imports also get compiled. Odin packages are usually not pre-compiled, so a package written purely in Odin has no binary dependencies.
- A package that contains bindings to a library written in another language needs to have some binary library files linked. The package that provides the bindings can specify which binary files to pass on to the linker as well as what procedures to expose from within that library.
- You can create platform specific files using `#+build windows` tags or a `_windows.odin` file suffix.
- If you need to compile several binaries as part of your build process, then just make a tiny batch / bash script that runs multiple `odin build` commands.
- For really complicated build set ups you *could* use something like CMake. But you're probably better off with a build script written in for example Python or Odin. The other points on this list simplifies the build process enough, making CMake "overkill" for the problem.

When you combine these points, then there is little need for a build system such as CMake or premake. Such a build system will probably cause more trouble than it is worth. Using such a build system will also make people who download your source code more likely to give up, due to them not having your build system installed.

Chapter 19

Reflection and Run-Time Type Information (RTTI)

Odin allows you to get information about the type of an object while the program is running. This is known as Run-Time Type Information (RTTI). We can use RTTI to ask questions about types, such as: What are the field names of a struct? What are the names of an enum's members? What variants does a union have? The act of using RTTI to get this information is sometimes referred to as *reflection*.

Let's look at some examples that use reflection. Many of the reflection features of Odin are implemented in `core:reflect`, which in turn uses the built-in RTTI features of the language.

Example: Enum member from string 19.1

In the following example, we'll see how to look up an enum member using a string. This is useful in many situations, for example when reading the name of an enum member from a configuration file.

```
import "core:reflect"

Cat_Color :: enum {
    None,   // 0
    Tabby,  // 1
    Orange, // 2
    Calico, // 3
}

main :: proc() {
    color_name := "Orange"

    if color, ok := reflect.enum_from_name(Cat_Color, color_name); ok {
        // Will print "2 Orange"
        fmt.println(int(color), color)
    }
}
```

`color_name` is a string with the value `"Orange"`. The enum `Cat_Color` has a member with the name `Orange`. We want to get the value associated with `Orange` based on the string. We use

`reflect.enum_from_name` to do that. This means that the result of `enum_from_name` will be equivalent to the identifier `Cat_Color.Orange`, which has the numeric value `2`.

There isn't any obvious string version `Cat_Color.Orange`. How is it that `reflect.enum_from_name` could figure this out? Let's take a look at what that procedure does:

```
enum_from_name :: proc($Enum_Type: typeid, name: string) -> (value: Enum_Type,
ok: bool) {
    ti := type_info_base(type_info_of(Enum_Type))
    if eti, eti_ok := ti.variant.(runtime.Type_Info_Enum); eti_ok {
        for value_name, i in eti.names {
            if value_name != name {
                continue
            }
            v := eti.values[i]
            value = Enum_Type(v)
            ok = true
            return
        }
    }
    return
}
```

Note the `$` on the parameter `$Enum_Type: typeid`. Many of Odin's reflection procedures are implemented by combining RTTI with **parametric polymorphism**.

This procedure takes the `typeid` we supply it, in our case `Cat_Color`, and fetches the `Type_Info` for it using the `type_info_of` procedure. `Type_Info` contains the run-time type information for a specific type. The procedure checks that this `Type_Info` actually represents an enum. If it does then it can iterate over the names in the `runtime.Type_Info_Enum` variant and look for an enum member with the name we were looking for (`"Orange"`).

You may have noted the `type_info_base` call on the line:

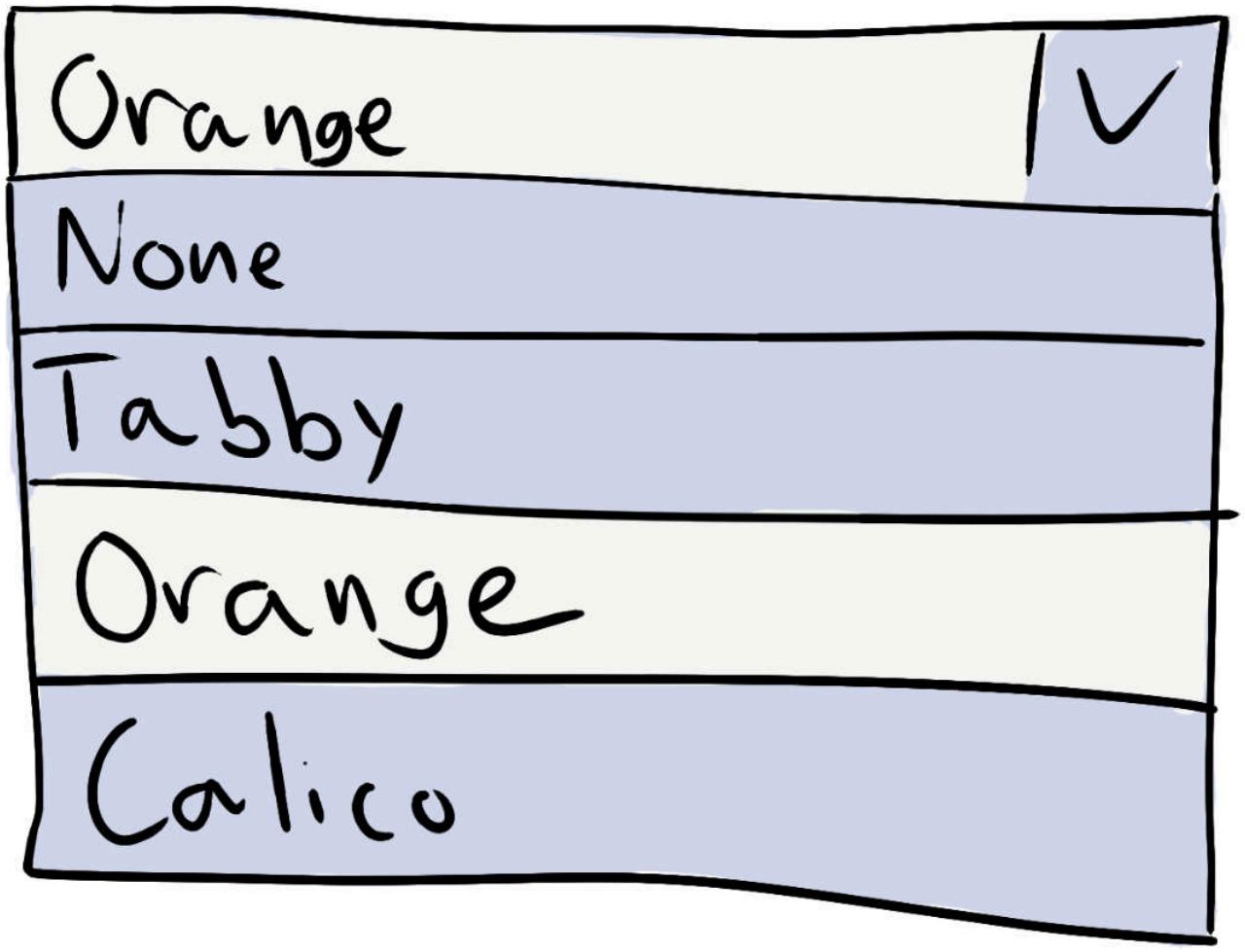
```
ti := type_info_base(type_info_of(Enum_Type))
```

It is there because the return value of `type_info_of(Enum_Type)` is a `Type_Info` of variant `Type_Info_Named`. This variant says that the type name is `Cat_Color`. It also contains a "base type". This base type is another `Type_Info` of variant `Type_Info_Enum`, which is what we are interested in. So `type_info_base` "unwraps" any `Type_Info_Named` and directly gives us the base type.

These `Type_Info` structs were created by the compiler while it compiled your program. It's just some extra information that the compiler stored as it went about its business.

Case study: Enum dropdown menu 19.2

Say that we have a program with an Immediate Mode UI where we want the user to be able to select one of the members of an enum. We need to display the enum names on the screen and let the user pick one of them. If the user picks a new value, then we need to store which one they picked. This can be done using a dropdown menu.



An example of how a dropdown menu can look.

We don't need to worry about how to write the dropdown GUI code here. I'll just present the signature of the dropdown GUI procedure:

```
gui_dropdown :: proc(rect: Rect, values: []$T, names: []string, cur: T) -> (T,
bool) {
    // GUI code goes here: Draw a list of
    // selectable values using `names`.

    // If the user selects one of the
    // names, then it will use the index of
    // the name to fetch the value from
    // the `values` slice. The new value
    // will be returned.

    // The proc returns `true` if a new
    // value was selected.
}
```

Think of `rect: Rect` as Rectangle that says where on the screen to draw the dropdown menu.

I included an example implementation of this procedure in **appendix C**. It's from my game CAT & ONION and is completely unedited. It's just there to show a real-world example.

The procedure above is general and knows nothing about enums. It has two return values. It returns `true` in the second return value if a new value was picked. The newly picked value will be available in the first return value.

Here's a procedure that uses `gui_dropdown` in combination with reflection to create a dropdown that works with enums:

```
gui_enum_dropdown :: proc(rect: Rect, cur: $T) -> (T, bool) where
intrinsics.type_is_enum(T) {
    names := reflect.enum_field_names(T)
    values := reflect.enum_field_values(T)
    values_i64 := slice.reinterpret([]i64, values)
    new_val, changed := gui_dropdown(rect, values_i64, names, i64(cur))
    return T(new_val), changed
}
```

Import `core:slice` and `base:intrinsics`.

This procedure has a polymorphic parameter `cur: $T`. This means that `T` can be of any type, given that it is an enum, which `where intrinsics.type_is_enum(T)` guarantees.

It uses reflection to fetch two slices:

- `names` is of type `[]string` and contains all the enum member names.
- `values` is of type `[]Type_Info_Enum_Value` and contains all the enum member values.

`Type_Info_Enum_Value` is defined as `Type_Info_Enum_Value :: distinct i64` .

When we call `gui_dropdown` it expects the parameters `values: []$T` and `cur: T` to use the same type `T`. That's why we use `slice.reinterpret` to turn the type of `values` into `[]i64`. We also cast `cur` into the type `i64`.

`slice.reinterpret` pretends that a slice contains elements of some other type.

We can't just cast `values` because of the `distinct` in the definition of `Type_Info_Enum_Value :: distinct i64`.

Instead of `slice.reinterpret([]i64, values)` one might try to instead use `slice.reinterpret([]T, values)` and not cast `cur`. But that is dangerous. `reinterpret` only works well when the types are of the same size. `T` is always some kind of enum, but it doesn't always have a 64 bit backing type.

The return value of `gui_dropdown` is saved in `new_val`. It is cast to the type `T` before returning it. This procedure should now work fine with a value of any enum type!

To draw a dropdown using `gui_enum_dropdown`, you'd do something like this:

```
Cat_Color :: enum {
    None,
    Tabby,
    Orange,
    Calico,
}

current_cat_color: Cat_Color

ui_update :: proc() {
    dropdown_rect := make_some_ui_rectangle()
    if new_color, changed := gui_enum_dropdown(dropdown_rect, current_cat_color);
changed {
        current_cat_color = new_color
    }
}
```

There is a global variable `current_cat_color` which stores a value of type `Cat_Color`. We display a dropdown by just passing in `current_cat_color` to `gui_enum_dropdown`. If the second return value of `gui_enum_dropdown` returns true, then we update `current_cat_color` to the newly picked value.

I find it oddly satisfying that a single parameter can be used to figure out both the current enum value, as well as all the other possible values and their names.

Reflection is *very useful* for making this kind of editors.

When to not use reflection 19.3

Avoid using reflection when there is a built-in way to do the same thing. Here's an example of something you shouldn't do:

In the JSON package (`core:encoding/json`) there is a `Value` type:

```
Value :: union {
    Null,
    Integer,
    Float,
    Boolean,
    String,
    Array,
    Object,
}
```

This type is a union. If you have a variable `val` that is of type `json.Value`, then you could use reflection to check the current variant type:

```
if reflect.union_variant_typeid(val) == json.Array {
    // process array
}
```

However, don't do the above! Instead, just do this:

```
if _, ok := val.(json.Array); ok {
    // process array
}
```

The reflection method is much more complicated and expensive. The second method is fast and intrinsic to the language.

If you can do something in two ways where one uses reflection, then pick the one that doesn't use reflection, simply because it is usually cheaper.

Learning more about reflection 19.4

Whenever you need anything reflection related, then search in `<odin>/core/reflect/reflect.odin`. There are things in there to get run-time type information about **structs**, **enums**, **unions**, **bit fields**, and more!

If you want a nice and big case study of how to use the reflection system, then check out the JSON marshaler that is part of the core collection. You'll find it in `<odin>/core/encoding/json/marshal.odin`. That file uses reflection in a bunch of different ways in order to turn almost any value into JSON. Also see `unmarshal.odin` in the same folder for more examples.

Chapter 20

Data-oriented design

Odin is marketed as a "data-oriented language" on its **website** . What does that mean and how does one write code in a data-oriented way?

Data-oriented design means writing code in such a way that the code can make efficient use of modern CPUs. This might sound like an obvious thing to do, but some programming paradigms lead to unnecessarily slow code. For example, it is hard to make object-oriented code efficiently utilize the CPU. We'll see some reasons as to why in this chapter.

A lot of data-oriented design revolves around writing code that is *cache friendly*. The CPU has a number of built-in caches called L1, L2 and L3. These caches are small amounts of memory that live on the chip of the CPU. A lower number means physically closer to the CPU core, and also faster, but smaller in size. So L1 is very fast, very close to the core, but also very small. If you can keep L1 filled with whatever data your program needs next, then your code will run very fast.

This is quite a big topic and I can only cover the basics here. But hopefully it gives you some good first ideas.

Avoid doing many small heap allocations 20.1

This is in my opinion the most easily approachable and important idea of data-oriented design.

If you separately heap allocate all the elements of an array, then the elements may get scattered in memory. When they are scattered it becomes very hard to keep the CPU caches filled with relevant data. This will make your program run slower.

> Just the act of dynamically allocating memory is in itself slow. However, in this section we'll focus on how it can be slow to iterate an array where the elements are separately dynamically allocated.

Let's look at two examples of different memory layouts. After we've looked at the examples we'll see why the second example can lead to faster code than the first one.

The first example shows code that does a lot of small heap allocations:

```
Person :: struct {
    health: int,
    age: int,
}

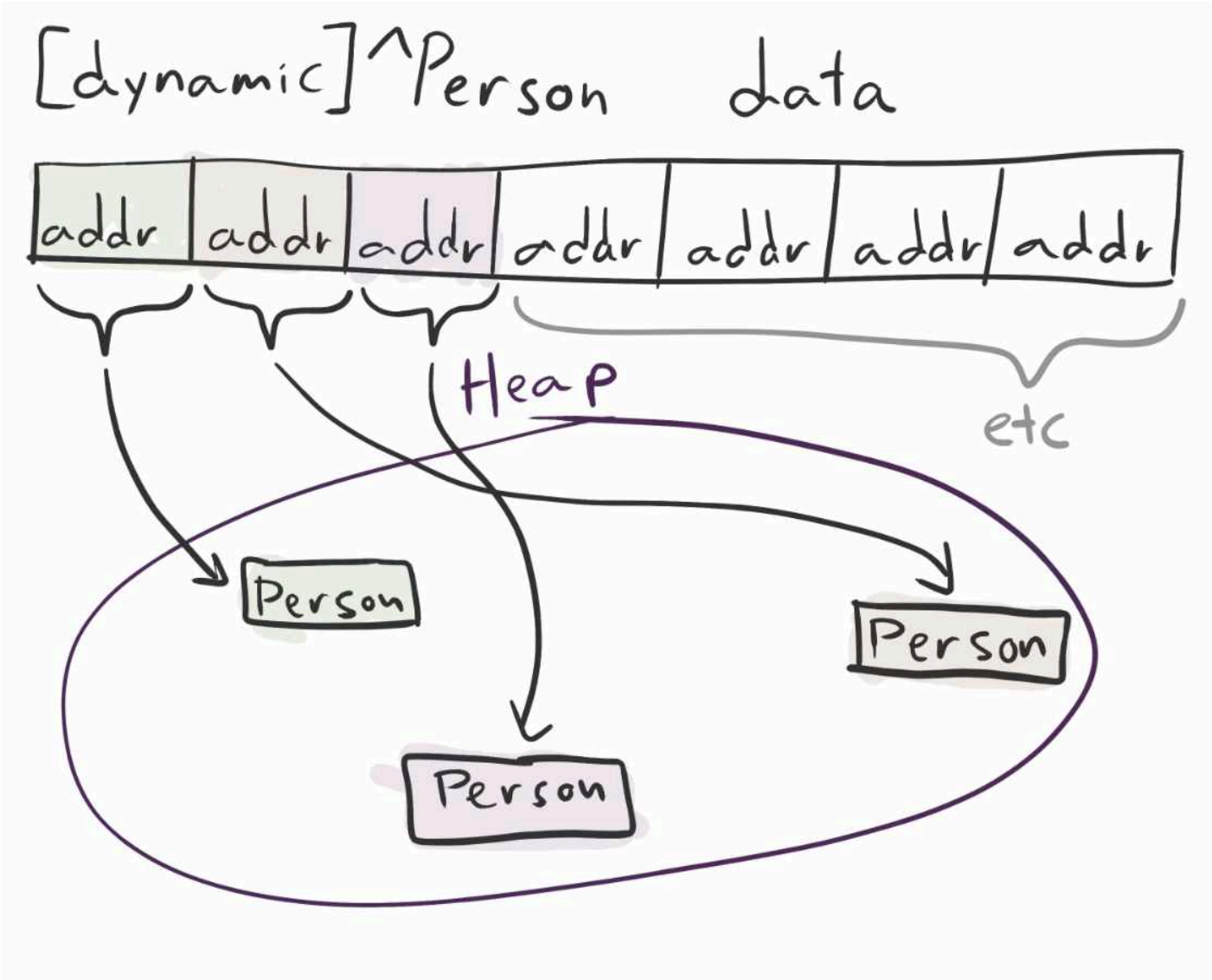
people: [dynamic]^Person

add_person :: proc(health: int, age: int) {
    p := new(Person)
    p^ = {
        health = health,
        age = age,
    }
    append(&people, p)
}
```

In this example, `people` has the type `[dynamic]^Person`, meaning that it is a dynamic array of pointers to `Person` structs. There is a procedure `add_person` that dynamically allocates an object of type `Person`. It does this by running `new(Person)`. The dynamically allocated `Person` object is given a value and then gets added to the `people` array.

This can make a newly created `Person` struct end up at a very different location in memory compared to the other ones in the `people` array. Below is an illustration that shows how each element of the array just points to somewhere else in the heap.

Here we assume that `context.allocator` is a default heap allocator.



The data of the `[dynamic]^Person` array is just a bunch of pointers (each containing a memory address, hence `addr` on each element of the array). Each of those pointers refer to some heap allocated memory. Note that the data of the dynamic array itself (the block of memory where the pointers are stored) is also heap allocated in our example, but that is not important.

Let's now look at an example where I've simplified the memory layout and removed all those small, separate heap allocations:

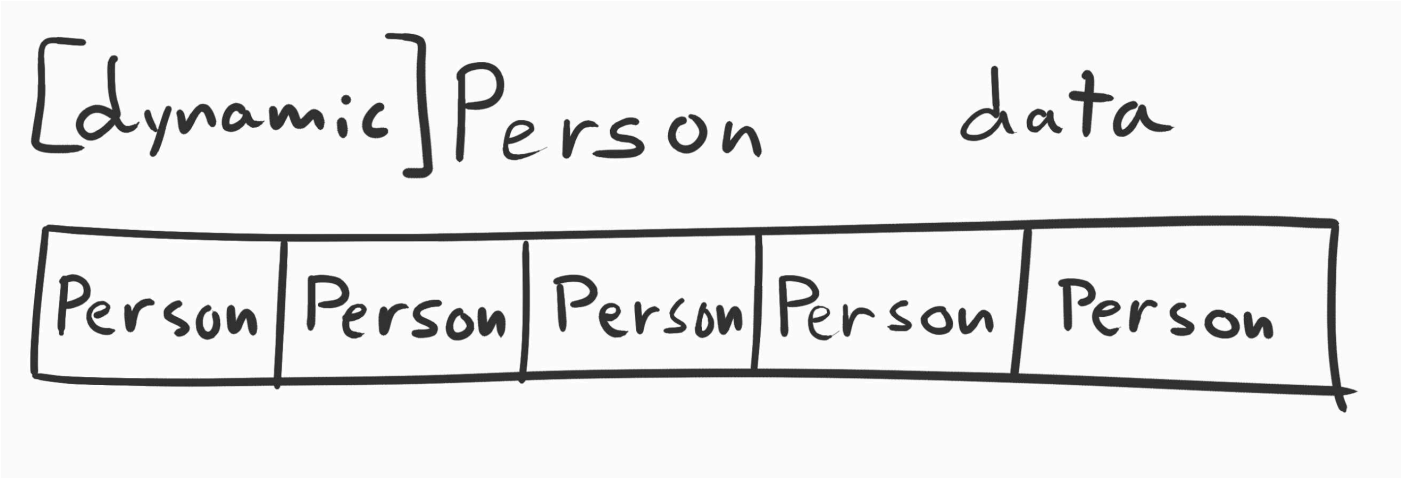
```
Person :: struct {
    health: int,
    age: int,
}

people: [dynamic]Person

add_person :: proc(health: int, age: int) {
    p := Person {
        health = health,
```

```
        age = age,
    }
    append(&people, p)
}
```

Note how I have removed the `^` in front of `Person` in `people: [dynamic]Person`. Also note that we no longer run `new` within `add_person`. This makes the memory layout much simpler, as illustrated in the image below. Each element of the array directly contains its own data without any indirection. All the elements are packed one-next-to-the-other in memory.



In the `[dynamic]Person` array all the `Person` structs live tightly packed within the array itself. There is no more indirection where each element points somewhere else.

Performance differences when iterating *20.1.1*

Now, let's say that our program has been running for a long time and elements have been added to `people` every now and then, resulting in a big array.

If we now want to calculate the average age of all the people, then this code would work regardless of which memory layout we chose earlier:

```
average_age := 0
for &p in people {
    average_age += p.age
}
average_age /= len(people)
```

In the case where we separately heap allocated each item of the array (the scattered case): For each lap of the loop `p.age` is fetched. But this `p` can be anywhere in memory, because of all the separate heap allocations. So for each lap of the loop, the computer is jumping around in memory in a very disorganized way. This will make it very hard for the computer to keep

the CPU caches filled, and thereby make it hard for the program to run at optimal speed. Why? The computer has two main ways to fill the CPU cache:

1. When the computer fetches some data from main memory, then it also brings along some additional data in the nearby memory regions. This block of data is known as a *cache line* or *cache block*.
2. When the computer fetches data several times, such as in a loop, then the computer starts guessing what will be fetched next. Based on these access patterns, it will try to *prefetch* data and put it into the CPU cache.

In the case where everything is separately heap allocated, it's unlikely that any of these two methods for filling the cache will work. The elements of the array may be scattered far apart, and in a disorganized way. So for each lap of the loop, it's likely that the cache has to be refilled, instead of immediately used. This is known as a *cache miss*. Going to the main memory and fetching data instead of using something readily available in the cache is *a lot* slower.

In the non-separately allocated case (the tight case): Each item of the array lives next to the other. If the array elements are small enough then point (1) above will ensure that several elements are in the cache at once. Since the elements are laid out in a very predictable way, point (2) may help fill the cache as well. So when the loop fetches `p.age` then it is likely already in the cache due to the previous laps of the loop having filled the cache.

Avoiding separate heap allocations for just one of the arrays in your whole program won't make a big difference. But if you try to program this way *in general*, then your programs will be much faster.

From C / C++

If you've used C++ with lots of inheritance, then you are essentially forced to do these separate allocations. You're probably used to seeing stuff like `std::vector<Person*>` etc. If this array stores sub-classes of `Person`, then you cannot just keep plain `Person` values in there because inheritance requires separate allocations, due to the varying memory size of each item.

How much faster is it? *20.1.2*

So how much faster will the cache friendly code be? It depends. Let's look at why.

Here's a naive benchmark that compares the two approaches:

```
package avoid_separate_heap_allocs

import fmt "core:fmt"
import "core:math/rand"
```

```
import "core:mem"
import "core:time"

Person :: struct {
    health: int,
    age: int,
}

NUM_ELEMS :: 10000
NUM_TEST_ITERS :: 10

make_person :: proc() -> Person {
    health := int(rand.int31_max(101))
    age := int(rand.int31_max(101))

    return {
        health = health,
        age = age,
    }
}

benchmark_scattered_array :: proc() -> f64 {
    people: [dynamic]^Person

    for _ in 0..<NUM_ELEMS {
        p := new(Person)
        p^ = make_person()
        append(&people, p)
    }

    age_sum: int
    start := time.now()
    for i in 0..<NUM_TEST_ITERS {
        for &p in people {
            age_sum += p.age
        }
    }
    end := time.now()
    fmt.println("Scattered array age sum:",
f32(age_sum)/(NUM_TEST_ITERS*NUM_ELEMS))

    return time.duration_milliseconds(time.diff(start, end))
}

benchmark_tight_array :: proc() -> f64 {
    people: [dynamic]Person
```

```
    for i in 0..<NUM_ELEMS {
        p := make_person()
        append(&people, p)
    }

    age_sum: int
    start := time.now()
    for i in 0..<NUM_TEST_ITERS {
        for &p in people {
            age_sum += p.age
        }
    }
    end := time.now()
    fmt.println("Tight array age sum:", f32(age_sum)/(NUM_TEST_ITERS*NUM_ELEMS))

    return time.duration_milliseconds(time.diff(start, end))
}

main :: proc() {
    time_scattered := benchmark_scattered_array()
    time_tight := benchmark_tight_array()
    fmt.printfln("Cache friendly method is %.2f times faster",
time_scattered/time_tight)
}
```

On my computer, running this program using `odin run . -o:none` displays:

```
Cache friendly method is 1.39 times faster
```

If you run with `odin run . -o:speed`, then it says:

```
Cache friendly method is 4.41 times faster
```

`-o:speed` optimizes the program more. I show `-o:none` first because that showcases the pure difference between the two methods with respect to cache friendliness. With `-o:speed` you'll also get additional optimizations that are related to *vectorization*. This means that our loop is able to add several items in a single operation, which speeds things up even more.

Also, never take a benchmark as an absolute truth. The conditions in the benchmark are specific. Changing the memory access patterns or the way you populate memory may change the outcome. Just see these benchmarks as a general indication.

OK, that's quite good! However, the procedure `benchmark_scattered_array` that does the tests for the cache unfriendly method actually has an unfair advantage: It allocates all the separate `People` objects in a loop. This makes them unusually likely to end up at memory

locations that are near each other. In a real program you'd probably add one person and then a bit later, another one, etc. They wouldn't all get created at the same time. During the in-between time, other things may get allocated by other parts of the program (imagine that this is part of a bigger program). This makes it less likely that two heap allocated `Person` objects end up nearby in memory: There may already be stuff near the previous one, so the new one has to go somewhere else!

We can simulate that by allocating some memory just before the `p := new(Person)` line:

```
for i in 0 ..< 100 {
    _, _ = mem.alloc(rand.int_max(8)+8)
}
```

The code above does 100 separate allocations where each allocation uses something between 8 and 16 bytes. This will make the calls to `new(Person)` less likely to get memory that is unusually close to the previous element.

So now the `benchmark_scattered_array` procedure looks like this:

```
benchmark_scattered_array :: proc() -> f64 {
    people: [dynamic]^Person

    for _ in 0..<NUM_ELEMS {
        for i in 0 ..< 100 {
            _, _ = mem.alloc(rand.int_max(8)+8)
        }

        p := new(Person)
        p^ = make_person()
        append(&people, p)
    }

    age_sum: int
    start := time.now()
    for i in 0..<NUM_TEST_ITERS {
        for &p in people {
            age_sum += p.age
        }
    }
    end := time.now()
    fmt.println("Scattered array age sum:",
f32(age_sum)/(NUM_TEST_ITERS*NUM_ELEMS))
```

```
    return time.duration_milliseconds(time.diff(start, end))
}
```

Note: It's only how much time it takes to loop through the data that is measured. This looping is situated in-between the lines `start := time.now()` and `end := time.now()`. The only thing we changed above is dynamically allocating some memory before the measurement even started. None of that extra memory is added to the array. The only effect this has on the `people` array is that the heap allocated elements of the array are less likely to be nearby each other.

On my computer, when I run the full program with this new version of `benchmark_scattered_array`, then it outputs, using `odin run . -o:none`:

```
Cache friendly method is 3.89 times faster
```

and using `odin run . -o:speed`:

```
Cache friendly method is 20.99 times faster
```

This is a very significant difference!

An interesting thing to note here is that the cache friendly array will always show similar performance characteristics since it will always have the data laid out in the same way. The cache unfriendly method will, as we saw, take different amounts of time depending on how we filled it. So not only is it slower, it is also unpredictably slow.

Problem: Pointers get invalidated when the array grows *20.1.3*

Using these tightly packed arrays is faster. But it also comes with some problems that you might not have when using arrays with separately allocated elements. One such problem is that any pointer to an array element may become invalid if the array grows.

When a dynamic array grows, then all the contents of the dynamic array may get moved to a completely new location in memory. I mentioned this **before**, but let's discuss it from a different angle.

The code below creates a dynamic array of `People` structs and adds a single element to it. It then fetches a pointer to that new element. Thereafter it adds 1024 more elements to the array. That is very likely to make the dynamic array grow. When a dynamic array grows, all of its contents may move to a new memory address.

```
package ptr_moved

import "core:fmt"

main :: proc() {
    Person :: struct {
        health: int,
        age: int,
    }

    people: [dynamic]Person
    append(&people, Person { health = 10, age = 65 })
    person_ptr := &people[0]

    for i in 0..<1024 {
        append(&people, Person{})
    }

    fmt.printfln("%p", &people[0])
    fmt.printfln("%p", person_ptr)
}
```

In this program, note how the last two lines print the memory address of `&people[0]` and `person_ptr`. `person_ptr` was given the value `&people[0]` before we added all those 1024 elements to the array. If you run the code, then it will very likely print two different memory addresses:

```
0x231D92A3018
0x231D92982D8
```

This means that the whole array has moved and that `person_ptr` is pointing at invalid memory. Using it can lead to crashes and irregular behavior, especially if you use `person_ptr` to modify the data it points to: You are then likely to stomp on memory that is now in use by something else!

If you print `person_ptr.age` at the end of the program, like so:

```
fmt.println(person_ptr.age)
```

Then it may still output the correct value `65`. But that's just because that region of memory hasn't been reused by anything yet. It is a disaster waiting to happen.

To counter this I recommend the following: Never permanently store pointers to things inside an array that may grow. You can have a pointer around for a short time, but make sure the

array absolutely cannot grow while you are using that pointer.

For storing permanent references to an array element, I advise you to instead use *handles*. A handle is essentially the index of the element plus a "generation" counter that tells you if the thing at the index has been destroyed. Such a handle can look like this:

There's a classic blog post by Andre Weissflog about how **"handles are the better pointers"**

```
Handle :: struct {
    index: u32,
    generation: u32,
}
```

See the **Appendix A: Handle-based array** for an example implementation of a generic array that supports handles.

Also, see **Appendix B: Using only fixed arrays** for an example on how to use even less memory allocations.

Structure of arrays (SoA) 20.2

Since the computer grabs memory in blocks called *cache lines* when it fills the cache, then it is beneficial if the data needed next is as close as possible to the data you're currently grabbing. You're then likely to fetch several pieces of relevant data for the price of fetching one.

Let's look at how iterating over data can become faster if we rearrange how arrays store data. We'll compare the default approach, called Arrays of Structures (AoS), to the Structure of Arrays (SoA) approach.

Let's begin by describing Arrays of Structures. If you have a struct like so:

```
Person :: struct {
    health: int,
    age: int,
}
```

And make a dynamic array of `Person`, filled with 1000 elements:

```
people: [dynamic]Person
```

```
for i in 0..<1000 {
    append(&people, Person { health = rand.int_max(101), age = rand.int_max(101)
})
}
```

then the memory layout of that array looks like this:

```
people[0].health
people[0].age
people[1].health
people[1].age
people[2].health
people[2].age
people[3].health
people[3].age
people[4].health
people[4].age
... etc
```

We see that everything related to `people[0]` comes first, and after that comes everything related to `people[1]`, etc.

Let's say that you loop through the array in order to sum the age of all the people. After the loop you use this sum to create an average age:

```
age_sum: int
for &p in people {
    age_sum += p.age
}
age_avg := f32(age_sum)/f32(len(people))
```

This loop is only accessing the `age` field. However, when it fetches `people[0].age` to put it in the cache, then it is also plucking out an area of memory around that value. So the cache line that gets fetched may also contain `people[1].health`. But our loop doesn't even need the health field! So you'll have to refill the cache unnecessarily often since half of the cache is filled with unneeded data.

Let's look at how you can rearrange this. You can change the declaration of the dynamic array to make it store the elements in a Structure of Arrays layout. There are two differences below: We added `#soa` in front of `[dynamic]` and we use `append_soa` instead of `append`.

```
people: #soa[dynamic]Person

for i in 0..<1000 {
```

```
    append_soa(&people, Person{ health = rand.int_max(101), age =
rand.int_max(101)})
}
```

The memory layout of `people` will now use a Structure of Arrays layout. That memory layout will look like this:

```
people[0].health
people[1].health
people[2].health
people[3].health
people[4].health
... and 995 more health items
people[0].age
people[1].age
people[2].age
people[3].age
people[4].age
... and 995 more age items
```

This is quite different from before! All the `health` fields are next to each other in memory and then comes all the `age` fields.

If you now re-run the same age-average-calculation code:

```
age_sum: int
for &p in people {
    age_sum += p.age
}
age_avg := f32(age_sum)/f32(len(people))
```

Then whenever `p.age` is fetched and placed in the cache, then the cache line will mostly contain `age` data of the current and surrounding elements, without any `health` data mixed in-between. This will make your loop run faster since it does not have to refill the cache as often.

When looping over data that is laid out in a predictable way, then the next values may be *prefetched*. This means that the computer sees a predictable access pattern in your code. It will try to fetch whatever data your code may need next, ahead of time.

However, the prefetcher may have to go all the way to main memory. That will take some time. If your loop does very little each lap, then the prefetcher won't be able to get the data before it is needed.

So prefetching can give good benefits for data that is laid out in a predictable way, even though it is not packed in any kind of SoA way. But your code may also outrun the prefetcher.

How much faster is SoA? *20.2.1*

How much faster will our cache friendly code be? Again, it depends.

Below is a benchmark program that you can try running:

```
package soa

import fmt "core:fmt"
import "core:math/rand"
import "core:mem"
import "core:time"

NUM_ELEMS :: 10000
NUM_TEST_ITERS :: 10

Person :: struct($N: int) {
    health: int,
    age: int,

    // We add in more and more extra bytes
    // by varying the parameter N. For
    // bigger `extra_data` you may see
    // bigger benefits of SoA because of
    // being able to fill cache lines with
    // more relevant data.
    extra_data: [N]byte,
}

make_person :: proc($N: int) -> Person(N) {
    health := int(rand.int31_max(101))
    age := int(rand.int31_max(101))

    return {
        health = health,
        age = age,
    }
}

benchmark_aos_array :: proc($N: int) -> f64 {
    people: [dynamic]Person(N)

    for i in 0..<NUM_ELEMS {
        p := make_person(N)
        append(&people, p)
    }
```

```
    age_sum: int
    start := time.now()
    for i in 0..<NUM_TEST_ITERS {
        for &p in people {
            age_sum += p.age
        }
    }
    end := time.now()
    fmt.println("Arrays of Structures age sum:",
f32(age_sum)/(NUM_TEST_ITERS*NUM_ELEMS))

    return time.duration_milliseconds(time.diff(start, end))
}

benchmark_soa_array :: proc($N: int) -> f64 {
    people: #soa[dynamic]Person(N)

    for i in 0..<NUM_ELEMS {
        p := make_person(N)
        append_soa(&people, p)
    }

    age_sum: int
    start := time.now()
    for i in 0..<NUM_TEST_ITERS {
        for &p in people {
            // This will become vectorized
            // with -o:speed. Run with
            // -o:none to see the
            // "pure SoA vs AoS numbers".

            // Also, if we add in lots of
            // extra instructions in this
            // loop, then prefetching might
            // start being as fast as SoA
            // because the prefetcher has
            // time enough to prefetch the
            // next thing from main memory.
            age_sum += p.age
        }
    }
    end := time.now()
    fmt.println("Structure of Arrays age sum:",
f32(age_sum)/(NUM_TEST_ITERS*NUM_ELEMS))
```

```
    return time.duration_milliseconds(time.diff(start, end))
}

soa_bench :: proc($N: int) {
    fmt.printfln("For %v bytes of extra data in each array element:", N)
    time_aos := benchmark_aos_array(N)
    time_soa := benchmark_soa_array(N)
    fmt.printfln("SoA is %.2f times faster than AoS", time_aos/time_soa)
    fmt.println()
}

main :: proc() {
    soa_bench(0)
    soa_bench(2)
    soa_bench(4)
    soa_bench(8)
    soa_bench(16)
    soa_bench(32)
    soa_bench(64)
    soa_bench(128)
    soa_bench(256)
    soa_bench(512)
    soa_bench(1024)
    soa_bench(1500)
    soa_bench(2000)
    soa_bench(3000)
}
```

This benchmark contains two procedures: `benchmark_aos_array` and `benchmark_soa_array`. They create an array in an AoS and SoA way, respectively. Both procedures add `10000` element to their array. Thereafter it measures how long it takes to loop through the arrays while summing the `age` field on each element. It repeats this loop `10` times.

Again, don't take these benchmarks as absolute truths. They are just a general indication. The performance characteristics can change *a lot* if you for example change what happens within the loops.

Note that `Person` has a field `extra_data: [N]byte` in it. If you look in `main` you can see that we vary how many bytes of extra data the `Person` struct should contain by feeding different values into `soa_bench`. We do this in order to investigate what difference the total size of the struct makes in our AoS-vs-SoA comparison.

When I ran with program using `odin run . -o:none`, then I got this output:

```
For 0 bytes of extra data in each array element:
SoA is 1.06 times faster than AoS
```

```
For 2 bytes of extra data in each array element:
SoA is 1.06 times faster than AoS

For 4 bytes of extra data in each array element:
SoA is 1.04 times faster than AoS

For 8 bytes of extra data in each array element:
SoA is 1.04 times faster than AoS

For 16 bytes of extra data in each array element:
SoA is 1.07 times faster than AoS

For 32 bytes of extra data in each array element:
SoA is 1.18 times faster than AoS

For 64 bytes of extra data in each array element:
SoA is 1.46 times faster than AoS

For 128 bytes of extra data in each array element:
SoA is 1.78 times faster than AoS

For 256 bytes of extra data in each array element:
SoA is 1.90 times faster than AoS

For 512 bytes of extra data in each array element:
SoA is 1.79 times faster than AoS

For 1024 bytes of extra data in each array element:
SoA is 2.24 times faster than AoS

For 1500 bytes of extra data in each array element:
SoA is 2.40 times faster than AoS

For 2000 bytes of extra data in each array element:
SoA is 2.07 times faster than AoS

For 3000 bytes of extra data in each array element:
SoA is 3.22 times faster than AoS
```

When I ran it with speed optimization on: `odin run . -o:speed`, then I got this output:

```
For 0 bytes of extra data in each array element:
SoA is 1.91 times faster than AoS

For 2 bytes of extra data in each array element:
```

```
SoA is 2.52 times faster than AoS

For 4 bytes of extra data in each array element:
SoA is 2.03 times faster than AoS

For 8 bytes of extra data in each array element:
SoA is 1.48 times faster than AoS

For 16 bytes of extra data in each array element:
SoA is 4.43 times faster than AoS

For 32 bytes of extra data in each array element:
SoA is 5.53 times faster than AoS

For 64 bytes of extra data in each array element:
SoA is 11.76 times faster than AoS

For 128 bytes of extra data in each array element:
SoA is 13.70 times faster than AoS

For 256 bytes of extra data in each array element:
SoA is 15.11 times faster than AoS

For 512 bytes of extra data in each array element:
SoA is 13.35 times faster than AoS

For 1024 bytes of extra data in each array element:
SoA is 7.71 times faster than AoS

For 1500 bytes of extra data in each array element:
SoA is 15.30 times faster than AoS

For 2000 bytes of extra data in each array element:
SoA is 25.99 times faster than AoS

For 3000 bytes of extra data in each array element:
SoA is 28.69 times faster than AoS
```

As we've seen previously, the reason for `-o:speed` giving such a *massive* speed increase isn't just SoA, but also something known as vectorization.

But even without the speed optimizations, we see that SoA is between `1.06` and `3.22` faster than AoS. The more "extra data" we add inside each array element, the further apart the `age` fields will be in the AoS setup, which will make it more and more cache unfriendly. The result that says `3.22 times faster` is perhaps a bit of a hyperbole since it requires `3000` bytes of

extra data per element. But at just 64 bytes of extra data we see that SoA is `1.46` times faster. That's 46% faster by just changing the memory layout! A struct using around 50-100 bytes of memory is not uncommon, so it's quite a realistic scenario.

Note that the SoA version is, for this specific test, always equally cache friendly since all the `age` fields are next to each other in memory regardless of how much bigger we make the `extra_data` field. The speed difference largely comes from the AoS version getting slower and slower with bigger `extra_data`.

When do I use SoA arrays? *20.2.2*

Use SoA when you think there is a benefit. I wouldn't use it everywhere since it makes the memory layout a bit trickier to inspect when debugging. But since it is so easy to use SoA in Odin you can easily add it whenever you feel like it. If you run into any technical issues with SoA, and you are unsure if you really need to use it, then don't.

From C / C++

In C using SoA can be a major pain because you manually have to split your struct into arrays. Odin does this for you when you use the `#soa` tag.

Conclusion and further reading 20.3

I would advise you to never separately heap allocate elements of a big array that you often iterate. It makes your arrays have slow and *unpredictable* performance characteristics.

I would advise you to use SoA arrays when there is a clear benefit. Don't use SoA arrays if it introduces friction without having a clear performance benefit. There are additional details about SoA in the **official overview** .

Going deeper into data-oriented design is outside the scope of this book. However, **here is an online book** that goes deep into this topic.

Chapter 21

Making C library bindings (Foreign Function Interface)

Odin is able to run pre-compiled C code. This makes it possible to use any C library, given that one creates the appropriate *bindings*. Bindings tell Odin how the code inside the C library looks, so that it knows how to run that code.

In this chapter I will first show the C code for an example library, and how to compile it. Thereafter I'll show how to create the bindings and how to use them in order to run our C code from an Odin program. Since this is a book on Odin, I don't expect you to understand the C code, but I will explain roughly what it does.

The code I show in this chapter is also available **here** .

The C library 21.1

Our C library code looks like this:

```
#include <stdio.h>

typedef struct TestStruct {
    int num;
    float flt_num;
} TestStruct;

typedef void (*Callback)(TestStruct);

Callback callback = NULL;

void set_callback(Callback c) {
    printf("Setting callback\n");
    callback = c;
}

void do_stuff(TestStruct ts) {
    printf("Doing stuff\n");
    ts.num += 1;
    ts.flt_num -= 1;

    // This is not important, but: This
    // will force flush the console buffer,
```

```
    // avoiding C and Odin prints to be out
    // of order. This is not a good
    // solution for this, instead you
    // should probably redirect the C
    // library's stdout to Odin's stdout.
    fflush(stdout);

    if (callback != NULL) {
        callback(ts);
    }
}
```

This code contains two functions:

- `set_callback` sets a global variable called `callback`. `callback` is a function pointer.
- `do_stuff` takes a struct `TestStruct` and modifies the fields of it. It then runs the callback previously set with `set_callback`, passing along the modified `TestStruct` to the callback.

Make a folder called `my_lib` and save the code above as `my_lib.c` inside it.

We now need to compile this C code into a static library. Use a terminal or command prompt to navigate to the `my_lib` folder.

On Windows, run this from a command prompt:

```
cl my_lib.c /c
mkdir windows
lib my_lib.obj /out:windows\my_lib.lib
```

You'll need to execute this from a command prompt that has access to the `cl` C compiler. Look for "x64 Native Tools Command Prompt" in the start menu.

On mac or Linux, you can do this instead:

```
clang -c my_lib.c
mkdir -p linux
llvm-ar rc linux/my_lib.a my_lib.o
```

Replace `linux` with `macos` if you're on a mac.

After this you should have a `my_lib.lib` file in the `windows` folder or a `my_lib.a` file in the `linux` or `macos` folder.

The Odin bindings 21.2

Let's now look at how to create the Odin bindings. These bindings enable our Odin code to call the C code in those library files we just compiled.

In the `my_lib` folder, create a `my_lib.odin` file and paste this code into it:

```
package my_lib

import "core:fmt"
import "core:c"

when ODIN_OS == .Windows {
    foreign import my_lib {
        "windows/my_lib.lib",
    }
} else when ODIN_OS == .Linux {
    foreign import my_lib {
        "linux/my_lib.a",
    }
} else when ODIN_OS == .Darwin {
    foreign import my_lib {
        "macos/my_lib.a",
    }
}

Callback :: proc "c"(TestStruct)

@(default_calling_convention="c")
foreign my_lib {
    set_callback :: proc(c: Callback) ---
    do_stuff :: proc(ts: TestStruct) ---
}

TestStruct :: struct {
    num: c.int,
    flt_num: c.float,
}
```

These lines tell the Odin compiler which pre-compiled C library files to load:

```
when ODIN_OS == .Windows {
    foreign import my_lib {
        "windows/my_lib.lib",
    }
```

```
} else when ODIN_OS == .Linux {
    foreign import my_lib {
        "linux/my_lib.a",
    }
} else when ODIN_OS == .Darwin {
    foreign import my_lib {
        "macos/my_lib.a",
    }
}
```

This code chooses the correct library file based on which platform we are on.

These lines say which functions inside the C code to expose to the Odin code:

```
@(default_calling_convention="c")
foreign my_lib {
    set_callback :: proc(c: Callback) ---
    do_stuff :: proc(ts: TestStruct) ---
}
```

There's a name `my_lib` on both the line `foreign my_lib {` and on the line `foreign import my_lib {`. This name can be anything, but must be the same on both lines. The `foreign import my_lib {}` block says which binary library file to load. The `foreign my_lib {}` block describes how the procedures within `my_lib` look, so that we can run them from our Odin code.

Note that it says `@(default_calling_convention="c")` before `foreign my_lib`. This tells the compiler that the things inside this `foreign` block should use the C calling convention. Remember: Odin calls procedures differently, for example Odin passes along an implicit context to all procedures. You can't pass that to C function, as it does not expect it. Therefore we must call C functions using the correct calling convention.

For a `foreign` block the C calling convention is actually the default, but most bindings specify it for clarity.

The `TestStruct` in the Odin code looks quite similar to the one in the C code:

```
TestStruct :: struct {
    num: c.int,
    flt_num: c.float,
}
```

vs

```
typedef struct TestStruct {
    int num;
    float flt_num;
} TestStruct;
```

In the Odin code we use types such as `c.int` and `c.float`. Those come from the `core:c` package.

These types are great to use when binding to C libraries, as they will use the correct amount of memory.

Testing our bindings 21.3

To test our bindings, create a file called `main.odin` and put the following code inside it. `main.odin` should live in the parent folder of `my_lib` so that it can import that folder as a package.

```
package binding_to_c_test

import "base:runtime"
import "core:fmt"

import "my_lib"

custom_context: runtime.Context

my_callback :: proc "c" (ts: my_lib.TestStruct) {
    // You can also do
    // context = runtime.default_context()
    // if you do not need any special
    // context.
    context = custom_context

    fmt.println("In the callback")
    fmt.println(ts)
}

main :: proc() {
    // Setup context however you want with
    // tracking allocators etc, then save
    // it to `custom_context` so you can
    // use it in `my_callback`.
```

```
    custom_context = context

    my_lib.set_callback(my_callback)

    ts := my_lib.TestStruct {
        num = 7,
        flt_num = 23.12,
    }

    my_lib.do_stuff(ts)
}
```

This code imports `my_lib`:

```
import "my_lib"
```

Since the `my_lib` package specifies which pre-compiled libraries to load, no additional setup is needed.

From C / C++

In C you would usually need to manually tell the linker what binary library files to use. In Odin that is not needed. The `foreign import` blocks take care of it.

If you run this program, then it should print:

```
Setting callback
Doing stuff
In the callback
TestStruct{num = 8, flt_num = 22.12}
```

What happens is the following:

- The program defines a procedure called `my_callback` and passes it to `my_lib.set_callback(my_callback)`.
- It then creates a `ts: TestStruct` object and runs `my_lib.do_stuff(ts)`. If you look back at the C code for `do_stuff`, then you see that it modifies the `num` and `flt_num` fields and then runs the callback we previously sent into `set_callback`. Into the callback it passes the modified `TestStruct`.
- Now `my_callback` will be called by the C code. We can see that it has indeed been modified by the C code: `num` has increased by `1` and `flt_num` has decreased by `1`.

Note that `my_callback` does this:

```
context = custom_context
```

We set `custom_context` at the beginning of `main`, like so:

```
custom_context = context
```

The reason for this is that `my_callback` uses the C calling convention, so there will be no `context` passed along to it. However, `fmt.println` is used inside `my_callback`. That procedure requires the `context`, so we have to set it.

Alternatively, you can remove the `custom_context` variable and do this within `my_callback`:

```
context = runtime.default_context()
```

The main reason to use a global `custom_context` variable instead of `runtime.default_context()` is in case you have anything custom within `context` that the default context won't have. For example, you might have set up tracking allocators or loggers.

More examples 21.4

If you prefer to use a dynamically linked library (DLL), instead of a static library, then I show how to set that up in the example code on GitHub. I only show how to do it for Windows.

You can also learn a lot from looking at the libraries in the `<odin>/vendor` folder. Many of them are actually just bindings to C libraries:

- raylib bindings
- box2D bindings
- x11 bindings

Video 21.5

This video contains mostly the same information as this chapter:

https://www.youtube.com/watch?v=BgBinHwJwgo

Chapter 22

Debuggers

Debuggers can be used to inspect what your code is doing as the program is running. The debugger can pause the program using breakpoints, and step the code line-by-line. As the program is paused you can inspect the values of variables.

There are a couple of different debuggers that have successfully been used with Odin. Three popular choices are:

- VS Code
- RemedyBG
- RAD Debugger

Of these VS Code runs on all major desktop operating systems, while RemedyBG and RAD Debugger only runs on Windows. VS Code and RAD Debugger are free. All three are good, so you can try them for yourself and see which one you prefer.

VS Code 22.1

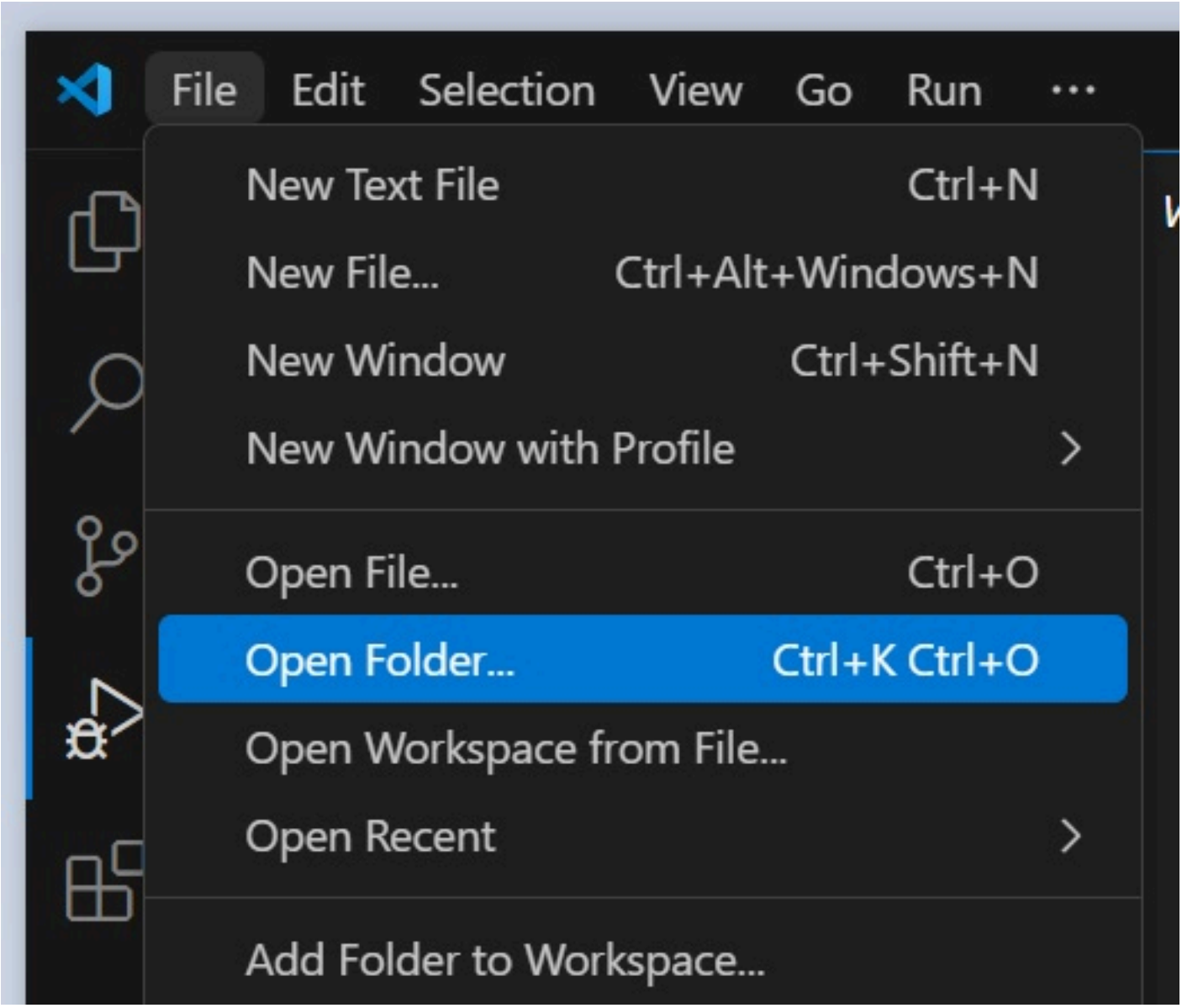
VS Code is an editor with a built in debugger. It's popular for editing Odin source code since you can edit and debug within the same application.

> VS Code is short for Visual Studio Code. However, it's a completely different program than the older Visual Studio editor. It's not a great name: It has just made it hard to search for things related to any of them.

To set VS Code up, you'll first need to install it. When you've done that, then you need to install some additional debugging tools. Depending on your platform, you should install one of these two:

- Windows: C++ build tools
- Linux / Mac: CodeLLDB

After you have the debugging tools installed, open the folder where your source code lives:



Open your folder with Odin source code using File -> Open Folder...

In order to build and debug your program you need to create a few configuration files. Within the folder you opened in VS Code, create a new subfolder called `.vscode` (with a period in front!). In it, add these three files:

`launch.json`:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "type": "cppvsdbg",
            "request": "launch",
            "preLaunchTask": "Build",
            "name": "Debug",
```

```
            "program": "${workspaceFolder}/${workspaceFolderBasename}",
            "args": [],
            "cwd": "${workspaceFolder}"
        },
    ]
}
```

Use `"type": "cppvsdbg"` on Windows and `"type": "lldb"` on macOS/Linux (it matches the additional debugging tools you installed earlier).

`tasks.json`:

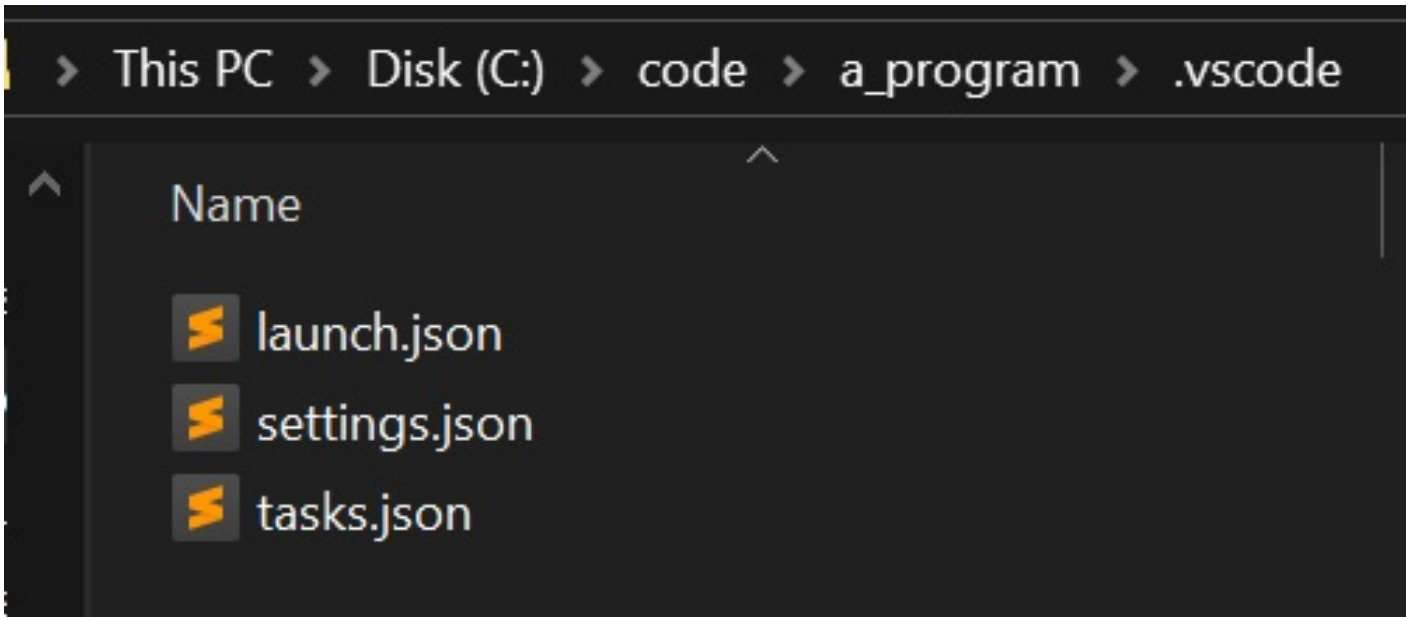
```
{
    "version": "2.0.0",
    "command": "",
    "args": [],
    "tasks": [
        {
            "label": "Build",
            "type": "shell",
            "command": "odin build . -debug",
            "group": "build"
        },
    ]
}
```

Note the `-debug` flag used here. That's what makes the Odin compiler generate the extra information that the debugger needs.

`settings.json`:

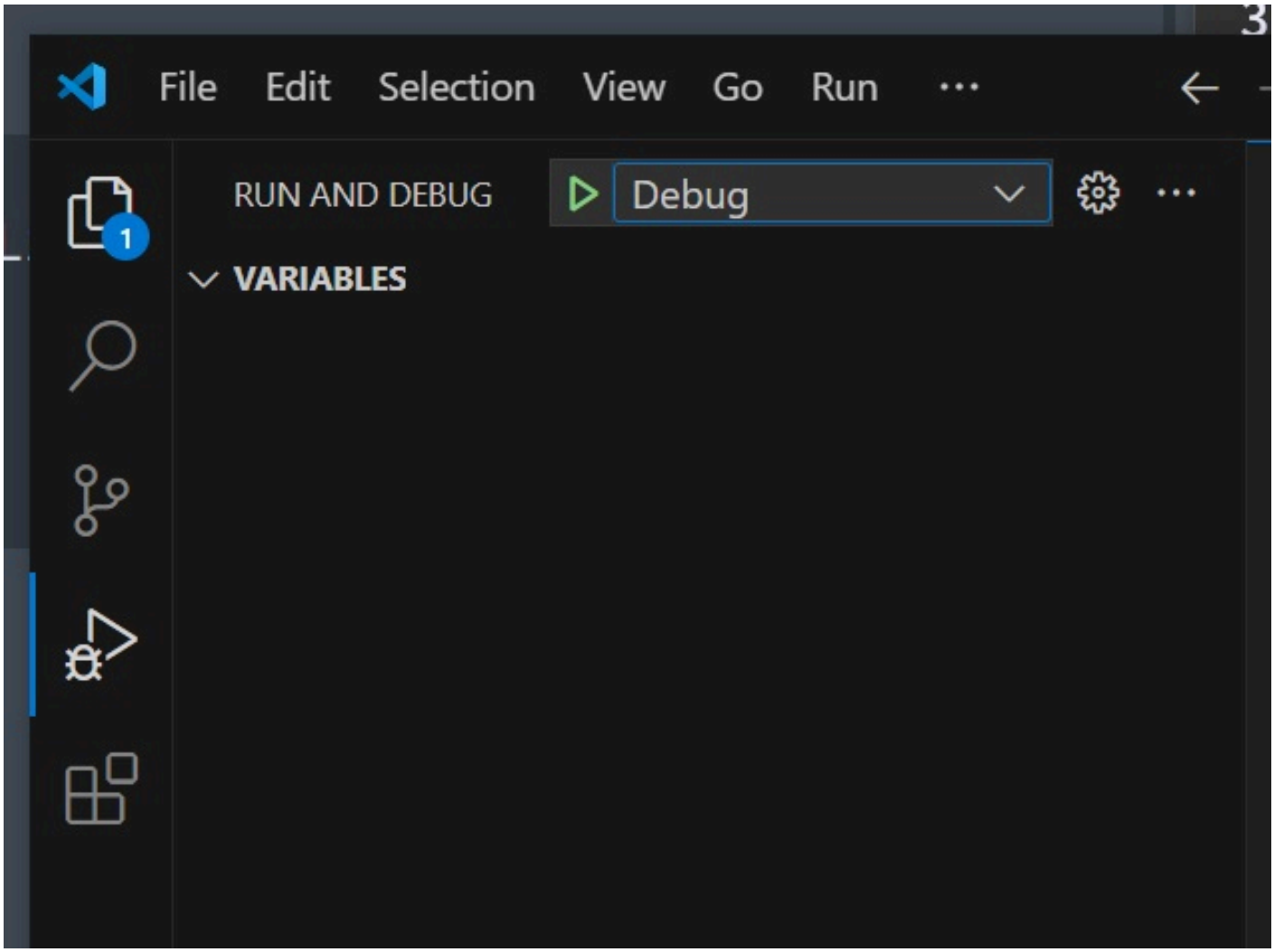
```
{
    "debug.allowBreakpointsEverywhere": true,
}
```

In the end, the `.vscode` folder should look something like this:



We've created a folder .vscode inside your project folder and put the three files above inside it.

When all that is done, then you'll be able to go to the "Run and debug" tab and press the green "play" button. It uses the stuff in `launch.json` and `tasks.json` to compile and start your program with a debugger attached.



Press the green play button to compile your program and start debugging

You can open `.odin` files and place breakpoints by putting your text cursor on a line and pressing F9.

If you wish to have syntax highlighting then I know of two alternatives:

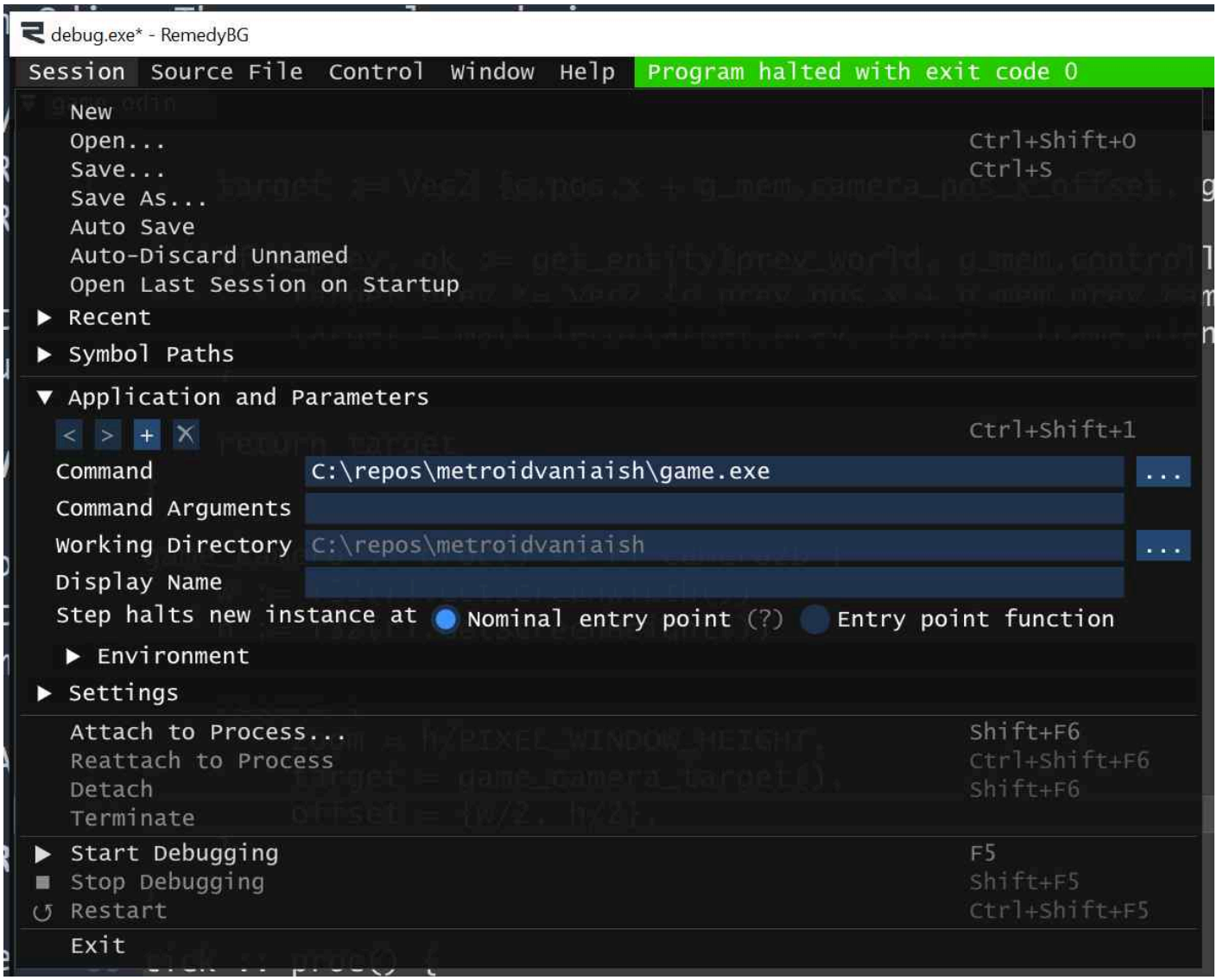
- Odin Language Server (OLS) : This is a complete language server. It gives syntax highlighting. But it also gives you code completion. To get the code completion to work, make sure that you have the Odin compiler in your PATH environment variable.
- Odin extension : This is a more minimalist alternative that just provides syntax highlighting.

If you install any of these two extensions, then you no longer need the `"debug.allowBreakpointsEverywhere": true` line in `settings.json`, as VS Code will properly recognize your `.odin` files as source code.

RemedyBG 22.2

RemedyBG is a very light weight debugger that works well with Odin. To use it, make sure you run the Odin compiler with the `-debug` flag:

```
odin build . -debug
```



To setup RemedyBG, start it and go into the 'Session' menu. From there, fill out 'Command' with the path to the program you just compiled.

When that is done you can press F5 to start debugging.

To set breakpoints: Open one of your source files using Ctrl+O and place breakpoints by having the text cursor on a line and pressing F9.

RAD Debugger 22.3

The RAD Debugger is a new debugger that is free, lightweight and powerful.

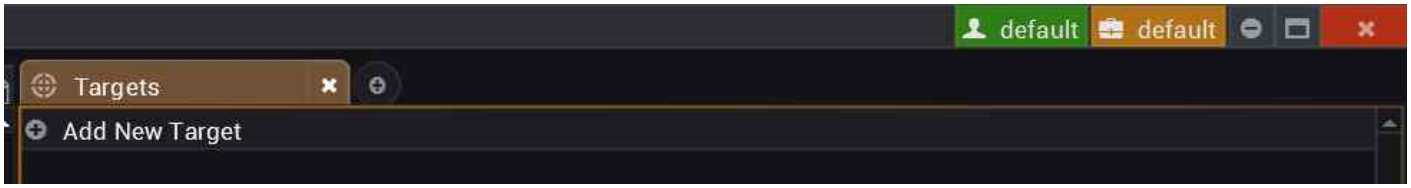
> At the time of writing, the RAD Debugger is alpha software and may contain lots of bugs. That said, I've successfully used it in my work.

To use it, make sure you compile your code in debug mode:

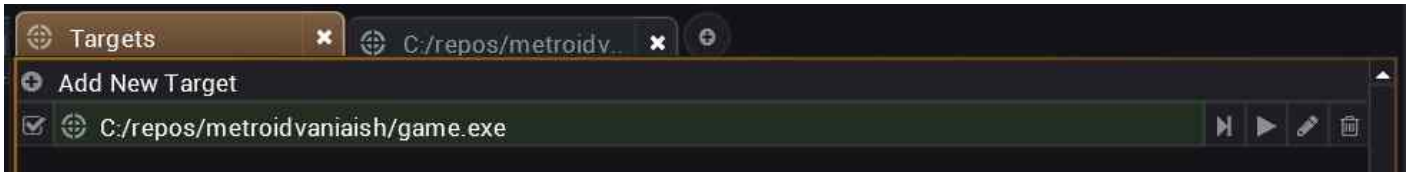
```
odin build . -debug
```

To set RAD Debugger up, download the latest release from GitHub .

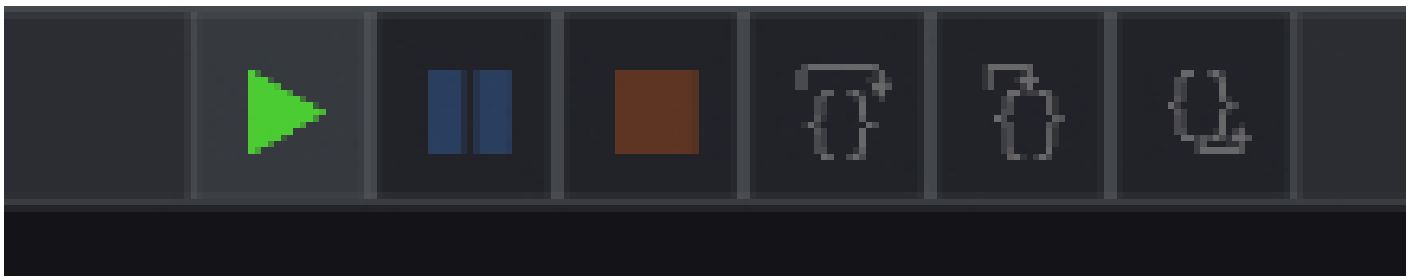
When you start the program find the "Add New Target" button. The target should point to the program you just compiled.



After choosing your target, make sure that it is checked in the Targets tab. This makes it the active target.



To start debugging, press F5 or use the green play button in the top-middle of the window:



To set breakpoints: Open a source file by pressing Ctrl+O. Within your source file, put your text cursor on a line and press F9.

Chapter 23

Odin features you should avoid

Let's look at a three things in Odin that you might want to avoid.

The fact that this chapter only contains three things is a testament to how well-designed Odin is.

Actually, the third thing in this chapter isn't even something you should avoid, but something that is disabled by default.

`using` on variables and parameters 23.1

You can write `using some_struct` to bring the fields of `some_struct` into the current scope. Here's an example:

```
My_Struct :: struct {
    number: int,
}

my_proc :: proc(s: My_Struct) {
    using s
    fmt.println(number)
}
```

Because of the `using s` line, I can use `number` directly without having to write `s.number`.

You can also put the `using` directly on the parameter:

```
My_Struct :: struct {
    number: int,
}

my_proc :: proc(using s: My_Struct) {
    fmt.println(number)
}
```

This feature exists for making refactoring of code easier. Using it to write permanent code is not recommended. Why? Because it creates hard-to-read code, where you are unsure where the variable names actually come from. Your code probably has some local variables and some

global variables. When you add `using` into the mix, then it can suddenly become too messy. In bigger procedures it can be very unclear if you're looking at a local variable, global variable or a variable related to some `using`.

Note that this usage of `using` has nothing to do with putting `using` on fields within a struct. I have talked about that usage of `using` before **here**. That usage of `using` is often very useful.

Gosh, it's awkward to write about using `using`!

any type 23.2

The `any` type contains a pointer and a typeid. `any` can be used to implicitly send a variable by pointer into a procedure. Here's a procedure with a parameter of type `any`:

```
my_proc :: proc(a: any) {
    // a.data is a pointer.
    // a.id is a typeid, it is the type of
    // the thing a.data points to.
}

number := 7
my_proc(number)
```

As I said, `any` contains a pointer. But we do not need to write `my_proc(&number)`, instead you just write `my_proc(number)`. Let's look at why. These two lines:

```
number := 7
my_proc(number)
```

Do the exact same thing as these lines:

```
number := 7
a: any
a.data = &number
a.id = typeid_of(type_of(number))
my_proc(a)
```

So `any` is really just a convenient way to implicitly fetch an address and a typeid and package them into a single object.

You should almost *never* use `any`. It was added to the language to make it easy to write printing procedures such as `fmt.println`. So if you want to make something similar to `fmt.println`, then you have a use-case for `any`, but otherwise, just avoid using it.

Do not use `any` to write generic procedures, instead use **parametric polymorphism**.

Dynamic literals 23.3

This isn't really something you should avoid, but rather something that is disabled by default.

Dynamic literals make it possible to create and initialize dynamic arrays and maps with a single statement:

```
ints := [dynamic]int { 1, 2, 3, 4, 5 }

name_to_age := map[string]int {
    "Karl" = 5,
    "Klucke" = 7,
}
```

Even though there is no call to `make` or `append`, these statements actually allocate dynamic memory. After all, the dynamic array needs some dynamically allocated memory in which to store the elements you are initializing it with. Same for the map.

This means that you need to `delete` this dynamic array and map when you no longer need them.

These kinds of implicit allocations may come as a surprise, and the dynamic literals are therefore disabled by default, with an error telling you why they are disabled. The error looks like this:

```
Error: Compound literals of dynamic types are disabled by default
ints := [dynamic]int { 1, 2, 3, 4, 5 }
        ^~~~~~~~~~~~~~~~~~~~~~~~~~~~^
Suggestion: If you want to enable them for this specific file, add '#+feature
dynamic-literals'at the top of the file
Warning: Please understand that dynamic literals will implicitly allocate using
the current 'context.allocator' in that scope
```

As we see in the error, you can enable dynamic literals by putting `#+feature dynamic-literals` at the top of the file where you need to use them. This ensures that all programmers

know about this potential memory allocation pitfall.

You can keep the `dynamic-literals` flag off and still create a dynamic array and initialize using a single statement, like this:

```
ints := slice.clone_to_dynamic([]int { 1, 2, 3, 4, 5 })

import "core:slice"
```

The result of this is something of type `[dynamic]int`. It has the same effect as the example at the start of this section, but it is more apparent that it allocates.

Dynamic map literals are harder to emulate. So if you have something that looks like the following:

```
name_to_age := map[string]int {
    "Karl" = 5,
    "Klucke" = 7,
}
```

Then you'll either have to put `#+feature dynamic-literals` at the top of the file, or split the code into three separate statements:

```
name_to_age: map[string]int
name_to_age["Karl"] = 5
name_to_age["Klucke"] = 7
```

Chapter 24

A tour of the core collection

The core collection is a big collection of libraries that comes with Odin. Some people call it the "standard library" of Odin. However, it is not meant to be thought of as a standard library. Instead, you should see it as a collection of useful libraries that you can use as they are, or modify according to your own needs. You'll find the core collection in `<odin>/core` and you import anything from in there by typing `import "core:path/to/library"`.

> If you like a package in the core library, but you want to modify it, then you can just copy it to your project and import that package instead.

In this chapter, I show some examples on how to use different parts of the core collection. Some of the things in here will be new while other things will refer back to other parts of the book. You can also see this chapter as a collection of additional examples on how to use the language.

As I mentioned **in the first chapter**, I really recommend that you pull in `<odin>/core` and `<odin>/base` into your text editor so that you can easily jump to the code in there. The code in the core collection is very readable and you can learn a lot from it!

> The `base` dir is a separate collection that contains basic things that many programs and core libraries use.

Reading and writing files (`core:os`) 24.1

You can easily read and write files using `core:os`. Import the functionality like so:

```
import "core:os"
```

Read an entire file called `my_file` like this:

```
if data, data_err := os.read_entire_file("my_file", context.allocator); data_err
== nil {
    // The file was successfully read.
    // data is of type []byte. It's a slice
    // containing the whole contents of the
    // file in the form of bytes.
} else {
```

```
    fmt.eprintfln("Failed reading from 'my_file'. Error: %v", data_err)
}
```

`data` will be of type `[]byte` and will be allocated using `context.allocator`. I often read files using the **temp allocator** since I will process the data into another format anyways:

```
if data, data_err := os.read_entire_file("my_file", context.temp_allocator);
data_err == nil {
    // use data
}
```

If you need a string based on `data` then you can write `string(data)`. It will not cause any extra allocations.

Use `strings.clone_from_bytes(data)` to get an allocated clone.

You can write to a file `my_file` like this:

```
write_err := os.write_entire_file("my_file", data)

if write_err != nil {
    fmt.eprintfln("Failed writing 'my_file'. Error: %v", write_err)
}
```

Here `data` needs to be of type `[]byte`. `os.write_entire_file` returns an error. It's `nil` if the write succeeded. In the example we print an error message if the error wasn't `nil`.

Read and write structs as JSON (`core:encoding/json`) 24.2

Odin comes with a JSON library that can convert structs into JSON (marshal) and then also convert that JSON back into structs (unmarshal).

JSON is an easy-to-read format for storing generic data.

Import the package like so:

```
import "core:encoding/json"
```

You can then convert a struct into JSON like this:

```
Some_Struct :: struct {
    some_field: int,
```

```
}

my_struct := Some_Struct {
    some_field = 7,
}

if json_data, json_err := json.marshal(my_struct, allocator =
context.temp_allocator); json_err == nil {
    // json_data is of type []byte,
    // so can write it directly to a file:
    write_err := os.write_entire_file("my_struct_file", json_data)
    if write_err != nil {
        fmt.eprintfln("Couldn't write file! Error: %v", write_err)
    }
} else {
    fmt.eprintfln("Couldn't marshal struct! Error: %v", json_err)
}
```

`json.marshal` will convert `my_struct` into JSON. Note that the second return value of `json.marshal` is a union. This union is `nil` if there was no error. I also included some code for writing out the JSON data to a file, using the `core:os` package.

In this example I used `context.temp_allocator` for the marshaling since I only need the data for immediately writing it to a file.

You can read back the data from the file, recreating your struct, like this:

```
if json_data, json_err := os.read_entire_file("my_struct_file",
context.temp_allocator); json_err == nil {
    my_struct: Some_Struct

    if unmarshal_err := json.unmarshal(json_data, &my_struct); unmarshal_err ==
nil {
        // `my_struct` now contains
        // the data from the file with the
        // filename "my_struct_file".
    } else {
        fmt.eprintfln("Failed to unmarshal JSON. Error: %v", unmarshal_err)
    }
} else {
    fmt.eprintfln("Failed to read my_struct_file. Error: %v", json_err)
}
```

Note that `json.unmarshal` may allocate memory if your struct contains slices, dynamic arrays or strings. It is up to you to free this memory when you no longer need the struct. One way to

group all the memory allocations done during unmarshal is by passing an arena allocator to `json.unmarshal`:

```
my_struct: My_Struct
arena: vmem.Arena
arena_allocator := vmem.arena_allocator(&arena)

if json.unmarshal(json_data, &my_struct, allocator = arena_allocator) == nil {
    // Any allocations done during
    // unmarshal are done into `arena`.
    // Save `my_struct` and `arena`
    // somewhere. When you need to
    // deallocate the memory, just do:
    // `vmem.arena_destroy(&arena)`
}
```

Needs `import vmem "core:mem/virtual"`

Don't store `arena_allocator` anywhere outside of this scope. It points to a stack variable `arena`. Just store the struct and the `arena`. If you need the allocator again, then calling `vmem.arena_allocator(&arena)` is a cheap operation.

Virtual memory arena allocators (`core:mem/virtual`) 24.3

See the chapter on arena allocators.

Working with strings (`core:strings`) 24.4

See the strings chapter.

Using the logger (`core:log`) 24.5

See the chapter on `context.logger`.

Start a new thread (`core:thread`) 24.6

To start a thread, you can do something like this:

```
import "core:thread"

Worker_Thread_Data :: struct {
    // run is optional, it's useful if your
    // thread needs to run in a loop until
    // you tell it to stop. If the thread
    // just needs to do a task and then
    // stop, then you can skip this bool
    // and the code that uses it.
    run: bool,

    // Add data your thread needs here.

    thread: ^thread.Thread,
}

worker_thread_proc :: proc(t: ^thread.Thread) {
    d := (^Worker_Thread_Data)(t.data)
    for d.run {
        // Let your thread do stuff!

        // You can make this thread run a
        // little slower using a sleep:
        // time.sleep(10*time.Millisecond)
        // (needs `core:time`)
    }
}

start_worker_thread :: proc(d: ^Worker_Thread_Data) {
    d.run = true
    if d.thread = thread.create(worker_thread_proc); d.thread != nil {
        d.thread.init_context = context
        d.thread.data = rawptr(d)
        thread.start(d.thread)
    }
}

stop_worker_thread :: proc(d: ^Worker_Thread_Data) {
    d.run = false
    thread.join(d.thread)
    thread.destroy(d.thread)
}
```

This will create a thread when you run `start_worker_thread`. It will run forever until you call `stop_worker_thread` with the same struct pointer.

The line `d.thread.init_context = context` makes the thread we are starting use the provided `context`. However, the thread will not use the same temp allocator. It will be given a new temp allocator automatically. This is because the temp allocator is not thread safe.

Each thread stores its temp allocator in thread local storage.

Note that Odin does not support things like co-routines and async / await. Those concepts are usually found in slightly higher-level languages, and don't fit nicely into Odin. Instead you can try using the threads in **`core:thread`** and the synchronization primitives in **`core:sync`**. You can also directly use the operating system's threading API, such as the thread related procedures in **`core:sys/windows`** or the pthreads-related procedures in **`core:sys/unix`**.

For an example on how to use threads, see **Jakub Tomšů's simple job system**.

Do things periodically (`core:time`) 24.7

One often wants to record the current time and then wait until a number of seconds has passed before doing an action. Here's a small program that uses `core:time` to periodically print a message:

```
package main

import "core:time"
import "core:fmt"

main :: proc() {
    start_time := time.now()
    last_print_time := time.now()

    for {
        current_time := time.now()

        if time.duration_seconds(time.diff(last_print_time, current_time)) > 1 {
            fmt.println("One second passed")
            last_print_time = current_time
        }

        if time.duration_minutes(time.since(start_time)) > 2 {
```

```
            fmt.println("Two minutes has passed, shutting down!")
            break
        }
    }
}
```

This code uses `time.now()` in order to fetch the current time. It uses `time.duration_seconds(time.diff(last_print_time, current_time)) > 1` to check if more than one second has passed since it last printed a message. In that case it prints a message and sets `last_print_time` to the current time.

It also checks if more than 2 minutes has passed since the program started:

```
if time.duration_minutes(time.since(start_time)) > 2 {
    fmt.println("Two minutes has passed, shutting down!")
    break
}
```

In which case the `for` loop will break and the program will finish.

This program uses both `time.diff` and `time.since`. Both these procedures return a `time.Duration` object. Such a duration can be converted into seconds, minutes or hours etc using procedures like `time.duration_second(some_duration)`.

`time.since(t)` just executes `time.diff(t, time.now())`.

See **<odin>/core/time/time.odin** for more examples on useful procedures to work with time.

Linear algebra math operations (`core:math/linalg`) 24.8

If you need to compute the length of vectors, normalize vectors, work with matrices or quaternions, then there is lots of helpful code in the `linalg` package:

```
import "core:math/linalg"
```

Below we use `linalg` to compute the length of a vector and normalize it:

```
Vector3 :: [3]f32
```

```
v := Vector3 {2, 3, 5}
v_length := linalg.length(v)
// normalize0 returns 0 if the length v is 0
v_normalized := linalg.normalize0(v)
```

See the **core:linalg docs** for an overview of what's available

The procedures in `linalg` that operate on vectors just need simple fixed arrays, defined like so: `[2]f32` and `[3]f32`. These simple arrays suffice as vector types thanks to Odin's great **array programming** system.

Create random numbers and shuffle arrays (`core:math/rand`) 24.9

The `rand` package lets you work with random numbers. Import it like so:

```
import "core:math/rand"
```

With this package you can generate a random number between `0` and `max(u32)` like this:

```
random_num := rand.uint32()
```

You can also make random numbers within a range of your choosing:

```
random_num := rand.int_max(128)
```

This will return a random number in the range from `0` to `127` (`128` not included).

The `rand` package also lets you shuffle slices:

```
arr: [128]int

for i in 0..<len(arr) {
    arr[i] = i
}

rand.shuffle(arr[:])
```

You can set the seed of the global random generator if you wish:

```
// Set seed based on current time. This
// gives a different seed each time you
// start the program.
```

```
seed := time.time_to_unix(time.now())
rand.reset(u64(seed))

// This will now use the global random
// generator's seed:
random_num := rand.uint32()
```

You can also create a random generator and explicitly pass it to the procedures in the `rand` package:

```
random_state := rand.create(42) // seed is 42
generator := runtime.default_random_generator(&random_state)
random_num := rand.uint32(generator)
```

Import `base:runtime`.

Also see the section on **`context.random_generator`**.

C types and using the C standard library `core:c` and `core:c/libc` 24.10

While the core collection comes with a lot of useful stuff, there may be cases where it would be nice to use some C standard library procedure. If you import `core:c/libc` then you can use most of the C11 standard library. For example, you could use the `libc` package to run system commands like so: `libc.system("build.bat")`.

If you import `core:c` (note: not `core:c/libc`. Different package!) then you'll get access to some useful types such as `c.int`. These are useful when **creating bindings to C libraries** because they are of the same memory size as C types.

Opening a window using the Windows API `core:sys/windows` 24.11

This is Windows specific. In `core:sys/windows` you'll find the so-called Win32 API. Here's how you can use it to open a Windows window:

```
package open_win32_window

import win "core:sys/windows"

main :: proc() {
    instance := win.HINSTANCE(win.GetModuleHandleW(nil))
```

```
    assert(instance != nil, "Failed to fetch current instance")
    CLASS_NAME :: "Windows Window"

    cls := win.WNDCLASSW {
        lpfnWndProc = win_proc,
        lpszClassName = CLASS_NAME,
        hInstance = instance,
    }

    class := win.RegisterClassW(&cls)
    assert(class != 0, "Class creation failed")

    hwnd := win.CreateWindowW(CLASS_NAME,
        "Windows Window",
        win.WS_OVERLAPPEDWINDOW | win.WS_VISIBLE,
        100, 100, 1280, 720,
        nil, nil, instance, nil)

    assert(hwnd != nil, "Window creation Failed")
    msg: win.MSG

    for win.GetMessageW(&msg, nil, 0, 0) > 0 {
        win.TranslateMessage(&msg)
        win.DispatchMessageW(&msg)
    }

    win.DestroyWindow(hwnd)
}

win_proc :: proc "stdcall" (hwnd: win.HWND, msg: win.UINT, wparam: win.WPARAM,
lparam: win.LPARAM) -> win.LRESULT {
    switch(msg) {
    case win.WM_DESTROY:
        win.PostQuitMessage(0)
    }

    return win.DefWindowProcW(hwnd, msg, wparam, lparam)
}
```

On GitHub I have **an example** on how to open a window and setup some basic software rendering.

Load symbols from a dynamic library `core:dynlib` 24.12

If you have a `.dll`, `.dylib` or `.so` file and you want to load symbols from it, so that you for example can run procedures inside that library, then you can use the `core:dynlib` package.

There's an example of this **in my Odin + Raylib + Hot Reload game template** where I load procedures from a dynamic library.

Chapter 25

Libraries for creating video games

Here's a list of some libraries that can help you create video games. Odin is a general-purpose language, but it has become quite popular specifically for making video games.

Also, I'm a biased game developer.

Some of these libraries are part of the "vendor" collection, which is included with the compiler.

Make a game using raylib `vendor:raylib` 25.1

Odin comes with bindings for raylib, a library for creating video games. raylib can open windows, draw graphics, play sound and process input. Here's how you open a window and start a game loop that constantly clears the screen with a blue color:

```
package game

import rl "vendor:raylib"

main :: proc() {
    rl.InitWindow(1280, 720, "Raylib Game")
    rl.SetTargetFPS(120)

    for !rl.WindowShouldClose() {
        rl.BeginDrawing()
        rl.ClearBackground(rl.BLUE)
        rl.EndDrawing()
    }

    rl.CloseWindow()
}
```

Go on, try copying this to a file and just run it. No further setup is required since Odin comes with the raylib bindings.

The code above results in this:



How the raylib example looks. It's just a blue background inside a window. But it's a start!

Going deep into raylib is outside the scope of this book. I encourage you to open `<odin>/vendor/raylib/raylib.odin` and explore the available procedures.

I have a **series on making games using Odin + Raylib** . I also have this 90 minute video tutorial where I create a little video game from start to finish:

https://www.youtube.com/watch?v=lfiQNCNUifI

Write platform-independent rendering code using SOKOL 25.2

SOKOL is a collection of cross-platform libraries for doing platform-independent real time rendering, window management and more. It can be used when you don't want to choose if you are going to use Vulkan, OpenGL, Metal or Direct3D. It provides an abstraction of those APIs. SOKOL offers official **bindings for Odin** .

The Odin SOKOL bindings come with many basic examples, such as **how to draw a triangle** .

Write rendering code using vulkan (`vendor:vulkan`) 25.3

Vulkan is a library for writing low level rendering code. In `vendor:vulkan` you'll find Odin bindings that map very closely to the C API of Vulkan. You can use any "Vulkan for C tutorial" and easily translate the examples in there over to Odin.

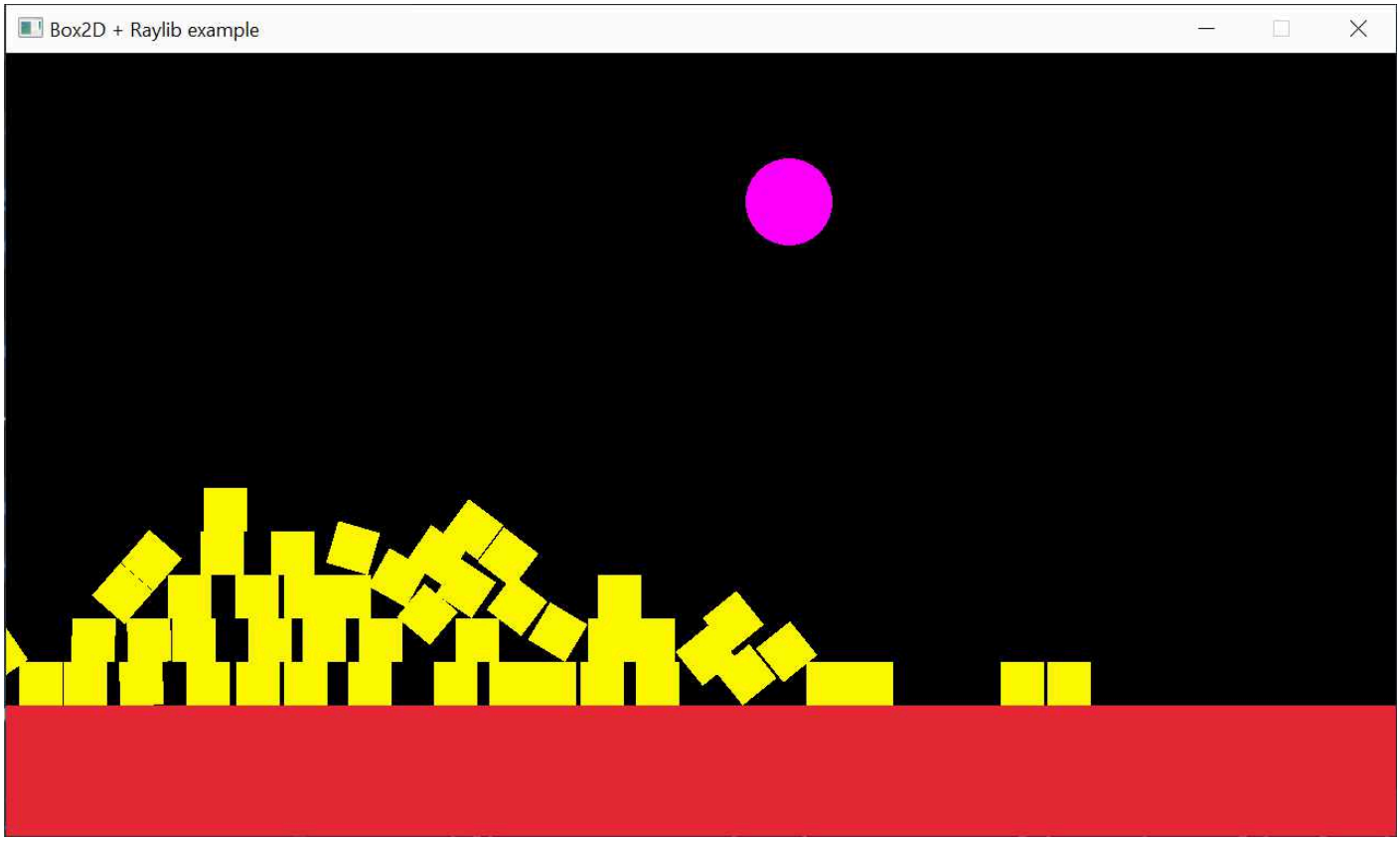
Here is a GitHub gist by laytan on how to draw a triangle using Vulkan.

Using less than 1000 lines! Wow!

Create 2D game physics using Box2D (`vendor:box2d`) 25.4

By importing `vendor:box2d` you can use the Box2D library, making it easy to create physics for 2D games.

In appendix D there is an example program showing how to create some stacked boxes and make it possible to smack them with a ball attached to the cursor. It looks like this when run:



An example program that uses Box2D and raylib to do a small physics simulation.

Note that the example is a single `.odin` file with no third-party dependencies, because Box2D and raylib both exist in `vendor`.

Chapter 26

A few more things...

We are almost at the end of the book!

When writing a book, you usually have to leave some things out due to time constraints. There are also some things that don't fit into any particular chapter. Here I just quickly mention some of those things, sometimes with links to where you can read more.

`#load` and `#load_directory` 26.1

You can load files at *compile time* using `#load`:

```
file_data := #load("my_file")
```

Since this happens at compile time, the loaded data is part of your program! The result of `#load` is a slice of bytes: `[]u8`. Since it's loaded straight into the data of your program, you should not try to deallocate it!

Read more in the **official overview** .

There is also `#load_directory`, which also happens at compile time. It loads all files in a directory. The result is of type `[]Load_Directory_File`, where each `Load_Directory_File` looks like this:

```
Load_Directory_File :: struct {
    name: string,
    data: []byte, // immutable data
}
```

`name` is the filename, and `data` contains the file's data.

The following video shows how to use `#load` to make an entire video game with everything baked into the executable:

https://www.youtube.com/watch?v=aNHr9ovD_N4

@rodata 26.2

If you have a constant array and try to index it using a variable, then it won't work:

```
NUMBERS :: [3]int {
    1, 2, 4,
}

main :: proc() {
    i := 1
    fmt.println(NUMBERS[i])
}
```

You *can* index it using a constant number: `fmt.println(NUMBERS[1])`.

But sometimes the index comes from a variable. Constants don't exist at run-time, so they cannot be indexed using the run-time value of a variable!

Instead, you can change `NUMBERS` into a normal global variable and put `@rodata` in front:

```
@rodata
numbers := [3]int {
    1, 2, 4,
}

main :: proc() {
    i := 1
    fmt.println(numbers[i])
}
```

Note how I use `:=` instead of `::` for `numbers` and how I added `@rodata` in front of it.

Now `numbers` will still be impossible to change, like a constant. But you can index it using a variable. `number`'s data will live in the read-only data block of the program.

Matrices 26.3

Odin comes with built-in support for small matrix types. For example, the type `matrix[4,4]f32` gives you a 4x4 matrix of `f32` values. Matrices are useful for linear algebra applications such as robotics as well as 2D and 3D rendering. Read more in the **official overview** and **core:math/linalg**.

Quaternions 26.4

Quaternions are often used as a compact way to represent 3D rotations. Read more in demo.odin .

mul in core:math/linalg is a good example of how to use quaternions. It lets you multiply quaternions by vectors, which can be seen as a way to rotate vectors.

Struct tags 26.5

Struct tags allow you to associate metadata with struct fields:

```
Cat :: struct {
    age: int "Information about this field",
}
```

Read more about it in the official overview .

You can extract struct field tags using reflection .

WASM 26.6

WASM stands for WebAssembly and is a way to build Odin programs for running in a web browser.

There are some nice examples of how to use WASM combined with WebGPU here .

Also, here is an example page with Odin + WebGL demos running the browser, with links to the source code.

I've made a template for how to build Odin + Raylib games for the web . Note that it uses *emscripten*, because raylib requires emscripten. Emscripten is a big hack that "automatically makes C web compatible". The previous two examples do not use emscripten.

Variadic procedures 26.7

Special syntax for making a procedure take an unspecified number of parameters. See **demo.odin** for an example.

`deferred_X` attributes 26.8

`deferred_in`, `deferred_out`, `deferred_none` and `deferred_in_out` are attributes you can give a procedure. When a procedure has one of these attributes, then another procedure will automatically be called when the scope within which you called the original procedure ends. Read more about it **here** .

These attributes are useful for working with for example Immediate Mode GUIs. They are also useful for integrating profilers.

Making libraries compile cleanly 26.9

If you are making a library that is going to be distributed as a package online, then it is a good thing if the library compiles without problems for the people using it. With that in mind, I'd recommend you make sure your library compiles with the `-vet -strict-style` compilation flags.

Odin libraries are distributed as source code and compiled as part of the program that imports it. So in order to not cause trouble, your library needs to be possible to compile with fairly strict rules.

If you cannot make your library work with those compilation flags, then you can disable specific "vetting rules" by putting for example `#+vet !unused-imports` at the top of the offending files. See `odin build -help` for a list of all the different vetting rules.

Code generation 26.10

Code generation can be seen as a form of "meta programming". This means that you generate some Odin code that you then compile as part of your program.

For example, you could generate some Odin code based on the contents of some files.

Code generation in Odin simply means running a program that somehow outputs an Odin source file. Thanks to Odin being simple enough, generating code in this way is actually feasible and straightforward.

I've made a basic code generation example for the Odin Examples repository. You can find it here .

Chapter 27

Where to find more Odin resources

There's a great list of resources, libraries and software on **Jakub Tomšů's Awesome Odin page**

If you're looking for reference material, then have a look at the **official overview** and also **demo.odin** .

There are a lot of useful examples in the official **examples repository** .

I also recommend reading the **Frequently Asked Questions page** .

Chapter 28

Thanks for reading!

Thank you for buying and reading my book! I hope you found it useful.

If you have any questions or want to discuss something I wrote, then you can **join my Discord server** . It's a friendly place to talk about Odin and also about game development.

Have a nice day!

/Karl Zylinski

Chapter 29

Appendix A: Handle-based array

Since I originally wrote the code in this appendix, a similar type has been added to the `core` collection. It's called a handle-based map. Read more about it **here** .

> Don't worry about the usage of "array" versus "map". You can think of it as a map because it "maps handles to values". But it also works mostly like an array internally. The handle-based map does not use Odin's `map` type.

Also, I have written two extensive blog posts on this topic: **Part 1** and **part 2** . As mentioned in part 2, I have set up a **repository with the implementations** . These blog posts and the implementations all predate the handle-based map in `core`, so in most cases I'd just recommend you to use that one. I hope you may still find the blog posts enlightening.

I'll keep this appendix here because it is a fairly simple implementation of the handle-based array. Here's the code:

```
// Handle-based array. Used like this:
//
// Entity_Handle :: distinct Handle
// entities: Handle_Array(Entity, Entity_Handle)
//
// It expects the type used (in this case
// Entity) to contains a field called
// `handle`, of type `Entity_Handle`.
//
// You can then fetch entities using
// ha_get() and add handles using ha_add().
//
// This is just an example implementation.
// There are many ways to implement this.
//
// Another example by Bill:
// https://gist.github.com/gingerBill/7282ff54744838c52cc80c559f697051
//
// Bill's example is a bit more
// complicated, but a bit more efficient.
// It doesn't have to loop through all items
// to "skip unoccupied holes".
//
// Jakub also has an implementation of
// something similar. But his version
```

```
    // doesn't use dynamic memory:
    // https://github.com/jakubtomsu/sds/blob/main/pool.odin

    package game

    Handle :: struct {
        idx: u32,
        gen: u32,
    }

    HANDLE_NONE :: Handle {}

    Handle_Array :: struct($T: typeid, $HT: typeid) {
        items: [dynamic]T,
        freelist: [dynamic]HT,
        num: int,
    }

    ha_clear :: proc(ha: ^Handle_Array($T, $HT)) {
        clear(&ha.items)
        clear(&ha.freelist)
    }

    ha_delete :: proc(ha: Handle_Array($T, $HT)) {
        delete(ha.items)
        delete(ha.freelist)
    }

    ha_add :: proc(ha: ^Handle_Array($T, $HT), v: T) -> HT {
        v := v

        if len(ha.freelist) > 0 {
            h := pop(&ha.freelist)
            h.gen += 1
            v.handle = h
            ha.items[h.idx] = v
            ha.num += 1
            return h
        }

        if len(ha.items) == 0 {
            append_nothing(&ha.items) // Item at index zero is always "dummy" used
    for zero comparison
        }

        idx := u32(len(ha.items))
```

```
        v.handle.idx = idx
        v.handle.gen = 1
        append(&ha.items, v)
        ha.num += 1
        return v.handle
    }

    ha_get :: proc(ha: Handle_Array($T, $HT), h: HT) -> (T, bool) {
        if h.idx > 0 && int(h.idx) < len(ha.items) && ha.items[h.idx].handle == h {
            return ha.items[h.idx], true
        }

        return {}, false
    }

    ha_get_ptr :: proc(ha: Handle_Array($T, $HT), h: HT) -> ^T {
        if h.idx > 0 && int(h.idx) < len(ha.items) && ha.items[h.idx].handle == h {
            return &ha.items[h.idx]
        }

        return nil
    }

    ha_remove :: proc(ha: ^Handle_Array($T, $HT), h: HT) {
        if h.idx > 0 && int(h.idx) < len(ha.items) && ha.items[h.idx].handle == h {
            append(&ha.freelist, h)
            ha.items[h.idx] = {}
            ha.num -= 1
        }
    }

    ha_valid :: proc(ha: Handle_Array($T, $HT), h: HT) -> bool {
        return ha_get(ha, h) != nil
    }

    // Iterators for iterating over all used
    // slots in the array. Used like this:
    //
    // ent_iter := ha_make_iter(your_handle_based_array)
    // for e in ha_iter_ptr(&ent_iter) {
    //
    // }

    Handle_Array_Iter :: struct($T: typeid, $HT: typeid) {
        ha: Handle_Array(T, HT),
        index: int,
```

```
}

ha_make_iter :: proc(ha: Handle_Array($T, $HT)) -> Handle_Array_Iter(T, HT) {
    return Handle_Array_Iter(T, HT) { ha = ha }
}

ha_iter :: proc(it: ^Handle_Array_Iter($T, $HT)) -> (val: T, h: HT, cond: bool) {
    in_range := it.index < len(it.ha.items)

    for in_range {
        cond = it.index > 0 && in_range && it.ha.items[it.index].handle.idx > 0

        if cond {
            val = it.ha.items[it.index]
            h = it.ha.items[it.index].handle
            it.index += 1
            return
        }

        it.index += 1
        in_range = it.index < len(it.ha.items)
    }

    return
}

ha_iter_ptr :: proc(it: ^Handle_Array_Iter($T, $HT)) -> (val: ^T, h: HT, cond:
bool) {
    in_range := it.index < len(it.ha.items)

    for in_range {
        cond = it.index > 0 && in_range && it.ha.items[it.index].handle.idx > 0

        if cond {
            val = &it.ha.items[it.index]
            h = it.ha.items[it.index].handle
            it.index += 1
            return
        }

        it.index += 1
        in_range = it.index < len(it.ha.items)
    }

    return
}
```

Chapter 30

Appendix B: Using only fixed arrays

Some people avoid dynamic memory allocations completely. How easy or hard this is depends on the type of program you are making. It is an approach people who crave control can try. It's not something I do personally, but my friend Jakub Tomšů has made an entire video game using this method. It's called **Solar Storm**

It's also **on Steam** , but the other link goes to itch and the itch version comes with Odin source code.

I've talked about why one should **avoid doing many small heap allocations**. If you avoid that, then most of your remaining dynamic memory allocations might go into allocating different forms of arrays. If you want to, then you can take things one step further: Don't use any dynamically allocated arrays at all.

A basic example is something like this:

```
package main

import "core:fmt"

MAX_NUMBERS :: 4096

numbers: [MAX_NUMBERS]int
num_numbers: int

add_number :: proc(n: int) {
    if num_numbers == len(numbers) {
        fmt.eprintln("Out of numbers")
        return
    }

    numbers[num_numbers] = n
    num_numbers += 1
}

main :: proc() {
    add_number(7)
    add_number(42)

    // cool program that uses these numbers
}
```

This example has a fixed array called `numbers` that contains `4096` integers. We also have a `num_numbers` variable to keep track of how many things in `numbers` we have actually used. `add_numbers` uses `num_numbers` to put a number into `numbers` and then increases `num_numbers`.

The code above is simple, but you probably want functionality for removing items etc. We've actually talked about a fixed array wrapper that implements exactly this: The **Fixed Capacity Dynamic Array** (FCD Array)! Here's how you can use it to do the same thing:

```
package main

MAX_NUMBERS :: 4096
numbers: [dynamic; MAX_NUMBERS]int

main :: proc() {
    append(&numbers, 7)
    append(&numbers, 5)
    append(&numbers, 42)

    // Removes the number `5` (which is at
    // index 1) from the array.
    unordered_remove(&numbers, 1)

    // cool program that uses these numbers
}
```

Using only fixed arrays and FCD Arrays requires you to figure out *reasonable upper estimates* of how big to make your arrays. In some types of programs this is easier. I think perhaps when making certain video games it's quite easy to come up with these "budget estimates". But if you are making for example a video editing software where the memory usage for a small project is tens of megabytes while the memory usage for a big project can be tens of gigabytes, then it is harder to come up with reasonable estimates. This is why I say that this method is more suited for some types of programs than others. With some experience you'll probably be able to tell if it's applicable or not.

It's also possible to create a version of the handle-based array that I showed in **appendix A** that instead of a dynamic array uses a fixed array. Jakub shows how in his **"Static Data Structures" repository** . With fixed arrays you will no longer have the issue that pointers get invalidated, since the array can't grow. But a handle-based array is still useful, since the generation counter on the handles make it possible to know if a slot has been reused.

Finally, one thing you can do to force yourself to work like this is to set the default allocator to the *panic allocator*:

```
context.allocator = runtime.panic_allocator()

import "base:runtime"
```

With the panic allocator, any allocation done using `context.allocator`, will make your program crash on purpose. You'll be able to see the line it crashed on and find the attempted allocation.

Many programmers who prefer fixed memory still use the temporary allocator. This way, you can still create temporary dynamic arrays, temporary strings etc. But the memory you use for more "permanent" storage is all non-dynamic.

Chapter 31

Appendix C: `gui_dropdown` from CAT & ONION

This shows how an implementation of `gui_dropdown` from the **dropdown case study** can look.

This code has dependencies on some other things in the CAT & ONION codebase, so it's not possible to copy-paste and directly use. If you buy CAT & ONION **on itch** then you get the full source code, which includes the editor and this dropdown.

```
gui_dropdown :: proc(rect: Rect, values: []$T, names: []string, cur: T, label:
string = "") -> (T, bool) #optional_ok {
    rect := gui_property_label(rect, label)

    rl.DrawRectangleRec(rect, ColorControlBackground)
    rl.DrawRectangleLinesEx(rect, 1, ColorControlBorder)

    cur_idx := -1

    for v, v_idx in values {
        if v == cur {
            cur_idx = v_idx
        }
    }

    id := ui_next_id()

    if cur_idx != -1 {
        n := names[cur_idx]
        rl.DrawTextPro(font, temp_cstring(n), pos_from_rect(rect) +
{MetricControlTextMargin, 0}, {}, 0, MetricFontHeight, 0, ColorControlText)

        rl.DrawTextPro(font, "v", pos_from_rect(rect) + {rect.width - 10, 0}, {},
0, MetricFontHeight, 0, ColorControlText)
    }

    if mouse_in_rect(rect) {
        ui.next_hover = id
    }

    if ui.active == id  {
        if ui.clicked_in_overlay == id || ui.clicked == id {
            ui.active = 0
        }
```

```
        active_data := transmute(^DropdownActiveData)(&ui.active_data[0])

        drect := rect
        drect.y += drect.height
        drect.height = f32(len(names) * 20 + (len(names) - 1) * 2)

        mwm := rl.GetMouseWheelMove()

        if mwm != 0 {
            active_data.scroll_offset += mwm * 5
        }

        drect.y += active_data.scroll_offset

        if drect.y + drect.height > f32(rl.GetScreenHeight()) {
            drect.y -= drect.height + 20
        }

        ui_begin_overlay()
        rl.DrawRectangleRec(drect, ColorControlBackground)

        if mouse_in_rect(drect) {
            ui.next_hover_in_overlay = id
        }

        for n, n_idx in names {
            row := cut_rect_top(&drect, 20, n_idx == 0 ? 0 : 2)

            if n_idx == cur_idx {
                rl.DrawRectangleRec(row, ColorControlBackgroundActive)
            } else {
                if mouse_in_rect(row) {
                    rl.DrawRectangleRec(row, ColorControlBackgroundHover)
                }
            }

            if mouse_in_rect(row) {
                if rl.IsMouseButtonPressed(.LEFT) {
                    ui_end_overlay()
                    ui.active = 0
                    return values[n_idx], true
                }
            }

            rl.DrawTextPro(font, temp_cstring(n), pos_from_rect(row)+
    {MetricControlTextMargin, 0}, {}, 0, MetricFontHeight, 0, ColorControlText)
```

```
            }

            ui_end_overlay()
        } else if ui.clicked == id {
            ui.active = id
            ui.active_data = {}
        }

        return cur, false
    }
```

Chapter 32

Appendix D: Box2D and raylib

```
// Odin + Box2D + Raylib example with
// stacking boxes and a shape attached to
// the cursor that can smack the shapes.

// Made during this stream:
// https://www.youtube.com/watch?v=LYW7jdwEnaI

// I have updated this to use the
// `vendor:box2d` bindings instead of the
// ones I used on the stream.

package game

import b2 "vendor:box2d"
import rl "vendor:raylib"
import "core:math"

Box :: struct {
    body: b2.BodyId,
    shape: b2.ShapeId,
}

create_box :: proc(world_id: b2.WorldId, pos: b2.Vec2) -> Box {
    body_def := b2.DefaultBodyDef()
    body_def.type = .dynamicBody
    body_def.position = pos
    body := b2.CreateBody(world_id, body_def)

    box := b2.MakeBox(20, 20)
    box_def := b2.DefaultShapeDef()
    shape := b2.CreatePolygonShape(body, box_def, box)

    return {
        body = body,
        shape = shape,
    }
}

main :: proc() {
    rl.InitWindow(1280, 720, "Box2D + Raylib example")
```

```
    world_def := b2.DefaultWorldDef()
    world_def.gravity = b2.Vec2{0, -1}
    world_id := b2.CreateWorld(world_def)

    ground := rl.Rectangle {
        0, 600,
        1280, 120,
    }

    ground_body_def := b2.DefaultBodyDef()
    ground_body_def.position = b2.Vec2{ground.x, -ground.y - ground.height}
    ground_body_id := b2.CreateBody(world_id, ground_body_def)

    ground_box := b2.MakeBox(ground.width, ground.height)
    ground_shape_def := b2.DefaultShapeDef()
    ground_shape := b2.CreatePolygonShape(ground_body_id, ground_shape_def,
ground_box)

    bodies: [dynamic]Box

    // This generates the initial placement
    // for the boxes.
    for x in 0..<10 {
        for y in 0..< 5 {
            px := f32(x*20 + 400)
            py := f32(y*20 - 400)
            b := create_box(world_id, {px, py})
            append(&bodies, b)
        }
    }

    // A circle that we'll move with the mouse.
    circle_body_def := b2.DefaultBodyDef()
    circle_body_def.type = .dynamicBody
    circle_body_def.position = b2.Vec2{0, 4}
    circle_body_id := b2.CreateBody(world_id, circle_body_def)

    circle_shape_def := b2.DefaultShapeDef()
    circle_shape_def.density = 1000

    circle: b2.Circle
    circle.radius = 40
    circle_shape := b2.CreateCircleShape(circle_body_id, circle_shape_def,
circle)

    TIME_STEP :: 1.0 / 60.0
```

```
        TIME_SUB_STEPS :: 4

        for !rl.WindowShouldClose() {
            rl.BeginDrawing()
            rl.ClearBackground(rl.BLACK)

            rl.DrawRectangleRec(ground, rl.RED)
            mouse_pos := rl.GetMousePosition()

            b2.Body_SetTransform(circle_body_id, {mouse_pos.x, -mouse_pos.y}, {})
            b2.World_Step(world_id, TIME_STEP, TIME_SUB_STEPS)

            for b in bodies {
                position := b2.Body_GetPosition(b.body)
                a := b2.Rot_GetAngle(b2.Body_GetRotation(b.body))

                // There's a negation of
                // position.y because Box2D is
                // Y up and Raylib is Y down.
                rl.DrawRectanglePro({position.x, -position.y, 40, 40}, {20, 20}, -
    a*math.DEG_PER_RAD, rl.YELLOW)
            }

            rl.DrawCircleV(mouse_pos, 40, rl.MAGENTA)
            rl.EndDrawing()
        }

        // You don't really need to destroy all
        // these things if you're going to destroy
        // the world. But in a simulation where you
        // have a persistent world and constantly
        // add and remove bodies you'll need to do
        // stuff like this.
        for b in bodies {
            b2.DestroyShape(b.shape, false)
            b2.DestroyBody(b.body)
        }

        b2.DestroyShape(ground_shape, false)
        b2.DestroyBody(ground_body_id)
        b2.DestroyShape(circle_shape, false)
        b2.DestroyBody(circle_body_id)

        b2.DestroyWorld(world_id)
```

```
        rl.CloseWindow()
    }
```

Chapter 33

About the author

Karl Zylinski is an independent game developer and programming educator. He is the author of Understanding the Odin Programming Language, a book that teaches Odin in an approachable way. Karl is the creator of the video game **CAT & ONION** . It was the first commercial video game made in the Odin Programming Language. He also runs **a YouTube channel** where he shares educational material on Odin and game development.

In the past Karl has worked as a game engine programmer at Our Machinery, Bitsquid and Autodesk. He has also worked as a game programmer at Hazelight (A Way Out) and Friendly Foe (SOULBOUND).

Karl has a bachelor's degree in astrophysics. In his free time he likes to play video games, hike, go bouldering and play piano.

Visit his website at **zylinski.se** .

Chat about Odin and game development on his **Discord Server** .